THIRSTY'S CODEX

STANDARDS EDITION

VOLUME III / 03 OF 20

REQUIREMENTS THROUGH HARDWARE ARCHITECTURE

Lifecycle Sections 019-060

NO PROOF, NO CLAIM

CONTINUOUS MASTER FOLIO / SOURCE-PRESERVING EDITION

Edition Statement

THIRSTY'S CODEX / STANDARDS EDITION / SOURCE-PRESERVING SET

This volume is one part of a continuous 20-volume edition. Canonical lifecycle section numbers remain stable across the set; no volume restarts or renumbers them.

The visual treatment has been rebuilt as a unified standards library: coordinated covers, volume architecture, source dividers, running headers, continuous master folios, local page numbers, and source-identity marks.

The source corpus remains authoritative over the presentation layer. Status terms are preserved exactly. CONTENT-COMPLETE does not silently become source-reconciled, hostile-reviewed, owner-accepted, ratified, or published.

Every registered source witness is identified by path, byte count, and SHA-256 digest. The complete source archive is preserved as an embedded attachment in Volume XX, while the two supplied foundational PDFs are preserved as attachments in Volume I.

CONTINUITY RULE

Read the master folio and canonical section sequence as one uninterrupted work. Each succeeding volume begins where the preceding volume ends.

Standards Edition Set Map

VOLUME I
Foundations and Governance
Doctrine, prompts, front matter, shared governance, roles, and output contracts

VOLUME II
Research, Strategy, and Risk
Lifecycle Sections 001-018

VOLUME III
Requirements through Hardware Architecture
Lifecycle Sections 019-060

VOLUME IV
Electrical Engineering through User Validation
Lifecycle Sections 061-106

VOLUME V
Software Systems through Smart Contracts
Lifecycle Sections 107-150

VOLUME VI
Data, Intelligence, and Human Oversight
Lifecycle Sections 151-188

VOLUME VII
Cybersecurity through Privacy Engineering
Lifecycle Sections 189-218

VOLUME VIII
Law, Assurance, and Validation
Lifecycle Sections 219-257

VOLUME IX
Configuration through Delivery Infrastructure
Lifecycle Sections 258-300

VOLUME X
Production Operations and Command
Lifecycle Sections 301-332

VOLUME XI
Support, Enterprise, and Roadmaps
Lifecycle Sections 333-373

VOLUME XII
Maintenance, Decommissioning, and Retirement
Lifecycle Sections 374-406

VOLUME XIII
Reference Registers and Working Documentation
Indexes, cross-references, source registers, checkpoints, and operational guides

VOLUME XIV
Standards Source Library
Preserved standards, command editions, blueprints, and source artifacts

VOLUME XV
Canonical Authority and Assurance
Authority, constitutions, policies, schemas, provenance, evaluation, and verifica...

VOLUME XVI
Applied Corpus and V3Q
Agent-output corpus, V3Q implementation, fixtures, tests, and governed examples

VOLUME XVII
Training Views and Indexes
Continuous-pretraining, evaluation, retrieval, and training views with canonical ...

VOLUME XVIII
Agents and Owner Command Standards
Agents 101 and Owner's Cockpit source-standard facsimiles

VOLUME XIX
Static Analysis and Tooling Evidence
Codacy configuration, rulesets, analysis logs, and repository tooling

Volume Contents

098 Section 019 - Requirements Engineering 6
099 Section 020 - Systems Engineering 20
100 Section 021 - Enterprise Architecture 37
101 Section 022 - Solution Architecture 46
102 Section 023 - Systems Architecture 55
103 Section 024 - Domain Architecture 65
104 Section 025 - Business Architecture 75
105 Section 026 - Capability Architecture 86
106 Section 027 - Process Architecture 97
107 Section 028 - Functional Architecture 109
108 Section 029 - Operational Architecture 122
109 Section 030 - Information Architecture 129
110 Section 031 - Data Architecture 136
111 Section 032 - Application Architecture 143
112 Section 033 - Software Architecture 150
113 Section 034 - Platform Architecture 155
114 Section 035 - Infrastructure Architecture 160
115 Section 036 - Cloud Architecture 165
116 Section 037 - Edge Architecture 170
117 Section 038 - Distributed Systems Architecture 174
118 Section 039 - Network Architecture 178
119 Section 040 - Communications Architecture 182
120 Section 041 - Integration Architecture 186
121 Section 042 - Api Architecture 190
122 Section 043 - Event Driven Architecture 194
123 Section 044 - Runtime Architecture 198
124 Section 045 - Security Architecture 202
125 Section 046 - Identity Architecture 206
126 Section 047 - Privacy Architecture 210
127 Section 048 - Governance Architecture 214
128 Section 049 - Compliance Architecture 219

Volume Contents

129 Section 050 - Safety Architecture 223
130 Section 051 - Reliability Architecture 227
131 Section 052 - Resilience Architecture 231
132 Section 053 - Accessibility Architecture 235
133 Section 054 - Content Architecture 239
134 Section 055 - Knowledge Architecture 243
135 Section 056 - Ai Architecture 247
136 Section 057 - Machine Learning Architecture 252
137 Section 058 - Robotics Architecture 256
138 Section 059 - Embedded Systems Architecture 260
139 Section 060 - Hardware Architecture 264

019

SECTION 019 - REQUIREMENTS ENGINEERING

CODEX/03_SECTIONS/019_REQUIREMENTS_ENGINEERING.txt

SHA-256 20a68a5720dba445cb319405ae738fbfdb7dbbad8bed2916713690bf54ba0755

47,130 BYTES / PRESERVED SOURCE IDENTITY

SECTION 019 — Requirements Engineering

STATUS: CONTENT-COMPLETE

PURPOSE

Transform authorized objectives, stakeholder and user needs, domain rules, feasibility conditions, risk treatments, governing constraints, and lifecycle obligations into a coherent, bounded, traceable, testable, change-controlled requirements system.

Requirements Engineering does not merely collect requests or produce a feature list. It establishes what outcomes, behaviors, qualities, constraints, controls, evidence, recovery paths, and lifecycle conditions must be satisfied; who has authority to require them; why they exist; where they apply; how they will be verified and accepted; and what downstream work must be reconsidered when they change.

REQUIRED OUTCOME

- A requirements context identifying the authorized objective, decision owners, affected parties, system boundary, environments, lifecycle scope, applicable authorities, constraints, and intended acceptance state.
- A controlled hierarchy connecting mission, business, stakeholder, user, domain, system, subsystem, component, interface, data, AI, governance, security, privacy, safety, accessibility, operational, support, maintenance, migration, deprecation, and retirement requirements as applicable.
- Atomic, unambiguous, necessary, feasible, consistent, verifiable, attributable, versioned requirements with stable identifiers, source, rationale, applicability, criticality, owner, acceptance criteria, verification method, evidence expectations, dependencies, and status.
- Explicit separation among needs, goals, requirements, constraints, assumptions, design decisions, implementation tasks, tests, preferences, recommendations, and unresolved questions.
- Bidirectional traceability from authoritative source and affected-party evidence through requirements, architecture, implementation, tests, evidence, acceptance, operation, change, and retirement.
- A conflict, dissent, ambiguity, gap, waiver, exception, and non-applicability record that prevents unresolved conditions from disappearing through synthesis.
- A governed baseline and change process that identifies impact, invalidates stale approvals and evidence, preserves history, and routes material changes to legitimate authority.
- A complete handoff to Systems Engineering and later architecture, design, implementation, testing, verification, operations, maintenance, and retirement work without forcing downstream roles to invent missing objectives or boundaries.

WHY THIS MATTERS

Systems often fail before implementation begins because the wrong thing is specified, a real need is converted into an assumed feature, a mandatory constraint is treated as a preference, or a vague requirement cannot be tested. A polished design can satisfy an undocumented interpretation while violating the owner's actual objective, excluding affected users, weakening a safety boundary, or creating an operating burden nobody accepted.

AI increases both the reach and the danger of requirements work. It can rapidly organize evidence, identify gaps, generate alternatives, and draft acceptance criteria. It can also make unsupported assumptions appear complete, silently broaden scope, normalize one stakeholder's preference into a universal rule, invent priorities, or translate model capability into permission to act. Governed Requirements Engineering keeps synthesis useful while preserving source identity, uncertainty, dissent, human authority, and the right to reject or revise a proposed requirement.

APPLICABILITY

- A product, service, system, platform, application, device, process, model, agent, integration, migration, deployment, operational capability, maintenance change, or retirement action must be defined before design or implementation.
- Multiple people, systems, environments, jurisdictions, data classes, interfaces, or lifecycle stages create different obligations or variants.

- Stakeholder statements, user findings, domain rules, feasibility conditions, or risk treatments must become actionable and verifiable.
- Work has material security, privacy, safety, accessibility, governance, legal, financial, environmental, reliability, recovery, or human-impact consequences.
- Existing requirements are incomplete, contradictory, stale, untraceable, implementation-specific, or disconnected from tests and evidence.
- A change may invalidate architecture, code, data, models, tests, approvals, procedures, training, support, or retirement plans.
- AI systems participate in analysis, recommendation, communication, content generation, tool use, autonomous action, or decisions affecting people or protected state.

NON-APPLICABILITY CONDITIONS

- The task is a small, reversible correction whose exact required behavior, source authority, acceptance criterion, test method, affected scope, and rollback condition are already current and verified.
- An authoritative requirements baseline already covers the exact objective, version, environment, user population, interfaces, lifecycle conditions, and change, and no material upstream condition has changed.
- The work is pure observation or research and no design, implementation, procurement, policy, operational, or lifecycle commitment is being made yet; findings must still be preserved for later requirements work.
- A requested statement is actually a legal prohibition, missing authority, active incident, confirmed defect, or hard dependency that requires direct handling rather than being disguised as a negotiable requirement.
- Required elicitation or validation would expose protected people, systems, data, intellectual property, or regulated activity without authority.

A non-applicability decision must identify the existing controlling requirement or evidence, its version and scope, the actor authorized to rely on it, and the trigger that would reopen Requirements Engineering. Schedule pressure, familiarity, or an AI-generated specification is not sufficient.

PREREQUISITES AND DEPENDENCIES

- Sections 001–018 as applicable, including the current objective, evidence base, future conditions, strategies, management boundaries, business analysis, market evidence, affected-party map, user research, domain model, feasibility conditions, and risk treatments.
- An authenticated human objective and legitimate authority to define, approve, prioritize, reject, amend, waive, or retire requirements.
- Declared scope, system boundary, environments, operating modes, lifecycle stages, affected parties, decision horizon, non-goals, and prohibited actions.
- Current source artifacts and system state, including contracts, policies, laws, standards, architecture, interfaces, data, models, incidents, support evidence, operations, maintenance, and retirement obligations where they exist.
- A controlled terminology set and method for distinguishing mandatory normative language from recommendation, permission, example, assumption, and commentary.
- A durable requirements repository or equivalent record supporting stable identifiers, versioning, provenance, relationships, review, status, history, and export.
- Defined verification and acceptance authority, evidence expectations, change control, conflict resolution, and escalation paths.

REQUIRED INPUTS

- Authorized mission, business, product, program, project, and operational objectives; desired outcomes; mission invariants; non-goals; success and failure conditions.
- Stakeholder identities, authority, rights, interests, commitments, conflicts, grievances, and affected non-users.
- User jobs, journeys, language, mental models, accessibility needs, consent expectations, trust conditions, failure experiences, recovery expectations, and research limitations.
- Domain terminology, bounded contexts, actors, entities, value objects, rules, invariants, events, states, transitions, exceptions, sources of truth, and jurisdictional conditions.
- Market, competitive, supplier, platform, integration, legal, regulatory, scientific, technical, operational, and organizational evidence relevant to the objective.

- Feasibility determinations, conditions, assumptions, disqualifiers, dependencies, resource limits, experiments, points of no return, and evidence that would change the conclusion.
- Risk statements, mandatory limits, treatments, controls, indicators, thresholds, contingencies, residual-risk decisions, expiry conditions, and safe-halt requirements.
- Existing requirements, specifications, contracts, service levels, schemas, policies, tests, defects, incidents, change records, waivers, exceptions, and acceptance evidence.
- Architecture and implementation constraints that are truly fixed, together with the authority and rationale making them constraints rather than preferences.
- AI and model identity, intended role, allowed and prohibited actions, data boundaries, evaluation needs, uncertainty behavior, human review, correction, appeal, fallback, revocation, monitoring, and provider dependencies where applicable.

HUMAN RESPONSIBILITIES AND AUTHORITY

Authorized humans retain authority over objectives, values, protected interests, mandatory constraints, priority, acceptable tradeoffs, approval, risk acceptance, commitments, and final acceptance. Requirements may be delegated for drafting or analysis, but legitimate authority is not transferred by fluent text or tool access.

- Confirm the objective, scope, system boundary, affected parties, environments, lifecycle horizon, non-goals, and decision owners.
- Identify which sources are authoritative for mission, law, policy, domain meaning, user need, technical state, budget, schedule, risk, and acceptance.
- Decide conflicts involving values, rights, safety, privacy, accessibility, environmental burden, employment, public impact, cost, schedule, and residual risk.
- Approve the normative language, criticality, priority method, baseline, variants, exceptions, waivers, and change authority.
- Ensure that affected people and qualified specialists are represented when their rights, safety, access, domain knowledge, or operating burden matter.
- Confirm that a proposed requirement expresses the intended need without silently embedding an unapproved design or excluding legitimate alternatives.
- Accept, reject, revise, defer, narrow, or retire requirements and record the rationale, conditions, dissent, evidence, and review trigger.
- Refuse downstream advancement when critical requirements, authority, acceptance criteria, or verification methods remain materially unresolved.
- Reopen the baseline when objective, evidence, law, domain, user population, architecture, risk, environment, model, provider, or operating conditions materially change.

AI RESPONSIBILITIES AND BOUNDARIES

The AI may organize source material, identify candidate needs, normalize terminology, detect ambiguity and conflict, propose requirement statements, generate traceability links, draft acceptance criteria, and expose missing evidence. It may not invent binding objectives, priorities, stakeholder consent, legal meaning, acceptable harm, resource commitments, or authority.

- Distinguish verbatim source statements, interpreted needs, derived requirements, proposed requirements, approved requirements, assumptions, constraints, preferences, design decisions, and unresolved questions.
- Preserve source identity, version, date, context, speaker or authority, evidence quality, and known limitations for every material derivation.
- Mark AI-produced language as proposed until an authorized review state is recorded.
- Do not turn frequency of mention, stakeholder status, model confidence, or persuasive wording into priority or legitimacy.
- Do not silently broaden a requirement from one user, environment, jurisdiction, version, or scenario to all contexts.
- Do not narrow a requirement merely to fit current architecture, available code, model limitations, budget, schedule, or implementation convenience.
- Identify ambiguous terms, hidden quantifiers, subjective adjectives, compound obligations, missing conditions, conflicting sources, unverifiable claims, and undefined actors.
- Propose alternatives where a need can be met through multiple designs; preserve the requirement at the outcome or constraint level unless design is legitimately fixed.

- Separate mandatory constraints from weighted preferences so prohibited options cannot win through aggregate scoring.
- State where legal, safety, medical, financial, regulatory, accessibility, scientific, or other specialist interpretation requires qualified human review.
- Treat missing evidence as unknown or unresolved rather than generating a plausible requirement to close the gap.
- Route value conflicts, scope changes, waivers, exceptions, and final acceptance to the authorized human owner.

APPLICABLE EXISTING AGENT ROLES

Primary roles:

- Analyst — structures needs, relationships, conflicts, dependencies, alternatives, and decision implications while separating evidence from interpretation.
- Architect — tests whether requirements are structurally coherent, allocable, interface-aware, governable, recoverable, and compatible with system boundaries without rewriting them to fit a preferred design.
- Governance Reviewer — checks authority, precedence, mandatory constraints, delegation, waivers, exceptions, status transitions, and acceptance legitimacy.
- Planner — turns the requirements effort into bounded elicitation, analysis, validation, baseline, change, and verification work with owners, dependencies, gates, and evidence.

Evidence, human, and domain roles:

- Researcher — obtains authoritative technical, legal, regulatory, scientific, market, supplier, and domain evidence for requirements and constraints.
- User Experience Reviewer — evaluates complete journeys, accessibility, cognitive load, trust, consent, error, denial, correction, and recovery requirements.
- Observer — captures current behavior, state, events, workflows, gaps, and operational conditions without unsupported interpretation.
- Security Auditor — identifies security, identity, authorization, data, supply-chain, evidence-integrity, and abuse requirements.
- Documentation Writer — maintains clear human-readable specifications, rationale, examples, decisions, and version history.
- Memory Curator — preserves requirement lineage, supersession, conflicts, decisions, change history, evidence, and reopening triggers.

Assurance and coordination roles:

- Coordinator — assigns bounded work, preserves provenance, reconciles compatible outputs, surfaces conflicts, and prevents parallel roles from silently producing different baselines.
- Critic — challenges ambiguity, hidden assumptions, missing requirements, contradictions, implementation bias, false completeness, and unsupported priority.
- Tester — converts requirements into verifiable conditions, identifies untestable statements, and maintains coverage and defect linkage.
- Verifier — independently checks requirement identity, source, quality, traceability, evidence, verification state, and acceptance boundaries.
- Red Team — attacks bypass paths, omitted adversarial conditions, unsafe defaults, privilege assumptions, evidence manipulation, and failure-recovery gaps.
- Decision Arbiter Advisory — structures recommendations on conflicting requirements or alternatives under authorized criteria while leaving binding value decisions with the authorized human.
- Implementer — provides current-state and implementation-deviation evidence and must not alter requirements without authorization.

Role selection must match the work, consequence, and required independence. Existing source-defined role contracts remain controlling and are not redefined by this section.

REQUIRED ACTIVITIES

1. Establish the requirements context

Define the authorized objective, scope, boundary, environments, lifecycle stages, affected parties, sources of authority, decision owners, constraints, non-goals, acceptance authority, and intended baseline.

2. Assemble and classify source material
Collect upstream findings, current system evidence, contracts, policies, standards, incidents, user and stakeholder evidence, feasibility conditions, risk treatments, and existing specifications. Classify each source by authority, scope, version, freshness, and limitation.

3. Define terminology and normative language
Establish canonical terms and controlled meanings for SHALL, SHALL NOT, SHOULD, SHOULD NOT, MAY, proposed, approved, verified, accepted, waived, superseded, and not applicable. Prevent ordinary prose from being mistaken for a binding obligation.

4. Identify needs and obligations
Extract mission needs, stakeholder needs, user needs, domain obligations, legal and policy duties, operational needs, risk treatments, lifecycle needs, and evidence needs without prematurely selecting implementation.

5. Separate statement types
Classify each item as objective, outcome, need, requirement, constraint, assumption, dependency, design decision, implementation task, acceptance criterion, test condition, preference, recommendation, issue, or unresolved question.

6. Define the requirements hierarchy
Organize requirements from mission and business through stakeholder, user, system, subsystem, component, interface, data, AI, governance, quality, operation, support, maintenance, migration, deprecation, and retirement levels as applicable.

7. Draft atomic requirements
Write one primary obligation per requirement. Identify actor or subject, required behavior or quality, object, triggering conditions, operating conditions, boundary, prohibited behavior, and measurable threshold where appropriate.

8. Assign stable identity and provenance
Give every requirement a unique stable identifier and record source, rationale, derivation, authority, author, owner, date, version, scope, and affected parties.

9. Define applicability and variants
State environments, jurisdictions, products, editions, user groups, roles, data classes, operating modes, lifecycle phases, feature states, and exceptions. Record legitimate non-applicability instead of deleting the requirement.

10. Classify requirement type and criticality
Distinguish functional, quality, interface, data, constraint, governance, security, privacy, safety, accessibility, reliability, performance, operational, support, maintenance, migration, deprecation, and retirement requirements. Record criticality separately from priority.

11. Analyze requirement quality
Check necessity, correctness, singularity, clarity, completeness, consistency, feasibility, verifiability, traceability, maintainability, boundedness, implementation neutrality where appropriate, and freedom from undefined terms.

12. Identify relationships and dependencies
Record parent-child, derives-from, satisfies, depends-on, conflicts-with, duplicates, refines, constrains, verifies, supersedes, and invalidates relationships. Expose circular, missing, and cross-boundary dependencies.

13. Resolve ambiguity and conflict
Return unclear statements to the appropriate source owner. Preserve competing interpretations, evidence, dissent, and value conflicts. Do not manufacture consensus or choose a value tradeoff without authority.

14. Prioritize under authorized criteria
Use mission, consequence, legal obligation, safety, security, affected-party need, dependency, evidence, cost, schedule, reversibility, and strategic value as authorized. Keep mandatory constraints outside tradeable scoring.

15. Define acceptance criteria

For every material requirement, state observable conditions for acceptance, including normal, boundary, denial, failure, degraded, recovery, rollback, migration, accessibility, security, governance, and lifecycle behavior where applicable.

16. Define verification methods
Assign inspection, analysis, demonstration, test, simulation, measurement, audit, owner acceptance, field observation, or qualified review appropriate to the claim. Identify environment, data, preconditions, threshold, evidence, and independent review needs.

17. Build bidirectional traceability
Connect each source and need to requirements, and each requirement to architecture, design, implementation, tests, evidence, acceptance, operations, maintenance, and retirement. Identify orphan sources, orphan requirements, orphan tests, and unsupported claims.

18. Validate with legitimate participants
Review requirements with authorized owners, affected stakeholders, users, operators, maintainers, support, domain experts, security, privacy, safety, accessibility, legal, and other specialists according to consequence and applicability.

19. Analyze feasibility and risk feedback
Ensure requirements preserve established feasibility conditions and risk treatments. When a requirement makes the option infeasible or creates new risk, reopen the relevant upstream decision rather than silently weakening the requirement.

20. Baseline the requirement set
Freeze an attributable version with included and excluded scope, open issues, accepted dissent, source and evidence versions, approvals, hashes or immutable identity, and effective date. A draft collection is not a baseline.

21. Control change
Require a change request with source, rationale, authority, affected requirements, impact, evidence, alternatives, migration, compatibility, cost, risk, test, documentation, and rollback implications.

22. Invalidate stale downstream state
When a material requirement changes, identify affected designs, code, configurations, data, models, interfaces, tests, evidence, approvals, procedures, training, releases, operations, maintenance, and retirement records. Do not preserve green status through a changed requirement.

23. Hostile-review the baseline
Search for omitted affected parties, unsafe assumptions, bypasses, hidden design commitments, untestable language, conflicting obligations, stale evidence, AI-generated inventions, inaccessible journeys, missing recovery, and retirement gaps.

24. Handoff and maintain continuity
Deliver the exact baseline, traceability, open questions, decisions, evidence, acceptance boundaries, change process, and safe next action so Systems Engineering and downstream roles can proceed without hidden conversational memory.

HUMAN-AI INTERACTION REQUIREMENTS

- The interface must visibly distinguish source text, human interpretation, AI synthesis, AI proposal, approved requirement, accepted baseline, and verified result.
- Users must be able to inspect the source, rationale, authority, affected parties, dependencies, acceptance criteria, evidence method, change history, and current status of every material requirement.
- AI-generated requirements must remain proposed until an authorized actor explicitly approves or adopts them within a defined scope.
- The AI must explain material ambiguity and present alternatives rather than forcing a single interpretation through confident wording.
- Humans must be able to correct terminology, reject derivations, preserve dissent, narrow scope, revoke approval, and reopen validation without losing history.
- The system must not use dark patterns, urgency, confidence scores, default selections, or conversational pressure to obtain requirement approval or risk acceptance.

- High-consequence requirements must expose uncertainty, missing evidence, affected-party implications, and the consequences of approval, rejection, delay, or change.
- User, owner, operator, maintainer, and affected non-user needs must remain distinguishable; one group must not silently stand in for another.
- Capability, permission, authority, requirement, implementation, test result, and acceptance must remain separate states.
- Critical stop, correction, appeal, evidence inspection, and fallback paths must remain available when an AI model or provider is unavailable or degraded.
- Synthetic personas, simulated stakeholders, model-generated user stories, and inferred needs must be labeled and cannot replace actual human evidence where actual human evidence is required.
- Requirement discussions that materially change objective, values, rights, commitments, cost, schedule, or risk must return to the authorized human decision owner.

ARCHITECTURE AND SYSTEM IMPLICATIONS

Requirements Engineering must produce durable structures that downstream systems can enforce and verify, not only prose that can be reinterpreted.

- Represent requirements as versioned objects with stable identifiers, type, normative force, text, source, rationale, scope, applicability, criticality, priority, owner, status, acceptance criteria, verification method, evidence, relationships, and change history.
- Maintain both human-readable and machine-readable representations with consistency checks between them.
- Use explicit state transitions such as PROPOSED, UNDER REVIEW, APPROVED, BASELINED, IMPLEMENTED, VERIFIED, ACCEPTED, DEFERRED, WAIVED, REJECTED, SUPERSEDED, RETIRED, and REOPENED, with authority and evidence rules for each transition.
- Preserve immutable baseline identities and bind approvals, architecture, builds, tests, evidence, and releases to the exact requirements version inspected.
- Provide bidirectional dependency graphs and invalidation propagation so changed upstream sources or requirements reopen affected downstream claims.
- Separate requirements from architecture decisions while supporting explicit allocation and satisfaction links.
- Encode mandatory security, privacy, safety, governance, accessibility, identity, authority, evidence, and lifecycle constraints at enforceable system boundaries where applicable.
- Support variants by environment, jurisdiction, product, role, data class, deployment mode, feature state, and lifecycle stage without uncontrolled copying or drift.
- Preserve source and dissent records even when a final requirement wording is selected.
- Ensure event, API, data, model, and interface requirements have versioned contracts, compatibility rules, error semantics, and evidence hooks.
- Instrument runtime outcomes, incidents, support, user behavior, model behavior, performance, recovery, and maintenance so post-implementation evidence can validate or challenge requirements.
- Provide export, backup, restore, migration, archival, and retirement mechanisms for the requirements corpus and its traceability.

OWNER COCKPIT AND USER-CONTROL REQUIREMENTS

The owner-facing operating surface or equivalent durable control record should expose:

- Current baseline identity, version, scope, effective date, owner, approval state, environments, variants, and open exceptions.
- Counts and lists by status, type, criticality, source, owner, subsystem, lifecycle stage, verification state, and affected party.
- Every requirement's source, rationale, authority, text, interpretation history, dependencies, acceptance criteria, verification method, evidence, dissent, and change history.
- Orphan needs, orphan requirements, orphan tests, missing evidence, unresolved ambiguity, conflicts, stale sources, expired approvals, and invalidated downstream artifacts.
- Mandatory constraints separately from preferences and priorities.
- Proposed AI derivations and the human action required before they become approved requirements.
- Impact analysis before change approval, including architecture, implementation, data, model, interface, test, release, operation, support, maintenance, migration, deprecation, and retirement effects.

- Controls to approve, reject, revise, defer, waive, supersede, retire, reopen, compare versions, request evidence, record dissent, and revoke stale approval.
- Traceability navigation from objective and source through requirement, implementation, test, evidence, acceptance, operation, and lifecycle outcome.
- Clear distinction between drafted, approved, baselined, implemented, verified, accepted, and operationally observed states.
- Safe-halt or advancement-block controls when critical requirements, authority, acceptance criteria, or required evidence are missing.
- Plain-language explanation of what changed, why it changed, what remains unresolved, who must decide, and the next safe action.

REQUIRED OUTPUTS AND ARTIFACTS

- Requirements Engineering plan and governance model.
- Requirements context, scope, boundary, authority, stakeholder, affected-party, and source registers.
- Controlled glossary and normative-language guide.
- Need, objective, obligation, constraint, assumption, dependency, issue, and unresolved-question registers.
- Requirements hierarchy and allocation model.
- Versioned requirements catalogue with stable identifiers and complete metadata.
- Functional, quality, interface, data, AI, governance, security, privacy, safety, accessibility, operational, support, maintenance, migration, deprecation, and retirement requirement sets as applicable.
- Applicability, variant, non-applicability, exception, waiver, and expiry records.
- Requirement-quality review results and correction history.
- Conflict, dissent, ambiguity, duplication, gap, and decision records.
- Priority and criticality model with authority and rationale.
- Acceptance-criteria catalogue and requirement-to-verification matrix.
- Bidirectional source-to-need-to-requirement-to-design-to-test-to-evidence-to-acceptance traceability matrix.
- Architecture allocation and interface obligation package.
- Baseline record with identity, scope, version, approvals, open findings, hashes or immutable reference, and effective date.
- Change requests, impact analyses, invalidation records, migration needs, compatibility decisions, and supersession history.
- Hostile-review findings, remediation, residual gaps, and escalation records.
- Systems Engineering and downstream handoff package with exact baseline and safe next action.

EVIDENCE REQUIREMENTS

- Every material requirement must identify its authoritative or evidentiary source, version, date, context, scope, owner, and derivation path.
- A stakeholder statement is evidence of that stakeholder's statement, not automatically proof of universal need, technical fact, legal obligation, or priority.
- User requirements must identify participant population, context, method, consent and privacy limits, accessibility coverage, sample limitations, and evidence confidence.
- Domain requirements must identify the controlling definition, rule, jurisdiction, effective date, source of truth, and qualified review where required.
- Feasibility and risk-derived requirements must link to the exact condition, treatment, control, threshold, or decision that created them.
- Legal, regulatory, safety, medical, financial, environmental, accessibility, security, scientific, and other specialist requirements require authoritative or qualified evidence proportionate to consequence.
- Requirement approval must identify the approving authority, exact version and scope, unresolved conditions, and evidence available at the time.
- Verification evidence must identify the requirement, artifact, version, environment, method, inputs, expected result, actual result, date, actor, and limitations.
- Evidence supporting a requirement must remain distinct from evidence showing implementation satisfaction.
- Superseded, failed, contradicted, stale, or withdrawn evidence must remain traceable rather than being deleted from history.

- Missing evidence must produce an explicit UNKNOWN, UNRESOLVED, PROPOSED, or BLOCKED state; it must not be repaired through polished language.
- AI-generated text, synthetic data, inferred needs, and model-based classifications must be labeled with model or method provenance and independently reviewed where material.

VERIFICATION AND ACCEPTANCE TESTS

- Sample every requirement class and verify stable identity, source, rationale, scope, authority, acceptance criteria, verification method, owner, status, and history.
- Trace representative objectives and source statements forward to requirements, architecture, implementation, tests, evidence, and acceptance; trace representative tests and artifacts backward to an authorized requirement.
- Remove or invalidate a source and confirm dependent requirements and downstream evidence are flagged for review rather than remaining silently approved.
- Change a material requirement after approval and confirm stale architecture, tests, evidence, releases, and approvals are invalidated or explicitly re-evaluated.
- Submit ambiguous, compound, subjective, implementation-biased, contradictory, and unverifiable statements and confirm the quality process rejects or returns them for correction.
- Attempt to approve or change a requirement without valid authority and verify denial, audit evidence, and preserved prior state.
- Have an AI propose a plausible requirement without an authoritative source and verify it remains proposed and cannot become binding automatically.
- Introduce conflicting stakeholder or specialist requirements and verify both positions, evidence, authority, and unresolved value conflict remain visible.
- Verify mandatory constraints cannot be outweighed by priority scoring or bypassed through UI, API, agent, automation, import, bulk edit, or administrative path.
- Confirm every mandatory requirement maps to at least one appropriate verification method and current result state, including untestable or blocked where truthful.
- Identify orphan needs, orphan requirements, orphan tests, orphan evidence, and implementation with no governing requirement.
- Test variant and applicability logic across required environments, jurisdictions, roles, data classes, operating modes, and lifecycle stages.
- Run owner and affected-user acceptance on complete journeys, including failure, denial, correction, recovery, accessibility, and fallback, rather than demonstration alone.
- Restart a downstream Systems Engineering or architecture role from the handoff package and verify it can reconstruct scope, baseline, obligations, open decisions, and safe next action without chat memory.

FAILURE STATES AND ANTI-PATTERNS

- Treating a backlog, feature list, prompt, user story collection, contract excerpt, or architecture diagram as a complete requirements baseline.
- Writing vague requirements such as fast, intuitive, secure, intelligent, seamless, scalable, robust, or user-friendly without defined conditions and evidence.
- Combining multiple obligations in one statement so partial satisfaction can masquerade as completion.
- Inventing requirements, priorities, consent, authority, budgets, deadlines, laws, integrations, or user needs to make the specification appear complete.
- Treating one stakeholder, one user, one model, or one environment as representative of all affected contexts.
- Converting the current implementation into a requirement merely because it already exists.
- Weakening or narrowing a requirement to fit a preferred architecture, available code, vendor, model, schedule, or budget without authorized change.
- Embedding a design solution where the underlying need should permit alternatives, or omitting a fixed design constraint that is genuinely mandatory.
- Using priority labels without criteria, owner, rationale, criticality, or dependency analysis.
- Allowing mandatory legal, safety, privacy, security, accessibility, or governance constraints to compete in weighted scoring.
- Accepting a demonstration, mockup, unit test, code review, or developer statement as proof of owner outcome or complete requirement satisfaction.
- Maintaining one-way traceability that cannot reveal orphan implementation, orphan tests, or stale approvals.

- Deleting superseded requirements, failed evidence, dissent, or rejected alternatives and thereby destroying decision history.
- Treating missing evidence as low priority, low risk, not applicable, or implicitly accepted.
- Approving a requirement set while critical ambiguity, conflict, authority, acceptance criteria, or verification methods remain unresolved.
- Allowing AI-generated wording to become binding through repetition, default selection, import, or conversational momentum.
- Failing to include denial, failure, degraded, recovery, rollback, migration, maintenance, deprecation, and retirement requirements.
- Reporting implementation complete when requirements are merely drafted, or reporting requirements verified when only their presence was inspected.

RECOVERY AND RECONSIDERATION

- Quarantine suspect requirements, baselines, approvals, or traceability links while preserving the last known accepted state and all evidence.
- Restore the last valid baseline when unauthorized edits, corruption, import defects, model hallucination, or uncontrolled merging occurs.
- Reopen elicitation and analysis with the appropriate source owner when ambiguity, conflict, missing affected parties, unsupported derivation, or changed context is discovered.
- Reclassify a statement when it was confused with a design decision, task, test, assumption, preference, or legal prohibition.
- Re-run impact analysis and revoke stale approvals, tests, evidence, releases, or conformance claims after material change.
- Narrow scope, create variants, defer, split requirements, stage a pilot, or return to feasibility and risk analysis where full satisfaction is not currently possible.
- Enter safe halt for affected downstream work when a critical requirement, authority boundary, safety condition, or evidence obligation cannot be trusted.
- Preserve failed interpretations and prior baselines with supersession links so recurring mistakes can be recognized.
- Define the evidence, authority, participant review, specialist review, test, or decision required to close each reopened condition.
- Resume only from a durable continuity record identifying exact baseline, changes, open findings, affected artifacts, owner decisions, and next safe action.

COMPLETION CRITERIA

Requirements Engineering is complete for the declared baseline only when:

- The objective, scope, boundary, environments, lifecycle stages, affected parties, non-goals, authority, and acceptance owners are explicit.
- All in-scope authoritative sources and material upstream findings have been reviewed, classified, and linked or explicitly excluded with rationale.
- Needs, requirements, constraints, assumptions, dependencies, design decisions, tasks, tests, and unresolved questions are distinguishable.
- Every material requirement has stable identity, source, rationale, scope, applicability, type, criticality, owner, status, acceptance criteria, verification method, and traceability.
- Requirements are sufficiently atomic, clear, consistent, feasible, necessary, bounded, and verifiable for downstream use.
- Mandatory constraints and prohibited conditions are separated from preferences and tradeable priorities.
- Critical ambiguity, conflict, missing authority, and missing acceptance criteria are resolved or visibly block affected advancement.
- Feasibility conditions and risk treatments are represented as requirements, controls, gates, tests, monitoring, recovery, or lifecycle obligations as appropriate.
- Human-AI interaction requirements preserve disclosure, correction, appeal, evidence inspection, authority, and non-deceptive fallback.
- Bidirectional traceability exposes source coverage, requirement coverage, test coverage, evidence state, and orphan artifacts.

- The baseline is versioned, attributable, preserved, reviewed by applicable roles, and approved by legitimate authority.
- Change, invalidation, supersession, waiver, exception, recovery, and reopening processes are defined and testable.
- The handoff allows Systems Engineering and downstream roles to proceed without inventing missing objectives, requirements, authority, or acceptance boundaries.

Completion does not mean every requirement is implemented or verified. It means the declared requirements baseline is sufficiently governed, bounded, and evidenced to control the next lifecycle stage truthfully.

HANDOFF AND CONTINUITY REQUIREMENTS

The Requirements Engineering handoff must include:

- Baseline identifier, version, hash or immutable reference, effective date, scope, environments, variants, lifecycle coverage, owner, and approval state.
- Objective, non-goals, affected parties, sources of authority, glossary, normative-language rules, and system boundary.
- Complete requirements catalogue with type, criticality, priority, owner, applicability, source, rationale, acceptance criteria, verification method, and status.
- Bidirectional traceability and explicit orphan, gap, conflict, dissent, ambiguity, waiver, exception, and non-applicability records.
- Feasibility conditions, risk treatments, mandatory constraints, thresholds, safe-halt conditions, recovery obligations, and points of no return.
- Open decisions, unresolved evidence, required specialists, review dates, change triggers, and evidence that would change the baseline.
- Downstream allocation expectations for Systems Engineering, architecture, data, interfaces, security, privacy, safety, accessibility, AI, testing, operations, support, maintenance, migration, deprecation, and retirement.
- Change authority, change-request process, impact-analysis method, invalidation rules, supersession history, rollback or restoration procedure, and audit trail.
- Exact next safe action and a statement of whether downstream work may proceed, proceed conditionally, or must remain blocked.

Downstream roles must report requirement conflicts, implementation deviations, impossible acceptance criteria, new risks, failed tests, incidents, and changed operating conditions back into the requirements system. Chat history is not an acceptable continuity mechanism.

RELATED CODEX SECTIONS

001 Research
002 Foresight / Futures
003 Innovation
004 Mission Strategy
005 Business Strategy
006 Product Strategy
007 Portfolio Management
008 Program Management
009 Project Management
010 Product Management
011 Business Analysis
012 Market Research
013 Competitive Intelligence
014 Stakeholder Management
015 User Research
016 Domain Analysis
017 Feasibility Analysis
018 Risk Management
020 Systems Engineering
021 Enterprise Architecture

022 Solution Architecture
023 Systems Architecture
024 Domain Architecture
025 Business Architecture
026 Capability Architecture
027 Process Architecture
028 Functional Architecture
029 Operational Architecture
030 Information Architecture
031 Data Architecture
032 Application Architecture
033 Software Architecture
041 Integration Architecture
042 API Architecture
044 Runtime Architecture
045 Security Architecture
046 Identity Architecture
047 Privacy Architecture
048 Governance Architecture
049 Compliance Architecture
050 Safety Architecture
051 Reliability Architecture
052 Resilience Architecture
053 Accessibility Architecture
054 Content Architecture
055 Knowledge Architecture
056 AI Architecture
057 Machine Learning Architecture
058 Robotics Architecture
059 Embedded Systems Architecture
060 Hardware Architecture
075 Human–Computer Interaction
106 User Validation
107 Software Engineering
151 Database Engineering
167 Machine Learning Engineering
178 Agent Systems Engineering
182 AI Safety
183 AI Governance
189 Cybersecurity Engineering
217 Trust and Safety
218 Privacy Engineering
219 Legal
220 Regulatory Compliance
222 Ethics
226 Audit
227 Internal Controls
231 Software Testing
246 Security Testing
247 Accessibility Testing
251 Compliance Testing
252 Safety Testing
253 Model Testing
256 Verification
257 Validation
289 Change Management

301 Site Reliability Engineering
303 Runtime Operations
319 Incident Management
326 Business Continuity
327 Disaster Recovery
331 Operational Readiness
332 Command and Control
333 Technical Support
341 Documentation Engineering
374 Maintenance Engineering
382 Data Migration Operations
383 Platform Migration
388 Version Management
389 Deprecation Management
390 Sunset Planning
405 Lifecycle Governance
406 System Retirement

SOURCE BASIS AND UNRESOLVED SOURCE GAPS

Primary supplied source basis:

- Thirsty's Standard v3 supplies the governing posture of truthful scope, current-state inspection, explicit unknowns, evidence before claims, minimal authorized change, complete work for the declared mode, verification, hostile review, continuity, and no fake completion.
- Thirsty's Agents 101 supplies the unchanged role contracts and governance stack: authorized human objective, global operating contract, bounded roles, task and authority envelope, execution and evidence, and independent review. Its Planner, Analyst, Architect, Researcher, Coordinator, Tester, Verifier, Governance Reviewer, Critic, Red Team, User Experience Reviewer, Documentation Writer, Memory Curator, Security Auditor, Implementer, and Decision Arbiter Advisory roles inform this section without being rewritten.
- Thirsty's Owner's Cockpit Standard supplies requirements-as-contract use, normative language, assignment of implementation owner and evidence method, requirements traceability, owner acceptance, authority and approval boundaries, versioned action and evidence contracts, change control, conformance scope, and owner ratification. That source is identified as a candidate for owner ratification and is not silently treated as universally ratified authority.
- Thirsty's Standard V4 Design Blueprint supplies draft concepts relevant to typed claims, obligations, evidence contracts, acceptance criteria, role assembly, rollback, reversal conditions, dissent preservation, artifact consistency, and trusted transition. The source is draft and unratified; these concepts are used as design input, not claimed as ratified governance.
- Sections 001–018 provide the current Codex working-draft handoff of evidence, objective, strategy, management, affected parties, user evidence, domain meaning, feasibility conditions, risk treatments, controls, acceptance boundaries, and human decision points.

Unresolved source gaps:

- The supplied corpus does not contain a complete, ratified, universal Requirements Engineering standard covering every industry, jurisdiction, safety classification, hardware domain, medical context, public-sector obligation, defense environment, financial system, or AI application.
- This section has not yet been reconciled against all current external Requirements Engineering standards, sector-specific regulations, contractual regimes, accessibility requirements, safety standards, or organizational procedures.
- Quantitative quality thresholds, document forms, traceability depth, approval models, and independent-review obligations must be specialized according to consequence, domain, law, lifecycle, and owner authority.
- Later architecture, engineering, testing, verification, operations, maintenance, and retirement sections may expose missing or conflicting requirements and must be permitted to reopen this baseline.
- STATUS remains CONTENT-COMPLETE. This section has not yet completed full source reconciliation, independent hostile review, owner acceptance, implementation, or operational verification.

020

SECTION 020 - SYSTEMS ENGINEERING

CODEX/03_SECTIONS/020_SYSTEMS_ENGINEERING.txt

SHA-256 a9ebeaa75fab0bd1a6d7c6b8e8e3642491f6f51523a65f22cda9eb297f82bbe7

58,875 BYTES / PRESERVED SOURCE IDENTITY

SECTION 020 — Systems Engineering

STATUS: CONTENT-COMPLETE

PURPOSE

Transform the approved requirements baseline and its upstream evidence into a coherent whole-system definition, lifecycle plan, technical baseline, integration strategy, verification strategy, validation strategy, transition path, and governed decision structure.

Systems Engineering coordinates the system of interest across people, processes, software, hardware, data, models, agents, facilities, suppliers, external services, operating environments, enabling systems, and lifecycle stages. It does not replace the specialist disciplines that design, implement, test, operate, maintain, or retire those elements. It ensures that their work remains connected to one authorized objective, one controlled set of requirements and interfaces, one visible system boundary, and one evidence-backed path from need to trusted operation and eventual retirement.

REQUIRED OUTCOME

- A declared system of interest with purpose, owner, boundary, environments, users, operators, affected parties, external systems, enabling systems, lifecycle stages, and non-goals.
- An operational concept describing how the system will be used, supervised, supported, degraded, recovered, changed, and retired through complete mission and owner journeys.
- A system context model showing external actors, systems, organizations, data sources, authorities, dependencies, interfaces, trust boundaries, failure domains, and sources of truth.
- A controlled allocation of approved requirements, constraints, risk treatments, feasibility conditions, and acceptance obligations to system functions, elements, interfaces, people, processes, and lifecycle activities.
- Functional, logical, physical, operational, information, data, authority, control, evidence, and lifecycle views that are consistent with one another and traceable to the requirements baseline.
- Defined system elements with responsibilities, inputs, outputs, dependencies, resources, interfaces, states, failure behavior, recovery behavior, configuration identity, ownership, and verification method.
- A versioned interface control model covering technical, human, organizational, data, event, model, authority, evidence, security, safety, privacy, accessibility, and operational interfaces as applicable.
- A lifecycle model integrating concept, development, acquisition, implementation, integration, verification, validation, transition, operation, support, maintenance, modernization, migration, deprecation, decommissioning, disposal, and retirement.
- A technical management system for baselines, configuration, decisions, risks, opportunities, changes, reviews, measures, evidence, deviations, waivers, nonconformities, and continuity.
- A system integration, verification, validation, transition, operations, support, recovery, and retirement strategy sufficient to guide downstream architecture and engineering without forcing those disciplines to invent missing system obligations.
- Explicit separation among system need, requirement, architecture, design, implementation, integration, verification, validation, acceptance, operation, and observed outcome.
- A bounded human-AI operating model in which AI capability, assigned role, delegated authority, execution permission, evidence, and final human decision authority remain distinct.

WHY THIS MATTERS

A collection of individually competent components can still fail as a system. Interfaces can disagree, requirements can be allocated to nobody, local optimizations can weaken the whole, recovery can depend on the failed component, operators can receive contradictory status, and a technically correct implementation can be unusable in the environment where it must work. These failures often emerge between disciplines, organizations, lifecycle stages, and sources of truth rather than inside one component.

Systems Engineering makes those relationships explicit before implementation cost and operational consequence make them difficult to change. It keeps the system objective, human outcome, requirements baseline, architecture, interfaces, evidence, risks, verification, validation, operation, and retirement connected as one governed technical thread.

AI intensifies both the opportunity and the risk. AI can map complex dependencies, compare alternatives, detect inconsistencies, maintain traceability, generate models, and assist analysis. It can also create convincing diagrams unsupported by implementation, invent interfaces, conceal uncertainty in synthesized language, treat model capability as authority, or optimize a subsystem while degrading safety, privacy, accessibility, resilience, human control, or mission success. Governed Systems Engineering uses AI as a bounded analytical and execution participant without surrendering human authority, evidence discipline, or whole-system accountability.

APPLICABILITY

- A product, platform, application, service, device, model, agent, robot, embedded system, infrastructure, organizational process, or socio-technical capability contains multiple interacting elements or lifecycle stages.
- The system crosses software, hardware, data, AI, human, organizational, physical, operational, supplier, or jurisdictional boundaries.
- Multiple teams, roles, vendors, models, repositories, environments, or authorities must produce one integrated outcome.
- Requirements must be allocated among system elements and verified at different levels.
- Interfaces, dependencies, trust boundaries, failure domains, external services, or sources of truth materially affect success.
- Security, privacy, safety, accessibility, governance, compliance, reliability, resilience, supportability, maintainability, recoverability, or environmental obligations span more than one component.
- The system must operate under degraded, disconnected, emergency, maintenance, migration, or retirement conditions.
- A change to one element may invalidate system behavior, interfaces, evidence, approvals, training, operations, or lifecycle plans.
- AI or autonomous functions can recommend, decide, communicate, control tools, modify state, affect people, or influence protected outcomes.
- Acquisition, procurement, manufacturing, deployment, transition, support, modernization, decommissioning, or disposal requires coordinated technical evidence.

NON-APPLICABILITY CONDITIONS

- The work is a bounded research question, text correction, or isolated reversible change whose system boundary, requirements, interfaces, affected elements, verification, recovery, and acceptance are already current and verified.
- An approved and current Systems Engineering baseline already covers the exact system version, environment, operating concept, requirements, interfaces, lifecycle stage, and proposed change, and impact analysis confirms no material system effect.
- The work has not yet reached a decision to create, change, acquire, integrate, deploy, operate, or retire a system; upstream research and requirements evidence must still be preserved for later use.
- A confirmed incident, unauthorized action, safety hazard, security compromise, data-loss condition, or failed mandatory control requires immediate containment before normal systems analysis continues.
- The requested analysis would require unauthorized access to protected systems, people, data, suppliers, facilities, or regulated information.

A non-applicability determination must identify the controlling baseline, exact scope, version, environment, interfaces, evidence, authorized decision owner, and reopening trigger. A small team, simple diagram, familiar technology, or AI-generated architecture is not sufficient reason to omit Systems Engineering.

PREREQUISITES AND DEPENDENCIES

- Sections 001-019 as applicable, including the authenticated objective, evidence base, future conditions, strategies, management boundaries, affected parties, user evidence, domain model, feasibility conditions, risk treatments, and approved requirements baseline.

- A legitimate human owner or delegated authority for system scope, values, resource commitments, acceptance, residual risk, and lifecycle decisions.
- A controlled requirements baseline with stable identity, traceability, acceptance criteria, verification methods, variants, open issues, and change authority.
- Declared system objective, non-goals, system of interest, preliminary boundary, environments, lifecycle horizon, users, operators, affected parties, and external dependencies.
- Current-state evidence for existing systems, interfaces, architecture, data, models, infrastructure, operations, incidents, support, maintenance, suppliers, contracts, and retirement obligations.
- Defined terminology and distinction among requirements, architecture, design, implementation, verification, validation, and acceptance.
- A durable technical-data environment or equivalent record for system models, baselines, interfaces, decisions, requirements, risks, evidence, changes, and continuity.
- Access to qualified specialist review proportionate to security, safety, privacy, accessibility, legal, financial, physical, environmental, medical, industrial, or other consequence.
- A change, configuration, evidence, review, escalation, and safe-halt process.

REQUIRED INPUTS

- Approved requirements baseline, including requirements, constraints, applicability, variants, criticality, acceptance criteria, verification methods, source evidence, approvals, waivers, exceptions, and unresolved conditions.
- Mission, business, product, program, project, operational, support, maintenance, modernization, and retirement objectives and constraints.
- Stakeholder and affected-party map, user jobs and journeys, owner outcomes, operator tasks, accessibility needs, consent expectations, trust conditions, and grievance or appeal paths.
- Domain model, terminology, bounded contexts, entities, value objects, rules, states, events, invariants, sources of truth, jurisdictions, and exceptions.
- Feasibility conditions, risk register, treatment plan, control objectives, thresholds, indicators, residual-risk decisions, safe-halt conditions, points of no return, and evidence that would change prior conclusions.
- Existing system context, architecture, configuration, interfaces, schemas, inventories, dependencies, suppliers, external services, environments, facilities, networks, hardware, software, data, models, and operational procedures.
- Incident, defect, performance, reliability, availability, security, privacy, safety, accessibility, support, maintenance, recovery, and user-outcome evidence.
- Resource, schedule, budget, procurement, manufacturing, deployment, staffing, training, support, licensing, supply-chain, and disposal constraints.
- Applicable law, regulation, contract, policy, standard, certification, assurance, and owner governance requirements.
- AI and model details including identity, version, provider, intended function, capabilities, limitations, data use, tool access, autonomy mode, evaluation evidence, fallback, monitoring, revocation, and human review.

HUMAN RESPONSIBILITIES AND AUTHORITY

Authorized humans retain authority over the system objective, values, protected interests, system boundary, acceptable tradeoffs, resource commitments, residual risk, approval, acceptance, deployment, operation, suspension, decommissioning, and retirement. Systems analysis may be delegated; legitimacy and accountability may not be manufactured by role title, model fluency, or technical access.

- Confirm the system of interest, system owner, decision owners, users, operators, affected parties, environments, lifecycle stages, non-goals, and protected invariants.
- Approve or reject changes to the requirements baseline, system boundary, operational concept, architecture direction, major interfaces, autonomy level, and lifecycle strategy.
- Decide value conflicts and tradeoffs involving safety, rights, privacy, accessibility, security, environmental impact, cost, schedule, employment, public effect, and residual risk.
- Ensure qualified specialists and affected participants are involved where consequence requires expertise or lived experience.
- Authorize resources, procurement, supplier commitments, data use, testing involving people or protected systems, deployment, and irreversible actions.
- Define which decisions may be delegated, the scope and duration of delegation, evidence required, review points, expiry, and revocation.

- Ratify technical baselines and transition decisions only for the exact version, environment, scope, evidence set, open findings, and residual conditions reviewed.
- Preserve the ability to stop, narrow, quarantine, roll back, supersede, or retire the system when authority, evidence, safety, or mission conditions fail.

AI RESPONSIBILITIES AND BOUNDARIES

AI systems may assist with decomposition, dependency analysis, model generation, interface discovery, alternative comparison, requirements allocation proposals, traceability, simulation support, anomaly detection, review preparation, and documentation within explicit authority and evidence boundaries.

The AI must:

- Restate the authorized objective, scope, requirements baseline, system boundary, assumptions, unknowns, and constraints before material analysis.
- Distinguish observed fact, supplied requirement, inference, assumption, model output, estimate, recommendation, decision, and verified result.
- Identify missing inputs, conflicts, unsupported interfaces, hidden dependencies, unallocated requirements, orphan elements, and evidence gaps.
- Preserve exact source, version, environment, artifact identity, and provenance for material system claims.
- Label generated models, diagrams, allocations, scenarios, estimates, and simulations as proposals until reviewed and accepted by authorized humans or qualified mechanisms.
- Remain within assigned role, tool, data, environment, and action boundaries.
- Escalate value judgments, material scope changes, residual-risk acceptance, irreversible choices, and decisions outside delegation.
- Support correction, dissent, alternative analysis, reproducibility, independent verification, and rollback.
- Degrade honestly when model, tool, provider, data, context, or evidence is unavailable or stale.

The AI must not:

- Redefine the objective, system boundary, user population, lifecycle, authority, or requirements to fit a preferred solution.
- Invent components, interfaces, integrations, resources, supplier capability, performance, certification, approval, test evidence, or operational readiness.
- Treat a diagram, model, simulation, prototype, digital twin, test stub, or generated specification as proof of implemented behavior.
- Convert technical capability into permission, approval, acceptance, or authority.
- optimize one element while silently degrading mission outcome, human control, security, privacy, safety, accessibility, resilience, evidence, supportability, or retirement.
- Approve its own architecture, verification result, validation result, risk acceptance, or release where independent human or specialist authority is required.
- Hide contradictory specialist findings or manufacture consensus.
- Continue consequential work through a failed prerequisite, invalid baseline, uncertain authority, or unresolved mandatory constraint.

APPLICABLE EXISTING AGENT ROLES

Use the existing role files unchanged. Select only the roles required for the declared systems task, define each assignment, and preserve separation among analysis, execution, review, verification, and decision.

- Architect: defines coherent system structure, elements, interfaces, data, authority, trust, failure, recovery, and verification paths from approved requirements.
- Researcher: acquires and evaluates authoritative technical, operational, scientific, supplier, regulatory, and lifecycle evidence.
- Analyst: performs option, tradeoff, sensitivity, dependency, consequence, and whole-system analysis without seizing decision authority.
- Critic: challenges completeness, internal consistency, hidden assumptions, operational realism, interfaces, evidence, and readiness claims.

- Security Auditor: examines assets, trust boundaries, identity, authorization, supply chain, runtime, evidence, recovery, and security claims.
- Implementer: performs separately authorized system-model, integration, prototype, configuration, or implementation work within approved architecture and requirements.
- Refactorer: improves structure without changing required behavior when a bounded behavior-preserving assignment exists.
- Tester: translates requirements and interfaces into reproducible normal, boundary, failure, recovery, compatibility, and governance tests.
- Verifier: independently determines whether material system claims are supported within exact scope, version, environment, and evidence limits.
- Documentation Writer: maintains accurate system descriptions, interface records, decisions, procedures, limitations, recovery, and lifecycle documentation.
- Teacher: explains system concepts, dependencies, tradeoffs, evidence, and operating procedures without replacing authority or qualified training.
- Planner: sequences system definition, integration, verification, validation, transition, operation, maintenance, and retirement work with explicit gates and dependencies.
- Coordinator: assigns and reconciles bounded specialist workstreams while preserving provenance, dissent, human authority, and one coherent system result.
- Observer: records system states, events, measurements, transitions, anomalies, failures, and evidence without unsupported interpretation.
- Memory Curator: preserves approved baselines, decisions, traceability, evidence, lessons, and continuity under retention and access rules.
- Governance Reviewer: evaluates authority, scope, delegation, policy, evidence, approval, exception, and transition compliance.
- Red Team: attacks assumptions, interfaces, authority, failure paths, recovery, bypasses, human factors, and readiness claims.
- Optimizer: improves measurable whole-system performance or cost while preserving protected properties and regression guardrails.
- User Experience Reviewer: evaluates complete owner, operator, maintainer, support, accessibility, failure, recovery, and control journeys.
- Decision Arbiter Advisory: compares system alternatives under authorized criteria, constraints, evidence, sensitivity, reversibility, and human approval boundaries.

REQUIRED ACTIVITIES

1. Accept the controlled upstream handoff
Verify the exact Requirements Engineering baseline, objective, scope, authority, sources, assumptions, variants, open findings, feasibility conditions, risk treatments, and acceptance boundaries. Do not proceed from an informal summary when the controlled baseline is available.

2. Establish the system of interest
Name the system, purpose, owner, intended outcomes, non-goals, lifecycle horizon, environments, users, operators, maintainers, affected parties, and preliminary boundary. Distinguish the system of interest from external systems and enabling systems.

3. Define the lifecycle model
Identify concept, research, acquisition, development, implementation, integration, verification, validation, transition, operation, support, maintenance, modernization, migration, deprecation, decommissioning, disposal, and retirement stages that apply. Define entry, exit, authority, evidence, and feedback for each stage.

4. Develop the operational concept
Describe complete mission, owner, user, operator, administrator, maintainer, support, auditor, incident, and retirement journeys. Include normal, denied, degraded, offline, emergency, recovery, maintenance, migration, and end-of-life conditions.

5. Model the system context

Identify external actors, organizations, systems, services, suppliers, facilities, environments, jurisdictions, sources of truth, data sources, communications, authorities, and dependencies. Define what crosses each boundary and under what contract.

6. Establish measures and success conditions
Define measures of effectiveness, measures of performance, technical performance measures, service levels, human outcomes, safety and security limits, evidence thresholds, and lifecycle success conditions. Identify owner, method, environment, range, uncertainty, and decision trigger for each measure.

7. Analyze and allocate functions
Identify functions required to achieve the operational concept. Decompose only to the level needed for responsibility, interface, risk, verification, and implementation decisions. Preserve end-to-end threads across functional boundaries.

8. Allocate requirements and constraints
Allocate each approved requirement, risk treatment, feasibility condition, and lifecycle obligation to system functions, elements, interfaces, people, processes, environments, or enabling systems. Record rationale, owner, verification level, and unresolved allocation.

9. Identify system elements and configuration items
Define the elements required to realize the functions. For each, record purpose, responsibility, inputs, outputs, dependencies, data, authority, trust, failure, recovery, owner, configuration identity, and verification method.

10. Develop and compare system alternatives
Create credible alternatives at the appropriate abstraction level. Compare them against mandatory constraints and authorized criteria including mission outcome, human impact, feasibility, risk, cost, time, complexity, performance, maintainability, supportability, interoperability, reversibility, and retirement.

11. Define coherent system views
Maintain consistent operational, functional, logical, physical, information, data, software, hardware, human, organizational, authority, trust, evidence, deployment, and lifecycle views as applicable. A view must identify its purpose, scope, source, version, assumptions, and relationship to other views.

12. Engineer interfaces
Define producer, consumer, purpose, data or resource exchanged, protocol, schema, units, timing, ordering, identity, authority, error behavior, retry, timeout, idempotency, versioning, compatibility, security, privacy, safety, observability, evidence, ownership, and change process for every critical interface.

13. Map system flows
Map material data, information, event, command, control, authority, identity, credential, model, evidence, audit, physical-energy, material, and human communication flows. Identify trust boundaries, transformations, custody, retention, and failure effects.

14. Analyze dependencies and failure domains
Classify hard, soft, informational, approval, evidence, resource, environmental, supplier, external, and review dependencies. Identify common-mode, cascading, correlated, circular, hidden, and single-point dependencies.

15. Integrate specialty engineering
Bring security, privacy, safety, accessibility, reliability, resilience, human factors, data, AI, software, hardware, embedded, network, compliance, quality, support, maintenance, environmental, manufacturing, and other applicable specialists into one traceable system model rather than appending isolated checklists.

16. Define human-AI teaming and autonomy
Specify AI identity, role, capabilities, limitations, autonomy mode, task envelope, allowed tools, data boundaries, human checkpoints, approval thresholds, explanations, uncertainty, monitoring, fallback, revocation, appeal, incident handling, and retirement. Preserve human control over values, consequential authority, and residual risk.

17. Develop integration strategy
Define integration sequence, environments, prerequisites, interface maturity, simulators, test doubles, hardware or service availability, data, configuration, rollback, evidence, defect handling, and exit criteria. Integrate according to dependency and risk, not convenience alone.

18. Develop verification strategy

Map each system requirement and interface obligation to verification level, method, environment, inputs, expected result, threshold, evidence, independence, and status. Distinguish analysis, inspection, demonstration, test, simulation, audit, and operational evidence.

19. Develop validation strategy
Define how authorized owners, users, operators, maintainers, affected parties, and qualified specialists will determine whether the integrated system solves the right problem in the intended context. Include complete journeys, realistic environments, failure, recovery, accessibility, and long-term effects.

20. Plan transition and deployment
Define readiness criteria, data and state migration, installation, configuration, credentials, training, support, monitoring, rollback, contingency, communications, owner control, and evidence required to move from development into operation.

21. Engineer operation and support
Define operating modes, roles, command and control, monitoring, alerting, incident response, service management, backup, restore, continuity, support, maintenance, capacity, configuration, documentation, training, and evidence retention.

22. Engineer change, modernization, and migration
Define change authority, impact analysis, compatibility, technical debt, dependency updates, model updates, schema changes, state migration, coexistence, rollback, forward recovery, re-verification, re-validation, and supersession.

23. Engineer decommissioning and retirement
Define service cessation, user transition, data export and deletion, record retention, credential and key revocation, dependency removal, contract termination, hardware disposition, environmental obligations, residual support, evidence preservation, and post-retirement monitoring.

24. Establish technical baselines
Create and control concept, requirements, functional, allocated, design, product, as-built, as-deployed, as-operated, and retirement baselines as applicable. Bind each to scope, version, configuration, approvals, evidence, open findings, and effective date.

25. Establish configuration and technical-data control
Identify configuration items, models, documents, schemas, software, hardware, data, models, policies, environments, procedures, and evidence under control. Define versioning, change, access, integrity, backup, restore, retention, and audit.

26. Manage technical decisions and tradeoffs
Record decision question, authority, criteria, options, evidence, assumptions, dissent, sensitivity, reversibility, selected path, conditions, rejected alternatives, and reopening triggers. Do not convert recommendations into decisions.

27. Conduct technical reviews and gates
Use evidence-based reviews appropriate to consequence, such as system requirements, functional, architecture, interface, integration readiness, test readiness, verification, validation, transition, operational readiness, and retirement reviews. A meeting or slide deck is not a passed gate.

28. Monitor technical performance and system risk
Track measures, margins, trends, assumptions, dependencies, interface maturity, defects, risks, control effectiveness, evidence status, and lifecycle readiness. Escalate threshold crossings and preserve uncertainty.

29. Control change and invalidation
When a requirement, interface, component, model, supplier, environment, authority, or lifecycle condition changes, identify all affected baselines, tests, evidence, approvals, procedures, training, deployments, operations, and retirement obligations. Reopen them as required.

30. Hostile-review and hand off
Attack the system model for orphan requirements, unallocated responsibility, hidden interfaces, circular recovery, stale baselines, unsupported performance, untested failure, autonomy drift, inaccessible control, supplier assumptions, evidence gaps, and retirement omissions. Deliver a durable package to architecture and engineering roles with exact next safe actions.

HUMAN-AI INTERACTION REQUIREMENTS

- The system must disclose when a human, AI model, agent, automation, rule engine, external service, or hybrid process produced a material recommendation, action, status, or evidence item.
- AI capability, assigned role, delegated authority, execution permission, approval, acceptance, and accountability must be represented as separate states.
- The owner must be able to inspect the system objective, boundary, operating mode, active actors, configuration, dependencies, evidence, risk, and next consequence before authorizing material action.
- AI-generated system models, allocations, interfaces, estimates, simulations, and architecture alternatives must remain proposals until reviewed under the appropriate authority.
- High-consequence recommendations must expose sources, assumptions, uncertainty, dissent, alternatives, affected parties, reversibility, and evidence that would change the recommendation.
- Humans must be able to correct terminology, reject system assumptions, narrow scope, change delegation, revoke authority, request independent review, stop execution, and restore a known-good baseline.
- No conversational path may bypass the same identity, authority, policy, state, evidence, and audit controls enforced by non-conversational interfaces.
- The interface must distinguish observed system state from inferred state, predicted state, simulated state, desired state, and planned state.
- Model or provider outage must not remove critical status, stop, diagnosis, evidence, recovery, or owner-control functions.
- Automated allocation, optimization, scheduling, routing, or decision support must preserve mandatory constraints and cannot hide minority findings or affected-party burdens.
- Complete human journeys must include denial, explanation, correction, fallback, accessibility, recovery, appeal, and exit rather than only successful task completion.
- Long-running or autonomous work must expose current assignment, elapsed state, authority expiry, resource use, evidence progress, stop conditions, and safe interruption behavior.
- Human oversight must be operationally possible, not merely documented; the human needs sufficient time, information, access, and control to intervene.
- Data collected for supervision, personalization, evaluation, or learning must follow declared privacy, consent, retention, access, and deletion requirements.
- The system must preserve dignity and non-deception for users and affected non-users even when they cannot directly operate the system.

ARCHITECTURE AND SYSTEM IMPLICATIONS

Systems Engineering must create a connected technical thread rather than a disconnected collection of documents.

- Represent the system of interest, external systems, enabling systems, actors, environments, lifecycle stages, requirements, functions, elements, interfaces, risks, decisions, tests, evidence, and baselines as versioned, attributable objects or equivalent controlled records.
- Maintain explicit relationships among objective, need, requirement, function, logical element, physical element, interface, implementation artifact, verification case, validation journey, evidence, acceptance, operation, incident, change, and retirement.
- Keep operational, functional, logical, physical, information, data, authority, trust, deployment, and lifecycle views consistent through automated or governed checks.
- Bind interface contracts to exact producer, consumer, schema, protocol, units, timing, identity, authority, error, compatibility, observability, and evidence semantics.
- Separate control plane, data plane, evidence plane, governance plane, and owner-control surfaces where doing so clarifies authority and failure behavior; do not assume separation without implementation evidence.
- Define system states and transitions for initialization, ready, active, degraded, blocked, awaiting authority, safe halt, recovery, maintenance, migration, rollback, shutdown, decommissioned, and retired as applicable.
- Design graceful degradation around essential owner and safety functions, with explicit dependency, timeout, retry, offline, reconciliation, and stale-data behavior.
- Ensure every consequential path is governed consistently through UI, API, agent, automation, scheduler, retry, administrative, and recovery interfaces.
- Preserve configuration identity across source, build, artifact, deployment, runtime, model, data, policy, hardware, environment, and evidence.

- Build observability that supports both owner situational awareness and engineering diagnosis without turning telemetry into surveillance or a substitute for verification.
- Treat AI models and external providers as bounded, versioned dependencies with failure, drift, fallback, evaluation, substitution, and retirement plans.
- Define interoperability and compatibility across supported versions, environments, platforms, data formats, protocols, and organizational boundaries.
- Include backup, restore, disaster recovery, business continuity, incident response, maintenance, supply-chain, migration, deprecation, and retirement architecture from the beginning.
- Preserve enough technical data and evidence to reproduce decisions, verify current state, investigate incidents, maintain the system, migrate it, and retire it safely.

OWNER COCKPIT AND USER-CONTROL REQUIREMENTS

The owner-facing cockpit or equivalent control record should expose:

- System name, purpose, owner, baseline, configuration, environment, operating mode, lifecycle stage, active version, and authority state.
- System context showing external systems, actors, dependencies, interfaces, trust boundaries, sources of truth, and failure domains.
- Operational journeys and mission threads, including normal, denied, degraded, recovery, maintenance, migration, and retirement paths.
- Requirement allocation and satisfaction status from mission objective through system element, interface, implementation, test, evidence, and acceptance.
- Interface health, version compatibility, ownership, recent changes, open defects, risk, and evidence status.
- Technical performance measures, margins, thresholds, trends, uncertainty, data freshness, and consequence of crossing limits.
- Current risks, controls, residual risk, treatment status, safe-halt triggers, and legitimate risk-acceptance authority.
- Integration, verification, validation, transition, operational-readiness, maintenance, and retirement progress as distinct states.
- AI and agent identity, role, model or provider, capabilities, limitations, current assignment, authority, tool access, evidence, and revocation controls.
- Baseline comparison, change impact, invalidated evidence, pending approvals, open dissent, unresolved decisions, and next safe action.
- Controls to request evidence, compare alternatives, authorize, reject, defer, narrow, stop, roll back, restore, quarantine, revoke, supersede, and retire within the owner's authority.
- Plain-language explanation of what is known, unknown, changed, degraded, blocked, failed, verified, unverified, and awaiting human decision.
- Recovery guidance that remains available when nonessential services, AI models, networks, or external providers fail.

REQUIRED OUTPUTS AND ARTIFACTS

- Systems Engineering management and lifecycle plan.
- System of interest definition, ownership record, boundary statement, non-goals, environments, lifecycle scope, and context model.
- Operational concept, concept of operations, mission threads, use cases, owner journeys, operator journeys, maintenance journeys, incident journeys, and retirement journeys as applicable.
- Stakeholder, affected-party, user, operator, maintainer, support, supplier, regulator, and external-system registers.
- Measures of effectiveness, measures of performance, technical performance measures, margins, thresholds, and monitoring plan.
- Functional analysis, functional decomposition, behavior models, state models, sequence models, and requirement allocation records.
- System element catalogue and configuration-item register.
- Operational, functional, logical, physical, information, data, authority, trust, deployment, and lifecycle architecture views at the systems level.
- Interface inventory, interface control documents or equivalent contracts, compatibility matrix, ownership, and change history.

- Data, event, command, control, authority, identity, evidence, audit, material, energy, and human communication flow models as applicable.
- Dependency, failure-domain, common-mode, cascade, and external-service analyses.
- Human-AI teaming, autonomy, capability, authority, fallback, oversight, monitoring, revocation, appeal, and retirement model.
- Specialty-engineering integration records for security, privacy, safety, accessibility, reliability, resilience, human factors, quality, supportability, maintainability, environmental, and other applicable disciplines.
- Alternative-system concepts, trade studies, sensitivity analysis, decisions, dissent, rejected alternatives, and reopening conditions.
- Integration strategy, sequence, environments, prerequisites, test resources, evidence requirements, defect process, rollback, and exit criteria.
- Requirements verification matrix and verification strategy.
- Validation plan connecting intended outcomes and complete journeys to realistic environments, participants, evidence, and acceptance authority.
- Transition, deployment, migration, training, support, operational-readiness, rollback, and continuity plans.
- Operations, maintenance, modernization, configuration, support, incident, backup, restore, disaster-recovery, and business-continuity concepts.
- Decommissioning, data disposition, credential revocation, dependency removal, hardware disposition, environmental, archival, and retirement plans.
- Controlled technical baselines, configuration records, change requests, impact analyses, waivers, deviations, nonconformities, and supersession history.
- Technical review records, gate criteria, evidence, findings, decisions, owners, and closure status.
- System risk and opportunity register linked to requirements, elements, interfaces, controls, tests, evidence, and lifecycle decisions.
- Hostile-review findings and durable downstream handoff package.

EVIDENCE REQUIREMENTS

- Every material system claim must identify system, version, configuration, environment, timeframe, scope, method, evidence source, limitations, and responsible actor.
- Each requirement allocation must identify the approved requirement baseline, allocated function or element, rationale, owner, verification level, and status.
- Each interface claim must identify both endpoints, exact versions, contract, data or resource exchanged, preconditions, error behavior, compatibility, and observed or tested evidence.
- Models and diagrams are evidence of the model or design record, not proof that the represented system exists or behaves as shown.
- Simulation results must identify model, assumptions, calibration, inputs, scenario, uncertainty, validation basis, and limits of applicability.
- Supplier, vendor, open-source, model-provider, hardware, certification, and service claims require current attributable evidence and must distinguish contractual promise from independently observed behavior.
- Verification evidence must identify requirement, test or analysis case, expected result, actual result, environment, configuration, data, actor, time, logs, deviations, and reproducibility.
- Validation evidence must identify intended users or owners, realistic context, complete journey, observed outcome, failure and recovery behavior, accessibility coverage, limitations, and acceptance authority.
- Runtime evidence must distinguish current measured state from cached, inferred, simulated, estimated, or predicted state.
- Risk controls must have implementation and effectiveness evidence; documentation or configuration presence alone is insufficient.
- Baseline and approval evidence must bind to exact artifacts, requirements, interfaces, configuration, environment, open findings, and authority.
- Failed, blocked, inconclusive, stale, superseded, or contradicted evidence must remain visible and linked to later resolution.
- Missing evidence must result in UNKNOWN, UNVERIFIED, BLOCKED, CONDITIONAL, or NOT APPLICABLE with rationale, not implied success.
- AI-generated content must retain model, provider, version, prompt or task context where permitted, tool outputs, source links, review state, and limitations proportionate to consequence.

VERIFICATION AND ACCEPTANCE TESTS

- Trace representative objectives from source through requirement, function, system element, interface, implementation, verification evidence, validation outcome, acceptance, and operational observation.
- Trace representative components and interfaces backward to authorized requirements and forward to tests, evidence, operation, maintenance, and retirement obligations.
- Verify every mandatory requirement is allocated or explicitly retained at system level with owner and rationale.
- Introduce an orphan requirement, orphan component, orphan interface, and orphan test and confirm the technical-data system identifies each.
- Change a material requirement, interface schema, external dependency, model version, authority rule, or operating environment and confirm affected baselines, evidence, approvals, and readiness states are invalidated or reviewed.
- Exercise normal, boundary, denied, degraded, offline, partial-failure, common-mode, recovery, rollback, migration, maintenance, and retirement scenarios.
- Disconnect or corrupt each critical external dependency in turn and verify essential status, stop, evidence, diagnosis, and recovery paths remain available or fail safely.
- Verify interface contracts reject malformed, unauthorized, stale, incompatible, duplicate, replayed, out-of-order, timed-out, and partially processed interactions as applicable.
- Confirm configuration identity can be traced from requirements and source through build, artifact, deployment, runtime, model, data, policy, hardware, test, and evidence.
- Have independent reviewers reproduce representative verification claims without relying solely on the producing role's interpretation.
- Validate complete owner, user, operator, maintainer, support, incident, and recovery journeys in realistic environments and with appropriate accessibility coverage.
- Disable AI models and external providers and confirm critical owner controls and deterministic safety boundaries remain operational.
- Attempt to let an AI-generated allocation, architecture decision, risk acceptance, or readiness recommendation become binding without authorized approval and verify rejection.
- Introduce conflicting specialist findings and confirm disagreement, evidence, scope, methods, authority, and decision owner remain visible.
- Verify technical reviews cannot pass when required evidence, authority, interface maturity, critical risk closure, verification, validation, rollback, or continuity is missing.
- Restore from backup or a previous baseline and verify configuration, data, authority, interfaces, evidence, and owner journeys after recovery.
- Restart an architecture or engineering role from the Systems Engineering handoff and verify it can reconstruct the system context, baseline, allocations, interfaces, risks, decisions, and next safe action without chat history.

FAILURE STATES AND ANTI-PATTERNS

- Treating Systems Engineering as diagram production, documentation after implementation, project administration, or a single architecture review.
- Defining the system boundary around the preferred solution while excluding inconvenient users, operators, suppliers, external systems, support, maintenance, or retirement.
- Decomposing components without preserving end-to-end mission and owner journeys.
- Allocating requirements to teams or documents instead of accountable system elements, interfaces, people, processes, and lifecycle activities.
- Leaving requirements, risks, controls, interfaces, or verification responsibilities unallocated.
- Using one view as though it represents operational, logical, physical, authority, trust, data, and lifecycle truth simultaneously.
- Treating models, simulations, prototypes, test doubles, dashboards, or digital twins as proof of production behavior.
- Inventing integrations, performance, capacity, supplier capability, certification, resources, schedules, or readiness.
- Freezing architecture before requirements, interfaces, feasibility conditions, risk treatments, human outcomes, and lifecycle obligations are understood.
- Local optimization that degrades whole-system mission, security, privacy, safety, accessibility, resilience, maintainability, supportability, operator control, evidence, or retirement.
- Interface-by-conversation: relying on informal agreement, shared assumptions, or chat history instead of versioned contracts.

- Circular recovery in which the recovery path depends on the failed component, credential, network, provider, model, or data source.
- Assuming external services, models, vendors, networks, users, operators, facilities, and data remain available, trustworthy, compatible, or unchanged.
- Conflating verification with validation, test completion with acceptance, execution success with verified outcome, or deployment with operational readiness.
- Allowing AI capability or autonomy to expand without explicit requirement, authority, monitoring, fallback, revocation, and evidence changes.
- Hiding conflicts among specialist disciplines through consensus language or aggregate scoring.
- Treating security, privacy, safety, accessibility, reliability, support, maintenance, and retirement as downstream add-ons.
- Maintaining stale baselines, undocumented deviations, untracked configurations, or evidence that cannot be bound to the exact system tested.
- Declaring the system complete while critical interfaces, dependencies, degraded modes, recovery, migration, support, or retirement remain undefined or unverified.

RECOVERY AND RECONSIDERATION

- Quarantine suspect baselines, models, interface contracts, allocations, evidence, approvals, and readiness claims while preserving the last known accepted state.
- Restore a known-good technical baseline when unauthorized change, corruption, incompatible merge, model hallucination, supplier drift, or uncontrolled configuration occurs.
- Reopen Requirements Engineering when system analysis exposes ambiguous, infeasible, conflicting, untestable, unallocated, or missing requirements.
- Reopen feasibility and risk analysis when architecture, interfaces, dependencies, scale, environment, suppliers, models, controls, cost, schedule, or lifecycle assumptions change.
- Re-run system context, dependency, interface, failure, security, privacy, safety, accessibility, resilience, support, maintenance, and retirement analyses after material change.
- Narrow scope, partition the system, create variants, stage integration, use a pilot, select a reversible experiment, or defer advancement when evidence is insufficient.
- Enter safe halt for affected work when authority, configuration identity, a mandatory requirement, critical interface, safety condition, evidence chain, or recovery path cannot be trusted.
- Preserve failed alternatives, invalid assumptions, prior baselines, rejected decisions, incidents, and recovery evidence with supersession links.
- Define exact evidence, specialist review, owner decision, test, validation journey, configuration restoration, or interface correction required to reopen advancement.
- Resume only from a durable continuity package that names the exact baseline, current state, invalidated artifacts, open findings, authority, and next safe action.

COMPLETION CRITERIA

Systems Engineering is complete for the declared system baseline and lifecycle decision only when:

- The system of interest, purpose, owner, objective, boundary, environments, lifecycle stages, users, operators, affected parties, external systems, enabling systems, non-goals, and authority are explicit.
- The exact approved requirements baseline and all material feasibility conditions, risk treatments, mandatory constraints, variants, waivers, exceptions, and open findings are identified.
- Operational concepts and complete journeys cover normal, denied, degraded, failure, recovery, maintenance, migration, and retirement conditions as applicable.
- Measures of effectiveness, performance, technical performance, human outcome, limits, margins, evidence methods, and decision thresholds are defined.
- Every material requirement is allocated or explicitly retained at system level with rationale, owner, interface effects, and verification method.
- System elements, responsibilities, interfaces, dependencies, trust boundaries, failure domains, flows, states, recovery, ownership, and configuration identity are defined at sufficient depth for downstream work.
- Operational, functional, logical, physical, information, data, authority, trust, deployment, and lifecycle views are consistent or their disagreements are visible.

- Security, privacy, safety, accessibility, reliability, resilience, human factors, governance, compliance, supportability, maintainability, environmental, and other applicable specialty obligations are integrated.
- Human-AI roles, capability, authority, autonomy, oversight, fallback, revocation, appeal, evidence, and incident behavior are bounded and testable.
- Integration, verification, validation, transition, operation, support, maintenance, change, migration, decommissioning, and retirement strategies are defined with entry, exit, evidence, and authority conditions.
- Technical baselines, configuration control, interface control, technical decisions, risk, change, review, evidence, and continuity processes are operationally defined.
- Critical ambiguity, missing authority, unallocated requirements, unresolved mandatory interface, unacceptable residual risk, missing recovery, and missing acceptance evidence block affected advancement.
- Applicable human owners and qualified specialists have reviewed the exact baseline within their authority.
- Downstream architecture and engineering roles can proceed without inventing system objective, requirements, interfaces, authority, evidence, or lifecycle conditions.

Completion does not mean the system has been implemented, integrated, verified, validated, deployed, accepted, or proven in operation. It means the declared Systems Engineering baseline is sufficiently coherent, governed, traceable, and evidenced to control those downstream stages truthfully.

HANDOFF AND CONTINUITY REQUIREMENTS

The Systems Engineering handoff must include:

- System name, identifier, baseline version, immutable reference or hash, owner, approval state, scope, environments, lifecycle stage, effective date, and safe-to-proceed determination.
- Authorized objective, non-goals, protected invariants, affected parties, system of interest, external systems, enabling systems, system context, and operating concept.
- Exact requirements baseline, allocation, variants, feasibility conditions, risk treatments, mandatory constraints, acceptance criteria, verification methods, waivers, exceptions, and open findings.
- System elements, configuration items, functions, behaviors, states, flows, interfaces, dependencies, trust boundaries, failure domains, sources of truth, and ownership.
- Alternative concepts, trade studies, technical decisions, dissent, assumptions, sensitivity, reversibility, rejected alternatives, and reopening conditions.
- Measures, thresholds, margins, monitoring, risks, controls, residual risk, safe-halt conditions, and decision authority.
- Integration, verification, validation, transition, deployment, operational-readiness, support, maintenance, migration, recovery, decommissioning, and retirement strategies.
- Technical baselines, configuration records, interface contracts, change history, invalidated artifacts, evidence register, review status, and unresolved decisions.
- Exact assignments for downstream Enterprise, Solution, Systems, Domain, Business, Capability, Process, Functional, Operational, Information, Data, Application, Software, Platform, Infrastructure, Security, AI, hardware, engineering, testing, verification, validation, operations, maintenance, and retirement work as applicable.
- Next safe action and conditions under which work may proceed, proceed conditionally, remain blocked, enter safe halt, or return upstream.

Downstream roles must return interface conflicts, impossible allocations, implementation deviations, new dependencies, performance shortfalls, failed tests, validation failures, incidents, operational evidence, maintenance findings, migration effects, and retirement constraints into the controlled systems record. Chat history is not a continuity mechanism.

RELATED CODEX SECTIONS

001 Research
002 Foresight / Futures
003 Innovation
004 Mission Strategy
005 Business Strategy
006 Product Strategy
007 Portfolio Management

008 Program Management
009 Project Management
010 Product Management
011 Business Analysis
012 Market Research
013 Competitive Intelligence
014 Stakeholder Management
015 User Research
016 Domain Analysis
017 Feasibility Analysis
018 Risk Management
019 Requirements Engineering
021 Enterprise Architecture
022 Solution Architecture
023 Systems Architecture
024 Domain Architecture
025 Business Architecture
026 Capability Architecture
027 Process Architecture
028 Functional Architecture
029 Operational Architecture
030 Information Architecture
031 Data Architecture
032 Application Architecture
033 Software Architecture
034 Platform Architecture
035 Infrastructure Architecture
036 Cloud Architecture
037 Edge Architecture
038 Distributed Systems Architecture
039 Network Architecture
040 Communications Architecture
041 Integration Architecture
042 API Architecture
043 Event-Driven Architecture
044 Runtime Architecture
045 Security Architecture
046 Identity Architecture
047 Privacy Architecture
048 Governance Architecture
049 Compliance Architecture
050 Safety Architecture
051 Reliability Architecture
052 Resilience Architecture
053 Accessibility Architecture
056 AI Architecture
057 Machine Learning Architecture
058 Robotics Architecture
059 Embedded Systems Architecture
060 Hardware Architecture
063 Computer Engineering
073 Control Systems Engineering
074 Robotics Engineering
075 Human–Computer Interaction
106 User Validation
107 Software Engineering

130 Microservices Engineering
136 Simulation Engineering
167 Machine Learning Engineering
178 Agent Systems Engineering
182 AI Safety
183 AI Governance
189 Cybersecurity Engineering
219 Legal
220 Regulatory Compliance
222 Ethics
226 Audit
227 Internal Controls
231 Software Testing
235 System Testing
256 Verification
257 Validation
258 Configuration Management
259 Source Control Management
260 Dependency Management
261 Package Management
262 Artifact Management
263 Environment Management
269 DevOps
272 Platform Engineering
289 Change Management
301 Site Reliability Engineering
303 Runtime Operations
311 Service Management
319 Incident Management
326 Business Continuity
327 Disaster Recovery
331 Operational Readiness
332 Command and Control
333 Technical Support
341 Documentation Engineering
350 Supply Chain Management
374 Maintenance Engineering
382 Data Migration Operations
383 Platform Migration
388 Version Management
389 Deprecation Management
390 Sunset Planning
405 Lifecycle Governance
406 System Retirement

SOURCE BASIS AND UNRESOLVED SOURCE GAPS

Primary supplied source basis:

- Thirsty's Standard v3 supplies the governing posture of truthful scope, current-state inspection, explicit unknowns, evidence before claims, minimal authorized change, complete work for the declared mode, independent verification, hostile review, continuity, and no fake completion.

- Thirsty's Agents 101 supplies the unchanged role contracts and governance stack. Its Architect role requires components, interfaces, dependencies, trust boundaries, data, authority, control, evidence, failure, recovery, implementation sequencing, and verification paths. Its Planner, Coordinator, Researcher, Analyst, Tester, Verifier, Governance Reviewer, Security Auditor, Critic, Red Team, User Experience Reviewer, Documentation Writer, Memory Curator, Implementer, Optimizer, Observer, and Decision Arbiter Advisory roles support this section without being rewritten.
- Thirsty's Owner's Cockpit Standard supplies system-boundary declaration, owner journeys, normative requirements, implementation ownership, evidence methods, vertical slices connecting control to execution and recovery, architecture separation, action state machines, versioned schemas, observability, configuration, reproducibility, resilience, backup and restore, lifecycle verification, change, conformance, and legitimate owner ratification. That source is a candidate for owner ratification and is not silently treated as universal ratified authority.
- Thirsty's Standard V4 Design Blueprint supplies draft concepts relevant to typed system claims, prerequisite-aware evidence, artifact consistency, role assembly, failure taxonomy, rollback, recovery, dissent, and trusted transition. The source is draft and unratified; it is design input, not claimed runtime authority.
- Sections 001-019 supply the current Codex working-draft chain from evidence and human purpose through strategy, affected parties, user and domain understanding, feasibility, risk, and the controlled requirements baseline.

Unresolved source gaps:

- The supplied corpus does not contain a complete ratified universal Systems Engineering standard for every software, hardware, aerospace, medical, industrial, infrastructure, defense, automotive, robotic, financial, public-sector, or AI system.
- This section has not yet been reconciled against all current external Systems Engineering, lifecycle, modeling, configuration, safety, security, reliability, quality, human-factors, acquisition, manufacturing, environmental, and sector-specific standards.
- The necessary lifecycle model, technical reviews, baselines, document forms, modeling notation, measurement thresholds, assurance independence, certification, and evidence depth must be specialized according to consequence, domain, jurisdiction, owner authority, and system type.
- Later architecture and specialist engineering sections may expose missing system allocations, interfaces, measures, dependencies, or lifecycle obligations and must be permitted to reopen this section.
- STATUS remains CONTENT-COMPLETE. This section has not completed full source reconciliation, independent hostile review, owner acceptance, implementation, integration, verification, validation, deployment, or operational verification.

021

SECTION 021 - ENTERPRISE ARCHITECTURE

CODEX/03_SECTIONS/021_ENTERPRISE_ARCHITECTURE.txt

SHA-256 f1810b348acd06f13c611aaa66db652356f159087fe3323de57fa162a78b7483

28,030 BYTES / PRESERVED SOURCE IDENTITY

SECTION 021 — Enterprise Architecture

STATUS: CONTENT-COMPLETE

PURPOSE

Translate the authorized mission, business strategy, product and portfolio direction, requirements baseline, system definition, operating realities, and lifecycle obligations into a coherent enterprise-wide structure for capabilities, value streams, organizations, information, applications, technology, governance, investment, change, and retirement.

Enterprise Architecture governs alignment across an enterprise or similarly broad socio-technical boundary. It does not merely catalogue technology, publish diagrams, or impose uniformity. It determines which capabilities the enterprise must sustain, where decision authority and accountability reside, how business and technical domains relate, which standards and shared services are justified, what variation is legitimate, how current and target states differ, how transitions will be governed, and how architecture claims will be proven.

REQUIRED OUTCOME

- A declared enterprise boundary identifying mission, owners, operating environments, business units, products, programs, platforms, partners, suppliers, regulators, users, operators, affected parties, external systems, and lifecycle horizon.
- An enterprise operating model showing how value is created, decisions are made, responsibilities are distributed, capabilities are owned, information flows, shared services are governed, and exceptions are handled.
- A capability map with stable names, definitions, owners, outcomes, maturity or condition, dependencies, criticality, evidence, and lifecycle state.
- Value-stream and journey views connecting mission and stakeholder outcomes to capabilities, processes, information, applications, technology, people, controls, evidence, and operational results.
- Current-state, target-state, and transition-state architectures that distinguish observed fact, verified condition, inference, assumption, recommendation, approved decision, and unresolved unknown.
- Explicit architecture principles, constraints, standards, reference patterns, decision rights, exception paths, and review triggers.
- Portfolio-level alignment among strategies, investments, programs, products, platforms, data, security, governance, operations, maintenance, modernization, migration, deprecation, and retirement.
- A traceable architecture roadmap with transition increments, dependencies, prerequisites, evidence gates, owner decisions, rollback or recovery conditions, and retirement obligations.
- A durable architecture repository containing decisions, models, standards, waivers, exceptions, evidence, dissent, risks, dependencies, and supersession history.
- A bounded human-AI operating model in which AI can analyze and propose enterprise structures without converting capability into authority, preferences into policy, or generated models into verified enterprise truth.

WHY THIS MATTERS

Enterprises often accumulate products, systems, data stores, vendors, processes, policies, teams, and operating practices faster than they accumulate shared understanding. Local decisions can be individually reasonable while collectively creating duplicate capabilities, contradictory rules, fragmented identity, incompatible data, hidden operational burden, unowned risk, brittle dependencies, and costly lock-in.

Enterprise Architecture provides a governed way to see and shape that whole. Its value is not the diagram itself. Its value is the ability to connect strategy to capabilities, capabilities to investments and systems, systems to evidence and operation, and change to accountable human decisions.

AI can help discover duplication, map dependencies, compare transition options, and maintain traceability. It can also confidently infer an enterprise that does not exist, erase legitimate local variation, recommend centralization without understanding human consequences, or present a generated target state as an approved plan. This section requires AI-supported architecture to remain source-bound, challengeable, reversible where practical, and subordinate to legitimate human authority.

APPLICABILITY

- Multiple products, programs, business units, platforms, applications, data domains, suppliers, or operating environments must support one mission or portfolio.
- Strategic objectives require coordinated changes across people, process, information, technology, governance, finance, security, operations, or lifecycle management.
- Shared capabilities, platforms, standards, services, data, identity, evidence, or infrastructure are being created, consolidated, separated, migrated, or retired.
- Existing architecture is fragmented, undocumented, inconsistent, duplicative, vendor-dependent, difficult to operate, or disconnected from strategy and owner outcomes.
- Investments or modernization decisions require comparison across current, target, and transition states.
- Regulatory, legal, privacy, safety, security, accessibility, resilience, or records obligations apply across more than one system or organizational boundary.
- AI systems, agents, models, or automation influence enterprise decisions, shared data, common services, workforce processes, customer outcomes, or protected state.
- A merger, acquisition, divestiture, partnership, outsourcing arrangement, major reorganization, or platform transition changes enterprise boundaries or dependencies.

NON-APPLICABILITY CONDITIONS

- The work is confined to one bounded solution whose enterprise dependencies, standards, shared capabilities, data, authorities, and lifecycle obligations are already current, explicit, and verified.
- A current approved enterprise baseline covers the exact scope and proposed change, and impact analysis demonstrates no material effect on shared capabilities, standards, data, identity, security, operations, investment, or retirement.
- The organization has not authorized enterprise-level change; exploratory analysis may continue, but it must be labeled advisory and may not be represented as policy or approved target state.
- An active incident, legal prohibition, missing authority, or unsafe condition requires containment before normal architecture planning continues.
- Required enterprise information cannot be accessed lawfully or safely.

A non-applicability decision must identify the controlling baseline, scope, version, environment, authority, evidence, and reopening trigger. Small organizational size, informal culture, or use of AI-generated diagrams is not sufficient reason to omit enterprise-level alignment when the consequences are enterprise-wide.

PREREQUISITES AND DEPENDENCIES

- Sections 001–020 as applicable, especially the authenticated objective, strategies, portfolio and program boundaries, affected parties, domain model, feasibility conditions, risk treatments, approved requirements baseline, system of interest, operational concept, interfaces, lifecycle model, and evidence obligations.
- Legitimate human authority to define enterprise scope, principles, standards, investment priorities, organizational responsibilities, shared services, exceptions, risk acceptance, and target-state commitments.
- Current-state evidence for capabilities, processes, applications, data, integrations, infrastructure, suppliers, contracts, policies, incidents, costs, operational performance, maintenance burden, and retirement obligations.
- A controlled vocabulary for enterprise, capability, value stream, product, platform, service, domain, application, data product, technology, standard, principle, exception, transition, and retirement.
- A durable architecture repository or equivalent record supporting versioning, provenance, relationships, decisions, exceptions, evidence, and supersession.
- Defined architecture governance, review, escalation, exception, and change-control mechanisms.

REQUIRED INPUTS

- Mission and business strategies, product and portfolio strategies, operating model, financial constraints, investment horizon, and explicit non-goals.
- Approved requirements and systems baselines relevant to the enterprise scope.
- Capability inventories, capability owners, performance evidence, criticality, maturity, dependencies, and known gaps.

- Value streams, owner and user journeys, business processes, organizational responsibilities, service relationships, and operating procedures.
- Information and data domains, sources of truth, classifications, quality evidence, lineage, retention, privacy, and sovereignty requirements.
- Application, platform, integration, API, event, infrastructure, cloud, edge, network, identity, security, observability, and operations inventories.
- Supplier, partner, vendor, licensing, procurement, service-level, support, and exit obligations.
- Risks, incidents, control findings, technical debt, resilience limits, accessibility gaps, and maintenance evidence.
- Existing architecture principles, standards, reference patterns, waivers, exceptions, roadmaps, and decision records.
- AI-system identities, model and provider dependencies, data boundaries, evaluation evidence, fallback paths, revocation mechanisms, and human-oversight obligations.

HUMAN RESPONSIBILITIES AND AUTHORITY

Authorized humans retain authority for mission, enterprise boundary, values, operating model, investment, organizational design, policy, standards, risk acceptance, exceptions, commitments, and target-state approval.

- Confirm the enterprise scope, owners, decision rights, affected parties, and intended architecture horizon.
- Decide which capabilities are strategic, differentiating, shared, commodity, outsourced, prohibited, transitional, or scheduled for retirement.
- Approve architecture principles, mandatory standards, exception criteria, transition priorities, and resource commitments.
- Resolve conflicts involving values, rights, workforce impact, customer impact, privacy, accessibility, safety, security, resilience, cost, schedule, and strategic tradeoffs.
- Ensure that architecture governance does not become unaccountable bureaucracy or a mechanism for hiding owner decisions behind committees, frameworks, or model output.
- Ratify enterprise target states and material transition decisions over exact versions and evidence.

AI RESPONSIBILITIES AND BOUNDARIES

AI may inventory, classify, compare, trace, model, simulate, summarize, detect inconsistency, propose alternatives, maintain records, and assist impact analysis within an explicit assignment.

AI must:

- Preserve the authorized objective and scope.
- Distinguish observed current state from inferred or proposed architecture.
- Identify source identity, currency, coverage, conflicts, and unknowns.
- Keep capability, organization, application, data, platform, and technology concepts distinct.
- Present alternatives, tradeoffs, dependencies, transition costs, and reversal conditions.
- Surface dissent, local constraints, and affected-party consequences rather than optimizing only for structural neatness.
- Avoid declaring an enterprise standard, target state, exception, investment, or retirement decision without human authority.
- Avoid treating tool access, model capability, or architecture-role assignment as permission to change enterprise state.
- Report when the evidence does not support a reliable enterprise view.

APPLICABLE EXISTING AGENT ROLES

The existing source-defined roles apply without modification:

- Architect for enterprise structure, boundaries, components, interfaces, flows, alternatives, and verification paths.
- Researcher for current-state, market, supplier, regulatory, technical, and organizational evidence.
- Analyst for capability, portfolio, cost, dependency, scenario, and transition analysis.
- Critic for contradictions, unsupported assumptions, hidden dependencies, and architecture theater.
- Security Auditor for enterprise trust, identity, access, supply-chain, evidence, and control boundaries.
- Planner for transition roadmaps, dependencies, gates, milestones, and contingencies.

- Coordinator for multi-domain workstreams, handoffs, conflicts, and provenance.
- Observer for neutral capture of current state, operating evidence, and changes.
- Verifier for architecture claims, baseline identity, traceability, and evidence sufficiency.
- Governance Reviewer for authority, policy, exception, and conformance boundaries.
- User Experience Reviewer for owner comprehension, accessibility, control, and enterprise-operability impacts.
- Decision Arbiter Advisory for structured comparison where criteria and authority are explicit.
- Documentation Writer and Memory Curator for durable architecture records and continuity.
- Red Team for adversarial challenge of centralization, bypass, dependency, migration, recovery, and governance assumptions.

REQUIRED ACTIVITIES

1. Establish the enterprise architecture charter: objective, scope, authority, decision owners, time horizon, exclusions, evidence expectations, and completion boundary.
2. Define the enterprise context and boundary, including external organizations, partners, regulators, users, communities, ecosystems, and dependencies.
3. Establish the current-state evidence baseline and identify confidence, freshness, ownership, and gaps.
4. Define or reconcile the enterprise operating model, decision rights, accountability, and service relationships.
5. Build the capability map and connect each capability to outcomes, owners, value streams, processes, information, applications, technology, controls, evidence, and lifecycle state.
6. Map value streams and critical journeys from initiating need through operational outcome, evidence, support, recovery, and closure.
7. Model current-state architecture across business, capability, process, information, data, application, integration, technology, security, governance, and operations views as applicable.
8. Identify duplication, fragmentation, gaps, concentration risk, unowned capability, hidden coupling, stale standards, unsupported technology, and retirement debt.
9. Define architecture principles and classify each as mandatory constraint, approved preference, recommendation, or experiment.
10. Develop credible target-state alternatives rather than one presumed answer.
11. Evaluate alternatives against mission alignment, requirements, feasibility, risk, human impact, cost, complexity, resilience, security, privacy, accessibility, maintainability, portability, supportability, and reversibility.
12. Define transition architectures that preserve continuity while moving between current and target states.
13. Produce a roadmap with increments, dependencies, decisions, evidence gates, migration and rollback conditions, training, support, and retirement work.
14. Define standards, reference architectures, shared services, and exception processes only where their enterprise value and enforcement authority are explicit.
15. Map investments, initiatives, products, platforms, and projects to capabilities and target-state outcomes.
16. Define architecture governance, review, exception, waiver, expiry, supersession, and reopening processes.
17. Establish architecture measures and anti-metrics that detect both progress and harmful optimization.
18. Hostile-review the architecture for bypass, false centralization, shadow systems, authority drift, vendor lock-in, stale data, migration failure, inaccessible control, and unowned recovery.
19. Verify that target-state claims are supported and that generated architecture is not represented as implemented or operational.
20. Preserve a durable handoff to Solution Architecture and the specialist architecture sections.

HUMAN-AI INTERACTION REQUIREMENTS

- The owner must be able to ask what the enterprise architecture is trying to accomplish, what evidence supports it, what is proposed, what is approved, what remains unknown, and what decision is required.
- AI-generated diagrams, capability maps, heat maps, scores, and roadmaps must disclose source coverage and uncertainty.
- Recommendations must remain distinct from enterprise decisions and policy.
- Material architecture alternatives and dissent must remain inspectable in owner language.
- Consequential changes require explicit authorization over exact scope, target, assumptions, and evidence.
- Delegation must be revocable, bounded by role and data access, and recorded in continuity artifacts.
- The owner must retain visible stop, defer, narrow, reject, and reopen paths.

- Architecture explanations must support both strategic comprehension and technical drill-down without hiding uncertainty behind jargon.

ARCHITECTURE AND SYSTEM IMPLICATIONS

Enterprise Architecture must connect, but not collapse, the following layers:

- Mission, strategy, policy, and operating model.
- Capabilities and value streams.
- Organizations, roles, authority, and accountability.
- Processes, services, products, and journeys.
- Information, data, knowledge, records, and evidence.
- Applications, platforms, integrations, APIs, and events.
- Technology, infrastructure, cloud, edge, networks, devices, and facilities.
- Security, identity, privacy, governance, compliance, safety, reliability, resilience, accessibility, and content.
- Delivery, operations, support, maintenance, migration, deprecation, and retirement.

Each layer must identify ownership, interfaces, dependencies, sources of truth, criticality, failure behavior, recovery, evidence, and lifecycle state. Shared architecture must not erase legitimate domain autonomy; local autonomy must not bypass enterprise constraints or create unowned enterprise risk.

OWNER COCKPIT AND USER-CONTROL REQUIREMENTS

The owner-facing architecture view must allow an authorized person to:

- See the current, target, and transition states and their evidence dates.
- Trace strategy to capabilities, initiatives, systems, dependencies, risks, and outcomes.
- Identify architecture decisions awaiting authority, approved exceptions, expiring waivers, and unresolved dissent.
- Understand which systems and capabilities are critical, degraded, duplicated, unsupported, or scheduled for retirement.
- Inspect the consequence, cost, reversibility, and recovery path of a proposed transition.
- Distinguish diagrams and plans from implemented, verified, and operational state.
- Stop or defer a transition without losing the current continuity record.
- Export the architecture record in durable, non-chat-dependent form.

REQUIRED OUTPUTS AND ARTIFACTS

- Enterprise architecture charter and scope statement.
- Enterprise context and boundary model.
- Operating-model and decision-rights map.
- Capability map and capability register.
- Value-stream and critical-journey maps.
- Current-state architecture baseline.
- Target-state alternatives and selected recommendation or approved decision record.
- Transition architecture and roadmap.
- Architecture principles and standards register.
- Shared-services and reference-architecture register.
- Investment-to-capability traceability map.
- Dependency, concentration, vendor, and lock-in analysis.
- Exception, waiver, dissent, and unresolved-decision ledgers.
- Architecture measures, anti-metrics, and review cadence.
- Evidence map and architecture claim register.
- Handoff package to Section 022 and later architecture sections.

EVIDENCE REQUIREMENTS

Evidence must be bound to source, version, environment, owner, date, method, scope, integrity, and claim. Relevant evidence may include:

- Approved strategies, requirements, systems baselines, policies, contracts, and decision records.
- Inventories, configuration records, repositories, schemas, runtime observations, telemetry, incidents, support records, and financial evidence.
- Capability-owner interviews, user and operator evidence, process observation, supplier evidence, and regulatory sources.
- Architecture repositories, dependency graphs, data lineage, application portfolios, interface inventories, and control evidence.
- Migration tests, pilot results, resilience tests, accessibility evaluations, security findings, and operational exercises.

A diagram, maturity score, vendor presentation, workshop consensus, or AI-generated synthesis is not sufficient evidence of current state, implementation, conformance, or readiness.

VERIFICATION AND ACCEPTANCE TESTS

- Trace a representative strategic objective through capability, value stream, system, initiative, evidence, and operational outcome.
- Select a capability and verify owner, definition, dependencies, supporting systems, evidence, risks, and lifecycle state.
- Enumerate consequential command or decision paths and confirm that authority and evidence boundaries remain consistent.
- Change a material source or dependency and confirm that affected architecture claims and approvals are invalidated or reopened.
- Test whether a local client, team, or AI agent can bypass enterprise policy or falsely claim an exception.
- Simulate loss of a major vendor, platform, identity service, data source, or operating site and inspect transition and recovery plans.
- Compare current-state evidence to architecture models and identify unsupported or stale entries.
- Have the owner explain the target state, material tradeoffs, unresolved decisions, and rollback conditions without relying on the architect's live interpretation.
- Verify that planned, implemented, verified, operational, and retired states remain distinct.

FAILURE STATES AND ANTI-PATTERNS

- Architecture theater: polished models without decision use, evidence, ownership, or operational consequence.
- Inventory masquerading as architecture.
- Target-state fiction unsupported by current-state evidence or transition capacity.
- Framework capture: forcing the enterprise into a template instead of preserving the actual objective and context.
- Centralization by default, or decentralization by default, without consequence analysis.
- Capability maps with no owners, outcomes, evidence, or lifecycle state.
- Standards with no authority, enforcement, exception process, or retirement path.
- Roadmaps that omit dependencies, migration, rollback, support, and human adoption.
- Technology-led strategy substitution.
- Vendor lock-in hidden as simplification.
- AI-generated certainty, invented inventories, or inferred authority.
- Architecture governance that blocks safe work while failing to control consequential work.
- Silent removal of dissent, local constraints, accessibility needs, workforce impacts, or affected-party concerns.
- Claiming enterprise conformance or readiness from workshop completion or diagram approval.

RECOVERY AND RECONSIDERATION

When enterprise architecture is incomplete, contradicted, stale, or rejected:

1. Identify the exact affected claim, model, standard, roadmap item, or decision.
2. Preserve the prior state, evidence, dissent, and decision history.
3. Classify the issue as source, scope, authority, data, modeling, feasibility, risk, governance, transition, implementation, or operational failure.
4. Determine what safe analysis or local work may continue.
5. Request the smallest evidence, experiment, owner decision, or scope reduction that can resolve the blocker.
6. Reopen affected capability, dependency, standard, investment, transition, and retirement decisions.

7. Reissue the architecture baseline with explicit supersession and downstream impact.

COMPLETION CRITERIA

Enterprise Architecture is DRAFT-complete only when:

- Scope, owner, authority, affected parties, and horizon are explicit.
- Current-state claims are source-bound and uncertainty is visible.
- Capabilities, value streams, responsibilities, information, applications, technology, and lifecycle obligations are connected.
- Target-state alternatives and tradeoffs are explicit.
- Transition architecture, dependencies, evidence gates, rollback, and retirement are defined.
- Principles, standards, exceptions, and governance have legitimate authority and review paths.
- Human and AI responsibilities remain bounded.
- Material decisions and dissent remain visible.
- Verification methods exist for material architecture claims.
- The owner can understand, challenge, approve, reject, or reopen the exact architecture state.

CONTENT-COMPLETE does not mean source-reconciled, hostile-reviewed, owner-accepted, implemented, verified, or operational.

HANDOFF AND CONTINUITY REQUIREMENTS

The handoff must include exact versions of the enterprise scope, current-state baseline, capability map, value streams, target alternatives, approved decisions, transition roadmap, principles, standards, exceptions, evidence, risks, dissent, open decisions, and next safe action.

Section 022 must receive enough context to design a bounded solution without inventing enterprise constraints, shared-service assumptions, data authority, identity, security, lifecycle, or target-state decisions. Any downstream discovery that invalidates an enterprise assumption must reopen this section through a durable change record.

RELATED CODEX SECTIONS

Primary upstream: 001–020.

Primary downstream: 022 Solution Architecture; 023 Systems Architecture; 024 Domain Architecture; 025 Business Architecture; 026 Capability Architecture; 027 Process Architecture; 028 Functional Architecture; 029 Operational Architecture; 030 Information Architecture; 031 Data Architecture; 032 Application Architecture; 033 Software Architecture; 034 Platform Architecture; 035 Infrastructure Architecture; 036 Cloud Architecture; 041 Integration Architecture; 045 Security Architecture; 048 Governance Architecture; 051 Reliability Architecture; 052 Resilience Architecture; 258 Configuration Management; 289 Change Management; 373 Roadmap Management; 384 Modernization; 389 Deprecation Management; 390 Sunset Planning; 405 Lifecycle Governance; 406 System Retirement.

SOURCE BASIS AND UNRESOLVED SOURCE GAPS

Primary supplied source basis:

- Thirsty's Standard v3 provides truthful scope, current-state inspection, unknown-means-unknown, evidence before claims, minimal authorized change, independent verification, hostile review, continuity, and no fake completion.
- Thirsty's Agents 101 supplies the unchanged role contracts. The Architect role requires explicit components, interfaces, dependencies, ownership, trust boundaries, data, authority, control, evidence, failure, recovery, alternatives, tradeoffs, sequencing, and verification.
- Thirsty's Owner's Cockpit Standard provides separation of presentation, governed action, domain services, agents, evidence, observability, and infrastructure; owner-visible authority; complete journeys; versioned contracts; resilience; change; and ratification boundaries. That source is a candidate for owner ratification and is not treated as universal ratified authority.

- Thirsty's Standard V4 Design Blueprint contributes draft concepts for prerequisite-aware evidence, artifact consistency, bounded role assembly, dissent, constructive denial, and trusted transition. It remains draft and unratified.
- Sections 001–020 supply the current Codex chain from evidence and strategy through requirements and whole-system definition.

Unresolved source gaps:

- The supplied corpus does not contain a complete ratified universal Enterprise Architecture standard for every organizational form, sector, jurisdiction, merger, acquisition, public institution, nonprofit, or multi-enterprise ecosystem.
- This draft has not been reconciled against all current external enterprise-architecture, portfolio, business-architecture, capability, data, security, governance, procurement, finance, records, and sector standards.
- Architecture methods, viewpoints, repository schemas, maturity methods, review bodies, and evidence depth must be specialized to actual consequence and authority.
- Later specialist sections may expose missing capability, data, platform, integration, security, operational, or lifecycle obligations and must be permitted to reopen this section.
- STATUS remains CONTENT-COMPLETE.

022

SECTION 022 - SOLUTION ARCHITECTURE

CODEX/03_SECTIONS/022_SOLUTION_ARCHITECTURE.txt

SHA-256 de0b92e05a788722df6bdbffefacfeb7e45c9f06d26ff3a03c98a7bf68e2b2ed

26,329 BYTES / PRESERVED SOURCE IDENTITY

SECTION 022 — Solution Architecture

STATUS: CONTENT-COMPLETE

PURPOSE

Transform an approved problem, requirements baseline, system definition, enterprise constraints, and current-state evidence into a coherent, implementable, operable, secure, recoverable, testable, and lifecycle-complete solution architecture.

Solution Architecture defines how a bounded solution will satisfy its authorized outcome within the surrounding enterprise and system context. It does not substitute preferred technology for the objective, treat diagrams as implementation, or silently decide unresolved values and risks. It connects owner journeys, requirements, domain behavior, components, interfaces, data, authority, security, operations, evidence, deployment, recovery, maintenance, migration, and retirement into one traceable design basis.

REQUIRED OUTCOME

- A bounded solution context identifying objective, owner, users, operators, affected parties, environments, external systems, enterprise services, constraints, non-goals, and lifecycle horizon.
- A requirements-to-architecture trace showing how every material requirement, risk treatment, feasibility condition, and acceptance obligation maps to one or more solution elements or remains visibly unresolved.
- Credible solution alternatives evaluated using consistent criteria, evidence, assumptions, cost, complexity, risk, human impact, reversibility, and operational burden.
- An approved or explicitly advisory architecture decision record that preserves decision owner, authority, rationale, rejected alternatives, conditions, dissent, and evidence that would change the outcome.
- Logical, physical, deployment, runtime, integration, data, security, identity, privacy, governance, observability, operational, and user-control views as applicable.
- Defined components, responsibilities, interfaces, contracts, states, dependencies, data handled, authority level, trust level, failure behavior, recovery behavior, ownership, and verification methods.
- Versioned interface and data contracts with validation, compatibility, idempotency, timeout, retry, cancellation, reconciliation, error, audit, and evidence semantics.
- Explicit separation between presentation, governed action, orchestration, domain logic, data, evidence, observability, and infrastructure boundaries.
- Deployment, configuration, secrets, migration, rollback, support, maintenance, deprecation, and retirement architecture.
- A solution assurance plan covering normal, boundary, failure, recovery, security, privacy, accessibility, performance, reliability, resilience, and owner-acceptance paths.
- A durable implementation handoff that does not require engineers or agents to invent missing scope, interfaces, authority, evidence, or lifecycle behavior.

WHY THIS MATTERS

A solution can satisfy individual feature requests while failing as an operable whole. Common failures include unowned components, implicit interfaces, frontend-only authority, duplicated business rules, inconsistent state, missing recovery, unsafe retries, stale evidence, unsupported deployment assumptions, and architecture that works only in a diagram or demonstration environment.

Solution Architecture makes the complete execution path visible before implementation hardens accidental choices. It connects user intent to governed action, domain behavior, data, evidence, operations, and recovery. It also creates the basis for independent challenge: a reviewer can ask whether the proposed solution actually satisfies the requirements, survives failure, preserves authority, and can be built, deployed, supported, changed, and retired.

AI can accelerate decomposition, option analysis, contract drafting, dependency mapping, and test design. It can also invent services, assume integrations, choose familiar technology without evidence, conflate provider capability with permission, or generate a persuasive architecture whose critical paths have never been proven. This section requires source-grounded architecture, explicit uncertainty, bounded delegation, and human control over consequential decisions.

APPLICABILITY

- A product, application, service, platform feature, integration, device, AI capability, migration, modernization, or operational workflow requires a concrete design before implementation.
- The solution crosses components, teams, repositories, data stores, networks, providers, environments, or trust boundaries.
- Requirements include security, privacy, safety, accessibility, reliability, resilience, performance, support, compliance, recovery, or lifecycle obligations.
- A solution must integrate with enterprise services, shared platforms, identity, data, policy, evidence, or observability.
- Multiple implementation approaches are credible and tradeoffs must be made explicitly.
- Existing systems are being extended, replaced, wrapped, migrated, decomposed, consolidated, or retired.
- AI agents, models, tools, or autonomous functions can affect decisions, communications, data, users, or protected state.
- Deployment, rollback, migration, support, and operation are material to success.

NON-APPLICABILITY CONDITIONS

- The work is a minimal reversible change inside an existing approved architecture, and impact analysis confirms no material change to requirements, interfaces, state, authority, data, security, operations, recovery, or lifecycle behavior.
- A current approved solution architecture covers the exact version, environment, scope, and change, with current evidence and no invalidated assumptions.
- The task is exploratory research or prototyping and no claim of implementability, readiness, or deployment is made; the prototype must still preserve its limitations and handoff evidence.
- An active incident, unauthorized request, legal prohibition, or failed mandatory control requires containment first.
- Required architecture work would need unauthorized access to protected systems, data, credentials, people, or suppliers.

The non-applicability record must identify the controlling architecture, exact scope, version, environment, evidence, authority, and reopening trigger. Familiar technology, small code size, or an AI-generated implementation plan is not sufficient.

PREREQUISITES AND DEPENDENCIES

- Sections 001–021 as applicable, especially approved requirements, system context, operational concept, allocations, enterprise principles, standards, shared services, current-state evidence, target-state direction, and transition constraints.
- Legitimate human decision authority for solution scope, architecture selection, resource commitment, risk acceptance, privacy, safety, security, accessibility, deployment, and retirement.
- Current-state evidence for existing components, interfaces, data, schemas, repositories, infrastructure, environments, providers, incidents, tests, operations, support, and maintenance.
- Defined terminology and a controlled architecture repository.
- Access to relevant specialist review and current implementation constraints.
- A versioned change, configuration, evidence, review, escalation, and safe-halt process.

REQUIRED INPUTS

- Authorized objective, scope, non-goals, users, operators, affected parties, environments, and lifecycle horizon.
- Approved requirements baseline, acceptance criteria, verification methods, variants, waivers, exceptions, and open decisions.
- Systems Engineering outputs: system context, operational concept, allocations, interfaces, measures, risks, baselines, integration, verification, validation, transition, operation, support, and retirement obligations.
- Enterprise Architecture outputs: principles, standards, shared services, target and transition states, capability dependencies, exceptions, and investment constraints.
- Domain rules, bounded contexts, entities, states, events, invariants, and sources of truth.

- Current software, data, platform, infrastructure, identity, security, privacy, governance, observability, deployment, and operational state.
- Workload, performance, capacity, availability, recovery, data-volume, latency, concurrency, compatibility, and cost evidence.
- Supplier, vendor, license, service-level, support, and exit constraints.
- AI-system identity, model, provider, data, evaluation, tool, autonomy, fallback, oversight, and revocation requirements where applicable.

HUMAN RESPONSIBILITIES AND AUTHORITY

Authorized humans retain final human authority over objective, values, user and affected-party commitments, architecture selection, material tradeoffs, risk acceptance, resource commitment, deployment, and final acceptance.

- Confirm solution scope, decision owner, architecture authority, and acceptance boundary.
- Approve mandatory constraints, architecture principles, standards, exceptions, and points of no return.
- Decide tradeoffs involving security, privacy, safety, accessibility, performance, cost, schedule, maintainability, operational burden, vendor dependence, and human impact.
- Confirm which architecture decisions are advisory, approved, conditional, deferred, or rejected.
- Approve consequential use of AI, data, external providers, privileged operations, and irreversible changes.
- Ensure the architecture remains understandable and operable by the intended owner and operators.

AI RESPONSIBILITIES AND BOUNDARIES

AI may inspect, model, compare, trace, simulate, draft, test hypotheses, generate contract candidates, identify risks, and maintain architecture records within an explicit assignment.

AI must:

- Restate the accepted objective without substitution.
- Separate requirements, observations, assumptions, inferences, recommendations, decisions, and unknowns.
- Identify source identity and evidence limits.
- Avoid inventing integrations, credentials, performance, availability, data quality, provider behavior, or implementation state.
- Present credible alternatives and tradeoffs.
- Keep architecture recommendation distinct from human approval.
- Never infer authority from technical capability or role title.
- Avoid treating generated code, diagrams, schemas, or tests as proof of deployed behavior.
- Preserve dissent, unresolved conditions, and evidence that would change the design.
- Stop or narrow affected work when authority, mandatory evidence, or safety conditions are missing while identifying safe work that may continue.

APPLICABLE EXISTING AGENT ROLES

The existing source-defined roles apply without modification:

- Architect for solution structure, interfaces, flows, trust boundaries, alternatives, tradeoffs, sequencing, and verification.
- Researcher for current-state, technology, provider, compatibility, operational, and regulatory evidence.
- Analyst for option comparison, workload, cost, complexity, risk, reversibility, and sensitivity analysis.
- Security Auditor for threat, identity, authorization, secrets, supply chain, evidence, and operational security.
- User Experience Reviewer for owner journeys, accessibility, comprehension, consequence, and recovery.
- Critic and Red Team for adversarial challenge of assumptions, bypasses, failure paths, dependencies, and readiness claims.
- Planner for implementation, migration, deployment, validation, rollback, and recovery sequencing.
- Tester and Verifier for traceability, contract, failure, recovery, and claim-verification design.
- Coordinator for specialist workstreams and conflict preservation.
- Governance Reviewer for authority, policy, compliance, and exception boundaries.
- Implementer for later authorized execution only; architecture assignment alone does not grant implementation authority.

- Documentation Writer and Memory Curator for durable records and continuity.
- Decision Arbiter Advisory for structured recommendations under explicit criteria and authority.

REQUIRED ACTIVITIES

1. Establish the solution architecture charter: objective, scope, authority, decision owners, environments, constraints, non-goals, and acceptance boundary.
2. Inspect the current state and bind architecture work to exact versions, environments, repositories, interfaces, providers, and evidence.
3. Define the solution context, external actors, enterprise services, systems, dependencies, trust boundaries, and sources of truth.
4. Trace approved requirements, risks, feasibility conditions, and lifecycle obligations into architecture concerns.
5. Identify mandatory constraints and disqualifying conditions before scoring options.
6. Generate credible architecture alternatives, including no-change, staged, reversible, or reduced-scope options when materially available.
7. Evaluate alternatives using the same visible criteria and identify sensitivity to assumptions.
8. Record the architecture decision with decision owner, authority state, evidence, rationale, dissent, conditions, and review trigger.
9. Decompose the selected or recommended solution into bounded components and responsibilities.
10. Define interfaces and contracts, including producer, consumer, protocol, identity, authorization, schema, validation, errors, timeout, retry, idempotency, cancellation, audit, and evidence.
11. Map data, control, authority, identity, event, and evidence flows.
12. Define state ownership, consistency model, transaction and compensation boundaries, reconciliation, and recovery semantics.
13. Separate presentation, action gateway, orchestration, domain logic, data, evidence, observability, and infrastructure.
14. Define security, privacy, governance, compliance, safety, accessibility, reliability, resilience, and content implications.
15. Define configuration, environment, secrets, dependency, build, artifact, deployment, migration, and rollback architecture.
16. Define observability for owner and engineer, including health, status, logs, metrics, traces, evidence, freshness, and incident signals.
17. Define operational ownership, support model, maintenance, upgrade, compatibility, deprecation, and retirement.
18. Produce implementation slices and sequence them according to dependencies and evidence gain.
19. Define verification, validation, owner acceptance, hostile review, and readiness gates.
20. Preserve a durable handoff to Systems Architecture and specialist architecture and engineering sections.

HUMAN-AI INTERACTION REQUIREMENTS

- The owner must be able to ask what the solution does, what it does not do, why this architecture was chosen, what alternatives exist, what is uncertain, and what decision is required.
- AI-generated architecture must show evidence basis, assumptions, unresolved references, and provider limitations.
- A conversational command must not bypass the same identity, policy, scope, approval, action-state, and evidence controls used by other interfaces.
- Consequential actions require inspectable objective, target, parameters, authority, expected effect, risk, and recovery before authorization.
- The system must expose when AI or provider capability is degraded and what controls remain available.
- Critical stop, status, evidence, and recovery paths must not depend solely on a model.
- Human correction, narrowing, revocation, and escalation must remain available and attributable.
- Owner-visible status must not exceed authoritative runtime and evidence truth.

ARCHITECTURE AND SYSTEM IMPLICATIONS

For each component define:

- Name, purpose, responsibilities, owner, inputs, outputs, dependencies, authority level, data, trust level, state, failure behavior, recovery, audit, and verification.

For each interface define:

- Producer, consumer, purpose, protocol, version, schema, authentication, authorization, validation, error, timeout, retry, idempotency, ordering, consistency, privacy, audit, evidence, compatibility, and deprecation.

For the solution as a whole define:

- System and deployment boundaries.
- Authoritative state and read models.
- Identity and authority flow.
- Data and evidence lifecycle.
- Failure domains and blast-radius controls.
- Degraded, offline, and recovery modes.
- Operational ownership and escalation.
- Build, artifact, environment, configuration, and release identity.
- Migration, rollback, support, maintenance, and retirement.

OWNER COCKPIT AND USER-CONTROL REQUIREMENTS

The owner must be able to:

- See the active solution version, environment, actor, mode, and evidence freshness.
- Understand what is running, degraded, blocked, awaiting authority, failed, or unverified.
- Trace a consequential action from intent through authority, execution, evidence, and terminal result.
- Inspect architecture decisions, alternatives, assumptions, and unresolved risks.
- See dependencies, failure consequences, recovery paths, and rollback readiness.
- Distinguish frontend presentation from authoritative state.
- Stop, cancel, defer, revoke, or recover within defined authority.
- Operate critical status and recovery controls during AI or provider outage.
- Export continuity and evidence without dependence on chat memory.

REQUIRED OUTPUTS AND ARTIFACTS

- Solution architecture charter and context diagram.
- Requirements-to-architecture traceability matrix.
- Architecture alternatives and tradeoff analysis.
- Architecture decision records.
- Component and responsibility model.
- Interface and contract catalogue.
- Data, authority, control, event, and evidence-flow models.
- State and consistency model.
- Security, privacy, safety, accessibility, reliability, resilience, and governance views.
- Deployment, environment, configuration, secrets, and dependency model.
- Observability and operational ownership model.
- Failure, degradation, recovery, migration, and rollback design.
- Implementation-slice roadmap and validation gates.
- Support, maintenance, compatibility, deprecation, and retirement plan.
- Evidence plan, hostile-review record, dissent ledger, and open-decision register.
- Durable handoff package.

EVIDENCE REQUIREMENTS

Evidence may include source inspection, current architecture and code, schemas, configuration, dependency and artifact records, provider documentation, runtime observations, traces, logs, tests, prototypes, benchmarks, failure exercises, security review, accessibility evaluation, owner research, support incidents, and operational records.

Every material architecture claim must identify scope, version, environment, method, result, limitations, and freshness. Vendor claims, design patterns, diagrams, code generation, static checks, passing happy-path tests, and demonstrations are not sufficient proof of production readiness or owner usability.

VERIFICATION AND ACCEPTANCE TESTS

- Trace every material requirement to an architecture element, interface, control, or visible unresolved state.
- Enumerate all consequential command paths and confirm uniform authority and evidence enforcement.
- Tamper with client state and confirm that security, approval, and completion remain server-authoritative.
- Validate representative valid and invalid contracts at trust boundaries.
- Follow one action across UI, gateway, orchestration, domain service, logs, and evidence.
- Force dependency failure, timeout, duplicate request, partial completion, and recovery.
- Complete execution while causing independent verification failure and confirm status remains distinct.
- Disable AI or external providers and test deterministic critical controls.
- Change a material architecture input after approval and confirm invalidation or review.
- Perform an owner walkthrough of critical journeys, error paths, and recovery without step-by-step coaching.
- Verify build, artifact, environment, configuration, migration, rollback, and continuity identity.

FAILURE STATES AND ANTI-PATTERNS

- Diagram-as-proof.
- Technology-first objective substitution.
- Frontend-only authority or business rules.
- Direct UI-to-side-effect bypass.
- Shared mutable state with no owner.
- Implicit interface, schema, timeout, retry, or error semantics.
- Hidden fallback that changes meaning.
- AI model output directly authorizing or executing consequential action.
- Provider availability or performance assumed without evidence.
- Happy-path-only design.
- Recovery depending on the failed component.
- Conflated execution, verification, validation, and acceptance.
- Unbounded retries, replay, duplicate effects, or stale approvals.
- Secrets or environment assumptions embedded in artifacts.
- Operational ownership, support, maintenance, or retirement omitted.
- Architecture decision recorded without alternatives, authority, or reversal conditions.
- Generated implementation represented as deployed or ready.

RECOVERY AND RECONSIDERATION

When the architecture is blocked or disproven:

1. Identify the exact requirement, assumption, interface, dependency, control, or decision affected.
2. Preserve the failed evidence, prior architecture, and dissent.
3. Determine whether the issue is scope, authority, source, requirement, design, provider, environment, feasibility, risk, security, operation, or lifecycle related.
4. Isolate affected work and identify safe work that may continue.
5. Select the smallest admissible experiment, evidence request, design change, scope reduction, or owner decision.
6. Re-evaluate alternatives, dependencies, migration, rollback, and downstream impact.
7. Supersede the architecture record and invalidate stale approvals and evidence.

COMPLETION CRITERIA

Solution Architecture is DRAFT-complete only when:

- Objective, scope, authority, context, environments, users, operators, and lifecycle are explicit.
- Requirements and risks map to architecture elements or visible unresolved states.
- Alternatives and tradeoffs are recorded consistently.
- Components, interfaces, data, state, authority, trust, evidence, failure, recovery, and operations are defined.
- Security, privacy, accessibility, resilience, deployment, migration, rollback, maintenance, and retirement are addressed as applicable.

- Human and AI authority boundaries are explicit.
- Owner control and critical journeys are architecturally possible.
- Verification, validation, hostile review, and acceptance paths are defined.
- The implementation handoff is durable and does not require invention of material boundaries.

CONTENT-COMPLETE does not mean implemented, source-reconciled, hostile-reviewed, owner-accepted, verified, validated, deployed, or production-ready.

HANDOFF AND CONTINUITY REQUIREMENTS

The handoff must include exact versions of the objective, requirements, system and enterprise baselines, selected or recommended architecture, decisions, components, interfaces, flows, data and state models, risks, evidence, tests, deployment, rollback, operations, support, open decisions, dissent, and next safe action.

Section 023 and later specialist architecture and engineering sections must be able to continue from the durable record without hidden chat memory. Downstream discoveries that invalidate requirements, system allocations, enterprise constraints, or solution assumptions must reopen the affected upstream section through controlled change.

RELATED CODEX SECTIONS

Primary upstream: 001–021.

Primary downstream: 023 Systems Architecture; 024 Domain Architecture; 025 Business Architecture; 028 Functional Architecture; 029 Operational Architecture; 030 Information Architecture; 031 Data Architecture; 032 Application Architecture; 033 Software Architecture; 034 Platform Architecture; 035 Infrastructure Architecture; 036 Cloud Architecture; 038 Distributed Systems Architecture; 041 Integration Architecture; 042 API Architecture; 043 Event-Driven Architecture; 044 Runtime Architecture; 045 Security Architecture; 046 Identity Architecture; 047 Privacy Architecture; 048 Governance Architecture; 050 Safety Architecture; 051 Reliability Architecture; 052 Resilience Architecture; 053 Accessibility Architecture; 056 AI Architecture; 107 Software Engineering; 128 API Engineering; 129 Integration Engineering; 167 Machine Learning Engineering; 178 Agent Systems Engineering; 231–257 testing, verification, and validation; 258–300 delivery and infrastructure; 301–332 runtime and operations; 374–406 maintenance through retirement.

SOURCE BASIS AND UNRESOLVED SOURCE GAPS

Primary supplied source basis:

- Thirsty's Standard v3 provides truthful scope, current-state inspection, evidence before claims, minimal authorized change, independent verification, hostile review, continuity, and no fake completion.
- Thirsty's Agents 101 supplies the unchanged role contracts. Its Architect role requires components, interfaces, dependencies, ownership, trust boundaries, data, authority, control, evidence, failure, recovery, alternatives, tradeoffs, implementation sequence, and verification plan.
- Thirsty's Owner's Cockpit Standard provides reference architecture and separation of concerns, governed action boundaries, authoritative state, action contracts, state machines, versioned schemas, correlation, observability, configuration, delivery, resilience, backup, verification, change, and owner ratification. It remains a candidate for owner ratification, not universal ratified authority.
- Thirsty's Standard V4 Design Blueprint contributes draft concepts for prerequisite-aware evidence, bounded roles, artifact consistency, constructive denial, dissent, and trusted transition. It remains draft and unratified.
- Sections 001–021 provide the current Codex chain from evidence and strategy through requirements, Systems Engineering, and enterprise constraints.

Unresolved source gaps:

- The supplied corpus does not contain a complete ratified universal Solution Architecture standard for every software, hardware, embedded, AI, cloud, edge, industrial, medical, financial, public, safety-critical, or regulated solution.
- This draft has not been reconciled against all current external architecture, software, data, security, privacy, safety, reliability, accessibility, operations, and sector standards.

- Required notation, review independence, assurance depth, performance thresholds, certification, and documentation must be specialized to the actual solution and consequence.
- Later specialist sections may expose missing interfaces, controls, operational behavior, or lifecycle obligations and must be permitted to reopen this section.
- STATUS remains CONTENT-COMPLETE.

023

SECTION 023 - SYSTEMS ARCHITECTURE

CODEX/03_SECTIONS/023_SYSTEMS_ARCHITECTURE.txt

SHA-256 389c9b97ea01b783b4bcbb8699f5492caa691b5e50524486cdeb8a4344c1cdac

32,647 BYTES / PRESERVED SOURCE IDENTITY

SECTION 023 — Systems Architecture

STATUS: CONTENT-COMPLETE

PURPOSE

Define the governed structural architecture of a system of interest so its elements, responsibilities, interfaces, dependencies, authority, state, data, control, evidence, trust, failure, recovery, deployment, operation, maintenance, and retirement can be understood and verified as one coherent whole.

Systems Architecture converts the whole-system definition established through Requirements Engineering, Systems Engineering, Enterprise Architecture, and Solution Architecture into explicit architectural views and contracts. It identifies what the system is made of, how the parts interact, where truth and authority reside, how failures are contained, how critical journeys remain controllable, and how the architecture can evolve without losing evidence or owner command.

This section does not treat diagrams as implementation, permit technology preference to replace requirements, or allow a model-generated architecture to become authoritative through fluency alone. It preserves the difference between proposed, approved, implemented, integrated, verified, validated, operational, and retired states.

REQUIRED OUTCOME

- A declared system-of-interest boundary with external actors, enabling systems, dependent systems, environments, jurisdictions, trust zones, lifecycle horizon, and excluded scope.
- A set of architecture drivers traceable to requirements, risks, constraints, mission threads, owner journeys, operational concepts, enterprise principles, solution decisions, and lifecycle obligations.
- Credible architecture alternatives evaluated against the same authorized criteria, mandatory constraints, evidence, assumptions, cost, complexity, human impact, reversibility, and operational burden.
- A selected, conditionally selected, or advisory systems architecture with decision owner, authority state, rationale, dissent, conditions, rejected alternatives, and reopening triggers.
- Logical, functional, behavioral, information, data, software, hardware, network, deployment, runtime, security, privacy, safety, reliability, resilience, observability, governance, and lifecycle views as applicable.
- A decomposition into bounded system elements with purpose, responsibilities, inputs, outputs, state ownership, data handled, authority level, trust level, dependencies, interfaces, failure behavior, recovery behavior, operational owner, and verification method.
- Explicit architectural invariants and forbidden states that cannot be traded away through local optimization or convenience.
- Versioned interface contracts covering semantics, identifiers, protocol, timing, units, authentication, authorization, validation, compatibility, idempotency, timeout, retry, cancellation, reconciliation, errors, evidence, and audit behavior.
- End-to-end mappings for critical owner, user, operator, safety, security, recovery, and evidence journeys across all participating elements.
- Failure-domain, dependency, degradation, containment, recovery, rollback, backup, restore, and continuity architecture.
- Deployment, configuration, secrets, environment, supply-chain, observability, support, maintenance, migration, modernization, deprecation, and retirement architecture.
- An architecture-to-requirement and architecture-to-evidence trace that exposes unresolved allocations, unproven claims, and stale assumptions.
- A durable handoff to specialist architecture and engineering sections without requiring downstream workers or agents to invent material structure, ownership, or authority.

WHY THIS MATTERS

A system may contain individually competent components and still fail as a system. Interfaces can disagree about meaning, timing, authority, state, or error handling. Multiple services can each believe they own the same truth. A local retry can duplicate an irreversible effect. A frontend can display completion while the governed operation remains unverified. A model can produce a valid-looking command outside its authority. Recovery can depend on the same failed dependency that caused the incident.

Systems Architecture makes these whole-system relationships visible before they are hardened into code, infrastructure, hardware, procedures, prompts, contracts, and organizational dependencies. It creates a reviewable basis for asking whether every requirement is structurally possible, every critical interface has an owner and contract, every consequential action crosses a governed boundary, every failure has a bounded consequence, and every material claim can be connected to evidence.

For human-AI systems, the architecture is also a relationship boundary. The architecture must keep proposal separate from authorization, capability separate from permission, orchestration separate from final authority, and conversational convenience separate from governed execution. Human control must remain available when models, providers, networks, or agents fail.

APPLICABILITY

- The system contains multiple components, services, devices, models, agents, repositories, data stores, networks, facilities, suppliers, environments, or operators.
- The system crosses trust, authority, identity, organizational, jurisdictional, physical, safety, security, privacy, or evidence boundaries.
- Requirements must be allocated across interacting elements and cannot be proven by one component alone.
- Critical behavior depends on sequencing, state transitions, concurrency, timing, consistency, availability, recovery, or external dependencies.
- The system includes AI-assisted decisions, agent delegation, autonomous behavior, tool execution, or model-generated action proposals.
- Existing architecture is being changed through migration, modernization, decomposition, consolidation, replacement, integration, or retirement.
- Failure could cause material harm, loss, deception, privilege escalation, data corruption, evidence loss, operational blindness, or loss of owner control.
- Architecture must support multiple environments or modes without weakening authority, evidence, or recovery boundaries.

NON-APPLICABILITY CONDITIONS

- A small, isolated, reversible change is fully covered by a current approved architecture, changes no system boundary, requirement allocation, interface, state owner, authority, data, failure domain, dependency, environment, recovery path, or lifecycle obligation, and impact analysis confirms that conclusion.
- The work is exploratory modeling or prototyping and makes no claim of implementation, integration, readiness, or operational suitability. The prototype must still identify its boundary and limitations.
- An active safety, security, privacy, integrity, or authority incident requires containment before architecture work can proceed safely.
- Required evidence or access is unavailable and the resulting architecture would be speculative. Such work may be retained only as a labeled hypothesis or option.
- The requested architecture would require unauthorized access to protected systems, data, credentials, facilities, suppliers, or people.

A non-applicability decision must identify the controlling architecture, exact version and environment, scope, evidence, authority, assumptions, and reopening trigger. Familiarity, low code volume, or generated diagrams are not sufficient.

PREREQUISITES AND DEPENDENCIES

- Sections 001-022 as applicable, especially the approved objective, requirements baseline, system context, operational concept, allocations, enterprise constraints, solution decisions, feasibility conditions, risk treatments, user journeys, domain model, and lifecycle obligations.

- A legitimate human architecture decision owner and explicit authority for scope, standards, exceptions, risk acceptance, deployment, migration, and retirement decisions.
- Current-state evidence for existing system elements, interfaces, data, state, software, hardware, infrastructure, environments, dependencies, providers, operations, incidents, and recovery.
- Controlled terminology, identifiers, configuration items, versions, architecture decisions, interface records, evidence, and change history.
- Access to relevant specialists and affected operators, users, maintainers, security, privacy, safety, reliability, accessibility, and governance reviewers.
- A process for architectural review, exception, experiment, change control, invalidation, rollback, safe halt, and owner ratification.

REQUIRED INPUTS

- Authorized objective, system-of-interest boundary, non-goals, lifecycle horizon, environments, users, operators, affected parties, and decision owner.
- Approved and proposed requirements, criticality, applicability, variants, waivers, acceptance criteria, verification methods, and unresolved conflicts.
- Systems Engineering outputs: context, operational concept, mission threads, system elements, allocations, interfaces, measures, baselines, integration, verification, validation, transition, support, and retirement obligations.
- Enterprise and Solution Architecture outputs: principles, standards, shared services, target and transition states, solution context, components, interface catalogue, deployment constraints, decisions, exceptions, and roadmaps.
- Domain Analysis and Domain Architecture inputs: terminology, bounded contexts, domain rules, invariants, entities, value objects, events, states, source-of-truth ownership, translations, and external authorities.
- Current code, repositories, schemas, APIs, events, data stores, hardware, network, identity, policy, configuration, secrets, build, deployment, runtime, observability, evidence, support, and incident records.
- Known workloads, timing, scale, capacity, performance, availability, reliability, resilience, security, privacy, safety, accessibility, and compliance conditions.
- External dependencies, supplier contracts, provider limits, service levels, jurisdictions, licenses, facilities, credentials, and continuity arrangements.
- Assumptions, unknowns, technical debt, constraints, prior decisions, rejected alternatives, open findings, accepted risks, and evidence that would change the architecture.

HUMAN RESPONSIBILITIES AND AUTHORITY

The authorized human remains responsible for the objective, architecture boundary, value tradeoffs, mandatory constraints, standards, exceptions, risk acceptance, resource commitments, irreversible choices, and final architecture decision.

- Confirm the system boundary, critical journeys, architecture drivers, mandatory constraints, and decision criteria.
- Identify which decisions are advisory, conditionally delegated, or reserved for human determination.
- Approve or reject material architecture decisions, exceptions, technology commitments, external dependencies, and transition plans.
- Resolve conflicts involving mission, values, safety, privacy, security, accessibility, legal obligations, operational burden, cost, and affected-party impact.
- Authorize experiments, prototypes, access, data use, provider evaluation, destructive testing, migration, and rollback.
- Ensure operational owners, maintainers, support, incident responders, and retirement authorities are identified.
- Accept residual risk only with visible evidence, scope, expiry, monitoring, and reopening conditions.
- Ratify only the exact architecture scope and evidence actually reviewed.

AI RESPONSIBILITIES AND BOUNDARIES

AI may inspect, extract, model, compare, trace, simulate, draft, challenge, and explain systems architecture within an explicit task and authority envelope. AI may not invent the objective, authority, implementation state, external dependency, performance result, risk acceptance, or final architecture decision.

- Separate observed current state, inherited requirement, inference, assumption, option, recommendation, decision, implementation, and verified evidence.

- Preserve exact source, version, environment, identity, provenance, freshness, and applicability for material architecture claims.
- Generate candidate decompositions, views, contracts, alternatives, failure hypotheses, and test paths as proposals rather than established truth.
- Detect missing owners, duplicated authority, conflicting sources of truth, circular dependencies, hidden coupling, weak interfaces, unbounded retries, single points of failure, stale evidence, and recovery paths that depend on failed components.
- Trace every proposed element and interface to requirements, journeys, domain meaning, risks, and verification obligations.
- Keep capability separate from permission. A model, agent, service, or tool may be technically capable while having no authority to act.
- Never allow model output to bypass deterministic validation, authorization, policy, action state, evidence, or audit controls.
- Preserve dissent and incompatible options instead of manufacturing one architecture for apparent completeness.
- State when evidence is insufficient and identify the smallest safe experiment or owner decision needed.
- Avoid selecting technologies solely because they are familiar, popular, available to the model, or convenient to generate.

APPLICABLE EXISTING AGENT ROLES

Primary roles:

- Architect
- Analyst
- Researcher

Common supporting roles:

- Planner
- Coordinator
- Security Auditor
- Governance Reviewer
- Observer
- Documentation Writer
- Memory Curator
- User Experience Reviewer

Challenge and assurance roles where consequence warrants:

- Critic
- Tester
- Verifier
- Red Team
- Decision Arbiter

Implementation remains separately authorized and belongs to the Implementer only when execution authority, scope, target, constraints, and evidence requirements are explicit. The role that proposes or implements an architecture must not be the sole independent verifier where consequence requires separation.

REQUIRED ACTIVITIES

1. Confirm architecture authority, scope, system boundary, environments, lifecycle horizon, and intended downstream use.
2. Inspect the actual current state before proposing a target state.
3. Identify architecture drivers from requirements, risks, journeys, operational concepts, enterprise principles, solution decisions, and lifecycle obligations.
4. Classify each driver as mandatory constraint, measurable objective, preference, assumption, unknown, or proposal.
5. Establish architecture evaluation criteria and the human authority behind any weighting or priority.

6. Define credible architecture alternatives, including no-change, staged, reversible, and reduced-scope options where materially available.
7. Decompose the system into bounded elements with single, explicit responsibilities and ownership.
8. Define state ownership, sources of truth, data authority, evidence authority, and control authority.
9. Define all critical interfaces and their contracts, including failure and recovery semantics.
10. Map end-to-end user, owner, operator, security, safety, evidence, recovery, migration, and retirement journeys.
11. Identify trust boundaries, privileged operations, external dependencies, failure domains, blast radius, and containment controls.
12. Model normal, boundary, degraded, denied, partial, timeout, retry, cancellation, recovery, rollback, and safe-halt behavior.
13. Define consistency, concurrency, ordering, idempotency, reconciliation, and duplicate-effect controls where state changes cross boundaries.
14. Define identity continuity, authentication, authorization, delegation, revocation, approval binding, and separation of duties.
15. Define evidence generation, correlation, provenance, integrity, freshness, supersession, export, and retention architecture.
16. Define observability for both owner comprehension and engineering diagnosis, including dependencies and degraded state.
17. Define deployment topology, environment separation, configuration, secrets, supply chain, build, release, migration, and rollback.
18. Define capacity, performance, availability, reliability, resilience, maintainability, supportability, accessibility, portability, compatibility, and cost implications.
19. Define verification, validation, hostile review, owner acceptance, and architecture conformance methods.
20. Record decisions, alternatives, assumptions, unknowns, dissent, exceptions, risks, experiments, evidence, and reopening triggers.
21. Create a dependency-aware implementation and transition sequence that preserves rollback and continuity.
22. Produce a durable handoff and confirm that downstream sections can proceed without hidden conversational memory.

HUMAN-AI INTERACTION REQUIREMENTS

- The AI must identify itself and its assigned role when its output may be mistaken for human judgment or institutional authority.
- Proposed architecture decisions must be labeled as proposals or recommendations until the legitimate decision owner acts.
- Material assumptions and unknowns must appear next to the affected architecture view or decision, not be buried in a general caveat.
- The human must be able to inspect alternatives, criteria, evidence, dissent, consequences, reversibility, and unresolved risks before authorization.
- AI-generated diagrams, matrices, scores, and models must expose their source basis and limitations.
- The AI must not pressure the owner to accept an option merely because it generated a complete-looking design.
- Consequential execution must require a separate governed action with explicit target, scope, authority, inputs, expected effect, evidence, and recovery.
- The human must be able to stop, defer, revoke, narrow, or supersede delegated architecture work.
- Critical status, stop, recovery, and evidence access must remain usable when AI or external providers are unavailable.
- Durable architecture records must preserve continuity outside the chat session.

ARCHITECTURE AND SYSTEM IMPLICATIONS

The systems architecture must preserve a coherent owner experience while enforcing separation of concerns. At minimum, it must distinguish:

- Owner and user experience surfaces from authoritative state and permission.
- Governed action gateway from presentation and conversational interfaces.
- Orchestration and agent coordination from final human authority.
- Domain services from duplicated frontend business logic.

- Operational state from evidence, audit, and historical truth.
- Observability from the authoritative state it describes.
- Repository-controlled components from external infrastructure, credentials, providers, and jurisdictions.
- Model-generated proposals from deterministic validation and governed execution.

The architecture must define minimum trust boundaries between client and governed backend, identity and authorization, model proposal and action validation, internal system and external infrastructure, operational state and evidence ledger, and imported content and executable instruction.

Every system element must have a bounded purpose, explicit owner, contract, failure behavior, recovery path, and verification method. Shared services and cross-cutting concerns must not become ownerless global state. Architecture views must remain mutually consistent or the artifact set must be classified as inconsistent until reconciled.

OWNER COCKPIT AND USER-CONTROL REQUIREMENTS

The owner must be able to:

- See the active system version, environment, mode, architecture baseline, current condition, degraded dependencies, and pending authority.
- Navigate from a critical journey or action to participating elements, interfaces, decisions, evidence, risks, and recovery paths.
- Distinguish current, proposed, approved, implemented, integrated, verified, validated, operational, deprecated, and retired architecture states.
- Inspect which component owns state, data, policy, authority, execution, evidence, and recovery for a consequential action.
- See assumptions, unknowns, exceptions, dissent, stale evidence, and unresolved architecture decisions.
- Review the consequence and reversibility of a proposed architecture or transition decision before approval.
- Stop, cancel, defer, revoke, quarantine, rollback, or request human review within defined authority.
- Operate essential status, stop, evidence, backup, restore, and recovery controls without dependence on conversational AI.
- Export architecture records, interface contracts, evidence, decisions, and continuity state in durable form.

REQUIRED OUTPUTS AND ARTIFACTS

- Systems architecture charter and boundary statement.
- Architecture driver and constraint register.
- Current-state, target-state, and transition architecture views.
- Architecture alternatives and tradeoff analysis.
- Architecture decision and dissent records.
- System element and responsibility catalogue.
- Interface control catalogue and versioned contracts.
- State ownership and source-of-truth model.
- Data, control, authority, identity, event, evidence, and trust-flow models.
- Critical journey and mission-thread maps.
- Dependency, failure-domain, blast-radius, degradation, recovery, rollback, backup, and restore models.
- Security, privacy, safety, reliability, resilience, accessibility, governance, and operational views.
- Deployment, environment, configuration, secrets, supply-chain, and external-dependency model.
- Observability, support, maintenance, migration, modernization, deprecation, and retirement model.
- Requirements-to-architecture and architecture-to-evidence traceability matrices.
- Verification, validation, hostile-review, and owner-acceptance plan.
- Exception, assumption, unknown, risk, open-decision, and reopening registers.
- Dependency-aware implementation and transition roadmap.
- Durable architecture handoff package.

EVIDENCE REQUIREMENTS

Evidence may include authoritative requirements and decisions, current architecture and configuration, source inspection, code, schemas, interface definitions, hardware records, deployment manifests, dependency and supplier records, runtime observations, traces, logs, metrics, incidents, tests, prototypes, benchmarks, failure exercises, recovery drills, security and privacy review, accessibility evaluation, owner journeys, support records, and operational history.

Every material architecture claim must identify the exact source or artifact, version, environment, method, result, limitations, freshness, and relationship to the claim. A diagram, design pattern, vendor statement, model output, static analysis result, passing unit test, or demonstration cannot by itself prove whole-system behavior, production operation, security, accessibility, resilience, or owner usability.

Evidence must be invalidated or reviewed when material requirements, architecture decisions, interfaces, policies, models, providers, environments, data flows, authority, or critical journeys change.

VERIFICATION AND ACCEPTANCE TESTS

- Trace every material requirement and risk treatment to one or more architecture elements, interfaces, controls, or visible unresolved states.
- Enumerate every consequential command path and verify uniform authority, policy, action-state, evidence, and audit enforcement.
- Tamper with client state and confirm it cannot create authority, approval, execution success, or verification success.
- Trace one critical journey end to end through identity, gateway, orchestration, domain behavior, state, evidence, observability, and recovery.
- Validate representative valid, invalid, stale, duplicate, unauthorized, and incompatible interface interactions.
- Force dependency outage, timeout, partial response, replay, version mismatch, and corrupted evidence; inspect containment and owner-visible state.
- Complete execution while independent verification fails and confirm those states remain distinct.
- Revoke delegated authority during work and verify safe stop, reconciliation, evidence, and residual-state handling.
- Disable AI and non-critical providers and confirm essential owner controls remain usable.
- Exercise backup, restore, rollback, reconnection, migration reversal, and recovery from a clean restart.
- Change a material architecture input after approval and confirm affected evidence, decisions, and downstream work are invalidated or reopened.
- Have an owner or authorized operator complete critical journeys, inspect evidence, and recover from failure without hidden coaching.
- Rebuild or reconstruct the architecture baseline from durable records without chat memory.

FAILURE STATES AND ANTI-PATTERNS

- Diagram-as-proof or model-output-as-authority.
- Technology-first decomposition that substitutes tools for requirements.
- Frontend authority, local success flags, or browser-held source of truth.
- Multiple owners for one authoritative state without reconciliation rules.
- Ownerless shared state or cross-cutting service.
- Implicit interface semantics, hidden retries, or unbounded fallback.
- Direct UI, model, agent, or alternate-client bypass around governed action controls.
- Capability treated as permission or role title treated as authority.
- Circular hard dependencies and recovery paths that depend on failed components.
- Split policy enforcement across inconsistent paths.
- Conflated proposal, approval, execution, verification, validation, and acceptance.
- Happy-path-only architecture with no degraded, denied, partial, or recovery states.
- Stale evidence or approval surviving a material architecture change.
- External provider, capacity, availability, or compliance assumed rather than evidenced.
- Security, privacy, safety, accessibility, reliability, resilience, support, or retirement deferred without an explicit owner and condition.
- Global architecture score that hides failed mandatory constraints.
- Unreviewed generated implementation represented as architecture conformance.

RECOVERY AND RECONSIDERATION

When the systems architecture is blocked, contradicted, or disproven:

1. Identify the exact requirement, decision, element, interface, dependency, assumption, authority, environment, or evidence affected.
2. Preserve the prior architecture, failed evidence, dissent, incident record, and supersession chain.
3. Classify the issue as source, scope, requirement, domain, architecture, interface, provider, environment, security, privacy, safety, reliability, resilience, governance, operation, or lifecycle related.
4. Contain affected work and identify safe work that may continue without compromising truth or authority.
5. Select the smallest admissible experiment, evidence request, architecture change, scope reduction, dependency replacement, rollback, or owner decision.
6. Re-evaluate alternatives, mandatory constraints, tradeoffs, migration, recovery, and downstream effects.
7. Update the architecture baseline, decision records, interface contracts, traceability, tests, and continuity map.
8. Invalidate stale approvals, evidence, and downstream claims.
9. Require renewed human authorization where scope, consequence, authority, risk, or irreversibility changes.

COMPLETION CRITERIA

Systems Architecture is DRAFT-complete only when:

- Scope, system boundary, lifecycle, environments, authority, architecture drivers, and mandatory constraints are explicit.
- Current-state evidence is separated from target and transition proposals.
- Credible alternatives and tradeoffs are recorded.
- Every major requirement maps to an architecture element or visible unresolved state.
- Every critical element has a responsibility, owner, interfaces, state and data authority, failure behavior, recovery path, and verification method.
- Every critical interface has a versioned contract and accountable owners.
- Trust, identity, authority, control, evidence, and failure boundaries are explicit.
- Critical journeys, degraded modes, safe halt, recovery, backup, restore, migration, rollback, support, maintenance, and retirement are addressed.
- Human and AI authority boundaries are explicit and owner control remains architecturally possible.
- Evidence, verification, validation, hostile review, and owner acceptance paths are defined.
- Assumptions, unknowns, dissent, exceptions, risks, and reopening triggers remain visible.
- The downstream handoff is durable and requires no invention of material structure or authority.

CONTENT-COMPLETE does not mean source-reconciled, hostile-reviewed, owner-accepted, implemented, integrated, verified, validated, deployed, or production-ready.

HANDOFF AND CONTINUITY REQUIREMENTS

The handoff must include the exact objective, scope, architecture baseline, requirements and allocations, drivers, decisions, alternatives, elements, interfaces, contracts, state ownership, data and authority flows, trust boundaries, failure and recovery models, deployment, operations, support, risks, evidence, tests, exceptions, dissent, open decisions, versions, hashes, and next safe action.

Section 024 and later specialist architecture and engineering sections must be able to continue from this durable record without hidden chat memory. Downstream discoveries that invalidate domain meaning, requirements, enterprise constraints, solution decisions, system allocations, or interface assumptions must reopen the affected upstream section through controlled change.

RELATED CODEX SECTIONS

Primary upstream: 001-022.

Primary downstream: 024 Domain Architecture; 025 Business Architecture; 026 Capability Architecture; 027 Process Architecture; 028 Functional Architecture; 029 Operational Architecture; 030 Information Architecture; 031 Data Architecture; 032 Application Architecture; 033 Software Architecture; 034 Platform Architecture; 035 Infrastructure Architecture; 036 Cloud Architecture; 037 Edge Architecture; 038 Distributed Systems Architecture; 039 Network Architecture; 040 Communications Architecture; 041 Integration Architecture; 042 API Architecture; 043 Event-Driven Architecture; 044 Runtime Architecture; 045 Security Architecture; 046 Identity Architecture; 047 Privacy Architecture; 048 Governance Architecture; 049 Compliance Architecture; 050 Safety Architecture; 051 Reliability Architecture; 052 Resilience Architecture; 053 Accessibility Architecture; 054 Content Architecture; 055 Knowledge Architecture; 056 AI Architecture; 057 Machine Learning Architecture; 058 Robotics Architecture; 059 Embedded Systems Architecture; 060 Hardware Architecture; 107-230 engineering; 231-257 testing, verification, and validation; 258-300 delivery and infrastructure; 301-332 runtime and operations; 374-406 maintenance through retirement.

SOURCE BASIS AND UNRESOLVED SOURCE GAPS

Primary supplied source basis:

- Thirsty's Standard v3 provides truthful scope, current-state inspection, evidence before claims, bounded execution, independent verification, hostile review, continuity, and no fake completion.
- Thirsty's Agents 101 supplies the unchanged role contracts. Its Architect role requires components, interfaces, dependencies, trust boundaries, data, authority, control, evidence, failure, recovery, alternatives, tradeoffs, implementation sequence, and verification plan.
- Thirsty's Owner's Cockpit Standard provides reference architecture and separation of concerns, governed action boundaries, authoritative state, action contracts, state machines, versioned schemas, correlation, observability, configuration, delivery, resilience, backup, restore, verification, and owner ratification. It remains a candidate for owner ratification.
- Thirsty's Standard V4 Design Blueprint contributes draft concepts for prerequisite-aware evidence, artifact consistency, bounded roles, constructive denial, dissent, and trusted transition. It remains draft and unratified.
- Sections 001-022 provide the current Codex chain from evidence and strategy through requirements, Systems Engineering, Enterprise Architecture, and Solution Architecture.

Unresolved source gaps:

- The supplied corpus does not contain a complete ratified universal Systems Architecture standard for every software, hardware, embedded, cyber-physical, AI, cloud, edge, industrial, medical, financial, public, safety-critical, or regulated system.
- This draft has not been reconciled against all current external systems, software, hardware, data, security, privacy, safety, reliability, resilience, accessibility, operations, or sector standards.
- Required notation, architecture framework, assurance independence, performance margins, certification, and artifact depth must be specialized to the actual system and consequence.
- Later specialist sections may expose missing elements, interfaces, constraints, or lifecycle obligations and must be permitted to reopen this section.
- STATUS remains CONTENT-COMPLETE.

024

SECTION 024 - DOMAIN ARCHITECTURE

CODEX/03_SECTIONS/024_DOMAIN_ARCHITECTURE.txt

SHA-256 9c587745d2a89f68a7fc53821c7b8aa65637e8ebabb9d7313444ed84b49ebdbd

30,870 BYTES / PRESERVED SOURCE IDENTITY

SECTION 024 — Domain Architecture

STATUS: CONTENT-COMPLETE

PURPOSE

Transform the source-grounded domain understanding established in Domain Analysis into a governed architecture of domain boundaries, responsibilities, language, rules, state, decisions, data ownership, services, events, authority, integrations, and lifecycle change.

Domain Architecture defines how domain meaning is partitioned and protected across the system and organization. It identifies bounded contexts, subdomains, model ownership, sources of truth, translations, contracts, invariants, decision rights, and integration boundaries so software, data, APIs, events, workflows, policies, AI systems, and human operations do not silently assign different meanings to the same concept.

This section does not declare a software class, database schema, organizational chart, or model-generated ontology to be the domain itself. It converts validated domain evidence into a controlled architectural basis while keeping disputed meaning, jurisdiction, effective date, authority, and unresolved gaps visible.

REQUIRED OUTCOME

- A declared domain-architecture scope identifying the domain, enterprise and system boundaries, decision owner, authoritative experts and sources, intended consumers, jurisdictions, lifecycle horizon, and exclusions.
- A domain landscape separating core, supporting, generic, external, regulatory, and enabling subdomains as applicable without assigning value labels unsupported by evidence.
- A bounded-context map showing each context's purpose, language, model owner, authority, source of truth, rules, state, services, events, data, evidence, interfaces, dependencies, and lifecycle.
- A context relationship map covering upstream and downstream responsibilities, published language, customer-supplier relationships, shared kernels or shared models, conformist dependencies, translation, anti-corruption layers, open-host services, and explicit isolation where applicable.
- Canonical terminology within each bounded context and controlled translation where the same word has different meanings across contexts.
- Domain models covering entities, value objects, aggregates or consistency boundaries where applicable, identifiers, classifications, policies, services, commands or intents, events, states, transitions, invariants, exceptions, time, and termination.
- Explicit distinction between domain rule, organizational policy, legal obligation, workflow convention, software behavior, data representation, model inference, recommendation, and human decision.
- Source-of-truth and authority ownership for every material domain fact, decision, state, rule, exception, and evidence object.
- Versioned domain contracts for APIs, events, data exchange, decisions, evidence, and human or AI handoffs.
- A change and evolution model covering effective dates, versioning, compatibility, migration, coexistence, deprecation, dispute, supersession, and retirement.
- A domain-to-requirement, domain-to-system, domain-to-data, domain-to-interface, domain-to-policy, and domain-to-test traceability model.
- A durable handoff to Business Architecture, Capability Architecture, Process Architecture, Functional Architecture, Operational Architecture, Information Architecture, Data Architecture, Application Architecture, Software Architecture, API Architecture, Event-Driven Architecture, AI Architecture, testing, operations, and lifecycle work.

WHY THIS MATTERS

Domain inconsistency is one of the most expensive forms of architectural debt because it spreads silently. A term can carry one meaning in a user journey, another in an API, another in a database, another in finance, and another in a model prompt. One service may treat an event as a fact while another treats it as a request. A workflow may encode an informal workaround as policy. A data record may be treated as the real-world entity. An AI classification may be treated as a decision even though no authority was delegated.

Domain Architecture confines meaning to explicit boundaries and defines how meaning crosses those boundaries. It allows different contexts to use different models where their responsibilities genuinely differ, while preventing accidental semantic collision. It also prevents architecture from turning every concept into one universal model that becomes too broad to own, verify, change, or retire.

For human-AI systems, domain architecture is a constitutional safeguard. The architecture must show who may define a term, who may observe a fact, who may recommend, who may decide, who may act, who is accountable, and what evidence is required. Fluency, prediction accuracy, or access to a tool cannot silently create domain authority.

APPLICABILITY

- The system spans multiple business, operational, scientific, legal, technical, organizational, or jurisdictional contexts.
- Terms are overloaded, disputed, translated, or used differently by users, operators, systems, regulators, or data sources.
- Multiple applications, services, teams, products, agents, models, or data stores represent the same real-world concepts.
- Domain rules, state transitions, exceptions, approvals, evidence, or authority are material to correctness.
- Integration, migration, modernization, merger, acquisition, platform consolidation, or data exchange requires semantic translation.
- AI systems classify, summarize, recommend, generate, route, or act on domain information.
- Domain errors could cause material harm, incorrect decisions, privacy or security violations, financial loss, safety failure, unfair treatment, or loss of accountability.
- Domain meaning changes over time through policy, law, contract, product, process, organization, or lifecycle events.

NON-APPLICABILITY CONDITIONS

- A small isolated change uses stable, current, authoritative domain definitions within one existing bounded context and changes no meaning, ownership, rule, state, event, interface, authority, jurisdiction, or lifecycle behavior.
- The work is user research about lived experience rather than domain structure; findings may inform but do not themselves establish domain authority.
- The work is only exploratory vocabulary collection and has not been validated by authoritative sources or experts. It must remain labeled as research output.
- An active incident requires containment before disputed domain meaning can be safely redesigned.
- Required domain knowledge is protected or unavailable and architecture would require unauthorized disclosure or speculation.

A non-applicability decision must identify the controlling domain model, exact context, version, jurisdiction, source, owner, evidence date, and reopening trigger.

PREREQUISITES AND DEPENDENCIES

- Section 016 Domain Analysis and relevant Research, Business Analysis, Stakeholder Management, User Research, Requirements Engineering, Systems Engineering, Enterprise Architecture, Solution Architecture, and Systems Architecture outputs.
- A legitimate human domain decision owner and identified authorities for definitions, rules, policy, data, decisions, exceptions, and jurisdiction.
- Current authoritative sources, domain experts, operational evidence, systems, schemas, APIs, events, workflows, records, incidents, and known exceptions.
- Controlled terminology, provenance, versioning, effective dates, disputes, approvals, and change history.
- Privacy, security, confidentiality, legal, licensing, records, retention, accessibility, and affected-party protections.
- A process for model review, domain decision, conflict resolution, exception, migration, compatibility, invalidation, and retirement.

REQUIRED INPUTS

- Authorized domain question, scope, intended use, decision owner, model owners, affected parties, environments, jurisdictions, and lifecycle horizon.
- Domain Analysis outputs: canonical terms, actors, responsibilities, authority, concepts, entities, value objects, events, states, rules, invariants, exceptions, decisions, processes, information, boundaries, dependencies, and unresolved disputes.
- Requirements and architecture outputs defining required capabilities, journeys, system elements, interfaces, data, policy, security, privacy, safety, reliability, accessibility, and evidence obligations.
- Authoritative laws, regulations, contracts, standards, policy, procedures, manuals, forms, data dictionaries, records, schemas, APIs, events, code, workflows, and reports.
- Evidence from domain experts, users, operators, owners, auditors, regulators, support, incidents, exceptions, and real transactions.
- Current applications, services, databases, integration maps, message flows, model prompts, agent tools, classification systems, and known semantic mismatches.
- Temporal and contextual conditions including effective dates, time zones, calendars, units, currencies, languages, markets, regions, and lifecycle states.
- Known synonyms, homonyms, aliases, deprecated terms, legacy identifiers, translation rules, migrations, and coexistence constraints.
- Assumptions, unknowns, disputed interpretations, model alternatives, accepted risks, and evidence that would change the architecture.

HUMAN RESPONSIBILITIES AND AUTHORITY

Authoritative humans and institutions retain responsibility for legitimate domain meaning, policy, law, values, exceptions, accountability, and decision rights.

- Confirm domain scope, subdomain boundaries, bounded contexts, model owners, canonical language, invariants, exceptions, and source hierarchy.
- Decide which differences are legitimate context-specific models and which are defects requiring convergence.
- Resolve disputes involving law, policy, contract, values, risk, harm, accountability, or institutional authority.
- Approve domain decisions, shared models, translations, context relationships, migration, deprecation, and retirement.
- Authorize access to protected knowledge and decide what may be encoded, automated, retained, shared, or used for AI.
- Ensure affected users, operators, data stewards, policy owners, security, privacy, accessibility, and governance authorities participate where required.
- Accept residual semantic or integration risk only with visible scope, evidence, expiry, monitoring, and reconsideration conditions.
- Ratify only the exact domain context, version, jurisdiction, and evidence reviewed.

AI RESPONSIBILITIES AND BOUNDARIES

AI may extract, compare, classify, model, map, translate, test, and document domain evidence within an explicit assignment. It may not invent domain truth, policy, legal meaning, authority, exception, consensus, source of truth, or final bounded-context decision.

- Preserve source identity, owner, date, version, jurisdiction, effective period, applicability, and confidence for every material statement.
- Separate observation, definition, rule, stored record, inference, prediction, recommendation, decision, instruction, exception, dispute, and unknown.
- Distinguish real-world concept from its representation in code, data, API, event, workflow, form, prompt, or model.
- Identify overloaded terms, inconsistent definitions, hidden translations, semantic drift, deprecated concepts, and context collisions.
- Propose candidate subdomains, bounded contexts, aggregates, services, events, contracts, and anti-corruption layers as options requiring review.
- Detect where one context is improperly controlling another context's meaning or authority.
- Keep capability separate from permission, responsibility, accountability, and authority for human, organizational, system, agent, and model actors.

- Never treat model confidence or classification as domain authority or verified fact without the required source and decision process.
- Preserve dissent, alternative models, and unresolved context boundaries instead of forcing a universal ontology.
- Avoid over-modeling concepts that do not affect decisions, behavior, evidence, interfaces, risk, or lifecycle.

APPLICABLE EXISTING AGENT ROLES

Primary roles:

- Architect
- Analyst
- Researcher

Common supporting roles:

- Observer
- Governance Reviewer
- Memory Curator
- Documentation Writer
- User Experience Reviewer
- Coordinator
- Planner

Challenge and assurance roles where consequence warrants:

- Critic
- Security Auditor
- Tester
- Verifier
- Red Team
- Decision Arbiter

The selected roles must operate within their existing contracts. Domain experts and legitimate human authorities are not replaced by an agent role. Implementation of domain structures remains separately authorized.

REQUIRED ACTIVITIES

1. Confirm the domain-architecture objective, scope, decision owner, jurisdictions, intended consumers, and lifecycle horizon.
2. Inspect authoritative domain evidence and the actual models embedded in current systems, data, workflows, policy, and operations.
3. Establish a source hierarchy for definitions, rules, decisions, states, exceptions, and evidence.
4. Identify subdomains and classify them only where the classification is useful and supported.
5. Define bounded contexts by responsibility, language, model ownership, authority, state, rules, data, services, events, and lifecycle.
6. Define canonical terminology inside each context and explicit translation across contexts.
7. Identify entities, value objects, identifiers, classifications, relationships, consistency boundaries, policies, services, commands or intents, events, states, transitions, invariants, exceptions, and termination.
8. Distinguish domain facts from records, derived values, predictions, recommendations, decisions, and evidence.
9. Identify the authoritative owner and source of truth for each material fact, state, rule, decision, exception, and evidence object.
10. Define context relationships, upstream and downstream obligations, published language, translations, anti-corruption layers, and isolation boundaries.
11. Define synchronous and asynchronous domain contracts, including semantic versioning, timing, ordering, duplication, correction, revocation, replay, reconciliation, and failure behavior.
12. Define identity, authentication, authorization, delegation, approval, accountability, and audit semantics for domain decisions and actions.
13. Map owner, user, operator, regulatory, exception, incident, recovery, migration, and retirement journeys across contexts.

14. Identify privacy, security, safety, accessibility, fairness, records, retention, localization, and jurisdiction implications of domain boundaries.
15. Define change, effective-date, coexistence, compatibility, migration, deprecation, supersession, and retirement rules.
16. Trace domain concepts and rules to requirements, architecture elements, data, APIs, events, policies, tests, and evidence.
17. Validate models with authoritative experts, representative examples, counterexamples, edge cases, and real operational scenarios.
18. Preserve disputes, alternative models, unresolved boundaries, assumptions, unknowns, and evidence gaps.
19. Define verification, hostile review, owner acceptance, and reopening triggers.
20. Produce a durable handoff that downstream sections can use without inventing domain meaning or authority.

HUMAN-AI INTERACTION REQUIREMENTS

- AI-generated domain models must be labeled as proposals until reviewed by legitimate domain authorities.
- The AI must show the source, context, jurisdiction, effective date, and uncertainty for material definitions and rules.
- The human must be able to inspect alternative context boundaries, translations, exceptions, dissent, and downstream consequences before approval.
- The AI must not collapse different stakeholder meanings merely to produce a cleaner model.
- The AI must identify when it is translating, inferring, simplifying, or applying an analogy rather than reproducing an authoritative definition.
- Model confidence must not be presented as legal, policy, clinical, financial, safety, or institutional authority.
- Consequential domain decisions and changes require explicit human authority and a durable decision record.
- The human must be able to stop, narrow, revoke, supersede, or defer delegated modeling work.
- Domain continuity must survive chat loss through versioned glossaries, context maps, decision records, examples, evidence, and open questions.

ARCHITECTURE AND SYSTEM IMPLICATIONS

Domain Architecture must define where domain behavior belongs and prevent accidental duplication of authoritative rules across presentation, orchestration, services, data, analytics, models, and external integrations.

- Presentation may explain domain state but must not become authoritative for rules, approval, or completion.
- Orchestration may coordinate work but must not silently redefine domain meaning or decision rights.
- Domain services own authoritative behavior only within their declared bounded context.
- Data stores represent domain state but do not automatically define real-world truth or policy.
- Events communicate attributable domain occurrences or declared state changes; event meaning, timing, source, correction, and revocation must be explicit.
- AI models may propose classification, extraction, prediction, recommendation, or content, but deterministic validation and authorized decision paths remain distinct.
- Integration boundaries must translate meaning deliberately and preserve provenance, authority, errors, and evidence.
- Shared models must be small, explicitly owned, versioned, and governed; broad enterprise-wide canonical models require strong evidence and ownership.
- Domain and system architecture views must remain consistent. A contradiction between terminology, rules, state, interfaces, data, policy, and tests must remain visible until resolved.

OWNER COCKPIT AND USER-CONTROL REQUIREMENTS

The owner must be able to:

- Inspect the active domain context, model version, jurisdiction, effective date, owner, authoritative sources, and known limitations.
- See which context owns a term, fact, state, rule, decision, exception, data object, event, and evidence object.
- Trace a visible label or action to its domain meaning, authoritative source, interface contract, and decision authority.
- Distinguish observed fact, stored record, inference, prediction, recommendation, decision, action, and verified evidence.

- Inspect translations between contexts and see where meaning changes or uncertainty is introduced.
- View unresolved disputes, exceptions, stale definitions, deprecated concepts, and migration state.
- Review consequences and affected downstream systems before approving a domain change.
- Stop, defer, revoke, quarantine, or supersede domain changes within defined authority.
- Export domain models, contracts, decisions, evidence, and continuity records without dependence on AI conversation history.

REQUIRED OUTPUTS AND ARTIFACTS

- Domain architecture charter and scope statement.
- Domain landscape and subdomain map.
- Bounded-context catalogue and context map.
- Canonical terminology and translation register.
- Domain model views for entities, value objects, identifiers, relationships, policies, services, commands or intents, events, states, transitions, invariants, and exceptions.
- Source-of-truth, state ownership, decision-right, responsibility, and accountability matrix.
- Context relationship and integration pattern catalogue.
- Published-language, API, event, data, decision, and evidence contracts.
- Domain journey, scenario, example, counterexample, and edge-case catalogue.
- Temporal, jurisdictional, localization, units, currency, calendar, and effective-date model.
- Privacy, security, safety, accessibility, fairness, records, retention, and governance implications.
- Change, compatibility, migration, coexistence, deprecation, supersession, and retirement plan.
- Domain-to-requirement, system, data, interface, policy, test, and evidence traceability matrix.
- Decision, dissent, exception, assumption, unknown, risk, and open-question registers.
- Verification, hostile-review, and owner-acceptance plan.
- Durable downstream handoff package.

EVIDENCE REQUIREMENTS

Evidence may include authoritative laws, regulations, contracts, standards, policy, procedures, manuals, forms, domain-expert review, user and operator research, records, transactions, schemas, APIs, events, code, workflows, incident history, audit findings, support cases, data lineage, examples, counterexamples, and observed state transitions.

Every material domain claim must identify its source, owner, version, jurisdiction, effective period, context, method, limitations, and relationship to the claim. Current software behavior, database design, majority usage, vendor terminology, AI output, or a single expert statement is not automatically authoritative.

Model validation must use representative examples, invalid examples, exceptions, boundary cases, conflicting sources, and change scenarios. Evidence must be invalidated or reviewed when definitions, law, policy, contracts, systems, interfaces, jurisdictions, data, authority, or effective dates change.

VERIFICATION AND ACCEPTANCE TESTS

- Give the same term to participants from different contexts and confirm the architecture exposes meaning differences rather than hiding them.
- Trace every material domain rule and invariant to an authoritative source, owner, context, implementation location, test, and evidence method.
- Trace a representative fact from observation or authoritative source through storage, transformation, interface, display, decision, and audit.
- Inject an invalid state transition, unauthorized decision, expired rule, revoked authority, and cross-jurisdiction case; confirm correct denial or escalation.
- Change an upstream definition or rule and confirm affected contexts, contracts, requirements, data, APIs, events, tests, and evidence are identified.
- Replay, duplicate, reorder, correct, and revoke representative domain events; verify downstream semantics and reconciliation.
- Introduce an external model using conflicting terminology and confirm the translation or anti-corruption boundary prevents semantic leakage.

- Ask an AI system to infer a domain fact or decision without evidence or authority and confirm it remains labeled as inference or unknown.
- Validate examples, counterexamples, edge cases, exceptions, and lifecycle transitions with authoritative domain reviewers.
- Reconstruct the domain model and its decision history from durable records without chat memory.
- Have an owner or operator trace a critical journey across contexts and identify authority, state, evidence, and recovery.

FAILURE STATES AND ANTI-PATTERNS

- One universal model forced across contexts with genuinely different responsibilities.
- Database schema, class hierarchy, API shape, event payload, or prompt treated as the domain itself.
- Same term used with different meanings without context or translation.
- Different terms used for the same concept without controlled mapping.
- Shared kernel or shared model with no owner, version, or change process.
- Domain rules duplicated across frontend, services, data, workflows, and models.
- Organizational reporting lines used as permanent domain boundaries without evidence.
- Current workaround encoded as legitimate policy.
- AI inference, prediction, or recommendation treated as observed fact or authorized decision.
- Capability treated as permission, authority, or accountability.
- Events used as commands or requests without explicit semantics.
- Missing effective dates, jurisdiction, units, currency, calendar, or temporal conditions.
- Cross-context data sharing without provenance, consent, authorization, purpose, or retention controls.
- Hidden translation, lossy mapping, or silent default at integration boundaries.
- Domain exceptions handled outside the model and therefore absent from tests and evidence.
- Deprecated concepts or identifiers retained indefinitely without migration and retirement.
- Apparent consensus created by deleting disputed interpretations.

RECOVERY AND RECONSIDERATION

When the domain architecture is contradicted, disputed, or invalidated:

1. Identify the exact term, context, rule, state, event, source, owner, jurisdiction, effective date, interface, or decision affected.
2. Preserve the prior model, conflicting evidence, dissent, examples, incidents, and supersession history.
3. Determine whether the conflict is factual, definitional, jurisdictional, temporal, procedural, policy-based, value-based, technical, or organizational.
4. Route factual gaps to Research or verification, requirement conflicts to the requirement owner, governance conflicts to Governance Reviewer, and value or authority conflicts to the legitimate human decision owner.
5. Contain affected integrations and identify safe work that can continue without propagating incorrect meaning.
6. Select the smallest admissible evidence request, expert review, example set, translation change, context split or merge, migration, rollback, or owner decision.
7. Update terminology, context maps, models, contracts, requirements, schemas, APIs, events, tests, data migrations, and continuity records.
8. Invalidate stale approvals, evidence, and downstream claims.
9. Re-authorize any change that alters authority, rights, obligations, risk, jurisdiction, or irreversible state.

COMPLETION CRITERIA

Domain Architecture is DRAFT-complete only when:

- Scope, contexts, jurisdictions, authority, owners, sources, lifecycle, and intended consumers are explicit.
- Subdomains and bounded contexts have documented responsibilities, language, models, state, rules, data, services, events, and ownership.
- Canonical terms and cross-context translations are controlled.
- Material entities, value objects, identifiers, policies, services, commands or intents, events, states, transitions, invariants, exceptions, and termination are addressed as applicable.
- Sources of truth, decision rights, accountability, and evidence ownership are explicit.

- Context relationships and integration contracts include semantic, authority, failure, recovery, compatibility, and lifecycle behavior.
- Human and AI authority boundaries are explicit.
- Domain models trace to requirements, architecture, data, interfaces, policy, tests, and evidence.
- Examples, counterexamples, edge cases, disputes, assumptions, unknowns, and limitations remain visible.
- Change, migration, coexistence, deprecation, supersession, and retirement are governed.
- The downstream handoff is durable and requires no invention of material domain meaning or authority.

CONTENT-COMPLETE does not mean source-reconciled, hostile-reviewed, owner-accepted, implemented, verified, validated, or universally authoritative.

HANDOFF AND CONTINUITY REQUIREMENTS

The handoff must include the exact domain scope, source hierarchy, terms, context map, model versions, owners, authority, rules, states, events, decisions, exceptions, integrations, contracts, data and evidence ownership, examples, disputes, risks, migrations, tests, approvals, hashes, and next safe action.

Sections 025 and later architecture and engineering sections must be able to continue from the durable record without hidden chat memory. Downstream discoveries that contradict domain meaning, requirements, system architecture, data, interfaces, policy, or operational behavior must reopen the affected upstream section through controlled change.

RELATED CODEX SECTIONS

Primary upstream: 001-023, especially 011 Business Analysis, 014 Stakeholder Management, 015 User Research, 016 Domain Analysis, 019 Requirements Engineering, 020 Systems Engineering, 021 Enterprise Architecture, 022 Solution Architecture, and 023 Systems Architecture.

Primary downstream: 025 Business Architecture; 026 Capability Architecture; 027 Process Architecture; 028 Functional Architecture; 029 Operational Architecture; 030 Information Architecture; 031 Data Architecture; 032 Application Architecture; 033 Software Architecture; 041 Integration Architecture; 042 API Architecture; 043 Event-Driven Architecture; 048 Governance Architecture; 049 Compliance Architecture; 054 Content Architecture; 055 Knowledge Architecture; 056 AI Architecture; 098 Documentation Architecture; 107 Software Engineering; 128 API Engineering; 129 Integration Engineering; 130 Microservices Engineering; 164 Data Science; 167 Machine Learning Engineering; 176 Ontology Engineering; 177 Semantic Systems Engineering; 178 Agent Systems Engineering; 231-257 testing, verification, and validation; 301-332 operations; 374-406 maintenance through retirement.

SOURCE BASIS AND UNRESOLVED SOURCE GAPS

Primary supplied source basis:

- Thirsty's Standard v3 provides truthful scope, source inspection, evidence before claims, bounded work, independent verification, hostile review, continuity, and no fake completion.
- Thirsty's Agents 101 supplies the unchanged role contracts. Its Architect, Researcher, Analyst, Governance Reviewer, Tester, Verifier, Documentation Writer, Memory Curator, and Coordinator roles provide bounded source, modeling, governance, evidence, and continuity behavior.
- Thirsty's Owner's Cockpit Standard requires authoritative domain services, explicit control and evidence boundaries, versioned schemas, traceability, owner-visible truth, and governed change. It remains a candidate for owner ratification.
- Thirsty's Standard V4 Design Blueprint contributes draft concepts for claim and obligation dependencies, bounded roles, dissent, artifact consistency, constructive denial, and trusted transition. It remains draft and unratified.
- Section 016 provides the source-grounded domain analysis basis. Sections 019-023 provide requirements, system, enterprise, solution, and systems architecture context.

Unresolved source gaps:

- The supplied corpus does not contain a complete ratified universal Domain Architecture or domain-modeling standard for every legal, financial, health, scientific, industrial, public, safety-critical, cultural, linguistic, or technical domain.

- This draft has not been reconciled against all current external domain modeling, data, software, semantic, policy, security, privacy, accessibility, AI, or sector standards.
- Terms such as bounded context, aggregate, published language, shared kernel, and anti-corruption layer are used as architectural concepts but require specialization and owner approval in each actual domain.
- Required notation, modeling depth, review independence, jurisdiction handling, ontology governance, and assurance evidence must be tailored to consequence.
- Later Business, Capability, Process, Information, Data, Application, Software, API, Event, AI, and operations sections may expose missing domain boundaries or contracts and must be permitted to reopen this section.
- STATUS remains CONTENT-COMPLETE.

025

SECTION 025 - BUSINESS ARCHITECTURE

CODEX/03_SECTIONS/025_BUSINESS_ARCHITECTURE.txt

SHA-256 31df9b46daa7a23894b2a79c6d01e44ee7861b63874758a1492f9275d2b41eb6

37,467 BYTES / PRESERVED SOURCE IDENTITY

SECTION 025 — Business Architecture

STATUS: CONTENT-COMPLETE

PURPOSE

Define the governed structure of how an enterprise or bounded organization creates, delivers, sustains, governs, changes, and retires value. Business Architecture connects mission and strategy to stakeholders, value propositions, outcomes, value streams, capabilities, organizations, roles, decisions, policies, information, products, services, resources, measures, risks, controls, investments, and transition work.

This section translates strategy and enterprise intent into an operating structure that can be understood before it is embedded in technology, staffing, contracts, procedures, or AI automation. It exposes who is affected, who owns which decisions, what the organization must be able to do, how value and evidence move, where accountability resides, and which conditions must remain true during change.

Business Architecture is not an organization chart, process inventory, software map, or consulting diagram. It does not treat an aspiration as an operating reality, an AI-generated model as authority, or a capability label as proof that the capability exists. It preserves the difference between current, proposed, approved, implemented, adopted, measured, effective, degraded, and retired states.

REQUIRED OUTCOME

- A declared enterprise or business boundary with included and excluded organizations, products, services, populations, jurisdictions, environments, partners, suppliers, channels, and lifecycle horizon.
- A source-grounded current-state business architecture that distinguishes observed operation from policy, planned work, informal workaround, and inference.
- A target-state business architecture traceable to mission, strategy, stakeholder outcomes, user evidence, domain meaning, requirements, risk, and authorized decisions.
- Explicit value propositions and outcome statements identifying intended beneficiaries, affected parties, obligations, harms to avoid, evidence, assumptions, and success boundaries.
- A value-stream architecture showing triggering need, participating stakeholders, capabilities, decisions, information, products, services, controls, evidence, support, recovery, and terminal outcome.
- A business capability map with stable definitions, ownership, outcomes, dependencies, required competencies, resources, information, controls, measures, evidence, lifecycle state, and realization mechanisms.
- An operating-model view defining how capabilities, organizations, roles, decision rights, processes, information, technology, partners, locations, and governance combine to deliver outcomes.
- A decision-rights and accountability model that distinguishes authority, responsibility, consultation, execution, verification, approval, risk acceptance, and final human control.
- A business information concept model identifying authoritative concepts, records, evidence, ownership, provenance, privacy, retention, and cross-domain translation needs.
- A product, service, channel, customer, user, partner, supplier, and ecosystem view linked to capabilities and value streams.
- A policy, rule, obligation, control, and exception view showing source, jurisdiction, owner, enforcement location, evidence, expiry, and escalation path.
- Current, target, and transition states with dependencies, sequencing, investments, benefits, risks, adoption, training, migration, rollback, continuity, and retirement obligations.
- A measurement architecture that distinguishes activity, output, outcome, benefit, harm, control effectiveness, resilience, owner operability, and evidence quality.
- A durable handoff to Capability Architecture, Process Architecture, Functional Architecture, Operational Architecture, Information Architecture, organization design, product and service design, technology architecture, delivery, operations, and lifecycle governance.

WHY THIS MATTERS

Strategy fails when it cannot be translated into an operating structure. Organizations may fund projects without knowing which enduring capability they improve. Teams may optimize local output while damaging an end-to-end value stream. Technology may automate an undocumented workaround or conceal a broken decision boundary. Metrics may reward volume while degrading user outcomes, safety, dignity, trust, or long-term viability.

Business Architecture gives the owner a coherent view of how intent becomes value and obligation becomes controlled action. It makes visible the relationships among mission, stakeholders, capabilities, decisions, processes, information, products, services, resources, controls, and evidence. It also exposes gaps between what the enterprise claims to do and what current evidence shows it can actually sustain.

For human-AI systems, Business Architecture prevents the substitution of model capability for institutional authority. AI may help discover patterns, model options, identify contradictions, and maintain traceability. It may not decide whose interests matter, redefine the mission, create binding policy, allocate undelegated authority, accept risk, or represent an unproven operating capability as real.

APPLICABILITY

- Mission, business, product, portfolio, program, project, or transformation strategy must be translated into an operating structure.
- The enterprise needs to understand how value is created across organizational, product, service, process, information, technology, partner, or jurisdiction boundaries.
- Capabilities, value streams, roles, decision rights, policies, controls, products, services, or investments are unclear, duplicated, fragmented, or changing.
- A merger, acquisition, divestiture, partnership, outsourcing, insourcing, reorganization, platform change, modernization, or retirement affects business operation.
- A digital, automation, AI, or agent initiative could change authority, accountability, work allocation, user experience, evidence, or risk.
- Business and technology teams disagree about what outcome is required, who owns it, or which operating constraint is binding.
- Regulatory, contractual, privacy, safety, security, accessibility, records, or public-interest obligations must be embedded in operating design.
- Portfolio and investment decisions require stable capability and value-stream context rather than project labels alone.
- Current-state operations include manual workarounds, shadow systems, undocumented decisions, or hidden dependencies.

NON-APPLICABILITY CONDITIONS

- A narrowly bounded change has no effect on value propositions, value streams, capabilities, operating model, decision rights, policy, organization, information, products, services, partners, measures, risk, controls, or lifecycle obligations, and current architecture evidence proves that conclusion.
- The work is a temporary exploratory exercise that makes no claim of organizational approval, implementation, adoption, benefit, or operating readiness. The exercise must still state its boundary and limitations.
- An active safety, security, privacy, integrity, labor, legal, or authority incident requires containment before business redesign can proceed safely.
- The requested modeling would require unauthorized access to protected strategy, personnel, customer, user, partner, supplier, contract, financial, security, or operational information.
- The legitimate business owner, affected-party authority, or decision boundary cannot be established. Work may continue only as a labeled research or option package.

A non-applicability decision must identify the exact current architecture, scope, version, evidence date, decision owner, assumptions, and reopening triggers. Convenience or small apparent size is not sufficient.

PREREQUISITES AND DEPENDENCIES

- Sections 001-018 provide research, foresight, strategy, product, portfolio, program, project, market, stakeholder, user, domain, feasibility, and risk context.
- Section 019 provides controlled requirements and acceptance boundaries.

- Section 020 provides the system-of-interest and lifecycle engineering context where business operation depends on systems or enabling systems.
- Sections 021-024 provide enterprise, solution, systems, and domain architecture constraints and current-state relationships.
- The mission, strategy owner, enterprise boundary, affected parties, decision rights, governing obligations, and evidence date must be known or explicitly unresolved.
- Current-state sources must include more than policy or aspiration when actual operating behavior matters.
- Material terminology must align with Domain Analysis and Domain Architecture or include explicit translations.
- Feasibility and risk conditions must be carried into target and transition design rather than treated as separate reports.
- Required human authority for organization, policy, investment, employment, contracting, risk acceptance, and external commitments must be established.

REQUIRED INPUTS

- Authorized business question, objective, scope, exclusions, decision owner, and decision horizon.
- Mission, business strategy, product strategy, portfolio priorities, program and project commitments.
- Stakeholder and affected-party maps, user research, market research, competitive intelligence, and partner or supplier context.
- Current and target value propositions, products, services, channels, customer and user segments, obligations, and harms to avoid.
- Current-state value streams, capabilities, processes, functions, organization structures, roles, locations, resources, competencies, information, applications, technologies, partners, and controls.
- Domain language, bounded contexts, business rules, decisions, states, events, records, and sources of truth.
- Requirements, acceptance criteria, risk register, feasibility conditions, architecture decisions, standards, exceptions, and technical constraints.
- Financial, capacity, workforce, contractual, regulatory, security, privacy, safety, accessibility, records, sustainability, and resilience constraints.
- Performance, outcome, benefit, harm, incident, support, audit, and control-effectiveness evidence.
- Current initiatives, investments, roadmaps, dependencies, technical debt, operational debt, contractual commitments, and retirement obligations.
- Known assumptions, unknowns, dissent, unresolved decisions, and evidence limitations.

HUMAN RESPONSIBILITIES AND AUTHORITY

The authorized human owner remains responsible for:

- Defining and approving mission, values, strategic outcomes, enterprise boundary, affected-party obligations, and acceptable tradeoffs.
- Establishing legitimate decision owners and confirming the authority of participating executives, managers, domain experts, operators, employees, contractors, partners, and reviewers.
- Approving value propositions, target operating model, capability priorities, organization changes, policies, investments, external commitments, risk acceptance, and transition authority.
- Determining when workforce, labor, accessibility, privacy, safety, security, legal, ethical, public-interest, or contractual consultation is required.
- Resolving value conflicts, policy conflicts, priority conflicts, and disputed interpretations that were not validly delegated.
- Preventing AI recommendations, scoring, simulations, or generated diagrams from becoming binding through presentation alone.
- Ensuring affected people have appropriate notice, participation, consent, grievance, appeal, accommodation, or recourse where required.
- Accepting, rejecting, conditioning, deferring, superseding, or reopening business architecture decisions.
- Ratifying claims of implementation, adoption, operating effectiveness, benefit, or readiness only when current evidence supports them.

AI may support but cannot inherit these responsibilities through tool access, system capability, role title, or prior conversational context.

AI RESPONSIBILITIES AND BOUNDARIES

AI may:

- Retrieve, classify, compare, and summarize authorized source material.
- Build current-state inventories, capability maps, value-stream models, decision maps, operating-model views, dependency graphs, traceability matrices, and option analyses.
- Detect contradictions, duplicated capabilities, fragmented ownership, missing decisions, unsupported claims, stale evidence, metric conflicts, and transition risks.
- Generate candidate target states, transition options, scenarios, questions, experiments, acceptance conditions, and evidence requests.
- Maintain provenance, assumptions, unknowns, dissent, dependencies, versions, and reopening triggers.
- Simulate bounded scenarios when the model, assumptions, data, and limitations are explicit.

AI must not:

- Redefine mission, values, stakeholders, outcomes, obligations, or enterprise scope.
- Invent organizational authority, policy, law, contract, budget, capacity, market evidence, user needs, consent, or operating state.
- Treat access to information or systems as authority to decide, approve, commit, reorganize, hire, terminate, contract, disclose, or deploy.
- Rank people, teams, partners, or populations using undisclosed or unauthorized criteria.
- Recommend harmful surveillance, coercion, manipulation, discrimination, dark patterns, or removal of meaningful human control.
- Convert a capability hypothesis into an implemented capability claim.
- Hide dissent, exceptions, weak evidence, affected-party harm, or uncertainty to create a clean target model.
- approve its own architecture, implementation, or verification where independent review is required.

When authority, evidence, or governing obligations are unclear, AI must return an explicit unknown, hold, request for owner decision, or bounded option set rather than fabricate closure.

APPLICABLE EXISTING AGENT ROLES

- Architect: structures the business architecture, views, boundaries, interfaces, dependencies, authority, failure, recovery, and verification path.
- Researcher: acquires and assesses authoritative business, market, operational, regulatory, contractual, and stakeholder evidence.
- Analyst: compares current and target states, options, outcomes, costs, dependencies, risks, and sensitivity to assumptions.
- Critic: challenges unsupported claims, hidden dependencies, incomplete stakeholder coverage, metric distortion, and operating-model contradictions.
- Security Auditor: evaluates security, identity, authority, supply-chain, information, partner, and control implications.
- Tester: converts business architecture claims into testable journeys, scenarios, controls, and acceptance conditions.
- Verifier: independently checks claim-to-evidence alignment, artifact identity, version, scope, and completion boundaries.
- Documentation Writer: maintains architecture descriptions, catalogs, decisions, glossary, traceability, and operating guidance.
- Planner: sequences discovery, modeling, decisions, experiments, transition, adoption, verification, and recovery work.
- Coordinator: assigns bounded work, preserves provenance, reconciles compatible outputs, and surfaces unresolved conflict.
- Observer: captures actual operating behavior, decisions, events, dependencies, workarounds, and evidence without unsupported interpretation.
- Memory Curator: preserves governed architecture state, decisions, source links, supersession, open questions, and continuity.
- Governance Reviewer: evaluates authority, rule hierarchy, policy, exception, approval, and safe-halt conditions.

- Red Team: pressure-tests incentives, bypasses, hidden authority, concentration, transition, resilience, and false-readiness claims.
- User Experience Reviewer: evaluates whether owner, employee, operator, customer, and user journeys remain understandable, accessible, controllable, and recoverable.
- Decision Arbiter Advisory: compares options against authorized criteria and provides advisory recommendations unless explicit decision authority exists.

Roles must be selected for the required function and given explicit assignments, inputs, authority, prohibited actions, outputs, evidence requirements, review gates, and completion conditions. The role corpus remains unchanged.

REQUIRED ACTIVITIES

1. Confirm the authorized question, enterprise boundary, decision owner, scope, exclusions, time horizon, environments, jurisdictions, and lifecycle state.
2. Establish the source hierarchy and separate observed operation, documented policy, contractual obligation, planned work, inference, assumption, and unknown.
3. Identify stakeholders, users, customers, beneficiaries, affected parties, partners, suppliers, regulators, operators, owners, and decision authorities.
4. Define value propositions and intended outcomes, including obligations, harms to avoid, anti-metrics, acceptance boundaries, and evidence methods.
5. Map current-state value streams from triggering need through decisions, capabilities, processes, information, products, services, controls, support, recovery, and outcome.
6. Identify and define business capabilities independently from current organization, project, product, application, provider, or process implementation.
7. Map capability dependencies, shared resources, competencies, information, controls, systems, partners, evidence, resilience, and lifecycle state.
8. Model the operating model: how value streams, capabilities, organization, roles, decision rights, processes, information, technology, locations, partners, and governance combine.
9. Define decision rights, accountability, delegation, approval, verification, risk acceptance, escalation, grievance, appeal, and revocation paths.
10. Model business information concepts, records, provenance, ownership, privacy, retention, quality, access, translation, and evidence relationships.
11. Map products, services, channels, segments, ecosystems, partners, suppliers, contracts, and external obligations to value streams and capabilities.
12. Identify policies, rules, controls, exceptions, standards, measures, incentives, and enforcement locations.
13. Evaluate architecture options against authorized criteria, mandatory constraints, evidence, cost, capacity, human impact, reversibility, resilience, and operational burden.
14. Define current, target, and transition architectures, including coexistence, migration, training, adoption, cutover, rollback, support, continuity, and retirement.
15. Define measurement architecture for activity, output, outcome, benefit, harm, control effectiveness, resilience, trust, accessibility, and owner operability.
16. Trace business architecture elements to mission, strategy, requirements, domain models, enterprise architecture, solution and systems architecture, initiatives, investments, risks, controls, tests, and evidence.
17. Validate models with legitimate owners, domain experts, operators, affected parties, representative users, and current evidence.
18. Run hostile review against metric gaming, hidden work, shadow authority, duplicated ownership, process theater, technology capture, provider lock-in, and AI overreach.
19. Record decisions, dissent, exceptions, assumptions, unknowns, evidence gaps, residual risks, and reopening triggers.
20. Produce a durable handoff to Capability Architecture and downstream architecture, design, engineering, change, adoption, operation, measurement, and retirement work.

HUMAN-AI INTERACTION REQUIREMENTS

- AI-generated business models are proposals until approved by the legitimate human authority.
- Every material AI claim must identify source, method, evidence date, scope, limitations, and whether it is observation, inference, assumption, recommendation, or unknown.

- The owner must be able to inspect current-state evidence, alternative models, dissent, tradeoffs, affected-party consequences, and unresolved questions before approval.
- The AI must disclose when a target model reflects a design choice rather than discovered fact.
- AI must not compress different stakeholder interests into one synthetic objective without authorization.
- Capability, maturity, performance, risk, or prioritization scores must expose criteria, weights, data quality, uncertainty, and decision authority.
- Consequential recommendations require evidence, alternatives, reversibility, conditions, and what would change the recommendation.
- Human approval must be bound to the reviewed architecture version, scope, assumptions, and evidence. Material change invalidates prior approval.
- The owner must be able to stop, narrow, revoke, supersede, or defer AI work and inspect the resulting state.
- Critical continuity must survive chat loss through versioned architecture artifacts, decisions, traceability, evidence, and next-safe-action records.

ARCHITECTURE AND SYSTEM IMPLICATIONS

Business Architecture establishes requirements for all downstream architecture and system work.

- Value streams and capabilities must map to domain, information, application, technology, solution, systems, process, functional, operational, security, privacy, safety, accessibility, data, integration, and lifecycle architecture as applicable.
- Each consequential business decision must have an authority path, policy basis, state model, evidence requirement, and owner-visible control.
- Each critical value stream must identify system and non-system dependencies, manual fallbacks, degradation, recovery, support, and continuity needs.
- Business information concepts must align with Domain Architecture and later Information and Data Architecture. Translation across contexts must be explicit.
- Technology may realize a capability but must not define the capability's enduring meaning or business ownership.
- AI and automation must be modeled as realization mechanisms with bounded role, authority, data, evidence, failure, fallback, monitoring, and revocation requirements.
- Current-state architecture must expose shadow systems, spreadsheets, manual decisions, tacit knowledge, external services, and human work rather than omitting them because they are not formal technology.
- Target-state architecture must preserve owner operability and human final authority where values, rights, obligations, risk, or irreversible consequences remain human decisions.
- Architecture changes must invalidate affected requirements, designs, tests, evidence, approvals, training, procedures, contracts, and measures.

OWNER COCKPIT AND USER-CONTROL REQUIREMENTS

The owner must be able to:

- Inspect the active business architecture version, scope, environment, jurisdiction, evidence date, decision owner, and approval state.
- Navigate from mission and strategy to outcomes, value streams, capabilities, decisions, processes, information, products, services, systems, initiatives, measures, risks, controls, and evidence.
- Distinguish current, proposed, approved, implemented, adopted, measured, effective, degraded, superseded, and retired states.
- See who owns each value stream, capability, decision, policy, measure, risk, exception, and transition obligation.
- Trace a capability claim to actual people, process, information, technology, partner, control, support, recovery, and evidence.
- Inspect affected parties, benefits, harms, assumptions, dissent, exceptions, and missing evidence before authorization.
- Compare architecture options and view consequences, dependencies, costs, reversibility, and residual risks.
- Stop, defer, revoke, quarantine, supersede, or reopen architecture and transition decisions within defined authority.
- Export architecture models, decisions, evidence, traceability, and continuity records without relying on AI chat history.
- Continue essential status, stop, decision, and recovery functions when AI or external providers are unavailable.

REQUIRED OUTPUTS AND ARTIFACTS

- Business architecture charter, scope, authority, and source register.
- Stakeholder, affected-party, ecosystem, partner, and supplier map.
- Value proposition and outcome catalog with obligations, harms to avoid, measures, anti-metrics, and evidence methods.
- Current, target, and transition business architecture views.
- Value-stream map and value-stream catalog.
- Business capability map, catalog, dependency model, ownership, lifecycle, and realization trace.
- Operating-model description and operating-model decision records.
- Organization, role, responsibility, authority, delegation, approval, and accountability views.
- Business information concept model and record/evidence ownership map.
- Product, service, channel, segment, customer, user, partner, supplier, and ecosystem views.
- Policy, rule, obligation, standard, control, exception, and enforcement catalog.
- Measure, incentive, outcome, benefit, harm, control-effectiveness, and resilience model.
- Initiative, investment, capability, value-stream, requirement, risk, and benefit traceability matrix.
- Transition roadmap including adoption, workforce, training, communication, migration, coexistence, rollback, continuity, support, and retirement.
- Architecture decision, dissent, assumption, unknown, exception, risk, and open-question registers.
- Verification, hostile-review, owner-acceptance, and reopening plan.
- Durable handoff package for Capability Architecture and downstream work.

EVIDENCE REQUIREMENTS

Evidence may include mission and strategy records, owner decisions, governance documents, organization records, role definitions, policies, contracts, laws, regulations, budgets, portfolio data, product and service records, customer and user research, stakeholder evidence, operational observation, process records, transaction records, system telemetry, incident history, support cases, audits, risk assessments, control tests, workforce evidence, training records, supplier evidence, financial records, and benefit measurement.

Each material claim must identify source identity, version, environment, jurisdiction, effective date, method, scope, owner, limitations, and relationship to the claim. A policy does not prove operational behavior. A software feature does not prove business capability. A project completion record does not prove adoption or benefit. A metric does not prove value unless its definition, data lineage, incentives, and validity are known. An AI-generated map is not evidence of the organization.

Current-state evidence should combine documented intent with direct operational evidence where practical. Target-state evidence must identify assumptions and design authority. Transition claims must bind to exact dependencies, resources, approvals, training, migration, support, rollback, and acceptance conditions.

VERIFICATION AND ACCEPTANCE TESTS

- Select a mission outcome and trace it through strategy, value proposition, value stream, capabilities, decisions, processes, information, products or services, systems, measures, controls, and evidence.
- Select a capability and prove that its outcome, owner, dependencies, competencies, resources, information, systems, controls, support, recovery, and current state are explicit.
- Select a critical value stream and run a representative end-to-end scenario, including exception, degradation, support, recovery, and evidence.
- Modify one current-state assumption and verify that affected architecture, requirements, initiatives, measures, risks, and decisions are identified.
- Introduce conflicting decision ownership and verify the architecture surfaces rather than silently resolves the conflict.
- Attempt to infer authority from an organization title, system permission, or AI capability and confirm that explicit delegation is still required.
- Remove or invalidate evidence supporting a capability or operating claim and confirm the claim degrades to unknown, unverified, or not effective.

- Compare current and target states and verify that every material gap has an owner, dependency, investment, transition action, evidence gate, and reopening trigger.
- Change a policy, contract, regulation, partner, supplier, or critical system and confirm impact propagation.
- Ask the owner to reconstruct the architecture and next safe decision from durable artifacts without chat memory.
- Verify that front-end, dashboard, or conversational claims cannot override authoritative business, system, authority, or evidence state.
- Hostile-review incentives and measures for gaming, displaced harm, hidden work, automation bias, and anti-metric failure.

FAILURE STATES AND ANTI-PATTERNS

- Organization chart presented as Business Architecture.
- Process inventory, application map, or project portfolio presented as a complete operating model.
- Capability names derived from current systems, departments, vendors, or initiatives without stable outcome definitions.
- Strategy slogans with no value stream, capability, decision, investment, measure, or evidence path.
- Current-state model built only from policy, leadership narrative, or documentation while actual work differs.
- Target state represented as current or inevitable.
- Value stream ending at internal output rather than stakeholder outcome.
- Metrics that reward throughput, engagement, or cost reduction while hiding harm, rework, exclusion, risk, or loss of control.
- Decision rights inferred from access, seniority, system role, or AI capability.
- Accountability assigned to a role that lacks authority, resources, information, or control.
- Technology chosen before business need, capability, domain, authority, and evidence are understood.
- AI automation used to conceal unresolved policy, staffing, labor, accessibility, privacy, safety, or governance choices.
- Capability duplication hidden by different labels or capability differences hidden by one broad label.
- Partner or supplier dependency omitted from capability and resilience models.
- Informal work, tacit knowledge, manual exceptions, shadow systems, or support burden omitted.
- Architecture approval treated as implementation, adoption, benefit, or operating proof.
- Transition plan without coexistence, migration, training, support, rollback, continuity, or retirement.
- Dissent, exceptions, and evidence gaps deleted to produce apparent alignment.

RECOVERY AND RECONSIDERATION

When Business Architecture is blocked, contradicted, or invalidated:

1. Identify the exact claim, model element, decision, source, or dependency affected.
2. Classify the issue as authority, source identity, scope, objective, stakeholder, domain, capability, value stream, decision right, information, policy, measure, evidence, feasibility, risk, implementation, adoption, or transition failure.
3. Freeze affected approvals, target-state claims, investment recommendations, or transition actions when continued use would be misleading or harmful.
4. Preserve prior versions, evidence, dissent, decisions, and change history.
5. Identify safe work that can continue without compromising truth, authority, or continuity.
6. Define the smallest admissible evidence request, experiment, owner decision, correction, or remapping action.
7. Reassess affected requirements, architecture, initiatives, contracts, controls, training, measures, tests, and evidence.
8. Reissue the architecture with explicit supersession, conditions, residual risk, and reopening triggers.
9. Escalate value conflicts, missing authority, affected-party harm, or unresolved governing obligations to the authorized human.

A denial or safe halt must identify the blocker, consequence, safe remaining work, minimum resolution, evidence required for reconsideration, and human decision point.

COMPLETION CRITERIA

Business Architecture is complete for its declared scope only when:

- Scope, authority, sources, current state, target state, transition state, assumptions, unknowns, and exclusions are explicit.
- Mission, strategy, stakeholder outcomes, value propositions, value streams, capabilities, decisions, processes, information, products, services, systems, resources, controls, measures, investments, and lifecycle obligations are traceable.
- Current-state claims are supported by current evidence and remain distinct from planned or inferred state.
- Capability and value-stream definitions are stable enough for downstream use and have owners, dependencies, evidence, and lifecycle states.
- Decision rights, accountability, delegation, approvals, exceptions, risk acceptance, and final human authority are explicit.
- Business information, records, evidence, privacy, retention, and translation responsibilities are defined.
- Target and transition decisions include alternatives, tradeoffs, dependencies, costs, risks, affected parties, reversibility, adoption, support, recovery, and retirement.
- Measures distinguish activity, output, outcome, benefit, harm, control effectiveness, resilience, and evidence quality.
- Required review, verification, hostile review, and owner acceptance have occurred or remain truthfully open.
- Dissent, exceptions, unresolved decisions, missing evidence, and reopening triggers remain visible.
- Downstream workers can proceed without inventing material business meaning, authority, ownership, or acceptance criteria.

HANDOFF AND CONTINUITY REQUIREMENTS

The handoff must include:

- Exact artifact identity, version, scope, environment, jurisdiction, evidence date, and authority state.
- Source register, architecture charter, current and target views, transition roadmap, decisions, assumptions, unknowns, dissent, exceptions, risks, and evidence gaps.
- Value proposition, value stream, capability, operating model, organization, decision-right, information, policy, product, service, ecosystem, measure, and investment artifacts.
- Traceability to Sections 001-024 and downstream sections.
- Approved, proposed, blocked, superseded, unverified, and retired states.
- Required human decisions, review gates, evidence thresholds, rollback conditions, and reopening triggers.
- Current safe-to-continue determination and next safe action.

The receiving role must acknowledge the exact version and identify conflicts before relying on the package. Chat history is not an acceptable continuity mechanism.

RELATED CODEX SECTIONS

Upstream: Sections 001-024, especially 004 Mission Strategy, 005 Business Strategy, 006 Product Strategy, 007 Portfolio Management, 011 Business Analysis, 014 Stakeholder Management, 015 User Research, 016 Domain Analysis, 017 Feasibility Analysis, 018 Risk Management, 019 Requirements Engineering, 020 Systems Engineering, 021 Enterprise Architecture, and 024 Domain Architecture.

Downstream: 026 Capability Architecture; 027 Process Architecture; 028 Functional Architecture; 029 Operational Architecture; 030 Information Architecture; organization, governance, product, service, data, application, technology, platform, integration, security, privacy, safety, accessibility, AI, delivery, operations, measurement, change, support, maintenance, modernization, deprecation, and retirement sections.

Reopen when mission, strategy, stakeholder, user, market, domain, requirement, capability, value stream, decision right, policy, organization, information, product, service, partner, supplier, cost, capacity, law, risk, evidence, system, incident, measure, adoption, or lifecycle state materially changes.

SOURCE BASIS AND UNRESOLVED SOURCE GAPS

Source basis:

- Thirsty's Standard v3 supplies truthful status, evidence-before-claims, minimum effective change, verification, hostile review, continuity, recovery, and no-fake-completion discipline.

- Thirsty's Agents 101 supplies bounded role contracts, explicit authority, role selection, assignment, evidence, independent review, disagreement preservation, and human final authority.
- Thirsty's Owner's Cockpit Standard supplies owner operability, governed action boundaries, source-of-truth state, evidence, identity, authorization, bounded delegation, recovery, continuity, accessibility, and owner acceptance expectations.
- The draft V4 design materials contribute prerequisite-aware obligations, bounded multi-agent assembly, proof-carrying work products, dissent preservation, constructive denial, and trusted advancement constraints. They remain draft and unratified.
- Sections 001-024 provide the current Codex chain from research and strategy through requirements, systems engineering, enterprise, solution, systems, and domain architecture.

Unresolved source gaps:

- The supplied corpus does not provide one ratified universal Business Architecture method for every enterprise size, sector, jurisdiction, ownership model, labor context, public institution, nonprofit, partnership, or regulated environment.
- No complete current external framework crosswalk has yet been performed for business architecture, operating-model design, capability mapping, value streams, organization design, benefits management, workforce design, public-interest governance, or sector-specific obligations.
- The corpus does not establish organization-specific capability maturity scales, financial thresholds, investment criteria, workforce agreements, or decision authorities beyond the evidence supplied for a particular application.
- Later architecture, design, engineering, security, privacy, safety, accessibility, legal, operational, measurement, and lifecycle sections may expose missing business allocations and must be allowed to reopen this draft.

This section is CONTENT-COMPLETE. It is not yet source-reconciled, hostile-reviewed, or owner-accepted.

026

SECTION 026 - CAPABILITY ARCHITECTURE

CODEX/03_SECTIONS/026_CAPABILITY_ARCHITECTURE.txt

SHA-256 6ad0c83feb6acb6a25272fc46fcf5f5c87e21708a956a6375ce29657e9916789

36,992 BYTES / PRESERVED SOURCE IDENTITY

SECTION 026 — Capability Architecture

STATUS: CONTENT-COMPLETE

PURPOSE

Define the governed architecture of the enduring abilities required to produce authorized outcomes. Capability Architecture identifies what an enterprise, system, product, service, organization, human-AI team, or ecosystem must be able to do, independent of the temporary projects, departments, applications, vendors, models, or processes used to realize that ability.

This section converts mission, strategy, business architecture, value streams, requirements, risk, and architecture constraints into a stable capability model. It defines capability outcomes, boundaries, ownership, dependencies, realization, competencies, information, technology, resources, controls, evidence, performance, resilience, maturity, lifecycle, gaps, increments, investments, transitions, and retirement.

Capability Architecture does not claim that naming a capability creates it. It does not confuse a feature, function, process, system, team, permission, or model output with an operationally sustained ability. It preserves the difference between required, existing, available, authorized, demonstrated, integrated, scalable, resilient, supported, effective, degraded, and retired capability states.

REQUIRED OUTCOME

- A declared capability-architecture scope, authority, enterprise or system boundary, stakeholders, environments, jurisdictions, and lifecycle horizon.
- A stable capability taxonomy and map that minimizes dependence on current organization, technology, provider, project, or process labels.
- A capability catalog with purpose, outcome, beneficiary, owner, authority, scope, inputs, outputs, dependencies, competencies, information, process, functions, organization, technology, partners, controls, measures, evidence, resilience, support, and lifecycle state.
- Capability levels or decomposition that preserve clear parent-child meaning and avoid arbitrary hierarchy.
- Traceability from mission, strategy, value propositions, value streams, requirements, risks, obligations, products, services, and architecture decisions to capabilities.
- A realization model showing how each capability is delivered through people, roles, skills, processes, functions, information, data, applications, technologies, facilities, suppliers, partners, AI, and governance.
- A dependency and enabling-capability model exposing shared services, bottlenecks, concentration, common-cause failure, sequence, and evidence dependencies.
- A current-state assessment that distinguishes documented existence, access, authorization, use, demonstration, performance, resilience, supportability, and effectiveness.
- A target-state capability architecture with authorized priorities, required levels, performance measures, capacity, resilience, control, evidence, and lifecycle obligations.
- A gap model separating absent capability, insufficient level, inadequate capacity, weak quality, missing authority, poor integration, unproven evidence, fragile dependency, unsupported operation, and retirement risk.
- Capability increments and transition states with dependencies, investment, ownership, acceptance criteria, evidence gates, adoption, training, migration, rollback, continuity, and retirement.
- A capability-to-initiative and capability-to-investment model that prevents project completion from being mistaken for capability delivery.
- A capability measurement and assurance model that distinguishes readiness, availability, authority, performance, quality, resilience, user outcome, benefit, harm, control effectiveness, and evidence confidence.
- A durable handoff to Process Architecture, Functional Architecture, Operational Architecture, Information Architecture, organization and workforce design, solution and technology design, portfolio and investment governance, testing, operations, and lifecycle management.

WHY THIS MATTERS

Enterprises often organize change around projects, products, systems, or departments because those objects are visible and funded. The enduring ability to achieve an outcome can remain unclear. A project can finish while the required capability is unavailable, unauthorised, poorly integrated, unsupported, inaccessible, or fragile. A system may contain a feature but lack the people, process, information, controls, recovery, or authority required to use it safely.

Capability Architecture creates a stable bridge between strategy and realization. It allows the owner to ask what must remain possible even when the organization, technology, provider, process, or staffing model changes. It reveals shared dependencies, investment gaps, duplicated solutions, hidden manual work, concentration risk, and capability claims unsupported by evidence.

For human-AI systems, this discipline is essential because model capability changes quickly and is easily overstated. A model's technical ability to generate, classify, plan, or execute does not establish permission, reliability, competence, safety, accountability, operational support, or institutional capability. Capability Architecture keeps capability separate from authority and turns AI functionality into a governed realization component rather than a sovereign operating claim.

APPLICABILITY

- Strategy, Business Architecture, Enterprise Architecture, product strategy, portfolio, or transformation work requires a stable view of enduring abilities.
- Investments, programs, projects, products, services, systems, platforms, teams, suppliers, or AI initiatives must be compared against common outcome requirements.
- Capability ownership, dependency, maturity, capacity, resilience, evidence, or lifecycle state is unclear.
- Multiple organizations or systems provide overlapping, fragmented, or duplicated capabilities.
- A capability depends on cross-functional work, shared information, common platforms, external providers, specialized expertise, or distributed authority.
- A merger, acquisition, divestiture, outsourcing, insourcing, reorganization, modernization, migration, or retirement changes how capability is realized.
- A high-consequence capability requires explicit security, privacy, safety, accessibility, legal, resilience, recovery, continuity, or owner-control conditions.
- AI or automation is proposed as a replacement or augmentation for human work, decision-making, analysis, operation, support, or control.
- Portfolio and investment decisions require evidence of capability gaps and benefits rather than project advocacy.

NON-APPLICABILITY CONDITIONS

- A bounded change neither creates, modifies, degrades, transfers, duplicates, depends on, or retires a capability, and current evidence supports that conclusion.
- The work is exploratory and makes no claim that a capability exists, is authorized, is operational, or can be sustained. Hypotheses must remain labeled.
- An active incident requires containment before capability assessment or redesign can proceed safely.
- Required operational, workforce, security, financial, partner, supplier, or system evidence is inaccessible or unauthorized, making the assessment speculative.
- Capability ownership or decision authority cannot be established. Work may continue only as a research package or proposed model.

A non-applicability decision must identify the controlling capability model, exact version, scope, evidence date, decision owner, assumptions, and reopening conditions.

PREREQUISITES AND DEPENDENCIES

- Sections 001-018 provide research, strategy, portfolio, program, project, market, stakeholder, user, domain, feasibility, and risk context.
- Section 019 provides requirements, acceptance criteria, and traceability.
- Section 020 provides system, enabling-system, lifecycle, verification, and validation context.
- Sections 021-025 provide enterprise, solution, systems, domain, and business architecture context.

- Business Architecture must provide sufficiently stable value propositions, outcomes, value streams, operating model, decision rights, information, products, services, and target-state context.
- Capability terms must align with Domain Architecture and Business Architecture or include explicit mappings.
- Current-state evidence must distinguish capability existence from performance, authority, availability, integration, resilience, and effectiveness.
- Feasibility and risk conditions must be represented in capability requirements, levels, increments, and acceptance criteria.
- Required owners, resource authorities, workforce authorities, risk owners, and final human decision points must be known or explicitly unresolved.

REQUIRED INPUTS

- Authorized capability question, scope, enterprise or system boundary, exclusions, decision owner, horizon, and lifecycle state.
- Mission, business strategy, product strategy, portfolio priorities, value propositions, outcomes, value streams, and operating-model decisions.
- Stakeholder, affected-party, customer, user, employee, operator, partner, supplier, and ecosystem evidence.
- Requirements, architecture drivers, domain models, business rules, policies, obligations, risks, controls, standards, exceptions, and acceptance criteria.
- Current products, services, systems, platforms, processes, functions, organizations, roles, competencies, information, data, facilities, vendors, partners, and contracts.
- Current capacity, demand, performance, quality, reliability, resilience, accessibility, support, incident, recovery, and benefit evidence.
- Workforce, skills, training, availability, succession, location, labor, accessibility, and human-factor constraints.
- Technology, model, data, integration, infrastructure, security, privacy, safety, governance, and operational constraints.
- Current initiatives, investments, budgets, dependencies, roadmaps, technical debt, operational debt, and retirement commitments.
- Known capability claims, gaps, assumptions, unknowns, dissent, evidence limitations, and prior assessments.

HUMAN RESPONSIBILITIES AND AUTHORITY

The authorized human owner remains responsible for:

- Approving capability scope, definitions, outcomes, owners, priority, target levels, investment, and lifecycle decisions.
- Determining which outcomes, obligations, rights, harms, and tradeoffs matter.
- Assigning legitimate capability ownership and ensuring owners have appropriate authority, resources, information, and accountability.
- Approving capability transfer, outsourcing, insourcing, automation, workforce change, supplier dependence, risk acceptance, and retirement.
- Resolving value, labor, policy, authority, privacy, safety, security, accessibility, legal, and public-interest decisions not validly delegated.
- Requiring affected-party, workforce, customer, user, regulator, partner, or supplier participation where appropriate.
- Preventing maturity scores, rankings, benchmarks, or AI recommendations from becoming binding without authorized criteria and evidence.
- Accepting, rejecting, conditioning, deferring, superseding, or reopening capability architecture decisions.
- Ratifying capability claims only for the exact scope, environment, demand, authority, evidence, and lifecycle state demonstrated.

AI capability, system permission, organizational access, or prior conversation does not transfer these responsibilities.

AI RESPONSIBILITIES AND BOUNDARIES

AI may:

- Extract candidate capabilities from authorized strategy, value streams, requirements, operations, products, services, systems, and evidence.
- Normalize labels, detect duplicates, map dependencies, compare realizations, trace gaps, and maintain catalogs.
- Analyze capability options, scenarios, capacity, bottlenecks, concentration, resilience, transition, and evidence coverage.
- Generate candidate definitions, decompositions, levels, measures, increments, acceptance criteria, and evidence requests.
- Identify unsupported capability claims, inconsistent maturity ratings, missing ownership, weak dependencies, stale evidence, and hidden manual work.
- Maintain versioning, provenance, assumptions, unknowns, dissent, and reopening triggers.

AI must not:

- Declare a capability operational, effective, safe, compliant, resilient, scalable, or ready without current evidence.
- Treat model benchmark performance, vendor claims, demonstrations, prototypes, or feature lists as business capability proof.
- Assign capability ownership, authority, budget, workforce accountability, or risk acceptance without explicit human decision.
- invent capability definitions, levels, weights, scores, demand, capacity, benefit, cost, or maturity evidence.
- Rank people or teams through undisclosed, discriminatory, or unauthorized criteria.
- Conceal human work, accessibility needs, labor implications, external dependencies, support burden, or failure recovery.
- Replace mandatory human judgment with model confidence.
- Approve its own capability model, realization, test, or assurance claim where independence is required.

When evidence or authority is insufficient, AI must mark the capability state unknown, unverified, unavailable, unauthorized, blocked, degraded, or proposed as appropriate.

APPLICABLE EXISTING AGENT ROLES

- Architect: defines capability structure, boundaries, dependencies, realization, interfaces, lifecycle, failure, recovery, and verification.
- Researcher: gathers current capability, market, workforce, operational, supplier, regulatory, and benchmark evidence.
- Analyst: compares capability options, levels, gaps, dependencies, costs, benefits, risks, capacity, and sensitivity.
- Critic: challenges vague definitions, duplicate labels, unsupported maturity, hidden realization, weak measures, and false readiness.
- Security Auditor: evaluates identity, authority, data, supply-chain, partner, resilience, and security implications.
- Tester: translates capability outcomes and conditions into scenarios, tests, acceptance criteria, and coverage models.
- Verifier: independently checks capability claims, evidence, configuration identity, environment, demand, and scope.
- Documentation Writer: maintains capability maps, catalogs, definitions, realization views, decisions, and guidance.
- Planner: sequences capability discovery, assessment, increments, investments, transitions, adoption, testing, and retirement.
- Coordinator: manages bounded workstreams, dependencies, conflict, evidence, handoffs, and synthesis.
- Observer: captures actual capability use, demand, performance, incidents, dependencies, workarounds, and state.
- Memory Curator: preserves definitions, model versions, evidence, decisions, supersession, and continuity.
- Governance Reviewer: checks authority, policy, obligations, exceptions, delegation, risk acceptance, and safe-halt conditions.
- Red Team: probes capability claims, concentration, bypass, degradation, demand spikes, supplier failure, and recovery.
- Optimizer: improves a measured capability realization without silently degrading protected outcomes, controls, or resilience.
- User Experience Reviewer: verifies that capability realization supports understandable, accessible, controllable, and recoverable journeys.
- Decision Arbiter Advisory: compares capability options and priorities against authorized criteria without assuming final authority.

The existing role definitions remain controlling and are not redefined by this section.

REQUIRED ACTIVITIES

1. Confirm scope, boundary, authority, decision owner, stakeholders, environments, jurisdictions, horizon, and lifecycle state.
2. Establish source hierarchy and distinguish observed capability, documented expectation, proposed target, vendor claim, inference, assumption, and unknown.
3. Derive candidate capabilities from mission outcomes, strategy, value propositions, value streams, requirements, obligations, risks, products, services, and operational evidence.
4. Normalize capability names around enduring outcome-producing ability rather than current organization, technology, project, feature, process, or supplier.
5. Define each capability's purpose, outcome, beneficiary, scope, exclusions, owner, authority, inputs, outputs, and acceptance boundary.
6. Decompose capabilities only where child capabilities have distinct outcomes, ownership, dependencies, realization, measures, or lifecycle decisions.
7. Map capability relationships, enabling capabilities, dependencies, sequence, shared resources, evidence dependencies, and common-cause failure.
8. Define capability realization through people, roles, skills, process, functions, information, data, applications, technology, facilities, partners, suppliers, AI, controls, and governance.
9. Assess current state across existence, access, authority, availability, capacity, performance, quality, integration, resilience, supportability, accessibility, control, evidence, and effectiveness.
10. Define target capability levels and conditions traceable to outcomes, demand, risk, requirements, architecture, and authorized strategy.
11. Classify gaps as absent, insufficient level, insufficient capacity, poor quality, missing authority, weak integration, fragile dependency, unsupported, inaccessible, unverified, ineffective, or retirement-related.
12. Identify capability redundancies, concentrations, single points of failure, shadow realizations, manual work, technical debt, operational debt, and supplier lock-in.
13. Define measures and evidence for capability readiness, availability, authority, performance, quality, resilience, accessibility, user outcome, benefit, harm, and control effectiveness.
14. Evaluate realization options and tradeoffs across build, buy, partner, share, outsource, insource, automate, augment, standardize, federate, consolidate, or retire.
15. Define capability increments with exact outcome, scope, dependencies, resources, authority, evidence, acceptance criteria, transition, rollback, support, and review triggers.
16. Map capabilities to initiatives, investments, products, services, systems, processes, functions, organizations, information, technology, risks, controls, and benefits.
17. Validate definitions and assessments with capability owners, operators, users, affected parties, domain experts, finance, workforce, technology, security, governance, and independent reviewers as applicable.
18. Hostile-review capability claims under demand surge, degraded dependency, provider outage, workforce loss, revoked authority, data failure, model failure, incident, and recovery.
19. Preserve dissent, exceptions, assumptions, unknowns, residual risks, missing evidence, and reopening triggers.
20. Produce a durable handoff for Process, Functional, Operational, Information, organization, solution, technology, investment, testing, operations, and lifecycle work.

HUMAN-AI INTERACTION REQUIREMENTS

- AI-generated capability names, maps, scores, and target levels remain proposals until human approval.
- The AI must show the source and reasoning for each capability definition and distinguish extraction, normalization, inference, recommendation, and unknown.
- The owner must be able to inspect realization, dependencies, evidence, authority, affected parties, costs, risks, and alternatives before deciding.
- Capability and maturity scoring must expose dimensions, criteria, weights, thresholds, evidence quality, uncertainty, and who authorized the method.
- The AI must not use fluency, benchmark scores, or provider claims to imply operational competence.
- AI-generated gap or investment recommendations must identify mandatory constraints, assumptions, sensitivity, reversibility, residual risk, and evidence that would change the result.

- Human approval must bind to the exact capability model version, scope, demand, environment, realization, and evidence state.
- Material changes to demand, authority, realization, provider, data, system, risk, or evidence invalidate affected approvals and assessments.
- The owner must be able to revoke delegation, stop analysis, narrow scope, supersede a model, and export all durable records.
- Critical capability state and control must remain available during model or provider outage.

ARCHITECTURE AND SYSTEM IMPLICATIONS

Capability Architecture constrains downstream design while remaining independent of any one realization.

- Capabilities must map to Business Architecture value streams and outcomes, Domain Architecture meaning, Requirements Engineering obligations, and Enterprise Architecture target states.
- Each capability realization must identify components, interfaces, information, data, authority, controls, evidence, failure, recovery, deployment, support, and retirement.
- Process and Functional Architecture must explain how capability behavior is performed without redefining capability outcomes.
- Operational Architecture must show how capabilities combine under real scenarios, environments, timing, resources, degraded states, and command relationships.
- Information, Data, Application, Technology, Platform, Infrastructure, Integration, Security, Privacy, Safety, Accessibility, AI, and Human Factors Architecture must trace to capability needs where applicable.
- Shared capabilities require explicit service boundaries, ownership, demand management, prioritization, availability, evidence, and recovery.
- AI realization must include model identity, version, data, context, tools, permissions, validation, monitoring, fallback, human oversight, revocation, and evidence.
- Capability changes must trigger impact analysis across requirements, business architecture, value streams, processes, functions, systems, workforce, suppliers, controls, tests, evidence, training, support, and retirement.
- Architecture must preserve the ability to change realization without losing capability meaning, ownership, continuity, or evidence.

OWNER COCKPIT AND USER-CONTROL REQUIREMENTS

The owner must be able to:

- Inspect the active capability map, catalog, version, scope, evidence date, owner, authority, and lifecycle state.
- Navigate from mission, strategy, outcomes, and value streams to capabilities, realization, dependencies, initiatives, investments, systems, measures, risks, controls, and evidence.
- Distinguish required, proposed, existing, available, authorized, demonstrated, integrated, operational, effective, degraded, unsupported, superseded, and retired states.
- Inspect the exact realization and evidence behind a capability claim.
- See capacity, demand, bottlenecks, shared dependencies, concentration, resilience, recovery, and support state.
- Compare capability options, target levels, increments, costs, risks, benefits, affected parties, and reversibility.
- Inspect who may define, fund, operate, approve, verify, accept risk for, or retire each capability.
- Stop, defer, revoke, quarantine, supersede, or reopen capability work within defined authority.
- Export capability artifacts, decisions, evidence, traceability, and continuity records without chat dependency.
- Retain essential capability status, stop, fallback, and recovery control during AI or provider failure.

REQUIRED OUTPUTS AND ARTIFACTS

- Capability architecture charter, scope, authority, and source register.
- Capability taxonomy, map, catalog, definitions, levels, and ownership.
- Capability-to-mission, strategy, value proposition, value stream, requirement, obligation, risk, and architecture traceability.
- Capability realization views covering people, roles, competencies, process, functions, information, data, applications, technology, facilities, partners, suppliers, AI, controls, and governance.
- Capability dependency, enabling-capability, shared-service, concentration, and failure-domain model.
- Current-state capability assessment with evidence and confidence boundaries.

- Target-state capability architecture and required levels.
- Capability gap, redundancy, fragility, capacity, support, and retirement analysis.
- Capability measure, demand, capacity, performance, quality, resilience, accessibility, outcome, benefit, harm, control, and evidence model.
- Capability increment definitions and transition roadmap.
- Capability-to-initiative, investment, product, service, system, organization, process, function, information, technology, supplier, and benefit matrix.
- Capability decision, dissent, assumption, unknown, exception, risk, and open-question registers.
- Verification, assurance, hostile-review, owner-acceptance, and reopening plan.
- Durable downstream handoff package.

EVIDENCE REQUIREMENTS

Evidence may include strategy and architecture records, owner decisions, value-stream observations, process and function records, organization and role data, workforce and competency evidence, product and service performance, system telemetry, capacity and demand data, incidents, support records, continuity exercises, control tests, audits, financial evidence, supplier and contract records, user research, accessibility testing, security and privacy assessments, safety evidence, operational demonstrations, recovery tests, and benefit measurement.

Every capability claim must bind to exact scope, outcome, owner, realization, environment, demand, authority, time, method, evidence, limitations, and lifecycle state. The following are insufficient alone: a capability label, a documented process, a trained employee, a software feature, an installed system, a model benchmark, a vendor promise, a successful demo, a completed project, or a passing unit test.

Evidence should distinguish availability from authorization, demonstration from sustained operation, nominal performance from resilience, technical success from user outcome, and current state from target design. When realization, demand, authority, system, supplier, workforce, model, data, environment, policy, risk, or evidence changes, affected capability claims must be re-evaluated.

VERIFICATION AND ACCEPTANCE TESTS

- Select a capability and trace it to mission, outcome, value stream, requirements, owner, realization, dependencies, measures, risks, controls, initiatives, and evidence.
- Replace a realization component with an alternative and verify that capability meaning remains stable while impacts are exposed.
- Remove one critical dependency and confirm that capability state, degraded behavior, fallback, recovery, and owner-visible consequence are correct.
- Increase demand, reduce capacity, revoke authority, expire evidence, disable an AI provider, and remove a critical operator; verify truthful state transitions.
- Attempt to claim capability from a feature, project completion, demo, benchmark, or vendor statement alone and confirm rejection or downgrade.
- Compare two capability assessments and verify that definitions, dimensions, scales, criteria, evidence dates, and environments are compatible.
- Change a capability definition or target level and confirm impact propagation to value streams, requirements, investments, processes, functions, systems, workforce, tests, and evidence.
- Validate capability increments against explicit outcomes, dependencies, authority, resources, acceptance criteria, evidence gates, rollback, and support.
- Run an end-to-end operational scenario that requires multiple capabilities and verify coordination, timing, shared resources, failure, recovery, and evidence.
- Verify that an AI system with technical ability but no authority cannot execute or approve a consequential capability action.
- Reconstruct capability state, decisions, evidence, and next safe action from durable artifacts without conversation history.
- Independently verify at least one high-consequence capability claim using evidence not controlled solely by the realization owner.

FAILURE STATES AND ANTI-PATTERNS

- Capability map copied from an organization chart, application portfolio, process hierarchy, vendor taxonomy, or project list.
- Capability names that describe current tools, departments, or initiatives rather than enduring outcomes.
- Capability declared present because a feature, employee, system, model, or provider exists.
- Capability maturity score with hidden dimensions, arbitrary weights, weak evidence, or no authority.
- One broad capability hiding materially different outcomes, owners, risks, or dependencies.
- Excessive decomposition producing a task inventory instead of an architecture.
- Duplicate capabilities hidden by synonyms or distinct capabilities collapsed under one label.
- Capability owner without authority, resources, evidence, or operational control.
- Missing human work, manual exceptions, accessibility support, training, tacit knowledge, or partner contribution.
- Capacity, demand, resilience, support, recovery, and continuity omitted.
- AI benchmark or demo presented as operational capability.
- Capability access confused with authorization or permission.
- Project completion, system deployment, or training attendance treated as capability outcome proof.
- Capability investments prioritized without value-stream, dependency, benefit, risk, or evidence context.
- Shared capability operated as an ungoverned bottleneck or hidden single point of failure.
- Outsourced capability treated as transferred accountability.
- Target state represented as current or funded state.
- Retirement of systems, skills, suppliers, or data without capability continuity.
- Dissent, weak evidence, or unsupported scores removed to create a clean portfolio view.

RECOVERY AND RECONSIDERATION

When Capability Architecture is blocked or invalidated:

1. Identify the exact capability, definition, owner, realization, dependency, measure, evidence, or decision affected.
2. Classify the issue as scope, authority, terminology, outcome, realization, dependency, demand, capacity, quality, integration, resilience, support, accessibility, control, evidence, effectiveness, transition, or retirement failure.
3. Downgrade affected capability claims to the truthful state and prevent unsupported investment, readiness, or operating claims.
4. Preserve prior model versions, assessments, evidence, dissent, decisions, and supersession history.
5. Identify safe work and unaffected capabilities that may continue.
6. Define the smallest evidence request, operational test, owner decision, realization correction, or model update needed.
7. Reassess affected value streams, requirements, architecture, initiatives, systems, workforce, suppliers, controls, tests, training, support, and benefits.
8. Reissue capability state with explicit conditions, residual risk, review date, and reopening triggers.
9. Escalate missing authority, value conflict, workforce impact, affected-party harm, or governing conflict to the authorized human.

A blocked state must include reason, consequence, evidence gap, safe work, minimum resolution, reconsideration condition, and human decision point.

COMPLETION CRITERIA

Capability Architecture is complete for its declared scope only when:

- Scope, authority, source hierarchy, current state, target state, transition state, assumptions, unknowns, and exclusions are explicit.
- Capability definitions are stable, outcome-based, non-duplicative, and sufficiently independent of current realization.
- Each material capability has an owner, outcome, beneficiary, scope, realization, dependencies, measures, evidence, risk, resilience, support, and lifecycle state.
- Capabilities trace to mission, strategy, value propositions, value streams, requirements, obligations, architecture, initiatives, investments, systems, and benefits.
- Current-state assessments distinguish existence, access, authority, availability, capacity, performance, quality, integration, resilience, supportability, accessibility, evidence, and effectiveness.
- Target levels and gaps use authorized criteria, visible evidence, and explicit uncertainty.

- Capability increments include dependencies, resources, authority, acceptance, evidence, adoption, rollback, support, and retirement.
- Shared, enabling, external, AI, and human capabilities have explicit boundaries and accountability.
- Required review, testing, verification, hostile review, and owner acceptance have occurred or remain truthfully open.
- Dissent, exceptions, residual risks, missing evidence, and reopening triggers remain visible.
- Downstream workers can proceed without inventing material capability meaning, ownership, state, authority, or acceptance criteria.

HANDOFF AND CONTINUITY REQUIREMENTS

The handoff must include:

- Exact capability model identity, version, scope, environment, demand basis, evidence date, and authority state.
- Capability map, catalog, definitions, ownership, realization, dependencies, assessments, gaps, measures, increments, decisions, risks, and evidence.
- Traceability to Business Architecture, value streams, requirements, domain models, enterprise and systems architecture, initiatives, investments, products, services, systems, organizations, processes, functions, information, technology, controls, and benefits.
- Proposed, approved, existing, available, authorized, demonstrated, operational, effective, degraded, unsupported, superseded, and retired states.
- Assumptions, unknowns, dissent, exceptions, missing evidence, human decisions, review gates, rollback conditions, and reopening triggers.
- Current safe-to-continue determination and next safe action.

The receiving role must acknowledge the exact version and expose conflicts before use. Conversation history is not a substitute for durable capability continuity.

RELATED CODEX SECTIONS

Upstream: Sections 001-025, especially 004 Mission Strategy, 005 Business Strategy, 006 Product Strategy, 007 Portfolio Management, 008 Program Management, 011 Business Analysis, 016 Domain Analysis, 017 Feasibility Analysis, 018 Risk Management, 019 Requirements Engineering, 020 Systems Engineering, 021 Enterprise Architecture, 023 Systems Architecture, 024 Domain Architecture, and 025 Business Architecture.

Downstream: 027 Process Architecture; 028 Functional Architecture; 029 Operational Architecture; 030 Information Architecture; organization and workforce design; product and service design; application, technology, platform, infrastructure, integration, security, privacy, safety, accessibility, AI, data, delivery, operations, measurement, capacity, resilience, support, maintenance, modernization, deprecation, and retirement sections.

Reopen when mission, strategy, outcome, value stream, requirement, capability definition, owner, authority, realization, demand, capacity, performance, quality, dependency, workforce, supplier, system, model, data, environment, law, policy, risk, control, incident, evidence, investment, benefit, or lifecycle state materially changes.

SOURCE BASIS AND UNRESOLVED SOURCE GAPS

Source basis:

- Thirsty's Standard v3 supplies truthful completion, evidence-before-claims, current-state inspection, verification, hostile review, continuity, recovery, and bounded reporting.
- Thirsty's Agents 101 supplies the existing role corpus, authority envelopes, assignment contracts, evidence requirements, independent review, disagreement preservation, and human final authority.
- Thirsty's Owner's Cockpit Standard supplies capability-versus-authority separation, owner control, governed execution, source-of-truth state, evidence, fallback, recovery, continuity, and owner acceptance.
- The draft V4 design materials contribute prerequisite-aware capability evidence, bounded role assembly, proof-carrying outputs, dissent ledgers, constructive denial, and trusted-state constraints. They remain draft and unratified.
- Sections 001-025 supply the current Codex chain from research and strategy through requirements, systems engineering, enterprise, solution, systems, domain, and business architecture.

Unresolved source gaps:

- The supplied corpus does not define one ratified universal capability taxonomy, decomposition rule, maturity scale, capacity method, or assurance level for every organization, system, sector, jurisdiction, or lifecycle.
- No complete crosswalk has yet been performed against current external capability-based planning, business architecture, enterprise architecture, systems engineering, workforce, resilience, benefits, portfolio, defense, public-sector, healthcare, financial, safety-critical, or other sector methods.
- Organization-specific capability priorities, ownership, resource authorities, demand forecasts, maturity criteria, financial thresholds, risk tolerances, workforce agreements, and supplier commitments must come from authoritative application evidence.
- Later Process, Functional, Operational, Information, Data, Application, Technology, AI, Security, Privacy, Safety, Accessibility, Testing, Operations, Support, Measurement, and Retirement sections may reveal missing capability conditions and must be allowed to reopen this draft.

This section is CONTENT-COMPLETE. It is not yet source-reconciled, hostile-reviewed, or owner-accepted.

027

SECTION 027 - PROCESS ARCHITECTURE

CODEX/03_SECTIONS/027_PROCESS_ARCHITECTURE.txt

SHA-256 b1da01a1954230b7d0ce1171dc7aa570ab6e5d2d1e7cb0be8dc67382e159ad35

41,492 BYTES / PRESERVED SOURCE IDENTITY

SECTION 027 — Process Architecture

STATUS: CONTENT-COMPLETE

PURPOSE

Define the governed structure of how work, decisions, controls, information, evidence, exceptions, and recovery move through a bounded organization, system, service, product, or socio-technical capability.

Process Architecture translates approved outcomes, capabilities, requirements, domain rules, operating concepts, and functional obligations into an intentional network of processes. It identifies triggers, actors, activities, decisions, states, handoffs, controls, inputs, outputs, evidence, measures, failure behavior, and lifecycle conditions without confusing a process with a capability, function, organization, application, procedure, or implementation.

Process Architecture is concerned with how work is coordinated in context. It does not authorize objectives, create business policy, decide values, replace domain authority, or claim that a modeled process is implemented or effective. A process model is a design and governance artifact until current behavior and evidence establish more.

REQUIRED OUTCOME

- A declared process-architecture scope identifying the organization, system, value stream, product, service, capability, population, environment, jurisdiction, lifecycle horizon, and authority boundary covered.
- A process taxonomy and catalog with stable identities, purposes, outcomes, triggers, owners, participants, boundaries, variants, states, dependencies, controls, evidence, measures, and lifecycle status.
- A value-stream-to-process view connecting desired outcomes and stakeholder journeys to capabilities, processes, functions, information, decisions, products, services, and evidence.
- Current-state process evidence that distinguishes observed behavior, documented procedure, policy expectation, informal workaround, proposed design, and unknown condition.
- A target-state process architecture that preserves approved requirements, domain meaning, risk treatments, mandatory constraints, accessibility, security, privacy, safety, reliability, resilience, and owner-control obligations.
- Process models showing start conditions, end conditions, activities, decisions, parallel work, synchronization, handoffs, waits, queues, exceptions, escalations, controls, evidence capture, and recovery paths as applicable.
- Explicit process ownership and decision rights for each material step, including who may initiate, approve, execute, observe, verify, override, suspend, resume, or retire a process.
- Process-to-function, process-to-capability, process-to-requirement, process-to-control, process-to-data, process-to-system, process-to-role, process-to-test, and process-to-evidence traceability.
- A process variant model for environments, jurisdictions, user groups, operating modes, products, data classes, risk classes, and lifecycle stages without uncontrolled copying or drift.
- A process performance and assurance model distinguishing activity completion, throughput, latency, quality, outcome, control effectiveness, user impact, resilience, recovery, and evidence freshness.
- A process change, exception, escalation, rollback, migration, deprecation, and retirement model that can be handed to Functional Architecture, Operational Architecture, Information Architecture, delivery, testing, operations, maintenance, and lifecycle governance.

WHY THIS MATTERS

Capabilities and systems do not produce outcomes by existing alone. People and machines act through sequences of work, decision points, information exchanges, controls, waits, handoffs, and recovery paths. A process that appears simple in a diagram may conceal unowned decisions, inaccessible steps, undocumented manual work, circular approval, weak evidence, or a recovery path that depends on the failed activity.

Process Architecture exposes the operating reality between strategy and implementation. It makes it possible to ask who performs each activity, what must be true before it begins, what evidence is created, who can decide, what happens when the normal path fails, and how a process changes without losing authority or continuity.

For human-AI systems, process boundaries are where model suggestions become human decisions, tool calls become external consequences, and generated content becomes a record or commitment. A model may participate in a process without owning the process, its values, its authority, its risk, or its acceptance. Process Architecture keeps those distinctions visible.

APPLICABILITY

- A capability, value stream, product, service, system, organization, or human-AI workflow requires multiple activities, actors, decisions, handoffs, states, controls, or lifecycle stages.
- Work crosses organizational, system, domain, supplier, partner, jurisdictional, physical, or data boundaries.
- Current work includes undocumented manual steps, queues, workarounds, approvals, escalations, exceptions, or shadow systems.
- A change may alter who performs work, which decisions are made, what evidence is created, how users or operators recover, or where authority is exercised.
- The process carries material security, privacy, safety, accessibility, legal, financial, ethical, reliability, resilience, support, maintenance, or retirement consequences.
- Automation, AI, agents, orchestration, or workflow tools are proposed to recommend, decide, route, transform, approve, execute, monitor, or recover work.
- Requirements or architecture must be allocated to end-to-end journeys and accountable process steps rather than to abstract components alone.
- Performance, quality, control effectiveness, user outcome, operational readiness, or benefit cannot be understood from component metrics alone.

NON-APPLICABILITY CONDITIONS

- The work is a bounded observation, text correction, or reversible change with no material effect on process sequence, decision rights, participants, controls, evidence, user journey, operating mode, or lifecycle state.
- An approved and current process architecture already covers the exact scope, version, environment, population, variant, and change, and impact analysis shows no material process effect.
- The work is exploratory and makes no claim that a process is approved, implemented, adopted, effective, compliant, or operational. The exploratory model must still record scope and limitations.
- An active safety, security, privacy, integrity, legal, labor, or authority incident requires containment before normal process redesign can safely proceed.
- Required current-state evidence is unavailable or unauthorized, making the process model speculative. Safe work may continue as a proposed model or evidence request, but not as an observed current-state claim.

A non-applicability decision must identify the controlling process record, exact scope, version, evidence date, decision owner, assumptions, and reopening trigger. Small size, familiarity, a flowchart, or an AI-generated workflow is not by itself sufficient reason to omit material process analysis.

PREREQUISITES AND DEPENDENCIES

- Sections 001-018 provide objective, strategy, stakeholder, user, domain, feasibility, and risk context.
- Section 019 provides the requirements baseline, acceptance boundaries, mandatory constraints, variants, and traceability obligations.
- Section 020 provides system, operational-concept, lifecycle, allocation, interface, verification, validation, transition, and recovery context.
- Sections 021-026 provide enterprise, solution, systems, domain, business, and capability architecture boundaries.
- Process terminology must align with Domain Analysis, Domain Architecture, Business Architecture, and Capability Architecture or include explicit translation records.
- Functional responsibilities, system elements, interfaces, information, data, authority, and control ownership must be known or visibly unresolved before a process is treated as a target design.
- Current-state process evidence must distinguish direct observation from procedure, policy, interview, system telemetry, inference, and proposal.
- Human owners and decision authorities for objectives, policy, risk, rights, staffing, commitments, exceptions, and acceptance must be attributable.

REQUIRED INPUTS

- Authorized process question, objective, scope, exclusions, decision owner, population, environment, jurisdiction, and lifecycle horizon.
- Mission, strategy, value propositions, value streams, capabilities, products, services, operating-model decisions, and affected-party outcomes.
- Requirements, acceptance criteria, risk treatments, feasibility conditions, domain rules, invariants, policies, contracts, standards, and architecture constraints.
- Current process maps, procedures, work instructions, tickets, records, logs, queues, event histories, service measures, incident records, support records, audit findings, and user or operator research.
- Actors, roles, organizations, systems, services, suppliers, partners, facilities, devices, data sources, authorities, and trust boundaries.
- Current and target functions, information, data, applications, interfaces, controls, evidence stores, owner cockpit paths, and operational dependencies.
- Accessibility, privacy, security, safety, reliability, resilience, continuity, workforce, training, support, maintenance, migration, deprecation, and retirement obligations.
- Known assumptions, unknowns, dissent, exceptions, workarounds, bottlenecks, failure modes, recovery conditions, and evidence limitations.

HUMAN RESPONSIBILITIES AND AUTHORITY

The authorized human owner retains authority over process purpose, protected outcomes, values, affected-party obligations, policy interpretation, decision rights, staffing or resource commitments, acceptable risk, exceptions, external commitments, automation boundaries, and final acceptance.

- Confirm the process objective, scope, boundaries, non-goals, affected parties, environments, variants, and lifecycle stages.
- Identify who legitimately owns the process outcome, each material decision, each control, each record, and each acceptance condition.
- Decide conflicts involving rights, safety, privacy, accessibility, dignity, labor, public impact, cost, schedule, risk, and operational burden.
- Approve target-state process changes, automation, delegation, outsourcing, role changes, policy changes, service commitments, and irreversible transitions.
- Ensure users, operators, maintainers, support staff, affected non-users, domain experts, security, privacy, safety, accessibility, legal, and governance specialists participate when consequence requires.
- Accept, reject, defer, constrain, waive, supersede, or reopen process architecture for the exact version, scope, evidence, environment, and unresolved conditions reviewed.
- Retain the ability to pause, narrow, quarantine, roll back, restore, or retire process work when authority, evidence, safety, or mission conditions fail.

AI RESPONSIBILITIES AND BOUNDARIES

AI may inspect authorized process records, compare current and target flows, identify missing steps and handoffs, propose models, detect duplicate or conflicting activities, generate traceability, analyze queues and dependencies, draft evidence requests, simulate scenarios, and document decisions within an explicit assignment.

AI must:

- restate the accepted objective, process boundary, source state, assumptions, unknowns, and consequence limits before material modeling;
- label observed behavior, supplied policy, interpreted meaning, inferred sequence, proposed design, simulation, recommendation, execution, and verified result separately;
- preserve the identity, version, environment, date, method, and limitations of process evidence;
- distinguish a participant's statement from a universal process fact and a model's suggestion from an authorized decision;
- expose missing actors, hidden work, unowned decisions, circular handoffs, inaccessible paths, weak controls, stale evidence, and recovery gaps;

- model AI participation as a bounded function or activity with identity, authority, tools, data, approval points, monitoring, fallback, revocation, and evidence;
- stop or escalate when process meaning, authority, protected interests, mandatory constraints, or current-state evidence are unresolved;
- preserve dissent and competing process interpretations rather than averaging them into a false consensus.

AI must not:

- invent process steps, policies, actors, approvals, service levels, staffing, system integrations, evidence, or current behavior;
- treat a generated diagram, simulation, workflow configuration, or successful test stub as proof of an implemented or effective process;
- assign ownership, accept risk, approve exceptions, create commitments, or change policy without delegated authority;
- optimize throughput, cost, or model utilization while silently degrading rights, safety, accessibility, quality, privacy, security, resilience, recovery, or human control;
- hide manual work, dissent, queueing, rework, denial, escalation, or failure to make the process appear simpler;
- route around identity, authority, audit, evidence, or recovery controls through conversational convenience.

APPLICABLE EXISTING AGENT ROLES

- Architect: defines process boundaries, responsibilities, dependencies, interfaces, controls, failure, recovery, and implementation implications.
- Researcher: gathers authoritative process, domain, user, operational, policy, legal, supplier, and current-state evidence.
- Analyst: examines flows, bottlenecks, variants, dependencies, tradeoffs, causes, consequences, and outcome relationships.
- Observer: records actual activities, states, events, waits, handoffs, workarounds, and exceptions without unsupported interpretation.
- Planner: sequences process discovery, design, validation, transition, adoption, and review work with gates and evidence.
- Coordinator: synchronizes bounded workstreams, participants, decisions, conflicts, and handoffs without redefining the objective.
- Critic: challenges missing actors, hidden work, circular control, false simplicity, unsupported effectiveness, and recovery gaps.
- Governance Reviewer: evaluates authority, policy, delegation, consent, exceptions, procedural compliance, and evidence admissibility.
- Security Auditor: examines trust boundaries, privileged steps, data handling, abuse paths, evidence integrity, and secure recovery.
- User Experience Reviewer: evaluates comprehension, accessibility, burden, denial, correction, recovery, dignity, and complete user or owner journeys.
- Tester: converts process obligations and journeys into reproducible normal, boundary, failure, recovery, and compatibility tests.
- Verifier: independently checks process claims against exact evidence, versions, environments, methods, and acceptance criteria.
- Documentation Writer: maintains clear process models, definitions, procedures, decisions, limitations, and recovery instructions.
- Memory Curator: preserves process identity, versions, decisions, evidence, dissent, supersession, and continuity.
- Implementer: realizes an approved process in authorized procedures, configuration, automation, code, or operational change without changing the process contract unilaterally.
- Optimizer: improves measured process performance while preserving protected outcomes, controls, authority, accessibility, and resilience.
- Red Team: attempts authorized adversarial pressure against process controls, handoffs, identity, evidence, denial, recovery, and human oversight.
- Decision Arbiter Advisory: compares process alternatives under authorized criteria while preserving final human authority.

Role selection must remain proportional to consequence. Existing role contracts remain controlling and are not redefined here.

REQUIRED ACTIVITIES

1. Establish the process context: objective, outcome, owner, boundary, population, environment, jurisdiction, lifecycle, variants, non-goals, and acceptance authority.
2. Identify the value stream, capability, product, service, system, or user journey the process supports.
3. Establish terminology and distinguish process, capability, function, activity, task, decision, event, state, procedure, policy, control, record, evidence, and outcome.
4. Inventory current-state sources and classify them as observation, policy, procedure, interview, telemetry, record, contract, inference, assumption, or proposal.
5. Identify participants, actors, roles, organizations, systems, suppliers, partners, affected non-users, authorities, and accountable owners.
6. Define process boundaries, triggers, entry conditions, exit conditions, terminal outcomes, handoff boundaries, and exclusions.
7. Build a process taxonomy and catalog with stable IDs, parent-child relationships, ownership, versions, applicability, and lifecycle state.
8. Map end-to-end value-stream and journey threads, including normal, denied, delayed, interrupted, degraded, emergency, maintenance, migration, and retirement paths.
9. Model activities, decisions, parallelism, synchronization, queues, waits, dependencies, escalations, approvals, controls, evidence capture, and human intervention.
10. Define process states and transitions, including actor, authority, trigger, precondition, guard, evidence, effect, failure, recovery, timeout, expiry, and resulting state.
11. Identify inputs, outputs, information, data, credentials, resources, physical materials, commands, events, decisions, records, and evidence crossing each boundary.
12. Allocate process activities and decisions to capabilities, functions, roles, people, systems, services, interfaces, facilities, suppliers, or enabling processes without creating authority by allocation.
13. Define process variants by environment, jurisdiction, population, role, product, data class, risk, operating mode, feature state, and lifecycle phase.
14. Identify controls, mandatory constraints, segregation of duties, approvals, audit points, evidence requirements, privacy, security, safety, accessibility, and human-control conditions.
15. Measure process performance and outcome using authorized criteria for time, quality, throughput, queue, cost, resource, user outcome, harm, control effectiveness, resilience, and evidence freshness.
16. Analyze failure modes, bottlenecks, rework, common-cause dependencies, single points of failure, unsafe defaults, bypasses, and circular recovery paths.
17. Design target and transition process states, including adoption, training, communication, coexistence, migration, rollback, fallback, support, maintenance, deprecation, and retirement.
18. Validate the model with accountable owners, participants, users, operators, maintainers, support, affected parties, and qualified specialists as applicable.
19. Hostile-review process claims and test whether controls, evidence, authority, accessibility, privacy, security, recovery, and owner visibility work under realistic conditions.
20. Baseline and hand off the process architecture with exact identity, open findings, dissent, evidence boundary, change authority, and next safe action.

HUMAN-AI INTERACTION REQUIREMENTS

- The human must be able to see whether each process element is observed, documented, proposed, approved, implemented, adopted, measured, effective, degraded, blocked, or retired.
- AI-generated process maps, sequence proposals, variants, simulations, and optimizations must remain proposed until an authorized review records adoption.
- The process surface must identify the acting human, AI, automation, rule engine, external service, or hybrid actor at each consequential step.
- A human must be able to correct intent, terminology, affected parties, process boundaries, role assignments, and decision ownership without losing prior versions or dissent.
- AI recommendations that alter rights, safety, privacy, accessibility, staffing, commitments, risk, cost, or external consequence must expose assumptions, alternatives, evidence, uncertainty, reversibility, and required approval.

- Users and operators must receive understandable explanations for denial, delay, escalation, correction, fallback, and recovery where those explanations affect action or trust.
- Model or provider failure must not remove essential stop, status, evidence, diagnosis, correction, or recovery paths.
- Automated routing, prioritization, scheduling, or optimization must preserve mandatory constraints and must not conceal minority findings or affected-party burden.
- No conversational path may bypass the identity, authority, policy, state, evidence, audit, accessibility, or recovery controls required by other process entry points.
- Long-running or autonomous process execution must expose assignment, current state, authority expiry, resource use, evidence progress, stop condition, and safe interruption behavior.

ARCHITECTURE AND SYSTEM IMPLICATIONS

- Represent process definitions, process instances, activities, decisions, states, transitions, actors, roles, policies, controls, evidence, measures, variants, and lifecycle states as versioned attributable records or equivalent controlled artifacts.
- Preserve explicit relationships among objective, outcome, value stream, capability, process, activity, function, requirement, control, information, data, system, interface, evidence, test, acceptance, incident, change, and retirement.
- Keep process definition separate from process execution state. A modeled target path is not proof of current runtime behavior.
- Define process states and transitions with guards, authority, idempotency, timeout, retry, compensation, cancellation, suspension, resumption, and audit semantics where applicable.
- Treat each material handoff as a contract identifying sender, receiver, purpose, input, output, timing, ordering, identity, authority, data, evidence, error, retry, compatibility, and ownership.
- Preserve human, AI, automated, and external actors as distinct identities with separate permissions, responsibility, evidence, and revocation paths.
- Support process variants without uncontrolled copying. Variant selection must be explicit, testable, attributable, and bound to environment, jurisdiction, population, role, data, risk, or lifecycle conditions.
- Make controls enforceable at the execution boundary where possible. A process diagram or procedure alone is not enforcement.
- Instrument sufficient events, measures, records, and evidence to distinguish work started, work completed, outcome achieved, control effective, and recovery successful.
- Design graceful degradation and safe halt for critical processes, including offline, stale data, unavailable actor, unavailable AI provider, expired authority, duplicate request, partial completion, and recovery conditions.
- Preserve process identity across requirements, architecture, configuration, implementation, deployment, runtime, evidence, migration, versioning, deprecation, and retirement.
- Ensure process changes trigger impact analysis for functions, data, interfaces, roles, training, support, controls, tests, evidence, approvals, commitments, and downstream lifecycle work.

OWNER COCKPIT AND USER-CONTROL REQUIREMENTS

- Show the process name, version, owner, authority state, scope, variant, environment, lifecycle state, and evidence freshness.
- Show the current process instance or aggregate state separately from the target process definition and planned state.
- Identify available, unavailable, prohibited, consequential, awaiting-approval, degraded, and recovery actions.
- Expose the acting identity, role, authority, tools, data boundary, consequence boundary, and approval requirement for each consequential step.
- Let the owner inspect triggers, participants, activities, decisions, queues, handoffs, controls, evidence, exceptions, risks, dependencies, and open findings.
- Provide controls to pause, narrow, approve, deny, revoke, retry, compensate, roll back, quarantine, restore, supersede, or retire within delegated authority.
- Explain what changed, why it changed, what is known, what is inferred, what is unknown, what failed, what remains unverified, and who must decide.
- Preserve essential status, stop, evidence, correction, and recovery controls when nonessential systems, networks, models, or providers fail.

- Allow export of the exact process definition, instance history, evidence, decisions, dissent, and continuity record without dependence on chat history.

The cockpit must not present a flowchart, completion count, throughput metric, model confidence, or green status as proof of successful outcome.

REQUIRED OUTPUTS AND ARTIFACTS

- Process Architecture charter with objective, scope, boundary, authority, owner, population, environments, variants, and lifecycle horizon.
- Process taxonomy, catalog, glossary, identifiers, ownership, versions, applicability, and lifecycle state.
- Current-state process inventory with source classification, observation method, evidence date, limitations, workarounds, and unknowns.
- Value-stream, capability, process, journey, organization, role, system, function, information, data, and control relationships.
- Process definition and instance models with triggers, states, activities, decisions, handoffs, queues, controls, evidence, and terminal outcomes.
- Process variant, applicability, non-applicability, exception, waiver, and supersession records.
- Process-to-requirement, process-to-function, process-to-capability, process-to-control, process-to-data, process-to-system, process-to-role, process-to-test, and process-to-evidence traceability.
- Process measures, outcome measures, queue and capacity analysis, control-effectiveness measures, user-impact measures, and evidence-freshness rules.
- Failure, degradation, denial, escalation, compensation, recovery, rollback, migration, deprecation, and retirement models.
- Human-AI process participation model with identity, authority, tools, data, approval, monitoring, fallback, revocation, appeal, and incident behavior.
- Target-state and transition process architecture with adoption, training, support, coexistence, migration, and operational readiness conditions.
- Decision, dissent, assumption, unknown, risk, evidence, change, and reopening registers.
- Validation, hostile-review, verification, owner-acceptance, and durable handoff package.

EVIDENCE REQUIREMENTS

- Every current-state process claim must identify the process, scope, version or observation period, environment, population, method, source, actor, evidence identity, and limitation.
- A policy, procedure, diagram, interview, ticket, log, or system trace proves only the bounded claim it can support; none automatically proves effective or adopted operation.
- Every process definition must identify its source, author, owner, version, applicability, assumptions, open findings, and approval state.
- Every material activity and decision must link to its outcome, requirement, role or actor, authority, control, input, output, evidence, and verification method as applicable.
- Every handoff claim must identify both parties, exact contract, data or resource exchanged, timing, identity, authority, error behavior, compatibility, and evidence.
- Process performance evidence must distinguish activity volume from outcome, nominal performance from resilience, and current measured state from target design or simulation.
- User and operator evidence must identify population, context, method, accessibility coverage, consent or handling limits, sample limitations, and unresolved interpretation.
- AI or automation evidence must identify model or tool identity, version, configuration, data, prompt or policy context where permitted, authority, human review, environment, result, and limitations.
- Control evidence must show implementation and effectiveness in the declared scope. Documentation presence alone is insufficient.
- Failed, blocked, stale, contradictory, superseded, and partial evidence must remain visible and linked to the resulting process status.
- Missing evidence must produce `UNKNOWN`, `NOT VERIFIED`, `BLOCKED`, `PROPOSED`, or another truthful bounded state; it must not be inferred as successful process operation.

VERIFICATION AND ACCEPTANCE TESTS

- Trace a representative outcome from mission or value stream through capability, process, activity, function, actor, control, system, evidence, test, acceptance, operation, maintenance, and retirement.
- Trace representative process steps and decisions backward to an authorized objective, requirement, domain rule, policy, or risk treatment.
- Introduce an unowned activity, unowned decision, orphan handoff, missing control, stale variant, and orphan evidence record and confirm each is detected or explicitly marked unresolved.
- Change a process definition, role, policy, interface, data source, authority rule, environment, or provider and confirm impacted functions, tests, evidence, training, approvals, commitments, and readiness states are flagged.
- Exercise normal, boundary, denied, delayed, duplicate, out-of-order, interrupted, degraded, offline, unauthorized, partial-completion, recovery, rollback, migration, maintenance, and retirement paths.
- Revoke authority during a long-running process and confirm new consequential actions stop, in-flight work is handled according to policy, evidence is preserved, and recovery remains possible.
- Remove or corrupt a critical actor, dependency, data source, network, model, or evidence store and confirm the process fails safely or exposes an explicit human decision point.
- Attempt to bypass a mandatory control through UI, API, agent, bulk operation, retry, administrative path, or recovery path and confirm consistent enforcement.
- Compare observed, documented, proposed, simulated, implemented, and measured process states and confirm they are not conflated.
- Validate complete user, owner, operator, maintainer, support, incident, and recovery journeys with appropriate accessibility and human-impact review.
- Verify AI-generated routing, prioritization, or automation cannot become binding without authorized adoption and required evidence.
- Independently reproduce a material process claim from the handoff package without relying on hidden conversation state.
- Confirm a process can be paused, narrowed, corrected, restored, superseded, and retired without destroying prior evidence or decision history.

FAILURE STATES AND ANTI-PATTERNS

- Treating a process diagram as an implemented or effective process.
- Defining the process around the preferred software, team, vendor, or current organization instead of the outcome and affected journey.
- Confusing a capability, function, task, procedure, policy, workflow configuration, or organization chart with a process architecture.
- Omitting manual work, queues, waits, rework, exceptions, denial, escalation, support, maintenance, or retirement.
- Assigning an activity without assigning authority, accountability, resources, inputs, outputs, evidence, and recovery.
- Making an approval step decorative, circular, unowned, or impossible to exercise in the real environment.
- Using one process model for materially different jurisdictions, populations, risk classes, operating modes, or lifecycle states without explicit variants.
- Optimizing throughput or cost while degrading user outcome, accessibility, safety, privacy, security, quality, resilience, or operator control.
- Treating a successful happy-path demonstration as evidence for boundary, failure, recovery, or long-term operation.
- Allowing AI to route, approve, commit, deny, or execute beyond its explicit authority and human-control boundary.
- Hiding dissent, workarounds, rejected paths, exception volume, or contradictory evidence to make the process appear simple.
- Claiming process compliance from procedure presence, or control effectiveness from configuration presence alone.
- Making recovery depend on the failed actor, credential, network, model, data source, or evidence path.
- Retiring a process without preserving capability continuity, records, user transition, support, obligations, evidence, and accountable ownership.

RECOVERY AND RECONSIDERATION

1. Quarantine the affected process claim, definition, variant, approval, evidence, or execution state while preserving the last trusted version.

2. Identify whether the issue concerns objective, boundary, terminology, ownership, authority, participant, activity, decision, control, data, interface, evidence, performance, accessibility, security, safety, resilience, recovery, or lifecycle state.
3. Downgrade the affected status to the truthful state and prevent unsupported readiness, compliance, effectiveness, or outcome claims.
4. Preserve the failed process path, prior versions, evidence, dissent, incidents, and supersession history.
5. Identify safe unaffected process work and any actions that must enter safe halt.
6. Request the smallest missing observation, owner decision, participant review, specialist assessment, configuration correction, test, or recovery exercise needed to reconsider.
7. Reassess affected capabilities, functions, requirements, roles, data, interfaces, controls, training, support, commitments, and downstream lifecycle obligations.
8. Restore, roll back, compensate, narrow, partition, stage, or redesign the process as authorized.
9. Record the new state, conditions, residual uncertainty, owner decision, review date, and reopening trigger.

A blocked process record must name the exact blocker, consequence, safe work, minimum resolution, authority required, and evidence needed for reconsideration.

COMPLETION CRITERIA

Process Architecture is complete for its declared scope only when:

- Objective, outcome, scope, boundary, owner, authority, population, environments, variants, non-goals, and lifecycle stages are explicit.
- Process meaning is distinguished from capability, function, activity, task, procedure, policy, organization, system, and implementation.
- Current-state evidence is classified and bounded, with observations, documentation, inference, assumptions, proposals, unknowns, and dissent visible.
- Material processes, activities, decisions, handoffs, states, controls, actors, roles, inputs, outputs, evidence, exceptions, and recovery paths are defined at sufficient depth.
- Every material process object has stable identity, version, owner, applicability, lifecycle status, authority, traceability, evidence, and change boundary.
- Process and function, capability, requirement, control, information, data, system, role, test, and evidence relationships are explicit or intentionally unresolved.
- Normal, boundary, denied, delayed, degraded, failure, recovery, rollback, migration, maintenance, deprecation, and retirement behavior is addressed where applicable.
- Human-AI participation, authority, approval, monitoring, fallback, revocation, appeal, and evidence conditions are bounded and testable.
- Performance and outcome measures do not silently substitute activity counts for user, mission, control, or benefit outcomes.
- Applicable owners, affected participants, operators, maintainers, and qualified specialists have reviewed the exact scope or their absence is recorded as a blocker or limitation.
- Verification, hostile review, acceptance, and reopening methods are defined; unresolved mandatory conditions block affected advancement.
- Downstream roles can reconstruct the process architecture, assumptions, evidence, decisions, open findings, and next safe action without chat history.

Completion does not mean the process is implemented, adopted, effective, compliant, verified in operation, or owner-accepted. It means the declared process architecture is sufficiently governed to control the next authorized step truthfully.

HANDOFF AND CONTINUITY REQUIREMENTS

The handoff must include:

- Process architecture identity, version, scope, owner, authority state, environments, variants, lifecycle stage, evidence date, and safe-to-proceed determination.
- Objective, outcomes, value streams, capabilities, journeys, boundaries, non-goals, terminology, affected parties, and decision rights.

- Process taxonomy, catalog, definitions, current-state sources, observations, target-state models, variants, applicability, and non-applicability records.
- Process instances or representative traces, states, activities, decisions, queues, handoffs, controls, inputs, outputs, data, evidence, and recovery behavior.
- Mappings to requirements, functions, capabilities, roles, systems, interfaces, information, data, controls, tests, acceptance, operations, maintenance, migration, deprecation, and retirement.
- Assumptions, unknowns, dissent, exceptions, risks, missing evidence, failed attempts, invalidated claims, and reopening triggers.
- Change authority, impact-analysis method, versioning, rollback, compensation, restoration, migration, support, training, and retirement conditions.
- Exact assignments and unresolved decisions for Functional Architecture, Operational Architecture, Information Architecture, Data Architecture, application and technology work, testing, operations, support, maintenance, and lifecycle governance.
- Current status, evidence boundary, human decisions required, safe work that may continue, blocker if any, and one next safe action.

Downstream workers must return impossible allocations, new dependencies, implementation deviations, control failures, user harm, failed tests, incidents, operational evidence, maintenance findings, migration effects, and retirement constraints to the controlled process record.

RELATED CODEX SECTIONS

001 Research
011 Business Analysis
014 Stakeholder Management
015 User Research
016 Domain Analysis
017 Feasibility Analysis
018 Risk Management
019 Requirements Engineering
020 Systems Engineering
021 Enterprise Architecture
022 Solution Architecture
023 Systems Architecture
024 Domain Architecture
025 Business Architecture
026 Capability Architecture
028 Functional Architecture
029 Operational Architecture
030 Information Architecture
031 Data Architecture
032 Application Architecture
041 Integration Architecture
042 API Architecture
043 Event-Driven Architecture
045 Security Architecture
047 Privacy Architecture
048 Governance Architecture
050 Safety Architecture
051 Reliability Architecture
052 Resilience Architecture
053 Accessibility Architecture
075 Human-Computer Interaction
078 Service Design
080 UX Research
106 User Validation
231 Software Testing

234 Integration Testing
235 System Testing
237 Acceptance Testing
249 Usability Testing
256 Verification
257 Validation
289 Change Management
301 Site Reliability Engineering
311 Service Management
319 Incident Management
331 Operational Readiness
332 Command and Control
341 Documentation Engineering
374 Maintenance Engineering
389 Deprecation Management
405 Lifecycle Governance
406 System Retirement

Reopen Process Architecture when outcome, value stream, capability, requirement, domain rule, actor, role, authority, activity, decision, control, data, interface, environment, evidence, user outcome, incident, recovery, or lifecycle obligation materially changes.

SOURCE BASIS AND UNRESOLVED SOURCE GAPS

Source basis:

- Thirsty's Standard v3 supplies the posture of truthful scope, current-state inspection, explicit unknowns, evidence before claims, independent verification, hostile review, continuity, recovery, and no fabricated completion.
- Thirsty's Agents 101 supplies the preserved role contracts, authority boundaries, evidence obligations, independent challenge, and human final authority.
- Thirsty's Owner's Cockpit Standard supplies owner visibility, action and evidence contracts, governed execution, state, recovery, versioning, and control requirements. It remains a candidate source for owner ratification rather than silently ratified universal authority.
- The draft V4 design materials supply design input for typed claims, evidence contracts, role assembly, dissent preservation, rollback, reversal, and trusted transitions. They remain draft and unratified.
- Sections 001-026 supply the current working-draft chain from research and strategy through requirements, systems engineering, enterprise, solution, systems, domain, business, and capability architecture.

Unresolved source gaps:

- The supplied corpus does not define one ratified universal process-architecture notation, taxonomy, process language, measurement model, workflow standard, or control framework for every organization, sector, jurisdiction, or system.
- No complete crosswalk has yet been performed against current external process, business-process management, service-management, safety, accessibility, privacy, security, records, labor, legal, or sector-specific standards.
- Process granularity, modeling notation, conformance thresholds, evidence freshness, participant protections, automation limits, and independent-review depth must be specialized according to consequence, domain, authority, and owner decision.
- Later Functional, Operational, Information, Data, Security, Testing, Operations, Support, Maintenance, and Retirement sections may expose missing process boundaries, controls, handoffs, or recovery obligations and must be allowed to reopen this section.

This section is CONTENT-COMPLETE. It is not yet source-reconciled, hostile-reviewed, owner-accepted, implemented, tested, or verified.

028

SECTION 028 - FUNCTIONAL ARCHITECTURE

CODEX/03_SECTIONS/028_FUNCTIONAL_ARCHITECTURE.txt

SHA-256 151bdd6e80ceb5640547dfee77c82e1ae138740e73a87ea98aeb7c24465cc2ee

44,233 BYTES / PRESERVED SOURCE IDENTITY

SECTION 028 — Functional Architecture

STATUS: CONTENT-COMPLETE

PURPOSE

Define the governed structure of what a system, service, product, organization, human, AI agent, device, or enabling element must do to produce authorized outcomes under declared conditions.

Functional Architecture decomposes approved requirements, operational concepts, process obligations, capabilities, domain rules, and system behavior into coherent functions and subfunctions. It identifies inputs, outputs, transformations, triggers, states, controls, dependencies, interfaces, allocations, failure behavior, recovery, evidence, and verification without prematurely selecting implementation technology or treating a functional model as proof of runtime behavior.

Functional Architecture specializes the whole-system functional analysis established by Systems Engineering. It preserves the difference between an enduring capability, a process sequence, a required function, a design component, an implementation artifact, and an observed outcome.

REQUIRED OUTCOME

- A declared functional-architecture scope identifying the system or organizational boundary, objectives, environments, users, operators, affected parties, lifecycle horizon, and authority.
- A stable functional taxonomy and hierarchy with clear parent-child meaning, function identity, purpose, inputs, outputs, conditions, constraints, owner, allocation, dependencies, evidence, and lifecycle state.
- A functional context model showing external actors, systems, services, people, devices, data, events, resources, commands, controls, authorities, trust boundaries, and evidence flows.
- A functional decomposition that covers normal, boundary, denied, degraded, failure, recovery, maintenance, migration, deprecation, and retirement behavior where applicable.
- Functional flows and behavior models connecting triggers, preconditions, transformations, decisions, outputs, states, timing, sequencing, concurrency, exceptions, and postconditions.
- Explicit allocation of functions and interfaces to human roles, AI roles, system elements, components, services, devices, organizations, external systems, processes, or enabling systems.
- Functional contracts defining inputs, outputs, units, schemas, identity, authority, timing, ordering, quality, error behavior, retry, idempotency, audit, evidence, and compatibility as applicable.
- Traceability from objectives, domain meaning, requirements, capabilities, processes, risks, controls, functions, allocations, implementations, tests, evidence, acceptance, operations, maintenance, and retirement.
- A functional-state model that distinguishes desired, defined, proposed, allocated, implemented, integrated, verified, validated, operational, degraded, blocked, superseded, and retired states.
- A functional verification and validation basis that makes claims reproducible and distinguishes analysis or model evidence from implementation and observed-behavior evidence.
- A durable handoff to Operational, Information, Data, Application, Software, Platform, Infrastructure, Integration, API, Event, Security, Privacy, Safety, Reliability, Resilience, Accessibility, Engineering, Testing, Operations, Maintenance, and Retirement work.

WHY THIS MATTERS

Requirements state what is authorized and required; process models state how work is coordinated in context. Functional Architecture makes the required behavior explicit enough to allocate, design, implement, test, verify, operate, recover, and retire. Without it, teams may implement features without a coherent behavior model, allocate a requirement to nobody, duplicate responsibility, or hide an essential control inside an assumed component.

Functional decomposition is also a powerful place for false certainty. A clean function tree can omit human judgment, authority, error handling, data provenance, accessibility, latency, recovery, or evidence. A function can be mathematically defined and still be unimplemented, untestable, unsafe, or unusable in context.

For human-AI systems, function boundaries determine whether an AI is classifying, recommending, deciding, executing, monitoring, or merely assisting. Those are materially different authorities and failure modes. Functional Architecture makes the boundary inspectable and prevents a model's ability to perform a function from becoming permission to perform it consequentially.

APPLICABILITY

- A system, service, product, organization, device, process, model, agent, or capability contains interacting behaviors, transformations, controls, decisions, or interfaces.
- Requirements must be decomposed and allocated across people, functions, systems, components, services, devices, external providers, or lifecycle activities.
- The system crosses software, hardware, data, human, organizational, domain, supplier, physical, or jurisdictional boundaries.
- Functional behavior has material security, privacy, safety, accessibility, reliability, resilience, performance, support, maintenance, recovery, or retirement consequences.
- Multiple implementations or architectural alternatives must satisfy the same outcome-level obligations.
- AI, automation, orchestration, or external services may perform, recommend, constrain, or supervise a function.
- Verification requires functional-level analysis, inspection, demonstration, simulation, testing, measurement, or traceability independent of implementation detail.

NON-APPLICABILITY CONDITIONS

- The work is a bounded observation, documentation correction, or reversible change with no material effect on required behavior, functional allocation, interface, control, state, evidence, user journey, or lifecycle obligation.
- A current approved functional architecture covers the exact objective, requirements baseline, version, environment, operating mode, interfaces, and change, and impact analysis confirms no material functional effect.
- The work is exploratory and makes no claim that functions are approved, allocated, implemented, integrated, verified, validated, or operational. The model must remain labeled as proposed.
- An active safety, security, privacy, integrity, legal, or authority incident requires containment before normal functional analysis can safely proceed.
- The necessary requirements, domain meaning, system boundary, or acceptance authority is unavailable. Work may continue as an evidence request or proposed analysis, but not as an accepted functional baseline.

A non-applicability decision must identify the controlling functional baseline, exact scope, version, environment, evidence date, authority, assumptions, and reopening condition. A component diagram, feature list, familiar implementation, or AI-generated decomposition is not sufficient reason to omit material functional analysis.

PREREQUISITES AND DEPENDENCIES

- Sections 001-018 provide objective, strategy, stakeholder, user, domain, feasibility, and risk context.
- Section 019 provides the controlled requirements baseline, acceptance criteria, verification methods, variants, mandatory constraints, and change obligations.
- Section 020 provides system context, operational concept, functional analysis, allocations, elements, interfaces, lifecycle, verification, validation, transition, and recovery context.
- Sections 021-027 provide enterprise, solution, systems, domain, business, capability, and process architecture context.
- Functional terminology must align with Domain Analysis, Domain Architecture, Systems Engineering, Capability Architecture, and Process Architecture or include explicit mappings.
- The boundary between function, process, capability, design component, implementation, and observed behavior must be agreed or marked unresolved.
- Human authority for requirements, safety, security, privacy, accessibility, policy, domain meaning, risk, architecture, allocation, acceptance, and deployment must be attributable.

REQUIRED INPUTS

- Authorized functional question, objective, scope, exclusions, owner, environments, operating modes, user and operator populations, and lifecycle horizon.

- Approved requirements baseline, acceptance criteria, verification methods, criticality, variants, constraints, risk treatments, feasibility conditions, and open findings.
- System context, operational concept, mission threads, process architecture, capability model, domain model, business and enterprise architecture.
- Actors, roles, organizations, systems, components, services, devices, facilities, suppliers, external dependencies, data sources, authorities, and trust boundaries.
- Current-state behavior, interface contracts, schemas, event records, logs, incidents, defects, performance, reliability, security, privacy, safety, accessibility, support, maintenance, and recovery evidence.
- Desired states, function quality, timing, capacity, resource, evidence, control, observability, compatibility, migration, deprecation, and retirement obligations.
- AI and model identity, intended function, capabilities, limitations, tool access, data boundary, autonomy mode, human review, fallback, monitoring, revocation, and provider dependencies where applicable.
- Known assumptions, unknowns, dissent, alternative decompositions, implementation constraints, supplier claims, and evidence limitations.

HUMAN RESPONSIBILITIES AND AUTHORITY

The authorized human owner retains authority over objectives, required outcomes, values, mandatory constraints, system boundary, functional criticality, acceptance, risk, safety, privacy, accessibility, security, resource commitments, autonomy, and irreversible allocation or deployment decisions.

- Confirm the functional scope, system of interest, operating modes, environments, affected parties, lifecycle stages, non-goals, and acceptance authority.
- Approve or reject derived functions, decomposition boundaries, criticality, allocations, variants, constraints, exceptions, and functional baselines.
- Decide value conflicts involving human judgment, rights, safety, privacy, accessibility, dignity, employment, public impact, cost, schedule, performance, and residual risk.
- Ensure qualified specialists and affected users, operators, maintainers, and support personnel review functions where consequence requires expertise or lived experience.
- Approve AI or automation roles, autonomy levels, tool permissions, human checkpoints, fallback, revocation, and appeal paths.
- Ratify functional claims only for the exact version, configuration, environment, scope, evidence, and unresolved conditions reviewed.
- Retain the ability to stop, narrow, quarantine, roll back, restore, supersede, or retire functional work when authority, evidence, safety, or mission conditions fail.

AI RESPONSIBILITIES AND BOUNDARIES

AI may inspect requirements and architecture, identify candidate functions, normalize terminology, compare decompositions, generate behavior models, propose allocations, analyze dependencies, draft functional contracts, produce test ideas, simulate bounded scenarios, and identify evidence gaps within its assignment.

AI must:

- restate the authorized objective, requirements baseline, system boundary, process context, assumptions, unknowns, and consequence limits before material decomposition;
- distinguish source requirement, derived function, inferred behavior, proposed allocation, implementation fact, simulation, recommendation, execution result, and verified behavior;
- preserve source identity, version, scope, environment, method, and limitations for each material functional claim;
- identify missing inputs, outputs, actors, authority, states, error behavior, timing, resources, controls, evidence, accessibility, and recovery;
- keep function definitions at the appropriate abstraction level and avoid narrowing them to a preferred implementation without authority;
- model AI participation with explicit identity, function, authority, tool access, data, human review, monitoring, fallback, revocation, and evidence;
- expose alternatives, tradeoffs, dissent, sensitivity, reversibility, and evidence that would change an allocation or recommendation;

- stop or escalate when requirements, domain meaning, authority, mandatory constraints, or acceptance boundaries are materially unresolved.

AI must not:

- invent requirements, functions, actors, interfaces, performance, resources, integrations, supplier capability, certification, or current behavior;
- convert a feature, API, prompt, component, model benchmark, prototype, simulation, or diagram into proof of functional satisfaction;
- assign authority, approve requirements, accept risk, decide values, or bind a functional baseline without valid delegation;
- optimize local function metrics while silently degrading mission outcome, user control, security, privacy, safety, accessibility, reliability, resilience, maintainability, evidence, or retirement;
- hide human judgment, exception handling, denial, error, degraded behavior, or recovery to make the functional model appear complete;
- allow conversational, administrative, retry, import, or recovery paths to bypass functional controls or authority boundaries.

APPLICABLE EXISTING AGENT ROLES

- Architect: defines functional boundaries, decomposition, contracts, allocations, dependencies, states, interfaces, failure, recovery, and lifecycle implications.
- Analyst: examines function relationships, alternatives, completeness, feasibility, quality, dependencies, tradeoffs, and consequences.
- Researcher: gathers requirements, domain, technical, human, operational, supplier, standard, and current-behavior evidence.
- Observer: records actual behavior, inputs, outputs, states, transitions, timings, failures, and system signals without unsupported causation.
- Planner: sequences functional analysis, allocation, prototyping, integration, verification, validation, transition, and review work.
- Coordinator: synchronizes functional workstreams and resolves routing or provenance issues without seizing authority.
- Critic: challenges decomposition gaps, hidden implementation bias, circular dependencies, false completeness, and unsupported behavior.
- Security Auditor: assesses identity, authorization, trust, data, control, abuse, evidence, and security behavior at functional boundaries.
- User Experience Reviewer: evaluates user, owner, operator, maintainer, support, accessibility, explanation, correction, denial, and recovery functions.
- Tester: designs functional tests across normal, boundary, failure, degraded, recovery, compatibility, security, accessibility, and governance conditions.
- Verifier: independently checks functional claims, allocations, contracts, evidence, reproducibility, and exact scope.
- Documentation Writer: maintains functional catalogs, models, contracts, decisions, assumptions, limitations, and handoffs.
- Memory Curator: preserves functional baseline identity, traceability, versions, evidence, dissent, decisions, and continuity.
- Governance Reviewer: checks authority, requirements precedence, mandatory constraints, delegation, exceptions, acceptance, and status transitions.
- Implementer: realizes approved functions within the allocated architecture and reports deviations rather than silently changing the functional baseline.
- Refactorer: improves internal structure while preserving approved functional behavior unless change is authorized.
- Optimizer: improves measured function performance while preserving protected properties and acceptance boundaries.
- Red Team: attempts authorized bypass, misuse, malformed input, privilege escalation, model manipulation, failure, recovery, and evidence attacks.
- Decision Arbiter Advisory: compares functional alternatives under authorized criteria and preserves owner decisions.

Role selection must preserve independence between implementation and verification where a false functional claim could create consequence. Existing role contracts remain controlling.

REQUIRED ACTIVITIES

1. Establish the functional context: objective, outcome, system boundary, environments, operating modes, users, operators, affected parties, lifecycle, non-goals, and authority.
2. Accept and inspect the exact requirements, process, capability, domain, systems, and architecture handoffs, including open findings and evidence limits.
3. Define terminology and distinguish capability, process, function, subfunction, activity, task, behavior, service, component, interface, control, requirement, and implementation.
4. Identify functional actors and subjects, including humans, AI agents, automation, rule engines, external systems, devices, services, facilities, and enabling organizations.
5. Establish the functional context and identify incoming and outgoing information, data, events, commands, resources, authority, identity, evidence, physical effects, and human communication.
6. Derive candidate functions from approved requirements, operational concepts, process obligations, capability outcomes, domain rules, risk treatments, and lifecycle needs.
7. Build a functional taxonomy and catalog with stable identities, purpose, scope, inputs, outputs, conditions, owner, criticality, status, dependencies, evidence, and lifecycle state.
8. Decompose functions using black-box and white-box views only as far as needed for responsibility, interface, risk, allocation, verification, implementation, and lifecycle decisions.
9. Define functional flows, behavior, sequences, timing, concurrency, state transitions, preconditions, guards, postconditions, exceptions, and compensation.
10. Define functional contracts for inputs and outputs, schema, units, identity, authorization, timing, ordering, quality, errors, retry, idempotency, audit, evidence, compatibility, and ownership.
11. Allocate functions to people, roles, processes, systems, components, services, devices, external providers, facilities, or enabling systems with rationale and authority.
12. Identify duplicated, missing, circular, conflicting, unowned, unreachable, unsafe, inaccessible, or unverifiable functions and allocations.
13. Allocate cross-cutting obligations for security, privacy, safety, accessibility, reliability, resilience, performance, maintainability, observability, support, migration, deprecation, and retirement.
14. Define functional variants by environment, jurisdiction, population, role, data class, risk, operating mode, product, feature state, and lifecycle stage.
15. Define function measures, thresholds, margins, evidence methods, test conditions, and invalidation rules for material quality or performance claims.
16. Develop and compare alternatives while preserving mandatory constraints outside weighted scoring and exposing sensitivity, reversibility, and residual uncertainty.
17. Define integration, transition, fallback, recovery, rollback, migration, coexistence, maintenance, deprecation, and retirement behavior at functional boundaries.
18. Validate functional meaning and allocation with owners, users, operators, maintainers, support, domain experts, and qualified specialists as applicable.
19. Hostile-review the functional architecture for omissions, unsafe assumptions, bypasses, inaccessible paths, unowned state, unsupported evidence, and implementation drift.
20. Baseline and hand off the functional architecture with exact identity, traceability, evidence boundary, unresolved decisions, change authority, and next safe action.

HUMAN-AI INTERACTION REQUIREMENTS

- Every consequential function must disclose whether it is performed by a human, AI model, agent, automation, rule engine, external service, or hybrid arrangement.
- The owner must be able to inspect each function's purpose, source, authority, inputs, outputs, state, allocation, criticality, controls, evidence, failure, recovery, and lifecycle status.
- AI-generated functions, decompositions, allocations, contracts, simulations, and optimizations remain proposals until authorized adoption.
- The human must be able to correct terminology, scope, function boundaries, affected parties, allocation, criticality, and acceptance without losing history or dissent.

- AI recommendations that affect rights, safety, privacy, accessibility, cost, commitments, autonomy, or external state must expose alternatives, assumptions, uncertainty, reversibility, affected parties, and evidence that would change the recommendation.
- The system must distinguish defined, proposed, allocated, implemented, integrated, verified, validated, operational, degraded, blocked, superseded, and retired function states.
- Critical stop, correction, appeal, evidence inspection, fallback, and recovery functions must remain available when an AI model, provider, network, or nonessential dependency fails.
- Automated function selection, routing, prioritization, or optimization must preserve mandatory constraints and cannot suppress minority findings or human review conditions.
- No conversational path may bypass the identity, authority, policy, state, evidence, audit, accessibility, or recovery controls required by other paths.
- Long-running or autonomous functions must expose assignment, authority expiry, current state, resource use, evidence progress, stop conditions, and safe interruption behavior.

ARCHITECTURE AND SYSTEM IMPLICATIONS

- Represent functions, subfunctions, actors, inputs, outputs, flows, states, contracts, allocations, controls, dependencies, evidence, tests, and lifecycle states as versioned attributable records or equivalent controlled artifacts.
- Preserve traceability among objective, requirement, capability, process, domain rule, function, allocation, interface, implementation artifact, test, evidence, acceptance, operation, incident, change, and retirement.
- Maintain separate functional context, functional hierarchy, functional flow, functional behavior, functional state, functional allocation, and functional contract views with explicit consistency rules.
- Use black-box functions to describe externally required behavior and white-box decomposition to expose internal structure only when that structure is needed for allocation, risk, interface, verification, or implementation decisions.
- Keep function identity separate from implementation component identity. Multiple implementations may realize one function; one component may realize multiple functions; neither relationship creates authority automatically.
- Define functional state transitions with actor, authority, trigger, precondition, guard, evidence, effect, error, timeout, retry, compensation, cancellation, and resulting state where applicable.
- Treat every material functional interface as a contract with producer, consumer, purpose, data or resource, identity, authority, timing, ordering, quality, error, compatibility, observability, evidence, and ownership.
- Allocate nonfunctional and cross-cutting obligations to functions, interfaces, actors, elements, environments, and lifecycle activities rather than leaving them as unowned global prose.
- Preserve human judgment, manual fallback, accessibility support, control, appeal, explanation, and recovery as explicit functions when they are required outcomes.
- Model AI and external providers as bounded functional realizations with version, data, context, tools, permissions, evaluation, monitoring, fallback, substitution, revocation, incident, and retirement behavior.
- Instrument functional execution and outcomes sufficiently to distinguish invocation, completion, correct result, accepted outcome, control effectiveness, and recovery.
- Ensure changes to requirements, domain rules, processes, data, interfaces, authorities, models, providers, environments, or implementations trigger functional impact analysis and evidence invalidation.
- Preserve compatibility, migration, deprecation, support, maintenance, and retirement semantics for functional contracts and historical evidence.

OWNER COCKPIT AND USER-CONTROL REQUIREMENTS

- Show the functional baseline identity, version, scope, environment, operating mode, owner, authority state, lifecycle state, and evidence freshness.
- Let the owner navigate from objective, requirement, capability, process, or domain rule to function, allocation, interface, implementation, test, evidence, acceptance, operation, and retirement.
- Identify available, unavailable, prohibited, consequential, awaiting-approval, degraded, failed, unverified, and recovery actions.
- Expose the acting identity, role, capability, authority, tools, data boundary, approval point, and consequence boundary for each consequential function.
- Show functional inputs, outputs, states, dependencies, controls, timing, quality, errors, retry, idempotency, observability, evidence, and owner.

- Provide controls to approve, reject, defer, narrow, allocate, revoke, pause, stop, roll back, restore, quarantine, supersede, and retire within authority.
- Explain what is defined versus implemented, what has been tested versus verified, what is simulated versus live, and what remains unknown.
- Preserve critical status, stop, correction, evidence, fallback, and recovery functions when nonessential dependencies fail.
- Export the functional baseline, contracts, allocations, traceability, evidence, decisions, dissent, and continuity record without hidden conversational state.

The cockpit must not equate a function catalog, component existence, benchmark, demo, or green dashboard with functional correctness or accepted outcome.

REQUIRED OUTPUTS AND ARTIFACTS

- Functional Architecture charter with objective, scope, system boundary, environments, operating modes, users, operators, affected parties, authority, and lifecycle.
- Functional taxonomy, hierarchy, catalog, glossary, identifiers, ownership, criticality, variants, applicability, and lifecycle status.
- Functional context, actor, trust, authority, data, information, event, command, control, evidence, and resource flow models.
- Functional decomposition with black-box and white-box views, purpose, inputs, outputs, conditions, transformations, dependencies, and boundaries.
- Functional behavior, sequence, state, timing, concurrency, exception, compensation, and recovery models.
- Functional contracts and interface-control records with identity, authority, schema, units, timing, ordering, quality, errors, retry, idempotency, audit, evidence, compatibility, and ownership.
- Process-to-function, capability-to-function, requirement-to-function, function-to-element, function-to-interface, function-to-implementation, function-to-test, function-to-evidence, and function-to-acceptance traceability.
- Functional allocation and responsibility matrix covering people, roles, AI, automation, systems, components, services, devices, providers, facilities, and enabling organizations.
- Functional variants, non-applicability, exception, waiver, supersession, migration, deprecation, and retirement records.
- Functional quality, performance, capacity, reliability, resilience, security, privacy, safety, accessibility, maintainability, observability, support, and evidence model.
- Human-AI functional participation and autonomy model with identity, authority, tools, data, review, fallback, monitoring, revocation, appeal, incident, and retirement behavior.
- Alternatives, tradeoffs, sensitivity, reversibility, dissent, assumptions, unknowns, risks, decisions, and reopening register.
- Integration, verification, validation, owner-acceptance, hostile-review, and durable handoff package.

EVIDENCE REQUIREMENTS

- Every functional claim must identify subject, function identity, version, scope, environment, operating mode, conditions, method, evidence source, timeframe, actor, and limitations.
- A requirement proves an obligation exists; a functional model proves how the obligation is represented; neither alone proves implementation or runtime behavior.
- Every function must identify its source requirement, capability, process, domain rule, risk treatment, or lifecycle obligation, or be explicitly marked proposed or unresolved.
- Every allocation must identify allocated function, receiving actor or element, responsibility, authority, rationale, interface effects, verification level, and status.
- Every functional contract must identify endpoints, exact versions, inputs, outputs, units, identity, authority, timing, ordering, quality, errors, retries, compatibility, observability, evidence, and ownership as applicable.
- Analysis, simulation, inspection, demonstration, prototype, test, measurement, runtime record, user observation, and owner acceptance must remain distinct evidence types.
- Performance and quality evidence must identify workload, data, configuration, environment, thresholds, uncertainty, method, result, and excluded claims.
- AI evidence must identify model or provider, version, configuration, context, data, tools, permissions, evaluation, human review, fallback, and limitations proportionate to consequence.

- Verification evidence must identify exact artifact or baseline, expected result, actual result, procedure, inputs, environment, actor, date, logs, deviations, reproducibility, and independence.
- Failed, blocked, inconclusive, stale, superseded, contradictory, or partial evidence must remain visible and must constrain the related functional state.
- Missing evidence must result in `UNKNOWN`, `NOT VERIFIED`, `BLOCKED`, `PROPOSED`, `CONDITIONAL`, or another bounded state; it must not be treated as functional success.

VERIFICATION AND ACCEPTANCE TESTS

- Trace representative objectives, requirements, capabilities, processes, and domain rules into functions, allocations, interfaces, implementations, tests, evidence, acceptance, operations, maintenance, and retirement.
- Trace representative implementations, components, interfaces, and tests backward to authorized functions and their source obligations.
- Introduce an orphan function, duplicate function, unallocated function, unreachable function, unowned state, incomplete contract, and orphan test and confirm each is detected or explicitly recorded.
- Change a requirement, process, domain rule, interface, data schema, authority rule, model, provider, environment, or implementation and confirm impacted functions, allocations, evidence, tests, approvals, and readiness are flagged.
- Exercise normal, boundary, invalid, unauthorized, malformed, duplicate, replayed, out-of-order, timed-out, partial, degraded, offline, failure, recovery, rollback, migration, maintenance, deprecation, and retirement behavior.
- Verify functional contracts reject or safely handle missing, stale, malformed, incompatible, unauthorized, duplicate, replayed, or partially processed inputs as applicable.
- Disable an AI model, external provider, network, data source, credential, or evidence store and confirm critical control, stop, status, fallback, and recovery functions remain available or fail safely.
- Attempt to let an AI-generated function, allocation, optimization, risk acceptance, or readiness claim become binding without authority and verify rejection.
- Confirm mandatory security, privacy, safety, accessibility, governance, reliability, resilience, and human-control obligations cannot be traded away through aggregate scoring or implementation convenience.
- Validate complete owner, user, operator, maintainer, support, incident, and recovery journeys in realistic environments with appropriate accessibility coverage.
- Independently reproduce representative functional claims from the exact handoff package and state what remains unverified.
- Restore a previous functional baseline and confirm identity, allocations, interfaces, evidence, compatibility, and owner controls remain coherent.

FAILURE STATES AND ANTI-PATTERNS

- Treating a function list, component diagram, API list, prompt, feature list, benchmark, prototype, or simulation as proof of functional behavior.
- Deriving functions from the current implementation and then calling them requirements or architecture without authorized analysis.
- Decomposing too little to expose ownership, interfaces, controls, states, failure, evidence, or verification.
- Decomposing too far into an ungoverned task inventory that obscures outcomes, responsibility, or change impact.
- Duplicating functions under different names or assigning one function to conflicting owners without an explicit allocation rule.
- Leaving human judgment, manual fallback, accessibility, explanation, appeal, recovery, evidence, or lifecycle work implicit.
- Allocating a function to a team, document, or component without defining authority, inputs, outputs, contract, resources, evidence, and verification.
- Treating a model's technical ability, provider claim, training result, or agent confidence as permission or operational competence.
- Local optimization that improves latency, cost, or throughput while weakening correctness, safety, privacy, security, accessibility, resilience, maintainability, support, or retirement.
- Hiding denied, malformed, adversarial, concurrent, stale, partial, degraded, or recovery behavior.
- Treating a test of one implementation, environment, input, or path as proof of the general functional claim.
- Allowing changes in requirements, process, domain, data, authority, provider, or environment to leave functional evidence and approvals falsely green.

- Making recovery depend on the failed function, component, credential, network, provider, model, data, or evidence store.
- Retiring a function or implementation without preserving required capability, user transition, compatibility, records, evidence, support, and accountability.

RECOVERY AND RECONSIDERATION

1. Quarantine the affected function, allocation, contract, baseline, evidence, approval, or readiness claim while preserving the last trusted state.
2. Classify the issue as objective, requirement, domain meaning, process, decomposition, actor, authority, input, output, state, contract, allocation, control, quality, evidence, implementation, interface, failure, recovery, or lifecycle defect.
3. Downgrade the affected functional state to `UNKNOWN`, `PROPOSED`, `BLOCKED`, `NOT VERIFIED`, `FAILED`, or another truthful state.
4. Preserve the prior functional model, implementation, evidence, dissent, failed attempt, incident, and supersession links.
5. Identify safe unaffected work and enter safe halt for actions that could create consequence from the invalid function.
6. Request the smallest missing requirement clarification, domain decision, owner approval, specialist review, observation, contract correction, implementation change, test, or recovery exercise needed.
7. Re-run impact analysis across processes, capabilities, requirements, architecture, allocations, interfaces, data, controls, tests, evidence, training, operations, support, maintenance, migration, deprecation, and retirement.
8. Narrow scope, split the function, create a variant, use a reversible realization, restore a prior baseline, or defer advancement as authorized.
9. Record the new state, conditions, residual uncertainty, owner decision, review date, and evidence that would change the conclusion.

A blocked functional record must name the exact blocker, consequence, safe work, minimum resolution, authority required, and evidence needed for reconsideration.

COMPLETION CRITERIA

Functional Architecture is complete for its declared scope only when:

- Objective, outcomes, requirements baseline, scope, system boundary, environments, operating modes, affected parties, authority, non-goals, and lifecycle stages are explicit.
- Function meaning is distinguished from capability, process, activity, task, procedure, policy, component, service, interface, implementation, and observed behavior.
- Every material function has stable identity, purpose, inputs, outputs, conditions, owner, criticality, dependencies, allocation, evidence, verification method, and lifecycle state.
- Functional context, hierarchy, flows, behavior, states, contracts, allocations, controls, and dependencies are coherent or disagreements are visible.
- Functions trace to authorized requirements, capabilities, processes, domain rules, risk treatments, and lifecycle obligations, or remain visibly proposed or unresolved.
- Material cross-cutting obligations for security, privacy, safety, accessibility, reliability, resilience, performance, maintainability, observability, support, migration, deprecation, and retirement are allocated.
- Normal, boundary, denied, malformed, concurrent, degraded, failure, recovery, rollback, migration, maintenance, deprecation, and retirement behavior is addressed where applicable.
- Human-AI roles, authority, autonomy, human checkpoints, fallback, revocation, appeal, monitoring, evidence, and incident behavior are bounded and testable.
- Functional contracts and allocations are versioned, compatible, traceable, and supported by appropriate evidence and independent verification.
- Applicable owners, users, operators, maintainers, support personnel, and qualified specialists have reviewed the exact scope or their absence is recorded as a limitation or blocker.
- Hostile review, verification, acceptance, change, invalidation, recovery, and reopening methods are defined; unresolved mandatory conditions block affected advancement.
- Downstream roles can reconstruct the functional baseline, assumptions, evidence, decisions, open findings, and next safe action without chat history.

Completion does not mean functions are implemented, integrated, verified, validated, accepted, operational, effective, or retired. It means the declared functional architecture is sufficiently coherent and governed to control the next authorized realization step truthfully.

HANDOFF AND CONTINUITY REQUIREMENTS

The handoff must include:

- Functional architecture identity, version, scope, owner, authority state, environment, operating modes, lifecycle stage, evidence date, and safe-to-proceed determination.
- Objective, outcomes, requirements baseline, capabilities, processes, domain meaning, actors, affected parties, non-goals, constraints, and acceptance authority.
- Functional taxonomy, hierarchy, catalog, context, flows, behavior, states, contracts, allocations, variants, applicability, and lifecycle status.
- Inputs, outputs, transformations, data, events, commands, resources, identity, authority, controls, evidence, errors, retries, timing, ordering, quality, and compatibility.
- Mappings to requirements, capabilities, processes, system elements, components, services, roles, interfaces, data, controls, tests, evidence, acceptance, operations, maintenance, migration, deprecation, and retirement.
- AI and automation participation, model and provider identity, tools, permissions, human review, fallback, monitoring, revocation, appeal, incident behavior, and retirement conditions.
- Assumptions, unknowns, dissent, alternatives, risks, missing evidence, invalidated claims, unresolved allocations, failed attempts, and reopening triggers.
- Change authority, impact analysis, versioning, compatibility, rollback, restoration, migration, support, training, and deprecation conditions.
- Exact assignments and open decisions for Operational, Information, Data, Application, Software, Platform, Infrastructure, Integration, API, Event, Security, Privacy, Safety, Reliability, Resilience, Accessibility, Engineering, Testing, Operations, Maintenance, and Retirement work.
- Current status, evidence boundary, human decisions required, safe work that may continue, blocker if any, and one next safe action.

Downstream workers must return impossible functions, allocation conflicts, interface defects, implementation deviations, failed tests, incidents, operational evidence, maintenance findings, migration effects, and retirement constraints into the controlled functional record.

RELATED CODEX SECTIONS

019 Requirements Engineering
020 Systems Engineering
021 Enterprise Architecture
022 Solution Architecture
023 Systems Architecture
024 Domain Architecture
025 Business Architecture
026 Capability Architecture
027 Process Architecture
029 Operational Architecture
030 Information Architecture
031 Data Architecture
032 Application Architecture
033 Software Architecture
034 Platform Architecture
035 Infrastructure Architecture
038 Distributed Systems Architecture
041 Integration Architecture
042 API Architecture
043 Event-Driven Architecture
044 Runtime Architecture

045 Security Architecture
046 Identity Architecture
047 Privacy Architecture
048 Governance Architecture
050 Safety Architecture
051 Reliability Architecture
052 Resilience Architecture
053 Accessibility Architecture
056 AI Architecture
057 Machine Learning Architecture
058 Robotics Architecture
059 Embedded Systems Architecture
060 Hardware Architecture
075 Human-Computer Interaction
106 User Validation
107 Software Engineering
117 Application Engineering
128 API Engineering
129 Integration Engineering
178 Agent Systems Engineering
182 AI Safety
183 AI Governance
231 Software Testing
234 Integration Testing
235 System Testing
246 Security Testing
247 Accessibility Testing
253 Model Testing
254 Data Testing
256 Verification
257 Validation
258 Configuration Management
289 Change Management
301 Site Reliability Engineering
303 Runtime Operations
313 Observability Engineering
319 Incident Management
327 Disaster Recovery
331 Operational Readiness
332 Command and Control
374 Maintenance Engineering
387 Compatibility Management
388 Version Management
389 Deprecation Management
405 Lifecycle Governance
406 System Retirement

Reopen Functional Architecture when objective, requirement, domain meaning, process, capability, system boundary, actor, authority, function, allocation, interface, data, model, provider, environment, risk, control, evidence, implementation, incident, or lifecycle obligation materially changes.

SOURCE BASIS AND UNRESOLVED SOURCE GAPS

Source basis:

- Thirsty's Standard v3 supplies truthful scope, current-state inspection, explicit unknowns, evidence before claims, independent verification, hostile review, continuity, recovery, and no fabricated completion.

- Thirsty's Agents 101 supplies the preserved role contracts, authority boundaries, architecture duties, implementation limits, independent verification, and human final authority.
- Thirsty's Owner's Cockpit Standard supplies owner visibility, governed actions, requirements and evidence contracts, state, fallback, recovery, versioning, and acceptance controls. It remains a candidate source for owner ratification rather than silently ratified universal authority.
- The draft V4 design materials supply design input for typed claims, evidence contracts, role assembly, dissent, rollback, reversal, and trusted transitions. They remain draft and unratified.
- Sections 019-027 supply the current working-draft chain from requirements and systems engineering through enterprise, solution, systems, domain, business, capability, and process architecture.

Unresolved source gaps:

- The supplied corpus does not define one ratified universal functional-architecture notation, decomposition rule, taxonomy, modeling language, allocation method, or assurance scale for every system, sector, jurisdiction, or lifecycle.
- No complete crosswalk has yet been performed against current external functional-analysis, systems-engineering, software, hardware, safety, security, privacy, accessibility, human-factors, reliability, or sector-specific standards.
- Functional granularity, criticality, timing, quality thresholds, independence, evidence freshness, model assurance, and acceptance depth must be specialized according to consequence, domain, authority, and owner decision.
- Later Operational, Information, Data, Application, Software, Platform, Infrastructure, Integration, Security, Testing, Operations, Maintenance, and Retirement sections may expose missing functions, allocations, interfaces, controls, or lifecycle obligations and must be allowed to reopen this section.

This section is CONTENT-COMPLETE. It is not yet source-reconciled, hostile-reviewed, owner-accepted, implemented, tested, or verified.

029

SECTION 029 - OPERATIONAL ARCHITECTURE

CODEX/03_SECTIONS/029_OPERATIONAL_ARCHITECTURE.txt

SHA-256 57ed6e065e3790f14c617fbaf0d0950c35fcbed5a19ebdb9de5dd6a009d1233e

20,501 BYTES / PRESERVED SOURCE IDENTITY

SECTION 029 — Operational Architecture

STATUS: CONTENT-COMPLETE

PURPOSE

Define how an approved capability, product, service, system, or organization is put into use and kept within its intended authority, performance, safety, security, support, continuity, and lifecycle boundaries.

Operational Architecture connects process and functional design to real operating modes. It describes people, procedures, systems, facilities, suppliers, controls, schedules, signals, decision points, support paths, maintenance, incidents, degradation, recovery, and retirement. It is an operating design, not proof that the design has been staffed, deployed, adopted, or effective.

REQUIRED OUTCOME

- A bounded operational concept with owners, users, operators, maintainers, support parties, environments, modes, dependencies, service boundaries, and lifecycle state.
- A catalog of operational services, responsibilities, procedures, runbooks, controls, resources, records, evidence, and escalation paths.
- Normal, boundary, emergency, degraded, maintenance, migration, recovery, and retirement operating models.
- Explicit operational decision rights for start, stop, approve, change, suspend, restore, fail over, communicate, and retire actions.
- Traceability from capabilities, processes, functions, requirements, risks, controls, systems, data, interfaces, tests, evidence, incidents, and owner acceptance.
- Measurable readiness, service, outcome, control, reliability, resilience, support, and recovery conditions with truthful evidence states.

WHY THIS MATTERS

A system can be architecturally coherent and still fail because no one knows who operates it, what happens outside the happy path, how authority expires, or how the service is restored after a dependency fails. Operational Architecture makes the lived operating environment explicit before implementation or launch is treated as success.

It also prevents operations from becoming an unowned afterthought. Human work, automation, alerts, maintenance, support, incident response, records, training, capacity, supplier obligations, and retirement all change the system's real behavior. For human-AI systems, operations must keep model, provider, tool, data, prompt, policy, and delegated-authority changes visible and reversible.

APPLICABILITY

- A product, service, system, capability, facility, process, or organization will be used by people or automated actors over time.
- Operation requires roles, schedules, procedures, monitoring, support, resources, approvals, controls, or service commitments.
- The system has production, field, offline, emergency, maintenance, degraded, multi-tenant, supplier, or jurisdictional operating modes.
- Operational failure could affect safety, security, privacy, accessibility, reliability, resilience, rights, finances, service continuity, or public trust.
- AI, automation, external providers, or long-running agents may act after design or deployment.

NON-APPLICABILITY CONDITIONS

- The artifact is a purely explanatory model with no claim of readiness, deployment, operation, support, or service effect.
- The change is bounded, reversible, and has no material effect on operating roles, procedures, controls, dependencies, evidence, support, recovery, or lifecycle state.

- An approved operational baseline covers the exact version, environment, population, mode, and change, and impact analysis finds no material operational effect.

The determination must name the controlling baseline, scope, evidence date, owner, assumptions, and reopening condition. Small size, test-environment use, or the presence of a runbook does not by itself establish non-applicability.

PREREQUISITES AND DEPENDENCIES

- Sections 019-020 provide requirements, operational concept, lifecycle, allocation, verification, validation, transition, and recovery context.
- Sections 021-028 provide enterprise, solution, systems, domain, business, capability, process, and functional boundaries.
- Current-state evidence identifies actual operators, environments, dependencies, support burden, incidents, capacity, and recovery behavior.
- Human owners exist for service outcomes, risk, security, privacy, safety, staffing, commitments, changes, and acceptance, or the absence is recorded as a blocker.

REQUIRED INPUTS

- Approved objective, scope, service or capability outcomes, users, affected parties, environments, jurisdictions, modes, and lifecycle horizon.
- Process and functional models, system elements, information and data obligations, interfaces, controls, requirements, risks, and acceptance criteria.
- Deployment topology, schedules, staffing, skills, facilities, vendors, contracts, credentials, tools, budgets, support channels, and communication paths.
- Monitoring, logging, tracing, metrics, alerting, incident, problem, change, maintenance, continuity, backup, restore, and retirement evidence where available.
- Availability, reliability, safety, security, privacy, accessibility, capacity, performance, service-level, recovery, and evidence-freshness obligations.
- AI and automation identities, models, providers, tools, data, policies, authority limits, review points, fallback, monitoring, and revocation paths.

HUMAN RESPONSIBILITIES AND AUTHORITY

The accountable human owner retains authority over service purpose, operating risk, commitments, staffing, resource allocation, policy interpretation, exceptions, release or launch readiness, incident severity, external communication, recovery tradeoffs, and final acceptance.

- Confirm operating scope, modes, populations, environments, non-goals, and service boundaries.
- Assign accountable operators, maintainers, support owners, security, privacy, safety, accessibility, and continuity authorities.
- Approve procedures, readiness gates, operational changes, risk treatments, emergency actions, exceptions, and retirement.
- Decide when evidence is sufficient to enter, remain in, or leave an operating mode.
- Ensure affected users and qualified specialists can report harm, challenge decisions, stop unsafe work, and obtain recovery or appeal.
- Retain the ability to pause, narrow, isolate, fail over, restore, revoke, or retire the service.

AI RESPONSIBILITIES AND BOUNDARIES

AI may inspect operational records, correlate signals, draft runbooks, classify alerts, propose schedules, simulate capacity or failure, summarize incidents, identify drift, and recommend bounded actions within explicit delegation.

AI must label observation, inference, forecast, recommendation, action, and verified result separately; preserve source and time boundaries; expose uncertainty and missing telemetry; and escalate when authority, safety, security, privacy, service commitment, or evidence is unresolved.

AI must not silently change production state, declare readiness, accept risk, suppress alerts, close incidents, communicate commitments, revoke access, or approve recovery beyond its delegated authority. Model confidence, automation success, and dashboard health are not operational acceptance.

APPLICABLE EXISTING AGENT ROLES

- Architect: defines operating modes, service boundaries, responsibilities, dependencies, controls, and lifecycle implications.
- Analyst: evaluates capacity, service behavior, incidents, tradeoffs, risk, and operational evidence.
- Observer: records actual operating conditions, work, signals, queues, failures, and operator behavior.
- Coordinator: synchronizes operators, suppliers, support, incidents, changes, and handoffs.
- Planner: sequences readiness, deployment, transition, maintenance, recovery, and retirement.
- Security Auditor: examines privileged operation, trust boundaries, monitoring, abuse, and secure recovery.
- Governance Reviewer: checks authority, commitments, exceptions, records, and status claims.
- Tester: exercises operational, degradation, recovery, and continuity scenarios.
- Verifier: checks readiness and operational claims against exact evidence.
- Documentation Writer: maintains runbooks, operating concepts, records, and handoffs.
- Memory Curator: preserves baseline identity, changes, incidents, evidence, and continuity.
- Critic: challenges hidden work, missing ownership, unsafe defaults, and false readiness.
- User Experience Reviewer: evaluates user, operator, support, accessibility, and recovery journeys.
- Implementer: performs authorized operational changes and reports deviations.
- Optimizer: improves measured service performance without weakening protected outcomes.
- Red Team: probes operational bypass, abuse, failure, and recovery boundaries.

Role selection remains proportional to consequence. Existing role contracts remain controlling.

REQUIRED ACTIVITIES

1. Establish the operational objective, owner, scope, service boundary, populations, environments, modes, schedule, jurisdiction, and lifecycle.
2. Identify users, operators, maintainers, support, suppliers, affected non-users, authorities, facilities, systems, and dependencies.
3. Define operational services, outcomes, responsibilities, procedures, controls, records, communications, and escalation paths.
4. Model normal, peak, reduced, emergency, offline, maintenance, migration, recovery, and retirement modes.
5. Define entry, exit, transition, trigger, guard, authority, evidence, timeout, and safe-state conditions for each mode.
6. Allocate operating work to people, teams, automation, AI, systems, suppliers, and facilities with explicit authority and fallback.
7. Define service, capacity, performance, availability, reliability, safety, security, privacy, accessibility, support, and recovery measures.
8. Design observability, alerting, diagnosis, incident, problem, change, communication, audit, and evidence paths.
9. Assess staffing, skills, training, fatigue, workload, accessibility, physical conditions, tool availability, and support burden.
10. Define deployment, promotion, rollback, backup, restore, failover, substitution, patching, maintenance, and retirement procedures.
11. Analyze dependencies, common failure domains, suppliers, credentials, data freshness, model or provider drift, and authority expiry.
12. Validate operating models with accountable owners, operators, users, maintainers, support, security, privacy, safety, and accessibility specialists.
13. Exercise failure, degradation, continuity, recovery, communication, and owner-control scenarios.
14. Baseline the operational architecture, record unresolved gaps, and hand it to delivery, testing, operations, maintenance, and governance.

HUMAN-AI INTERACTION REQUIREMENTS

- Show whether each operational status is observed, inferred, simulated, recommended, executed, or verified.

- Identify the acting human, AI, automation, service, provider, authority, tool, data scope, and consequence for each consequential action.
- Require human review for changes to protected outcomes, safety, rights, privacy, security, commitments, staffing, or risk.
- Keep stop, pause, revoke, escalation, diagnosis, evidence, communication, fallback, and recovery paths available during model or provider failure.
- Expose uncertainty, stale telemetry, missing coverage, conflicting signals, and the evidence that would change a recommendation.
- Never let a conversational command bypass change, identity, authorization, audit, accessibility, or recovery controls.

ARCHITECTURE AND SYSTEM IMPLICATIONS

- Represent operational modes, services, roles, responsibilities, procedures, dependencies, schedules, controls, events, evidence, metrics, and lifecycle states as versioned records.
- Separate intended operating design from current runtime state and measured outcome.
- Give every operational handoff an owner, input, output, timing, authority, identity, evidence, error, and recovery contract.
- Design safe states, graceful degradation, isolation, retry, compensation, failover, restore, and manual fallback for consequential services.
- Preserve operator and owner visibility into state, freshness, capacity, authorization, active changes, incidents, and unresolved conditions.
- Ensure operational changes invalidate or requalify affected tests, evidence, approvals, training, commitments, and runbooks.
- Treat models, providers, prompts, policies, tools, and data as operational dependencies with versioning, monitoring, substitution, and revocation.

OWNER COCKPIT AND USER-CONTROL REQUIREMENTS

- Show baseline identity, current mode, environment, owner, authority state, freshness, service state, and degraded conditions.
- Show available, prohibited, consequential, awaiting-approval, and recovery actions with acting identity and required authority.
- Expose active changes, incidents, dependencies, capacity, alerts, evidence, decisions, communications, and open findings.
- Permit authorized pause, narrow, approve, deny, revoke, isolate, fail over, restore, rollback, and retire actions.
- Explain what is known, inferred, stale, unverified, blocked, failed, and awaiting human decision.
- Preserve durable status, evidence, and recovery controls when nonessential systems fail.

REQUIRED OUTPUTS AND ARTIFACTS

- Operational Architecture charter and operational concept.
- Service, mode, environment, responsibility, dependency, and authority catalogs.
- Operating procedures, runbooks, escalation, communication, support, maintenance, and recovery models.
- Readiness, capacity, availability, reliability, safety, security, privacy, accessibility, and service-measure definitions.
- Observability, incident, problem, change, evidence, audit, backup, restore, failover, and retirement designs.
- Human-AI operating participation, authority, monitoring, fallback, and revocation model.
- Operational risk, assumption, dependency, decision, dissent, exception, and reopening registers.
- Validation, verification, owner-acceptance, and continuity handoff package.

EVIDENCE REQUIREMENTS

- Each operational claim identifies the exact service, version, environment, mode, population, time, method, actor, source, and limitation.
- A runbook, dashboard, drill, simulation, or readiness review does not by itself prove effective operation.
- Readiness evidence identifies required condition, test or inspection method, result, exceptions, owner, date, and validity period.

- Performance and availability evidence states workload, configuration, observation period, exclusions, uncertainty, and whether it is target, simulated, or measured.
- Incident and recovery evidence preserves timeline, state, authority, action, communication, result, residual impact, and reproducibility.
- AI evidence identifies model, provider, version, data, policy, tool, authority, human review, result, and limitation.
- Failed, stale, partial, contradictory, and missing evidence constrains operational status and remains visible.

VERIFICATION AND ACCEPTANCE TESTS

- Trace an outcome through process, function, service, operator, system, evidence, support, maintenance, and retirement.
- Verify every operational service and mode has an owner, entry condition, exit condition, authority, evidence, and recovery path.
- Exercise normal, peak, degraded, offline, emergency, maintenance, dependency-loss, authority-expiry, and retirement scenarios.
- Disable a provider, model, network, credential, operator, data source, monitoring path, and evidence store and verify safe behavior or explicit human escalation.
- Revoke authority during a long-running action and verify consequential work stops or follows the approved safe procedure.
- Inject stale telemetry, duplicate alerts, false positives, missed alerts, and conflicting signals and verify diagnosis and escalation.
- Verify backup, restore, failover, rollback, communication, and return-to-service claims with recorded results.
- Independently reproduce readiness, service, incident, and recovery claims from the handoff package.

FAILURE STATES AND ANTI-PATTERNS

- Treating deployment, uptime, a green dashboard, or a completed runbook as proof of operational success.
- Leaving operators, support, suppliers, facilities, staffing, training, or communication unmodeled.
- Designing only the normal path and omitting degraded, emergency, offline, maintenance, recovery, and retirement modes.
- Making alerts noisy, unactionable, unowned, or impossible to investigate.
- Allowing AI or automation to suppress evidence, close incidents, change state, or accept risk without authority.
- Optimizing service metrics while increasing user harm, operator fatigue, accessibility barriers, security exposure, or recovery time.
- Depending on the failed component, unavailable credential, stale data, or inaccessible cockpit for recovery.

RECOVERY AND RECONSIDERATION

- On uncertainty, loss of authority, unsafe condition, major drift, or material evidence failure, stop or narrow affected actions and preserve state and evidence.
- Identify the exact blocker, safe work that may continue, authority needed, evidence required, recovery owner, and review deadline.
- Use approved fallback, isolation, failover, manual operation, rollback, restore, provider substitution, or service reduction paths.
- Reassess user impact, commitments, security, privacy, safety, accessibility, and residual risk before resuming.
- Reopen the operational baseline when incidents, changes, new dependencies, changed populations, or invalidated evidence alter operating assumptions.

COMPLETION CRITERIA

- Scope, owners, authorities, modes, dependencies, services, procedures, controls, evidence, measures, and lifecycle states are explicit.
- Normal, boundary, degraded, emergency, maintenance, recovery, migration, and retirement paths are addressed or justified as not applicable.
- Readiness, support, observability, security, privacy, safety, accessibility, continuity, and human-control conditions are traceable.

- Verification and owner acceptance identify exact versions, environments, evidence, unresolved gaps, and validity limits.
- The result is marked CONTENT-COMPLETE until source reconciliation, hostile review, and owner acceptance are separately completed.

HANDOFF AND CONTINUITY REQUIREMENTS

- Preserve the operational baseline identity, version, scope, environment, modes, owners, dependencies, decisions, evidence, dissent, and open gaps.
- Hand off service contracts, operating procedures, runbooks, readiness criteria, telemetry, incidents, recovery, support, maintenance, and retirement obligations.
- Record changes that require reopening Process, Functional, Information, Data, Application, Security, Reliability, Resilience, Testing, or Governance sections.
- State the next safe action, blocked actions, required authority, evidence needed, and exact continuation point.
- Do not claim operational readiness, adoption, effectiveness, or owner acceptance from this draft alone.

RELATED CODEX SECTIONS

- 019 Requirements Engineering; 020 Systems Engineering.
- 021-028 Enterprise, Solution, Systems, Domain, Business, Capability, Process, and Functional Architecture.
- 030-044 Information, Data, Application, Software, Platform, Infrastructure, Cloud, Edge, Distributed, Network, Communications, Integration, API, Event-Driven, and Runtime Architecture.
- 045-053 Security, Identity, Privacy, Governance, Compliance, Safety, Reliability, Resilience, and Accessibility Architecture.
- 301-331 Site Reliability, Production, Runtime, Cloud, Network, Security, Data, AI, Application, Platform Operations, Service Management, Observability, Incident, Capacity, Availability, Performance, Reliability, Service Level, Continuity, Disaster Recovery, Backup, Restore, Failover, and Operational Readiness.
- 332-406 Command and Control through System Retirement.

SOURCE BASIS AND UNRESOLVED SOURCE GAPS

- This draft is based on the Canonical Section Contract, shared governance files, Sections 001-028, the Owner Cockpit Requirements, the Status and Evidence Lexicon, and the Remaining-Section Integration Roadmap.
- The supplied Codex does not provide a single ratified operational standard for every sector, service level, safety regime, staffing model, jurisdiction, or technology environment.
- Sector-specific legal, contractual, safety, labor, accessibility, security, continuity, and service obligations remain to be identified and reconciled for each actual system.
- Current operating evidence, staffing authority, supplier commitments, recovery objectives, and owner acceptance are unresolved until supplied and independently verified.

030

SECTION 030 - INFORMATION ARCHITECTURE

CODEX/03_SECTIONS/030_INFORMATION_ARCHITECTURE.txt

SHA-256 e34d3401ba6c6fc9d3f1a9750746c3ffc3e554de4bc65591795ea6232b608f59

19,118 BYTES / PRESERVED SOURCE IDENTITY

SECTION 030 — Information Architecture

STATUS: CONTENT-COMPLETE

PURPOSE

Define how information is named, organized, related, described, discovered, presented, exchanged, governed, and retired so that people and systems can understand and use it within authorized context.

Information Architecture concerns meaning and access across journeys, processes, products, services, applications, records, interfaces, and operating environments. It establishes structures, vocabularies, labels, navigation, metadata, provenance, permissions, representations, and lifecycle states without confusing information with raw data, a storage technology, a user interface, or a claim of user comprehension.

REQUIRED OUTCOME

- A bounded information landscape with audiences, purposes, contexts, domains, owners, authorities, sensitivity, jurisdictions, and lifecycle.
- Canonical terms, definitions, identifiers, taxonomies, ontologies, relationships, metadata, labels, and translation rules.
- Information flows and views connecting users, operators, processes, functions, systems, data stores, interfaces, records, controls, and evidence.
- Discoverability, navigation, search, filtering, accessibility, localization, presentation, and explanation structures appropriate to each audience.
- Explicit ownership for meaning, quality, provenance, access, retention, correction, publication, and retirement.
- Traceability from requirements, domain rules, processes, functions, data, applications, policies, controls, tests, evidence, and user outcomes.

WHY THIS MATTERS

People and systems cannot make reliable decisions from information that is ambiguous, stale, inaccessible, misclassified, or impossible to find. Poor information structure creates duplicate records, hidden authority, unsafe defaults, privacy leakage, inconsistent terminology, and user journeys that cannot be completed.

Information Architecture makes meaning and context deliberate. It separates a term from its definition, a record from its presentation, a source from a derivative, and discoverability from permission. For AI systems, it also determines what context may be retrieved, what provenance is shown, how uncertainty is communicated, and how generated information is corrected or withdrawn.

APPLICABILITY

- People or systems must find, interpret, compare, exchange, update, publish, retain, or retire information.
- Multiple domains, products, services, organizations, languages, jurisdictions, roles, or applications use related information.
- Information affects decisions, rights, safety, privacy, security, accessibility, compliance, service, operations, or trust.
- Search, navigation, recommendations, generated answers, dashboards, records, content, APIs, events, or knowledge systems are designed or changed.
- Terms, schemas, metadata, provenance, access, retention, or presentation vary by context.

NON-APPLICABILITY CONDITIONS

- The work has no material effect on meaning, labels, context, discoverability, access, representation, provenance, retention, user journey, or lifecycle.
- An approved information architecture covers the exact domain, audience, version, environment, and change, with impact analysis showing no material effect.

- The artifact is exploratory and makes no claim that its vocabulary, navigation, accessibility, permissions, or user interpretation is approved or effective.

The record must identify scope, baseline, owner, evidence date, audience, assumptions, and reopening condition. A simple screen, document, or database table is not automatically exempt when it changes meaning or access.

PREREQUISITES AND DEPENDENCIES

- Sections 016, 019, and 020 provide domain meaning, requirements, system boundaries, information obligations, verification, validation, and lifecycle context.
- Sections 021-029 provide enterprise, solution, systems, domain, business, capability, process, functional, and operational context.
- Data ownership, privacy, security, records, accessibility, localization, and legal constraints are known or marked unresolved.
- Human authorities for definitions, classification, access, publication, correction, retention, and acceptance are attributable.

REQUIRED INPUTS

- Objectives, audiences, affected parties, user and operator journeys, domains, contexts, languages, jurisdictions, environments, and lifecycle stages.
- Domain terms, business rules, requirements, process and functional models, data inventories, records schedules, policies, contracts, and standards.
- Current content, taxonomies, catalogs, schemas, metadata, search behavior, navigation, interfaces, logs, support records, user research, and accessibility evidence.
- Source systems, owners, provenance, sensitivity, permissions, quality, freshness, retention, archival, deletion, and evidence obligations.
- AI retrieval, generation, ranking, summarization, citation, context, prompt, tool, provider, evaluation, and human-review requirements where applicable.

HUMAN RESPONSIBILITIES AND AUTHORITY

The accountable human owner retains authority over domain meaning, definitions, classification, audience obligations, rights, publication, access, privacy, retention, deletion, correction, exceptions, and acceptance.

- Confirm information scope, audiences, purposes, contexts, boundaries, terminology, and non-goals.
- Assign owners for meaning, source, quality, provenance, access, presentation, records, and lifecycle.
- Resolve conflicting definitions, translations, classifications, accessibility needs, privacy interests, and stakeholder interpretations.
- Approve canonical terms, taxonomies, access rules, publication claims, retention, deletion, and material generated-information behavior.
- Ensure users, operators, domain experts, affected parties, privacy, security, legal, records, and accessibility specialists participate as consequence requires.
- Retain the ability to correct, withdraw, quarantine, restrict, supersede, or retire information.

AI RESPONSIBILITIES AND BOUNDARIES

AI may inventory information, compare vocabularies, propose taxonomies, identify duplicates, map concepts, analyze journeys, draft metadata, test findability, and recommend context or presentation within explicit authority.

AI must preserve source identity, provenance, version, audience, context, uncertainty, and permission; distinguish source information from inference and generation; expose conflicts and missing definitions; and escalate material semantic or rights questions.

AI must not invent canonical meaning, cite inaccessible or nonexistent sources, disclose restricted information, alter records, publish consequential claims, or treat retrieval, ranking, fluency, or user engagement as proof of accuracy or comprehension.

APPLICABLE EXISTING AGENT ROLES

- Architect: defines information boundaries, relationships, ownership, flows, context, and lifecycle.
- Researcher: gathers domain, user, content, records, legal, accessibility, and source evidence.
- Analyst: compares meanings, structures, usage, findability, quality, and impacts.
- Observer: records actual language, navigation, search behavior, and information use.
- Documentation Writer: maintains definitions, taxonomies, metadata, content structures, and explanations.
- User Experience Reviewer: evaluates comprehension, findability, accessibility, language, burden, and correction.
- Governance Reviewer: checks authority, classification, access, records, provenance, privacy, and publication.
- Security Auditor: examines disclosure, trust boundaries, access, integrity, and abuse paths.
- Verifier: checks information claims, provenance, version, permission, and reproducibility.
- Critic: challenges ambiguity, hidden assumptions, duplicate meaning, false precision, and exclusion.
- Memory Curator: preserves canonical terms, supersession, decisions, sources, and continuity.
- Implementer: realizes approved information structures and controls without changing meaning unilaterally.
- Tester: validates navigation, search, labels, access, context, error, correction, and lifecycle behavior.

Role selection remains proportional to consequence. Existing role contracts remain controlling.

REQUIRED ACTIVITIES

1. Establish audiences, purposes, contexts, domains, scope, jurisdictions, languages, environments, and lifecycle.
2. Inventory information sources, products, records, views, interfaces, repositories, generated outputs, and existing structures.
3. Capture canonical terms, definitions, synonyms, identifiers, relationships, translations, classification, and unresolved ambiguity.
4. Model information journeys and flows through people, processes, functions, applications, data stores, interfaces, and evidence.
5. Define information types, metadata, provenance, ownership, sensitivity, freshness, quality, retention, access, and lifecycle states.
6. Design taxonomies, hierarchies, facets, labels, navigation, search, filtering, recommendations, and contextual views.
7. Define representations for user, operator, machine, audit, accessibility, localization, offline, and degraded contexts.
8. Allocate responsibility for meaning, source, creation, transformation, publication, correction, approval, access, retention, and retirement.
9. Assess duplicate, conflicting, stale, inaccessible, misleading, overexposed, orphaned, and untraceable information.
10. Define generated, summarized, ranked, translated, or inferred information behavior, provenance, review, correction, and withdrawal.
11. Validate structures with users, operators, domain experts, owners, maintainers, support, privacy, security, records, legal, and accessibility specialists.
12. Test normal, ambiguous, denied, stale, missing, conflicting, multilingual, inaccessible, offline, generated, and recovery journeys.
13. Baseline the information architecture and hand off definitions, structures, flows, controls, tests, evidence, and open gaps.

HUMAN-AI INTERACTION REQUIREMENTS

- Show whether information is source content, transformed content, inference, recommendation, generation, or verified record.
- Expose source, timestamp, scope, provenance, confidence limits, permissions, freshness, and unresolved conflict where material.
- Require human review for canonical definitions, rights-affecting classification, restricted disclosure, consequential publication, and irreversible deletion.
- Let users correct labels, context, identity, interpretation, and generated content without erasing source history or dissent.
- Preserve accessible, non-AI, status, citation, correction, appeal, and recovery paths when retrieval or generation fails.

- Never let a fluent answer, ranking, engagement signal, or citation count substitute for source and meaning verification.

ARCHITECTURE AND SYSTEM IMPLICATIONS

- Represent terms, concepts, identifiers, taxonomies, relationships, metadata, provenance, permissions, views, translations, records, and lifecycle states as versioned attributable artifacts.
- Separate information meaning from storage schema, interface presentation, query syntax, model embedding, and generated wording.
- Preserve lineage across source, transformation, derivative, publication, retrieval, generation, correction, supersession, archival, and deletion.
- Make context, audience, jurisdiction, language, accessibility, sensitivity, freshness, and permission part of information selection and presentation.
- Ensure search, navigation, ranking, caching, replication, offline views, and generated context cannot bypass access, retention, or provenance controls.
- Trigger impact analysis when terms, structures, classifications, schemas, source systems, policies, permissions, languages, or lifecycle rules change.

OWNER COCKPIT AND USER-CONTROL REQUIREMENTS

- Show information baseline, audience, context, vocabulary version, source, freshness, permission, and lifecycle state.
- Let owners inspect definitions, lineage, dependencies, access, uses, representations, conflicts, evidence, and open decisions.
- Show available, restricted, prohibited, generated, inferred, stale, conflicting, inaccessible, and correction actions.
- Provide authorized controls to approve, correct, classify, restrict, publish, withdraw, supersede, archive, and retire.
- Explain why information was shown, omitted, ranked, transformed, translated, generated, or blocked where action or trust depends on it.
- Export the information structure, definitions, lineage, decisions, evidence, dissent, and continuity record.

REQUIRED OUTPUTS AND ARTIFACTS

- Information Architecture charter, audience and context map, and scope record.
- Canonical vocabulary, taxonomy, ontology references, identifiers, metadata, labels, translation, and classification rules.
- Information-flow, journey, navigation, search, representation, accessibility, and content-structure models.
- Ownership, provenance, lineage, quality, freshness, access, privacy, records, retention, and lifecycle registers.
- Generated-information, retrieval, ranking, citation, correction, appeal, and withdrawal model where applicable.
- Traceability to requirements, domains, processes, functions, data, applications, controls, tests, evidence, and acceptance.
- Ambiguity, conflict, assumption, decision, dissent, exception, and reopening register.

EVIDENCE REQUIREMENTS

- Each information claim identifies source, owner, version, scope, audience, context, date, method, permission, and limitation.
- A label, taxonomy, search result, citation, diagram, or user statement proves only its bounded claim and not universal comprehension or correctness.
- Definitions and classifications require authoritative owner, rationale, effective date, applicability, review condition, and supersession path.
- Findability, comprehension, accessibility, and usability evidence identifies population, context, method, coverage, tasks, limitations, and result.
- Generated or transformed information records source material, transformation, model or tool, configuration, reviewer, output, and limitations proportionate to consequence.
- Missing, stale, conflicting, inaccessible, restricted, or withdrawn evidence must remain visible and constrain status.

VERIFICATION AND ACCEPTANCE TESTS

- Trace representative outcomes from domain and requirement through term, information type, source, transformation, view, user action, evidence, and lifecycle.
- Search for an authorized item, an ambiguous term, a synonym, a translation, a restricted item, a stale item, and a superseded item and verify expected behavior.
- Introduce duplicate definitions, conflicting sources, missing provenance, incorrect labels, broken links, unauthorized access, and stale cache and verify detection or safe handling.
- Verify accessibility, localization, reading order, keyboard or alternative input, plain-language, and nonvisual information paths as applicable.
- Disable a source, index, model, provider, network, citation service, or permission service and verify safe degradation and truthful status.
- Correct, withdraw, supersede, archive, and delete information and verify lineage, permissions, caches, records, and user notification behavior.
- Independently reproduce representative information claims from the baseline and evidence package.

FAILURE STATES AND ANTI-PATTERNS

- Treating discoverability, fluency, search rank, citation, or visual polish as proof of truth or comprehension.
- Mixing source, interpretation, inference, recommendation, and generation without labels or provenance.
- Creating a universal taxonomy that ignores domain, audience, language, jurisdiction, accessibility, or lifecycle context.
- Letting navigation, cache, index, analytics, or generated context bypass permission, retention, or deletion controls.
- Hiding ambiguity, contradictory sources, unavailable information, or affected-party language to create false simplicity.
- Making correction, appeal, withdrawal, or non-AI access impossible.

RECOVERY AND RECONSIDERATION

- On semantic conflict, provenance loss, permission uncertainty, stale source, harmful disclosure, or inaccessible path, restrict affected use and preserve the prior record.
- Name the blocker, safe uses that continue, owner, authority, evidence, review date, and affected audiences.
- Use approved quarantine, source substitution, prior version, manual review, restricted presentation, rollback, withdrawal, or correction paths.
- Reopen the architecture when domain meaning, audience, law, policy, source, classification, language, model, or lifecycle changes invalidate assumptions.

COMPLETION CRITERIA

- Scope, audiences, contexts, definitions, structures, sources, lineage, ownership, permissions, accessibility, lifecycle, and evidence are explicit.
- Ambiguous, conflicting, generated, stale, restricted, denied, and recovery conditions are addressed or justified.
- Users and owners can inspect, correct, restrict, withdraw, and recover information within authority.
- Verification and acceptance state exact baselines, evidence, limits, open gaps, and validity.
- The section remains CONTENT-COMPLETE until source reconciliation, hostile review, and owner acceptance are separately completed.

HANDOFF AND CONTINUITY REQUIREMENTS

- Preserve vocabulary, taxonomy, identifiers, contexts, audiences, lineage, permissions, sources, decisions, dissent, versions, and open gaps.
- Hand off information contracts to Data, Application, API, Content, Knowledge, Search, AI, Privacy, Security, Records, Accessibility, Testing, and Operations work.
- Record changes requiring reopening Domain, Process, Functional, Data, Application, Privacy, Governance, Content, Knowledge, or AI sections.
- State next safe action, blocked actions, authority, evidence needed, and continuation point.

RELATED CODEX SECTIONS

- 016 Domain Analysis; 019 Requirements Engineering; 020 Systems Engineering.
- 021-029 Enterprise through Operational Architecture.
- 031-060 Data, Application, Software, Platform, Infrastructure, Cloud, Edge, Distributed, Network, Communications, Integration, API, Event-Driven, Runtime, Security, Identity, Privacy, Governance, Compliance, Safety, Reliability, Resilience, Accessibility, Content, Knowledge, AI, Machine Learning, Robotics, Embedded Systems, and Hardware Architecture.
- 151-188 Data and AI engineering through Human-in-the-Loop Operations.
- 225 Records Management; 247 Accessibility Testing; 250 Localization Testing; 391-395 Data Retention through Data Deletion.

SOURCE BASIS AND UNRESOLVED SOURCE GAPS

- This draft uses the Canonical Section Contract, shared governance files, Sections 001-029, Owner Cockpit Requirements, Status and Evidence Lexicon, and the Remaining-Section Integration Roadmap.
- No universal supplied vocabulary, taxonomy, information classification, records schedule, accessibility standard, retention rule, or sector-specific disclosure regime applies to every future system.
- Actual domain authorities, source inventories, user populations, translations, legal obligations, permissions, and deletion requirements remain unresolved until supplied and verified.

031

SECTION 031 - DATA ARCHITECTURE

CODEX/03_SECTIONS/031_DATA_ARCHITECTURE.txt

SHA-256 2143aee856d982cfab3f9d912ecc8236785f1ce291fa1d89fbc818f1fadc1e6b

18,962 BYTES / PRESERVED SOURCE IDENTITY

SECTION 031 — Data Architecture

STATUS: CONTENT-COMPLETE

PURPOSE

Define the governed structure and lifecycle of data: what data exists, what it means, where it comes from, how it moves and changes, who owns it, how it is protected and qualified, and when it is retained, archived, migrated, deleted, or destroyed.

Data Architecture turns information obligations, domain rules, processes, functions, requirements, and operational needs into data entities, relationships, flows, stores, contracts, controls, quality rules, lineage, and lifecycle behavior. It is not a database product choice and does not prove that data is accurate, complete, available, lawfully held, or safe in runtime.

REQUIRED OUTCOME

- A bounded data landscape with domains, entities, events, records, sources, consumers, owners, custodians, authorities, sensitivity, jurisdictions, and lifecycle.
- Conceptual, logical, and physical data views with explicit mappings and intentional variation.
- Data-flow, lineage, provenance, transformation, synchronization, replication, integration, and dependency models.
- Data quality, validation, reconciliation, observability, access, privacy, security, resilience, backup, restore, retention, archival, migration, and deletion obligations.
- Versioned data contracts and ownership for meaning, schema, quality, access, evidence, change, and retirement.
- Traceability from objectives, requirements, domains, processes, functions, information, applications, controls, tests, evidence, decisions, and outcomes.

WHY THIS MATTERS

Data is often treated as inert material when it is actually a source of decisions, obligations, identity, automation, records, and harm. Wrong lineage, stale values, incompatible schemas, missing consent, weak deletion, or unowned quality can make an otherwise functional system untrustworthy.

Data Architecture makes data behavior explicit across its full life rather than at one storage boundary. It preserves the distinction between a value and its interpretation, a copy and its source, a record and a cache, a generated feature and an observed fact, and a successful transfer and a valid outcome.

APPLICABILITY

- A system creates, receives, stores, transforms, exchanges, analyzes, publishes, generates, retains, archives, migrates, or deletes data.
- Data influences identity, rights, safety, privacy, security, finance, compliance, operations, models, decisions, or user experience.
- Multiple systems, domains, jurisdictions, environments, suppliers, or lifecycle stages share or derive data.
- Data quality, freshness, lineage, availability, confidentiality, integrity, portability, retention, or deletion is material.
- AI, analytics, search, recommendation, simulation, or automation uses data beyond a single bounded transient operation.

NON-APPLICABILITY CONDITIONS

- The change has no material effect on data meaning, source, schema, flow, quality, access, lifecycle, evidence, or user or system outcomes.
- An approved data architecture covers the exact scope, data classes, environment, version, population, and change, with impact analysis showing no material effect.
- The artifact is exploratory and is clearly labeled as proposed, synthetic, sample, or unverified rather than as production data design or evidence.

The decision must identify scope, baseline, owners, evidence date, assumptions, and reopening condition. A small table, payload, file, or model input is not exempt when it carries material meaning or consequence.

PREREQUISITES AND DEPENDENCIES

- Sections 016, 019, 020, and 030 provide domain, requirements, systems, information, lifecycle, evidence, and context obligations.
- Sections 021-029 provide architecture, capability, process, functional, and operational ownership and boundaries.
- Privacy, security, records, legal, quality, accessibility, resilience, and AI governance constraints are known or explicitly open.
- Human authorities for data meaning, access, quality, retention, deletion, migration, and acceptance are attributable.

REQUIRED INPUTS

- Objectives, domains, entities, data subjects, users, operators, jurisdictions, environments, sensitivity, purposes, and lifecycle.
- Business rules, requirements, information structures, processes, functional contracts, application boundaries, interfaces, events, and controls.
- Source inventories, schemas, metadata, lineage, data dictionaries, catalogs, records schedules, access policies, quality results, incidents, and migration history.
- Availability, integrity, confidentiality, freshness, accuracy, completeness, timeliness, retention, portability, recovery, and deletion obligations.
- AI, analytics, model, feature, training, evaluation, synthetic, prompt, retrieval, and generated-data requirements where applicable.

HUMAN RESPONSIBILITIES AND AUTHORITY

The accountable human owner retains authority over purpose, collection, classification, consent or other lawful basis, sharing, quality thresholds, access, retention, correction, migration, deletion, risk, and acceptance.

- Confirm data scope, purpose, subjects, domains, jurisdictions, environments, sensitivity, and non-goals.
- Assign data owners, custodians, stewards, processors, consumers, quality authorities, privacy and security authorities, and lifecycle owners.
- Approve schemas, canonical sources, transformations, sharing, retention, archival, deletion, migration, and exceptions.
- Resolve conflicting meanings, quality thresholds, access interests, privacy rights, operational needs, and evidence requirements.
- Ensure affected people and qualified domain, privacy, security, records, legal, accessibility, and safety specialists participate as needed.
- Retain the ability to restrict, quarantine, correct, restore, migrate, withdraw, delete, or destroy data within authority.

AI RESPONSIBILITIES AND BOUNDARIES

AI may inventory data, compare schemas, identify lineage gaps, propose models or quality rules, analyze distributions, draft mappings, detect anomalies, and generate migration or test ideas within authorized data access.

AI must preserve data provenance, permission, sensitivity, purpose, version, transformations, uncertainty, and representativeness; identify missing or biased evidence; and stop or escalate when use, sharing, retention, or deletion authority is unclear.

AI must not invent data, infer permission, expose restricted values, silently change records, treat synthetic or generated values as observed facts, approve data sharing, or declare quality from a sample, score, or successful pipeline run alone.

APPLICABLE EXISTING AGENT ROLES

- Architect: defines data boundaries, ownership, flows, contracts, controls, lineage, and lifecycle.
- Analyst: evaluates data relationships, quality, dependencies, transformations, tradeoffs, and impact.
- Researcher: gathers source, domain, legal, user, operational, and data-use evidence.
- Observer: records actual data behavior, flows, failures, quality, and user or system effects.
- Governance Reviewer: checks purpose, authority, access, retention, records, privacy, and policy.
- Security Auditor: examines confidentiality, integrity, identity, access, secrets, abuse, and recovery.
- Verifier: independently checks lineage, quality, contracts, controls, and evidence.
- Tester: exercises schema, quality, access, migration, deletion, recovery, and compatibility behavior.
- Documentation Writer: maintains models, dictionaries, catalogs, contracts, and evidence records.
- Memory Curator: preserves source, transformations, versions, decisions, and continuity.
- Critic: challenges false precision, missing lineage, hidden copies, biased samples, and unowned quality.
- Implementer: realizes approved data structures, pipelines, controls, and migrations.
- Optimizer: improves data performance and cost without weakening meaning, quality, rights, or controls.
- Red Team: probes exfiltration, poisoning, corruption, bypass, re-identification, and deletion failures.

Role selection remains proportional to consequence. Existing role contracts remain controlling.

REQUIRED ACTIVITIES

1. Establish data purpose, subjects, domains, scope, environments, jurisdictions, sensitivity, owners, and lifecycle.
2. Inventory data sources, entities, records, events, copies, caches, derivatives, stores, pipelines, consumers, and external parties.
3. Define canonical meanings, identifiers, keys, relationships, metadata, classifications, and ownership.
4. Model conceptual, logical, and physical structures and their controlled mappings.
5. Map data creation, collection, validation, transformation, transfer, storage, use, publication, backup, archival, and deletion.
6. Define data contracts for schema, semantics, identity, authority, quality, timing, ordering, encoding, errors, compatibility, and evidence.
7. Establish quality dimensions, rules, thresholds, reconciliation, quarantine, correction, stewardship, and invalidation behavior.
8. Define lineage, provenance, source authority, derivation, replication, cache, feature, synthetic, and generated-data status.
9. Allocate confidentiality, integrity, availability, privacy, access, retention, backup, restore, and deletion controls.
10. Assess bias, missingness, drift, leakage, duplication, corruption, re-identification, overcollection, and unauthorized reuse.
11. Design migration, coexistence, rollback, reconciliation, archival, retention, disposal, and evidence preservation.
12. Validate with data owners, domain experts, users, operators, privacy, security, records, legal, accessibility, and AI specialists.
13. Test normal, malformed, stale, duplicate, unauthorized, partial, incompatible, lost, corrupted, migrated, restored, and deleted data paths.
14. Baseline and hand off the data architecture with unresolved gaps and evidence limits.

HUMAN-AI INTERACTION REQUIREMENTS

- Show observed, sourced, transformed, inferred, synthetic, generated, cached, replicated, and verified data states distinctly.
- Expose source, lineage, purpose, permissions, sensitivity, freshness, quality, model or tool use, and retention status where material.
- Require human authorization for new purposes, restricted access, sharing, canonical-source changes, deletion exceptions, and consequential generated data.
- Let owners and affected people correct, restrict, appeal, export, withdraw, or request deletion within applicable authority.
- Keep non-AI, manual review, status, evidence, and recovery paths available when data services or models fail.
- Never let an anomaly score, schema match, benchmark, or pipeline completion become a universal quality claim.

ARCHITECTURE AND SYSTEM IMPLICATIONS

- Represent entities, records, events, schemas, metadata, lineage, provenance, contracts, quality, permissions, retention, and lifecycle as versioned records.
- Separate source data, derived data, cache, replica, backup, archive, synthetic data, feature, model artifact, and evidence copy.
- Make purpose, subject, sensitivity, jurisdiction, identity, authority, provenance, freshness, and retention enforceable at data boundaries.
- Design idempotency, ordering, reconciliation, versioning, compatibility, quarantine, correction, rollback, backup, restore, and deletion semantics.
- Ensure data access, replication, indexes, logs, caches, exports, backups, models, and generated contexts respect restriction and deletion obligations.
- Trigger impact analysis when meaning, schema, source, flow, owner, permission, purpose, quality rule, model, or retention changes.

OWNER COCKPIT AND USER-CONTROL REQUIREMENTS

- Show data baseline, source, purpose, owner, sensitivity, lineage, quality, freshness, access, retention, and lifecycle state.
- Let owners inspect consumers, transformations, copies, dependencies, incidents, quality failures, permissions, and evidence.
- Show available, restricted, prohibited, quarantined, stale, corrupted, synthetic, generated, retention, and deletion actions.
- Provide authorized approve, correct, restrict, quarantine, restore, migrate, export, archive, delete, and destroy controls.
- Explain why data was collected, transformed, retained, shown, hidden, shared, generated, or blocked.
- Export the data model, contracts, lineage, quality, decisions, evidence, dissent, and continuity record.

REQUIRED OUTPUTS AND ARTIFACTS

- Data Architecture charter, data landscape, domain and ownership map.
- Conceptual, logical, physical, event, state, flow, storage, and lineage models.
- Data dictionary, catalog, metadata, classification, provenance, source-of-truth, and data-contract records.
- Quality, observability, reconciliation, access, privacy, security, backup, restore, retention, archival, migration, and deletion designs.
- AI, analytics, feature, training, evaluation, synthetic, and generated-data governance where applicable.
- Traceability to requirements, information, processes, functions, applications, controls, tests, evidence, and acceptance.
- Quality, bias, assumption, risk, exception, decision, dissent, and reopening registers.

EVIDENCE REQUIREMENTS

- Each data claim identifies source, schema or version, purpose, population, scope, environment, time, method, owner, permission, and limitation.
- A sample, schema, pipeline success, checksum, dashboard, or model score proves only its bounded claim.
- Quality evidence identifies dimensions, population, sampling, rules, thresholds, missingness, uncertainty, result, and invalidation condition.
- Lineage evidence identifies source, transformations, tools, versions, operators, destinations, timing, and unresolved gaps.
- Migration and deletion evidence identifies scope, records, precondition, authority, procedure, result, exceptions, verification, and residual copies.
- AI or generated-data evidence identifies model, provider, version, data, context, tools, permissions, review, result, and limitations.
- Failed, partial, stale, contradictory, inaccessible, and missing evidence remains visible and constrains status.

VERIFICATION AND ACCEPTANCE TESTS

- Trace representative data from source and purpose through schema, transformation, storage, use, output, evidence, retention, and deletion.

- Introduce missing, duplicate, malformed, stale, unauthorized, poisoned, incompatible, corrupted, and out-of-order data and verify safe handling.
- Change a schema, definition, source, owner, permission, purpose, retention, quality rule, model, or provider and verify impact detection.
- Verify lineage, access, encryption or equivalent protection, retention, backup, restore, archival, deletion, and residual-copy behavior.
- Exercise migration, coexistence, rollback, reconciliation, partial completion, interrupted transfer, and recovery.
- Disable a source, catalog, quality service, network, key, model, or evidence store and verify safe status and fallback.
- Verify generated, synthetic, inferred, cached, and replicated data cannot be mistaken for authoritative source data.
- Independently reproduce representative quality, lineage, access, migration, and deletion claims.

FAILURE STATES AND ANTI-PATTERNS

- Treating a schema, successful load, checksum, dashboard, or clean sample as proof of data correctness.
- Confusing storage location with ownership, source authority, meaning, permission, retention, or deletion.
- Creating hidden copies in logs, caches, backups, exports, models, prompts, or support tools.
- Ignoring missingness, bias, drift, duplicate identity, incompatible versions, and common-cause corruption.
- Allowing AI to infer permission, alter records, publish sensitive values, or make generated data appear authoritative.
- Optimizing volume, latency, or cost while weakening data rights, provenance, quality, recoverability, or lifecycle control.

RECOVERY AND RECONSIDERATION

- On provenance loss, quality failure, unauthorized access, corruption, poisoning, schema break, retention uncertainty, or deletion failure, quarantine affected data and preserve evidence.
- Name the blocker, safe uses, affected subjects, owner, authority, containment, evidence, review time, and recovery path.
- Use approved source restoration, prior version, reconciliation, correction, reprocessing, rollback, restricted mode, manual review, or deletion procedures.
- Reassess downstream reports, decisions, models, records, notifications, permissions, and evidence before resuming.
- Reopen when domain meaning, purpose, law, source, schema, quality, model, population, or lifecycle changes invalidate the baseline.

COMPLETION CRITERIA

- Purpose, scope, subjects, meaning, owners, sources, structures, flows, lineage, quality, access, protection, retention, recovery, and deletion are explicit.
- Normal, malformed, stale, unauthorized, partial, corrupted, migrated, restored, and deleted states are addressed or justified.
- Data contracts, evidence, verification, user control, privacy, security, records, and lifecycle obligations are traceable.
- Exact baseline, validity, unresolved gaps, and owner acceptance state are recorded.
- The section remains CONTENT-COMPLETE until source reconciliation, hostile review, and owner acceptance are separately completed.

HANDOFF AND CONTINUITY REQUIREMENTS

- Preserve data identities, definitions, sources, lineage, mappings, contracts, quality, permissions, retention, decisions, evidence, dissent, and open gaps.
- Hand off to Application, Software, Platform, Infrastructure, Integration, API, Event, Security, Identity, Privacy, Records, AI, Testing, Operations, Migration, and Retirement work.
- Record changes requiring reopening Domain, Information, Functional, Application, Security, Privacy, Governance, AI, Testing, or Retirement sections.
- State the next safe action, blocked action, authority, evidence request, and exact continuation point.

RELATED CODEX SECTIONS

- 016 Domain Analysis; 019 Requirements Engineering; 020 Systems Engineering; 030 Information Architecture.
- 021-029 Enterprise through Operational Architecture; 032-044 Application through Runtime Architecture.
- 045-060 Security through Hardware Architecture.
- 151-188 Database and Data Engineering through Human-in-the-Loop Operations.
- 225 Records Management; 254 Data Testing; 285 Database Administration; 307 Data Operations; 391-395 Data Retention through Data Deletion.

SOURCE BASIS AND UNRESOLVED SOURCE GAPS

- This draft uses the Canonical Section Contract, shared governance files, Sections 001-030, Owner Cockpit Requirements, Status and Evidence Lexicon, and the Remaining-Section Integration Roadmap.
- No universal supplied data classification, quality standard, records schedule, privacy regime, retention rule, storage technology, or sector-specific data authority applies to every future system.
- Actual data inventories, lawful purposes, subjects, source authorities, quality thresholds, contracts, jurisdictional constraints, and deletion evidence remain unresolved until supplied and verified.

032

SECTION 032 - APPLICATION ARCHITECTURE

CODEX/03_SECTIONS/032_APPLICATION_ARCHITECTURE.txt

SHA-256 86bf5b0f8f6a189b50753b9777e137144be0692eb140bcd1ed6b39f31bb054da

20,162 BYTES / PRESERVED SOURCE IDENTITY

SECTION 032 — Application Architecture

STATUS: CONTENT-COMPLETE

PURPOSE

Define the governed structure of application systems and application boundaries: which applications exist, what responsibilities they hold, how they interact with people and other systems, how they are deployed and operated, and how they change across their lifecycle.

Application Architecture translates functional, information, data, process, operational, security, and user obligations into application responsibilities, boundaries, modules, services, interfaces, state, dependencies, user surfaces, deployment units, and lifecycle controls. It is not a product catalog or implementation proof.

REQUIRED OUTCOME

- An application landscape with stable identities, purposes, owners, users, environments, responsibilities, dependencies, criticality, data, interfaces, and lifecycle state.
- Explicit application boundaries and allocation of functions, information, data, workflows, controls, and user journeys.
- Application context, interaction, integration, state, deployment, configuration, observability, support, and recovery views.
- Application contracts for interfaces, identity, authorization, data, events, timing, errors, compatibility, audit, and evidence.
- Traceability from requirements, capabilities, processes, functions, information, data, security, privacy, operations, tests, evidence, and retirement.
- A migration, coexistence, versioning, deprecation, support, and retirement model that preserves authority and continuity.

WHY THIS MATTERS

Applications become dangerous when boundaries are implicit. Responsibilities are duplicated, data is copied without ownership, users are forced across inconsistent workflows, and failures propagate because no one defined state, contracts, or recovery. A catalog alone cannot show whether an application is fit for purpose, secure, accessible, supported, or operational.

Application Architecture keeps behavior and ownership visible across user-facing and back-office surfaces, services, jobs, agents, integrations, and supporting applications. For AI-enabled applications, it makes model, provider, prompt, tool, data, human-review, and fallback behavior part of the application boundary rather than hidden implementation detail.

APPLICABILITY

- One or more applications, services, clients, jobs, agents, or user surfaces realize material functions or information journeys.
- Application boundaries, integration, state, identity, data, user experience, deployment, support, or lifecycle affect outcomes.
- Applications are composed, replaced, integrated, migrated, exposed to external users, or operated across environments.
- Failure, unauthorized action, inaccessible behavior, data loss, dependency drift, or unsupported versions could cause material consequence.

NON-APPLICABILITY CONDITIONS

- The change has no material effect on application responsibility, boundary, interface, state, data, identity, user journey, operation, evidence, or lifecycle.

- An approved application architecture covers the exact scope, version, environment, population, and change, and impact analysis confirms no material effect.
- The artifact is exploratory and makes no claim that an application is approved, implemented, deployed, secure, accessible, supported, or accepted.

The determination must identify the baseline, scope, owner, evidence date, assumptions, and reopening trigger. A single executable or vendor package is not automatically exempt when it carries material responsibility.

PREREQUISITES AND DEPENDENCIES

- Sections 019-020 provide requirements, system boundary, functions, interfaces, lifecycle, verification, validation, transition, and recovery context.
- Sections 021-031 provide enterprise, solution, systems, domain, business, capability, process, functional, operational, information, and data context.
- User, operator, security, privacy, accessibility, reliability, support, deployment, and retirement obligations are known or marked unresolved.
- Human authorities for application ownership, release, data, access, risk, acceptance, and retirement are attributable.

REQUIRED INPUTS

- Objectives, users, operators, affected parties, journeys, environments, operating modes, jurisdictions, and lifecycle horizon.
- Requirements, functions, processes, information and data contracts, domain rules, controls, service levels, risks, and acceptance criteria.
- Application inventory, source repositories, dependencies, runtime environments, interfaces, configurations, credentials, logs, incidents, support, and deployment records.
- Identity, authorization, privacy, security, accessibility, reliability, resilience, observability, backup, migration, deprecation, and retirement obligations.
- AI models, providers, prompts, tools, data, policies, autonomy, human review, evaluation, fallback, monitoring, and revocation requirements where applicable.

HUMAN RESPONSIBILITIES AND AUTHORITY

The accountable human owner retains authority over application purpose, boundaries, architecture, data use, access, release, risk, user impact, service commitments, exceptions, support, and retirement.

- Confirm application scope, responsibilities, users, environments, modes, non-goals, and criticality.
- Assign application owner, product or service owner, data owner, security, privacy, accessibility, operations, support, and retirement authorities.
- Approve boundaries, integrations, releases, migrations, provider use, automation, exceptions, risk treatments, and decommissioning.
- Resolve conflicts among usability, cost, performance, security, privacy, safety, accessibility, reliability, resilience, maintainability, and continuity.
- Ensure affected users, operators, maintainers, support, and qualified specialists review material application behavior.
- Retain the ability to stop, narrow, revoke, roll back, isolate, restore, replace, or retire an application.

AI RESPONSIBILITIES AND BOUNDARIES

AI may inventory applications, map dependencies, analyze traces and interfaces, propose boundaries, draft contracts, identify drift, generate test cases, and recommend migration or remediation within explicit authority.

AI must preserve artifact identity, version, environment, source, configuration, evidence, and uncertainty; distinguish application behavior from proposed design; and escalate when access, data, safety, security, privacy, user impact, or release authority is unresolved.

AI must not invent application capability, claim deployment or support, change production configuration, approve release, expose secrets, alter user rights, or treat a successful build, demo, test, or model response as application acceptance.

APPLICABLE EXISTING AGENT ROLES

- Architect: defines application boundaries, responsibilities, interfaces, dependencies, state, deployment, and lifecycle.
- Analyst: evaluates application behavior, alternatives, dependencies, cost, risk, and impacts.
- Researcher: gathers user, domain, technical, operational, supplier, and current-state evidence.
- Implementer: realizes approved application structure and reports deviations.
- Refactorer: improves internal structure while preserving approved contracts and behavior.
- Tester: validates interfaces, workflows, failure, security, accessibility, performance, recovery, and compatibility.
- Verifier: independently checks application claims, versions, configurations, evidence, and acceptance.
- Security Auditor: examines identity, authorization, secrets, dependencies, data, attack surface, and recovery.
- User Experience Reviewer: evaluates user, operator, accessibility, explanation, correction, and recovery journeys.
- Governance Reviewer: checks authority, release, records, licensing, risk, and status.
- Planner: sequences design, implementation, integration, transition, support, and retirement.
- Documentation Writer: maintains application catalogs, contracts, runbooks, decisions, and limitations.
- Memory Curator: preserves versions, dependencies, decisions, evidence, and continuity.
- Critic: challenges boundary leakage, hidden coupling, false completeness, and unsupported claims.
- Optimizer: improves measured behavior without weakening protected outcomes or controls.
- Red Team: probes application misuse, integration, privilege, failure, and recovery paths.

Role selection remains proportional to consequence. Existing role contracts remain controlling.

REQUIRED ACTIVITIES

1. Establish application purpose, users, scope, responsibilities, environments, modes, criticality, and lifecycle.
2. Inventory applications, modules, services, jobs, agents, clients, packages, data stores, providers, interfaces, and dependencies.
3. Define application boundaries and allocate functions, processes, information, data, controls, user journeys, and operations.
4. Model application context, interaction, sequence, state, deployment, configuration, identity, trust, and failure domains.
5. Define interfaces and application contracts for inputs, outputs, identity, authority, schema, timing, errors, retries, compatibility, and evidence.
6. Define user, operator, administrative, support, API, batch, event, offline, and recovery surfaces.
7. Address security, privacy, safety, accessibility, reliability, resilience, observability, support, maintainability, and cost obligations.
8. Assess dependency, version, provider, license, configuration, secret, data, runtime, and supply-chain risks.
9. Define build, release, deployment, configuration, promotion, rollback, migration, coexistence, support, deprecation, and retirement behavior.
10. Define AI and automation boundaries, model or provider identity, tool permissions, review, fallback, monitoring, and revocation where applicable.
11. Validate with owners, users, operators, maintainers, support, domain, data, security, privacy, accessibility, and operations specialists.
12. Test normal, invalid, unauthorized, concurrent, degraded, offline, dependency-loss, migration, rollback, recovery, and retirement paths.
13. Baseline and hand off application responsibilities, contracts, evidence, open gaps, and reopening conditions.

HUMAN-AI INTERACTION REQUIREMENTS

- Identify whether an application action is performed by a human, rule, automation, AI model, agent, external service, or hybrid.
- Show application, model, provider, prompt, policy, tool, data, authority, review, and evidence identity for consequential generated behavior.

- Require authorized human review for release, privilege, data-use, externally binding, rights-affecting, safety, privacy, or irreversible actions.
- Keep status, evidence, correction, non-AI, manual fallback, stop, and recovery paths available during AI or provider failure.
- Expose assumptions, uncertainty, alternatives, limitations, and evidence that would invalidate a recommendation.
- Never allow conversational convenience, administrative access, retry, import, or recovery to bypass application controls.

ARCHITECTURE AND SYSTEM IMPLICATIONS

- Represent application identities, responsibilities, boundaries, modules, services, clients, jobs, agents, interfaces, states, configurations, dependencies, evidence, and lifecycle as versioned artifacts.
- Separate application definition, source, build artifact, deployed instance, runtime state, user data, logs, cache, backup, and evidence.
- Define application contracts with identity, authority, schema, timing, ordering, quality, errors, retry, idempotency, compatibility, audit, observability, and ownership.
- Make authentication, authorization, privacy, accessibility, safety, resilience, and recovery enforceable across UI, API, batch, event, administrative, and emergency paths.
- Preserve independent stop, diagnosis, evidence, support, fallback, rollback, and recovery capabilities.
- Trigger impact analysis for changes to requirements, functions, data, interfaces, dependencies, providers, identity, policies, environments, or configurations.

OWNER COCKPIT AND USER-CONTROL REQUIREMENTS

- Show application baseline, version, environment, owner, status, dependencies, data scope, authority, lifecycle, and evidence freshness.
- Let owners navigate from requirement, process, function, information, or data to application boundary, interface, deployment, test, evidence, and incident.
- Show available, prohibited, consequential, awaiting-approval, degraded, failed, unverified, and recovery actions.
- Expose acting identity, role, authority, tools, data boundary, user impact, and approval requirement.
- Provide authorized approve, deny, release, pause, revoke, isolate, roll back, restore, migrate, supersede, and retire controls.
- Explain what is designed, built, deployed, running, measured, verified, accepted, degraded, or unknown.
- Export the application architecture, contracts, dependency map, configuration identity, evidence, decisions, dissent, and continuity record.

REQUIRED OUTPUTS AND ARTIFACTS

- Application Architecture charter, landscape, ownership, criticality, and lifecycle catalog.
- Application context, boundary, responsibility, user journey, interaction, state, deployment, trust, and dependency models.
- Application interface, data, event, identity, authorization, configuration, observability, and support contracts.
- Application security, privacy, accessibility, safety, reliability, resilience, backup, restore, migration, deprecation, and retirement model.
- Build, release, deployment, promotion, rollback, coexistence, support, incident, and operational-readiness handoff.
- Human-AI application participation, autonomy, review, provider, tool, data, monitoring, fallback, and revocation model.
- Traceability, evidence, decision, assumption, risk, exception, dissent, and reopening registers.

EVIDENCE REQUIREMENTS

- Every application claim identifies artifact, version, source, environment, configuration, scope, actor, method, date, result, and limitation.
- A source tree, build, deployment, demo, uptime signal, or test proves only its bounded claim and not general fitness or acceptance.
- Boundary and responsibility evidence identifies requirement, function, data, interface, owner, allocation rationale, and status.

- Interface evidence identifies endpoints, versions, schemas, identity, authority, timing, errors, compatibility, observed result, and limitations.
- Deployment and runtime evidence identifies artifact, configuration, environment, change, timing, telemetry, result, and rollback or recovery state.
- AI evidence identifies model, provider, version, context, data, tools, permissions, review, evaluation, and limitation.
- Failed, stale, partial, contradictory, insecure, inaccessible, and missing evidence constrains application status.

VERIFICATION AND ACCEPTANCE TESTS

- Trace representative requirements, processes, functions, information, and data through application boundaries, interfaces, deployments, tests, evidence, and retirement.
- Introduce duplicate, missing, unowned, over-broad, unreachable, incompatible, unauthorized, and undocumented application responsibilities and verify detection.
- Exercise user, operator, API, batch, event, administrative, offline, degraded, dependency-loss, rollback, recovery, migration, and retirement paths.
- Verify identity, authorization, data minimization, audit, privacy, accessibility, safety, observability, and recovery across every material entry path.
- Disable dependencies, credentials, providers, models, network, data sources, configuration, and evidence stores and verify safe behavior.
- Change a contract, schema, dependency, model, provider, configuration, role, policy, or environment and verify impact and evidence invalidation.
- Verify release, promotion, rollback, restore, migration, and deprecation procedures with recorded results.
- Independently reproduce representative application claims from the exact baseline and handoff package.

FAILURE STATES AND ANTI-PATTERNS

- Treating an application catalog, source repository, build, deployment, demo, or uptime metric as proof of correctness.
- Letting application boundaries follow team charts or vendor packaging instead of responsibility, data, authority, and outcome.
- Hiding shared databases, privileged paths, background jobs, manual work, configuration, secrets, or provider dependencies.
- Supporting the happy path while omitting invalid, unauthorized, concurrent, stale, offline, degraded, migration, and recovery behavior.
- Allowing AI or administrators to bypass user, data, identity, audit, accessibility, security, or recovery controls.
- Optimizing local application performance or cost while weakening user outcome, privacy, safety, reliability, maintainability, or retirement.

RECOVERY AND RECONSIDERATION

- On a critical defect, dependency loss, unauthorized state, data issue, security event, inaccessible path, evidence failure, or authority conflict, stop or isolate the affected application behavior and preserve state.
- Name the blocker, safe operation, authority, affected users, evidence, owner, communication path, and review condition.
- Use approved fallback, manual operation, feature reduction, provider substitution, rollback, restore, failover, migration, or retirement paths.
- Reassess security, privacy, accessibility, safety, data, commitments, support, and downstream evidence before resuming.
- Reopen the architecture when application boundaries, functions, data, interfaces, dependencies, environments, or lifecycle assumptions change.

COMPLETION CRITERIA

- Application purpose, boundary, responsibilities, users, data, interfaces, identity, deployment, dependencies, controls, operation, recovery, and lifecycle are explicit.
- Normal, boundary, failure, degraded, offline, migration, rollback, support, deprecation, and retirement behavior is addressed or justified.

- Application claims are traceable to requirements and bounded by evidence, verification, acceptance, and unresolved gaps.
- Owner controls and handoff artifacts support continuation without hidden conversation state.
- The section remains CONTENT-COMPLETE until source reconciliation, hostile review, and owner acceptance are separately completed.

HANDOFF AND CONTINUITY REQUIREMENTS

- Preserve application identities, boundaries, responsibilities, contracts, versions, dependencies, configurations, decisions, evidence, incidents, dissent, and open gaps.
- Hand off to Software, Platform, Infrastructure, Cloud, Edge, Distributed, Network, Communications, Integration, API, Event, Runtime, Security, Identity, Privacy, Testing, Operations, Maintenance, and Retirement work.
- Record changes requiring reopening Functional, Information, Data, Operational, Security, Privacy, Reliability, Resilience, Accessibility, Testing, or Governance sections.
- State next safe action, blocked action, authority, evidence request, exact baseline, and continuation point.

RELATED CODEX SECTIONS

- 019 Requirements Engineering; 020 Systems Engineering; 021-031 Enterprise through Data Architecture.
- 033-044 Software, Platform, Infrastructure, Cloud, Edge, Distributed, Network, Communications, Integration, API, Event-Driven, and Runtime Architecture.
- 045-060 Security through Hardware Architecture.
- 107-150 Software Engineering through Smart Contract Engineering.
- 255-300 Test, delivery, deployment, packaging, provisioning, and platform lifecycle sections.
- 301-406 Operations, support, maintenance, preservation, governance, and retirement sections.

SOURCE BASIS AND UNRESOLVED SOURCE GAPS

- This draft uses the Canonical Section Contract, shared governance files, Sections 001-031, Owner Cockpit Requirements, Status and Evidence Lexicon, and the Remaining-Section Integration Roadmap.
- The supplied Codex does not specify a universal application reference architecture, technology stack, licensing regime, deployment model, service-level regime, or sector-specific acceptance standard.
- Actual application inventory, owners, users, dependencies, environments, contracts, accessibility obligations, support model, security posture, and retirement authority remain unresolved until supplied and verified.

033

SECTION 033 - SOFTWARE ARCHITECTURE

CODEX/03_SECTIONS/033_SOFTWARE_ARCHITECTURE.txt

SHA-256 08479920f17a22ce35c364d97c63d80f2596701418275253539 1e3e9ee3451bd

10,601 BYTES / PRESERVED SOURCE IDENTITY

SECTION 033 — Software Architecture

STATUS: CONTENT-COMPLETE

PURPOSE

Define the structure, responsibilities, interfaces, state, quality attributes, and lifecycle of software that realizes approved application and functional obligations. Software Architecture keeps design intent distinct from source code, build output, runtime behavior, and evidence of acceptance.

REQUIRED OUTCOME

- A bounded software context with responsibilities, components, interfaces, data, control flow, state, dependencies, environments, owners, and lifecycle.
- Versioned design views and contracts covering composition, quality attributes, security, observability, failure, recovery, compatibility, and change.
- Traceability from requirements, functions, applications, data, interfaces, implementation, tests, evidence, operation, maintenance, and retirement.
- Explicit human and AI participation, authority, tool access, review, fallback, and revocation where software makes or supports consequential decisions.

WHY THIS MATTERS

Software structure determines how change, failure, privilege, data, and responsibility propagate. A clean module diagram can still conceal shared state, unsafe defaults, untestable coupling, unsupported dependencies, or a recovery path that depends on the failed component.

APPLICABILITY

- Software realizes material functions, application responsibilities, data handling, interfaces, automation, or user and operator journeys.
- Composition, dependency, concurrency, state, performance, security, accessibility, reliability, maintainability, or lifecycle has material consequence.
- Software is changed, reused, generated, integrated, deployed, migrated, or retired across environments.

NON-APPLICABILITY CONDITIONS

- The work is a documentation-only or reversible change with no material effect on software responsibility, contract, state, control, evidence, or lifecycle.
- An approved baseline covers the exact software, version, environment, and change and impact analysis confirms no material effect.
- The artifact is exploratory and is labeled proposed, simulated, or unverified.

The record must name the baseline, scope, owner, evidence date, assumptions, and reopening trigger.

PREREQUISITES AND DEPENDENCIES

- Sections 019-032 provide requirements, systems, information, data, operational, and application boundaries.
- Source, build, dependency, runtime, security, privacy, accessibility, testing, deployment, and support authorities are identifiable or recorded as gaps.

REQUIRED INPUTS

- Requirements, functions, application boundaries, data and interface contracts, users, operators, environments, risks, and acceptance criteria.
- Source repositories, build definitions, dependencies, configuration, runtime targets, performance budgets, security controls, observability, recovery, and retirement obligations.

- Model, provider, prompt, tool, data, evaluation, and human-review constraints where AI is embedded.

HUMAN RESPONSIBILITIES AND AUTHORITY

The accountable owner approves software boundaries, quality attributes, dependencies, security and privacy posture, release risk, exceptions, AI authority, and lifecycle decisions. Humans resolve tradeoffs, accept residual risk, approve material changes, and retain stop, isolate, rollback, restore, and retire authority.

AI RESPONSIBILITIES AND BOUNDARIES

AI may inspect authorized source and design artifacts, propose decomposition, identify coupling and gaps, draft contracts and tests, and analyze bounded traces. It must preserve versions and evidence, label inference and proposal, and escalate unresolved authority or safety issues. It must not invent behavior, expose secrets, approve releases, change production, or treat a build, benchmark, or generated code sample as acceptance.

APPLICABLE EXISTING AGENT ROLES

- Architect, Analyst, Researcher, Implementer, Refactorer, Tester, Verifier, Security Auditor, Governance Reviewer, Critic, Planner, Documentation Writer, Memory Curator, Optimizer, and Red Team.

Role selection is proportional to consequence; existing role contracts remain controlling.

REQUIRED ACTIVITIES

1. Establish scope, owners, environments, quality attributes, criticality, and lifecycle.
2. Inventory components, responsibilities, data, interfaces, dependencies, configurations, and trust boundaries.
3. Model composition, control flow, state, concurrency, resource use, failure, recovery, and compatibility.
4. Define contracts for identity, authority, schema, timing, errors, retry, idempotency, observability, and ownership.
5. Allocate requirements and cross-cutting controls to components and interfaces without creating authority by allocation.
6. Analyze dependency, supply-chain, configuration, secret, migration, and technical-debt risks.
7. Validate with users, operators, maintainers, security, privacy, accessibility, and domain specialists.
8. Test normal, invalid, concurrent, degraded, offline, recovery, migration, rollback, and retirement behavior.
9. Baseline the design and record unresolved decisions, dissent, evidence limits, and handoff.

HUMAN-AI INTERACTION REQUIREMENTS

- Identify whether each consequential software action is human, rule-based, automated, AI, external, or hybrid.
- Show model, provider, prompt, tool, data, authority, review, and evidence identity for generated behavior.
- Require human approval for rights, safety, privacy, security, release, privilege, or irreversible effects.
- Preserve status, stop, correction, manual fallback, evidence, and recovery paths when models or providers fail.

ARCHITECTURE AND SYSTEM IMPLICATIONS

- Keep design, source, build artifact, deployed instance, runtime state, logs, cache, backup, and evidence distinct.
- Represent components, contracts, state transitions, dependencies, versions, ownership, and lifecycle as attributable records.
- Enforce identity, authorization, validation, audit, accessibility, and recovery across UI, API, batch, event, administrative, and emergency paths.
- Define substitution, rollback, migration, deprecation, and compatibility semantics before implementation.

OWNER COCKPIT AND USER-CONTROL REQUIREMENTS

- Show baseline, version, environment, owner, dependencies, current state, authority, evidence freshness, and open risks.
- Expose consequential actions, required approvals, acting identity, data boundary, and recovery options.
- Permit authorized pause, revoke, isolate, rollback, restore, migrate, supersede, and retire actions.
- Export design views, contracts, traceability, decisions, evidence, dissent, and continuation state.

REQUIRED OUTPUTS AND ARTIFACTS

- Software context, composition, dependency, data-flow, control-flow, state, deployment, quality, security, and recovery views.
- Component and interface catalog, decision record, traceability matrix, risk and assumption register, test basis, and lifecycle handoff.
- Human-AI software participation, authority, evaluation, monitoring, fallback, and revocation record where applicable.

EVIDENCE REQUIREMENTS

- Each claim identifies artifact, version, environment, configuration, method, actor, date, result, and limitation.
- Source, build, test, benchmark, deployment, trace, and runtime evidence proves only its bounded claim.
- AI evidence identifies model, provider, context, data, tools, permissions, review, and limitations.
- Failed, stale, partial, contradictory, or missing evidence constrains status and remains visible.

VERIFICATION AND ACCEPTANCE TESTS

- Trace requirements and functions through components, contracts, builds, deployments, tests, runtime evidence, and retirement.
- Detect duplicate, orphaned, unowned, unreachable, incompatible, and over-privileged components.
- Exercise invalid, unauthorized, concurrent, stale, dependency-loss, degraded, rollback, restore, migration, and retirement paths.
- Disable models, providers, credentials, networks, data, configuration, and evidence stores and verify safe behavior.
- Reproduce representative claims from the exact baseline without hidden chat state.

FAILURE STATES AND ANTI-PATTERNS

- Treating source, build success, test pass, benchmark, or diagram as proof of general correctness.
- Hiding shared state, manual work, configuration, privilege, dependencies, or recovery coupling.
- Letting generated code or AI confidence become authorization.
- Optimizing local performance or cost while weakening security, privacy, accessibility, reliability, maintainability, or control.

RECOVERY AND RECONSIDERATION

- On defect, authority conflict, dependency loss, security event, data issue, or evidence failure, stop or isolate affected behavior and preserve state.
- Name the blocker, safe operation, owner, authority, evidence, communication, and review condition.
- Use approved fallback, feature reduction, substitution, rollback, restore, migration, or retirement paths.
- Reopen when requirements, interfaces, dependencies, models, environments, or lifecycle assumptions change.

COMPLETION CRITERIA

- Responsibilities, boundaries, contracts, state, dependencies, controls, quality, evidence, recovery, and lifecycle are explicit.
- Failure, degraded, migration, rollback, support, and retirement behavior is addressed or justified.
- Exact baseline, validity, unresolved gaps, verification, and acceptance authority are recorded.
- The section remains CONTENT-COMPLETE until source reconciliation, hostile review, and owner acceptance are separately completed.

HANDOFF AND CONTINUITY REQUIREMENTS

- Preserve identities, versions, contracts, decisions, evidence, dissent, incidents, open gaps, and next safe action.
- Hand off to Platform, Infrastructure, Testing, Security, Operations, Maintenance, Deployment, and Retirement sections.

- Record changes requiring reopening Application, Data, Interface, Security, Reliability, or Governance architecture.

RELATED CODEX SECTIONS

- 019-032 Requirements through Application Architecture; 034-044 Platform through Runtime Architecture.
- 045-060 Security through Hardware Architecture; 107-150 Software and systems engineering; 255-300 testing and delivery; 301-406 operations and lifecycle.

SOURCE BASIS AND UNRESOLVED SOURCE GAPS

- Based on the Canonical Section Contract, shared governance files, Sections 019-032, Owner Cockpit Requirements, Status and Evidence Lexicon, and local architecture baseline.
- No universal language, runtime, licensing, supply-chain, performance, accessibility, or sector standard is supplied. Actual technology, owners, contracts, threat model, support model, and acceptance evidence remain unresolved.

034

SECTION 034 - PLATFORM ARCHITECTURE

CODEX/03_SECTIONS/034_PLATFORM_ARCHITECTURE.txt

SHA-256 461edd0f774df80956fe0971629f64ab27fd814c04c95e75e5f06bc0f5c01a6b

9,781 BYTES / PRESERVED SOURCE IDENTITY

SECTION 034 — Platform Architecture

STATUS: CONTENT-COMPLETE

PURPOSE

Define the shared platform capabilities, boundaries, interfaces, tenancy, policy, runtime services, developer surfaces, and lifecycle that support applications without silently becoming application authority.

REQUIRED OUTCOME

- A platform landscape with capabilities, consumers, owners, trust boundaries, service contracts, quotas, versions, dependencies, and lifecycle.
- Explicit separation between platform defaults, tenant or application configuration, user data, and governance policy.
- Availability, isolation, security, accessibility, observability, migration, recovery, and support obligations with evidence limits.
- Traceability from application and software needs through platform services to infrastructure, operations, tests, and retirement.

WHY THIS MATTERS

Shared platforms amplify both benefit and failure. A default identity rule, quota, logging path, runtime service, or control-plane action can affect many applications at once. Platform Architecture makes blast radius, tenancy, ownership, and safe change visible.

APPLICABILITY

- Multiple applications or teams depend on shared runtime, delivery, identity, data, observability, policy, developer, or integration capabilities.
- Platform changes can alter isolation, availability, cost, security, privacy, performance, accessibility, or recovery for consumers.
- The platform provides managed automation, AI services, containers, orchestration, or external provider access.

NON-APPLICABILITY CONDITIONS

- A bounded application-local component has no shared consumers or material platform contract.
- An approved platform baseline covers the exact service, tenant, environment, version, and change with no material impact.
- The artifact is exploratory and makes no claim of platform readiness, support, isolation, or effective enforcement.

Record the baseline, scope, owner, consumers, evidence date, assumptions, and reopening trigger.

PREREQUISITES AND DEPENDENCIES

- Sections 019-033 define requirements, applications, software, data, operations, security, and lifecycle context.
- Consumer inventory, tenancy model, capacity, provider contracts, identity, policy, support, and recovery authorities are known or marked open.

REQUIRED INPUTS

- Consumer applications, workloads, environments, service objectives, data classes, jurisdictions, criticality, and lifecycle.
- Platform APIs, quotas, policies, identity model, configuration, deployment, observability, cost, support, incident, and recovery requirements.
- Supplier, license, model/provider, compliance, accessibility, migration, deprecation, and retirement obligations.

HUMAN RESPONSIBILITIES AND AUTHORITY

The platform owner approves shared capabilities, defaults, tenant isolation, quotas, provider use, policy enforcement, risk, service commitments, changes, exceptions, and retirement. Consumer owners retain authority over application purpose and data use. Humans can pause, restrict, isolate, revoke, restore, or retire platform functions.

AI RESPONSIBILITIES AND BOUNDARIES

AI may inventory consumers, analyze dependencies and capacity, draft contracts, detect drift, and propose platform changes. It must preserve scope, tenant, version, evidence, and uncertainty. It must not change shared defaults, grant access, declare isolation, accept platform risk, or execute control-plane changes without delegated authority.

APPLICABLE EXISTING AGENT ROLES

- Architect, Analyst, Researcher, Planner, Coordinator, Implementer, Tester, Verifier, Security Auditor, Governance Reviewer, Critic, Documentation Writer, Memory Curator, Optimizer, and Red Team.

REQUIRED ACTIVITIES

1. Inventory consumers, platform capabilities, dependencies, tenants, trust boundaries, and owners.
2. Define service contracts, quotas, policies, identities, configuration surfaces, and compatibility rules.
3. Model control plane, data plane, management plane, failure domains, and blast radius.
4. Allocate security, privacy, accessibility, reliability, resilience, observability, cost, and support controls.
5. Define onboarding, isolation, promotion, scaling, downgrade, migration, recovery, deprecation, and retirement.
6. Validate with consumer owners, operators, security, privacy, accessibility, finance, support, and suppliers.
7. Exercise tenant failure, quota exhaustion, provider loss, stale policy, unauthorized administration, and recovery.
8. Baseline contracts, evidence, decisions, open gaps, and handoff.

HUMAN-AI INTERACTION REQUIREMENTS

- Identify platform AI, automation, policy, and administrative actions and their exact authority.
- Require human review for shared defaults, access, isolation, data use, cost commitments, safety, and irreversible changes.
- Preserve consumer status, stop, evidence, fallback, and recovery paths when platform models or providers fail.
- Prevent conversational or tenant-admin shortcuts from bypassing platform controls.

ARCHITECTURE AND SYSTEM IMPLICATIONS

- Separate control plane, data plane, management plane, tenant configuration, platform state, and consumer state.
- Represent services, contracts, policies, quotas, dependencies, versions, and blast radius as attributable records.
- Enforce tenant isolation, identity, authorization, policy, audit, rate limits, and safe defaults at shared boundaries.
- Provide compatibility, migration, rollback, provider substitution, capacity, and recovery semantics.

OWNER COCKPIT AND USER-CONTROL REQUIREMENTS

- Show platform baseline, consumers, tenants, services, versions, policy state, capacity, incidents, dependencies, and evidence freshness.
- Expose shared-impact actions, approvals, actor identity, scope, blast radius, and recovery options.
- Permit authorized pause, restrict, revoke, isolate, restore, rollback, migrate, supersede, and retire actions.
- Export contracts, consumer impact, decisions, evidence, dissent, and continuity state.

REQUIRED OUTPUTS AND ARTIFACTS

- Platform landscape, capability and consumer catalog, tenancy and trust model, service contracts, quota and policy model.
- Dependency, capacity, cost, observability, security, privacy, accessibility, support, recovery, migration, and retirement designs.

- Impact, decision, risk, exception, evidence, test, and handoff package.

EVIDENCE REQUIREMENTS

- Claims identify platform service, consumer scope, tenant, version, configuration, environment, method, date, result, and limits.
- A dashboard, successful onboarding, policy file, benchmark, or isolated test proves only its declared scope.
- Isolation, availability, performance, quota, security, recovery, and support evidence states workload, conditions, coverage, and validity.
- Missing or failed consumer-impact evidence constrains shared-platform status.

VERIFICATION AND ACCEPTANCE TESTS

- Trace consumer requirements through platform contracts, implementation, deployment, runtime evidence, and retirement.
- Verify isolation, authorization, quotas, policy enforcement, compatibility, audit, accessibility, and blast-radius boundaries.
- Exercise noisy neighbor, quota exhaustion, provider loss, control-plane failure, stale policy, credential loss, migration, rollback, and restore.
- Confirm one tenant cannot bypass controls or corrupt another tenant's state or evidence.
- Reproduce representative platform claims from the baseline and handoff package.

FAILURE STATES AND ANTI-PATTERNS

- Treating a shared service, default, or green dashboard as universally safe.
- Hiding consumer dependencies, tenant boundaries, quotas, provider concentration, or control-plane privilege.
- Changing defaults without impact analysis, migration, communication, or recovery.
- Optimizing platform utilization while weakening isolation, accessibility, security, reliability, or consumer control.

RECOVERY AND RECONSIDERATION

- On shared failure, policy conflict, tenant isolation risk, provider loss, capacity exhaustion, or evidence failure, restrict the affected scope and preserve records.
- Identify safe consumers, owner, authority, communications, evidence, and review condition.
- Use approved isolation, capacity reduction, fallback, substitution, rollback, restore, migration, or retirement.
- Reopen on consumer, provider, policy, quota, architecture, or lifecycle changes.

COMPLETION CRITERIA

- Consumers, services, boundaries, contracts, policies, quotas, owners, risks, evidence, recovery, and lifecycle are explicit.
- Shared failure, tenant isolation, migration, support, and retirement are addressed or justified.
- Exact validity and unresolved gaps are recorded; draft status does not imply readiness or effective enforcement.

HANDOFF AND CONTINUITY REQUIREMENTS

- Preserve consumer maps, contracts, versions, policies, decisions, incidents, evidence, dissent, and open gaps.
- Hand off to Infrastructure, Cloud, Network, Security, Identity, Operations, Testing, Deployment, and Retirement sections.
- Record changes requiring reopening Application, Software, Data, Reliability, Resilience, or Governance architecture.

RELATED CODEX SECTIONS

- 019-033 Requirements through Software Architecture; 035-044 Infrastructure through Runtime Architecture.
- 045-060 Security through Hardware Architecture; 255-300 delivery and platform lifecycle; 301-406 operations and retirement.

SOURCE BASIS AND UNRESOLVED SOURCE GAPS

- Based on the Canonical Section Contract, shared governance files, Sections 019-033, Owner Cockpit Requirements, Status and Evidence Lexicon, and local architecture baseline.
- No universal tenancy, service-level, quota, platform, provider, pricing, accessibility, or compliance standard is supplied. Actual consumers, capacity, contracts, and acceptance evidence remain unresolved.

035

SECTION 035 - INFRASTRUCTURE ARCHITECTURE

CODEX/03_SECTIONS/035_INFRASTRUCTURE_ARCHITECTURE.txt

SHA-256 35d1607cd9b603d5852bf8a4cb8a7587f950cc0cd117dbd822f6e6dcb429d5bb

9,585 BYTES / PRESERVED SOURCE IDENTITY

SECTION 035 — Infrastructure Architecture

STATUS: CONTENT-COMPLETE

PURPOSE

Define the physical and virtual compute, storage, host, facility, power, environmental, provisioning, and infrastructure-management structure that supports approved systems and platforms.

REQUIRED OUTCOME

- A bounded infrastructure topology with resources, locations, ownership, dependencies, trust and failure domains, capacity, interfaces, controls, and lifecycle.
- Explicit provisioning, configuration, patching, monitoring, backup, restore, failover, replacement, disposal, and evidence obligations.
- Traceability from workloads and platform contracts through infrastructure resources, operations, security, testing, continuity, and retirement.

WHY THIS MATTERS

Infrastructure failures can affect availability, data integrity, safety, security, cost, and recovery across many services. Naming resources without modeling failure domains, physical constraints, ownership, or replacement leaves the real system ungoverned.

APPLICABILITY

- Software, platforms, devices, facilities, or data depend on compute, storage, host, network, power, environmental, or provisioning resources.
- Infrastructure changes affect capacity, isolation, resilience, physical access, energy, cost, security, privacy, or continuity.

NON-APPLICABILITY CONDITIONS

- No material infrastructure resource, dependency, physical effect, control, evidence, or lifecycle obligation changes.
- An approved baseline covers the exact environment, resource class, version, and change with no material impact.
- The artifact is explicitly exploratory and not a readiness or capacity claim.

Record baseline, scope, owner, evidence date, assumptions, and reopening trigger.

PREREQUISITES AND DEPENDENCIES

- Sections 019-034 provide requirements, applications, software, platform, data, operations, security, and resilience context.
- Facility, supplier, asset, capacity, energy, identity, access, network, backup, continuity, and disposal authorities are known or open.

REQUIRED INPUTS

- Workloads, service objectives, data classes, locations, environments, operating modes, capacity, performance, physical, environmental, and lifecycle needs.
- Resource inventories, topology, configuration, images, credentials, dependencies, maintenance, monitoring, incidents, backup, restore, failover, and retirement records.

HUMAN RESPONSIBILITIES AND AUTHORITY

Infrastructure owners approve topology, placement, capacity, physical access, security, environmental risk, suppliers, maintenance, exceptions, continuity, and retirement. Workload owners approve workload needs and risk. Humans retain authority to isolate, power down, fail over, restore, replace, and dispose.

AI RESPONSIBILITIES AND BOUNDARIES

AI may inventory authorized resources, compare topology, detect drift, analyze capacity, draft infrastructure-as-code and tests, and recommend remediation. It must label observation and proposal, preserve resource and environment identity, and never change infrastructure, expose credentials, declare resilience, or accept physical or service risk without authority.

APPLICABLE EXISTING AGENT ROLES

- Architect, Analyst, Researcher, Planner, Implementer, Tester, Verifier, Security Auditor, Governance Reviewer, Critic, Documentation Writer, Memory Curator, Optimizer, Coordinator, and Red Team.

REQUIRED ACTIVITIES

1. Inventory resources, locations, owners, workloads, dependencies, physical and virtual boundaries, and failure domains.
2. Define provisioning, configuration, identity, access, placement, capacity, performance, energy, and lifecycle policies.
3. Model normal, maintenance, degraded, facility-loss, resource-loss, recovery, migration, and disposal states.
4. Allocate security, privacy, safety, accessibility, observability, backup, restore, failover, and support controls.
5. Validate topology and operations with workload owners, operators, facilities, security, suppliers, and continuity specialists.
6. Exercise resource loss, corruption, capacity exhaustion, credential loss, network isolation, restore, replacement, and disposal.
7. Baseline topology, evidence, decisions, open gaps, and handoff.

HUMAN-AI INTERACTION REQUIREMENTS

- Show resource identity, environment, owner, placement, acting authority, change scope, and physical or service consequence.
- Require human review for destructive, shared, physical, security-sensitive, cost-bearing, or irreversible changes.
- Preserve console, status, evidence, isolation, manual recovery, and emergency paths when automation or AI fails.
- Prevent chat, break-glass, or provisioning shortcuts from bypassing authorization and audit.

ARCHITECTURE AND SYSTEM IMPLICATIONS

- Separate desired infrastructure, declared configuration, provisioned resource, observed state, backup, evidence, and retired asset.
- Model failure domains, blast radius, placement, capacity, quotas, physical constraints, and supplier dependencies.
- Define idempotent provisioning, drift detection, patching, rollback, restore, failover, replacement, and secure disposal.
- Protect credentials, logs, images, snapshots, backups, and management paths under the same lifecycle and access rules.

OWNER COCKPIT AND USER-CONTROL REQUIREMENTS

- Show topology, resources, location, owner, capacity, health, dependencies, cost, authority, evidence freshness, and lifecycle.
- Expose shared-impact and physical-impact actions with approval, actor, scope, and recovery details.
- Permit authorized isolate, pause, fail over, restore, replace, migrate, quarantine, and retire actions.
- Export topology, configuration identity, evidence, incidents, decisions, dissent, and continuation state.

REQUIRED OUTPUTS AND ARTIFACTS

- Infrastructure topology, asset and resource catalog, placement and failure-domain model, capacity plan, configuration and provisioning contracts.
- Security, physical access, energy, observability, backup, restore, failover, maintenance, migration, replacement, and disposal plans.
- Verification, readiness, impact, risk, decision, evidence, and handoff package.

EVIDENCE REQUIREMENTS

- Claims identify resource, location, configuration, workload, environment, method, observation time, actor, result, and limits.
- Provisioning success, inventory, uptime, capacity, benchmark, failover drill, or backup listing proves only its bounded scope.
- Physical, environmental, security, resilience, restore, and disposal evidence states coverage, conditions, exceptions, and validity.
- Stale, missing, failed, or contradictory evidence constrains readiness and health status.

VERIFICATION AND ACCEPTANCE TESTS

- Trace workloads and platform contracts through resources, provisioning, operation, evidence, recovery, and retirement.
- Verify identity, least privilege, isolation, configuration integrity, capacity, monitoring, backup, restore, failover, and disposal controls.
- Exercise host, storage, power, facility, credential, network, image, provider, and evidence-store loss.
- Verify drift, unauthorized placement, quota exhaustion, stale configuration, and incomplete replacement are detected.
- Reproduce topology and recovery claims from the exact baseline.

FAILURE STATES AND ANTI-PATTERNS

- Treating resource existence, inventory, uptime, or a green health signal as resilience or capacity proof.
- Ignoring physical access, facility, energy, supplier, image, snapshot, backup, or management dependencies.
- Making infrastructure changes without blast-radius, maintenance, rollback, and communication analysis.
- Leaving credentials, snapshots, logs, or retired assets outside lifecycle control.

RECOVERY AND RECONSIDERATION

- On resource loss, corruption, physical hazard, capacity exhaustion, unauthorized change, or evidence failure, isolate affected scope and preserve records.
- Identify safe workload operation, owner, authority, communications, evidence, and recovery window.
- Use approved failover, restore, replacement, reduced service, migration, or secure disposal paths.
- Reopen on location, supplier, workload, capacity, topology, security, or lifecycle change.

COMPLETION CRITERIA

- Resources, owners, placement, dependencies, capacity, controls, evidence, recovery, maintenance, replacement, and disposal are explicit.
- Resource and facility failure, migration, restore, and retirement are addressed or justified.
- Exact validity and unresolved gaps are recorded; CONTENT-COMPLETE does not imply provisioned or resilient.

HANDOFF AND CONTINUITY REQUIREMENTS

- Preserve asset identities, topology, configurations, locations, dependencies, incidents, decisions, evidence, dissent, and gaps.
- Hand off to Cloud, Network, Security, Operations, Reliability, Resilience, Testing, Provisioning, and Retirement sections.
- Record changes requiring reopening Platform, Software, Data, Cloud, Network, or Governance architecture.

RELATED CODEX SECTIONS

- 019-034 Requirements through Platform Architecture; 036-044 Cloud through Runtime Architecture.
- 045-060 Security through Hardware Architecture; 255-300 delivery and infrastructure lifecycle; 301-406 operations and retirement.

SOURCE BASIS AND UNRESOLVED SOURCE GAPS

- Based on the Canonical Section Contract, shared governance files, Sections 019-034, Owner Cockpit Requirements, Status and Evidence Lexicon, and local architecture baseline.
- No universal facility, hardware, energy, capacity, asset, supplier, or infrastructure standard is supplied. Actual topology, locations, service objectives, physical controls, and acceptance evidence remain unresolved.

036

SECTION 036 - CLOUD ARCHITECTURE

CODEX/03_SECTIONS/036_CLOUD_ARCHITECTURE.txt

SHA-256 d8104fedc3dbae7130b3a0f66fca43dd68327847dbbcc7fb3fc52ac44a42a9cd

9,813 BYTES / PRESERVED SOURCE IDENTITY

SECTION 036 — Cloud Architecture

STATUS: CONTENT-COMPLETE

PURPOSE

Define the governed use of cloud regions, accounts, subscriptions, tenants, managed services, control planes, data planes, identity, networking, elasticity, provider dependencies, and exit paths for approved workloads.

REQUIRED OUTCOME

- A cloud scope and landing model with owners, regions, tenants, accounts, environments, workloads, data, trust boundaries, service contracts, and lifecycle.
- Explicit decisions for placement, residency, identity, tenancy, scaling, cost, availability, resilience, portability, provider risk, and recovery.
- Evidence-bounded handoff to platform, infrastructure, network, security, operations, testing, continuity, and retirement.

WHY THIS MATTERS

Cloud control planes and managed services create powerful shared dependencies. Convenience can hide provider lock-in, region concentration, opaque failure behavior, cost exposure, data-residency issues, or administrative paths that bypass application authority.

APPLICABILITY

- Workloads use hosted compute, storage, identity, networking, managed data, AI, observability, delivery, or other provider services.
- Cloud decisions affect jurisdiction, availability, security, privacy, cost, scalability, portability, resilience, or retirement.

NON-APPLICABILITY CONDITIONS

- No cloud service, provider, region, tenant, data-residency, identity, cost, or lifecycle condition is affected.
- An approved cloud baseline covers exact workload, provider, environment, region, and change with no material impact.
- The artifact is exploratory and explicitly not a deployment, compliance, resilience, or provider-capability claim.

Record baseline, scope, owner, evidence date, assumptions, and reopening trigger.

PREREQUISITES AND DEPENDENCIES

- Sections 019-035 provide requirements, applications, software, platform, infrastructure, data, operations, security, and recovery context.
- Provider terms, region availability, identity, network, billing, data residency, support, exit, and continuity authorities are known or open.

REQUIRED INPUTS

- Workloads, data classes, users, jurisdictions, environments, service objectives, criticality, recovery targets, and lifecycle.
- Provider services, accounts, subscriptions, policies, quotas, topology, configuration, contracts, cost limits, telemetry, incidents, backup, restore, and exit requirements.

HUMAN RESPONSIBILITIES AND AUTHORITY

Cloud owners approve provider, region, residency, account structure, identity, shared services, cost commitments, exceptions, resilience, support, and exit. Workload and data owners approve use and risk. Humans retain authority for provisioning, suspension, failover, restore, migration, and account closure.

AI RESPONSIBILITIES AND BOUNDARIES

AI may compare provider offerings, inspect authorized configuration, model cost or capacity, draft infrastructure-as-code, identify policy drift, and propose migration. It must label estimates and evidence, preserve account and region identity, and must not provision, grant access, transfer data, accept terms, or declare compliance or resilience without authority.

APPLICABLE EXISTING AGENT ROLES

- Architect, Analyst, Researcher, Planner, Implementer, Tester, Verifier, Security Auditor, Governance Reviewer, Critic, Documentation Writer, Memory Curator, Optimizer, Coordinator, and Red Team.

REQUIRED ACTIVITIES

1. Establish workload, provider, account, region, tenant, jurisdiction, data, owner, and lifecycle scope.
2. Assess provider capability, shared responsibility, residency, identity, support, cost, lock-in, and exit.
3. Model control plane, data plane, management plane, failure domains, quotas, and blast radius.
4. Define landing zones, policy, network, access, secrets, logging, backup, restore, scaling, and cost controls.
5. Define deployment, promotion, rollback, failover, region substitution, migration, portability, and account closure.
6. Validate with workload, data, security, privacy, finance, operations, continuity, legal, and supplier authorities.
7. Exercise provider, region, credential, quota, network, service, billing, data, and control-plane failures.
8. Baseline decisions, evidence, assumptions, open gaps, and handoff.

HUMAN-AI INTERACTION REQUIREMENTS

- Show provider, account, region, resource, data boundary, authority, actor, cost, and external consequence for consequential actions.
- Require human review for provisioning, residency, access, cost commitment, data transfer, policy, security, and destructive actions.
- Preserve provider-independent status, evidence, stop, fallback, and recovery paths where feasible.
- Prevent chat, break-glass, or tenant-admin paths from bypassing policy and audit.

ARCHITECTURE AND SYSTEM IMPLICATIONS

- Separate provider control plane, workload data plane, management plane, customer configuration, and evidence.
- Represent provider services, regions, contracts, quotas, policies, dependencies, costs, and exit mappings as versioned records.
- Enforce identity, authorization, residency, encryption or equivalent protection, logging, backup, and deletion at cloud boundaries.
- Design provider substitution, region failover, portability, migration, restore, and account closure before dependence becomes irreversible.

OWNER COCKPIT AND USER-CONTROL REQUIREMENTS

- Show provider, account, region, workload, data, owner, policy, cost, capacity, health, authority, evidence freshness, and lifecycle.
- Expose shared-impact, data-transfer, cost-bearing, and destructive actions with approval and blast radius.
- Permit authorized restrict, suspend, fail over, restore, migrate, revoke, rollback, and close actions.
- Export cloud topology, policies, contracts, evidence, decisions, dissent, and continuity record.

REQUIRED OUTPUTS AND ARTIFACTS

- Cloud context, account and landing model, region and residency decision, provider responsibility matrix, topology, service contracts, and cost model.
- Identity, network, security, privacy, observability, backup, restore, failover, portability, migration, exit, and retirement designs.
- Impact, risk, verification, evidence, acceptance, and operational handoff package.

EVIDENCE REQUIREMENTS

- Claims identify provider, account, region, resource, workload, data class, configuration, observation period, method, actor, and limitation.
- Provider documentation, deployment success, cost estimate, policy scan, uptime, or failover drill proves only its declared scope and conditions.
- Residency, security, availability, cost, portability, recovery, and support evidence identifies coverage and validity.
- Stale, missing, failed, or provider-controlled evidence remains visible and constrains status.

VERIFICATION AND ACCEPTANCE TESTS

- Trace workload and data obligations through provider services, identity, regions, controls, operation, evidence, recovery, and exit.
- Verify account isolation, least privilege, residency, policy, quotas, cost alerts, logging, backup, restore, and deletion.
- Exercise region loss, provider-service loss, credential compromise, quota exhaustion, billing anomaly, network partition, stale policy, and account lockout.
- Verify portability or explicitly record the accepted dependency and recovery limit.
- Reproduce representative cloud claims from the exact configuration and evidence package.

FAILURE STATES AND ANTI-PATTERNS

- Treating managed-service availability, provider documentation, or deployment success as general resilience or compliance proof.
- Hiding shared responsibility, provider concentration, region dependency, opaque managed state, data transfer, or cost exposure.
- Granting broad cloud administration to simplify automation or allowing provider defaults to become policy without approval.
- Designing no exit because migration is inconvenient.

RECOVERY AND RECONSIDERATION

- On provider, region, account, identity, policy, cost, residency, or evidence failure, restrict affected resources and preserve records.
- Identify safe service, affected data, owner, authority, communication, evidence, and recovery window.
- Use approved reduced service, region failover, restore, provider substitution, migration, export, or closure paths.
- Reopen when provider terms, regions, data, workload, policy, cost, or lifecycle changes.

COMPLETION CRITERIA

- Provider, account, region, workload, data, authority, controls, cost, evidence, recovery, portability, and lifecycle are explicit.
- Cloud failure, migration, exit, and retirement are addressed or justified.
- Exact validity and unresolved gaps are recorded; CONTENT-COMPLETE does not imply deployed, compliant, or resilient.

HANDOFF AND CONTINUITY REQUIREMENTS

- Preserve provider identities, accounts, regions, configurations, contracts, dependencies, costs, incidents, decisions, evidence, dissent, and gaps.
- Hand off to Platform, Infrastructure, Network, Security, Identity, Data, Operations, Testing, Continuity, Migration, and Retirement sections.

- Record changes requiring reopening Application, Software, Data, Reliability, Resilience, or Governance architecture.

RELATED CODEX SECTIONS

- 019-035 Requirements through Infrastructure Architecture; 037-044 Edge through Runtime Architecture.
- 045-060 Security through Hardware Architecture; 255-300 delivery and cloud lifecycle; 301-406 operations and retirement.

SOURCE BASIS AND UNRESOLVED SOURCE GAPS

- Based on the Canonical Section Contract, shared governance files, Sections 019-035, Owner Cockpit Requirements, Status and Evidence Lexicon, and local architecture baseline.
- No universal provider, region, residency, pricing, service-level, portability, or compliance regime is supplied. Actual workloads, contracts, account controls, recovery objectives, and exit evidence remain unresolved.

037

SECTION 037 - EDGE ARCHITECTURE

CODEX/03_SECTIONS/037_EDGE_ARCHITECTURE.txt

SHA-256 2e60c958f6021d83573d59c0f504b80c0a0796eda216265d38d73298f5485c6e

8,883 BYTES / PRESERVED SOURCE IDENTITY

SECTION 037 — Edge Architecture

STATUS: CONTENT-COMPLETE

PURPOSE

Define computation, storage, sensing, control, and decision boundaries placed near users, devices, networks, or physical environments, including synchronization with central services and operation under intermittent connectivity.

REQUIRED OUTCOME

- A node, site, trust, data, workload, connectivity, identity, and lifecycle model for edge resources.
- Explicit local versus central authority, freshness, synchronization, offline, safety, privacy, security, update, and recovery behavior.
- Traceability from workloads, data, applications, networks, devices, operations, tests, evidence, and retirement.

WHY THIS MATTERS

Edge placement changes latency, availability, physical exposure, data residency, consistency, and recovery. A disconnected or compromised node can continue acting after central policy or authority has changed, so local autonomy and synchronization must be deliberate.

APPLICABILITY

- Workloads execute near users, devices, facilities, sensors, vehicles, or constrained networks.
- Local processing affects safety, privacy, security, latency, continuity, physical action, or data movement.

NON-APPLICABILITY CONDITIONS

- No local resource, offline mode, physical exposure, synchronization, or edge-specific lifecycle effect changes.
- An approved baseline covers the exact sites, nodes, workloads, and change with no material impact.
- The artifact is exploratory and not a deployment, coverage, or safety claim.

Record baseline, sites, owner, evidence date, assumptions, and reopening trigger.

PREREQUISITES AND DEPENDENCIES

- Sections 019-036 provide requirements, application, software, platform, infrastructure, cloud, data, security, and recovery context.
- Site, device, network, identity, update, physical safety, privacy, and operational authorities are known or recorded as gaps.

REQUIRED INPUTS

- Sites, nodes, devices, users, workloads, data classes, connectivity, latency, power, environmental, safety, jurisdiction, and lifecycle constraints.
- Synchronization, identity, policy, update, observability, backup, restore, failover, support, and retirement requirements.

HUMAN RESPONSIBILITIES AND AUTHORITY

Edge owners approve local autonomy, physical placement, safety, data handling, authority expiry, update, isolation, and service risk. Site and domain authorities approve physical and operational conditions. Humans retain stop, quarantine, disconnect, restore, and retire authority.

AI RESPONSIBILITIES AND BOUNDARIES

AI may analyze edge topology, telemetry, synchronization, and capacity, and draft bounded deployment or test proposals. It must identify node, site, model, policy, and connectivity context. It must not infer offline authority, change nodes, issue physical commands, or declare safe operation without delegated authority and current evidence.

APPLICABLE EXISTING AGENT ROLES

- Architect, Analyst, Researcher, Observer, Planner, Implementer, Tester, Verifier, Security Auditor, Safety Auditor, Governance Reviewer, Critic, Documentation Writer, Memory Curator, and Red Team.

REQUIRED ACTIVITIES

1. Inventory sites, nodes, workloads, data, users, physical conditions, dependencies, and owners.
2. Define local and central responsibilities, authority, policy freshness, synchronization, and conflict resolution.
3. Model connected, intermittent, offline, degraded, maintenance, emergency, recovery, migration, and retirement modes.
4. Define identity, secure update, local storage, privacy, safety, observability, and evidence controls.
5. Validate with site operators, users, network, security, privacy, safety, and continuity authorities.
6. Exercise disconnection, stale policy, clock drift, node compromise, data conflict, update failure, and restore.
7. Baseline and hand off decisions, evidence, open gaps, and next safe action.

HUMAN-AI INTERACTION REQUIREMENTS

- Show node, site, connectivity, policy age, identity, acting authority, model, data, and physical consequence.
- Require review for local autonomy, physical effect, privacy, safety, security, updates, and irreversible actions.
- Preserve local stop, status, evidence, manual control, and recovery when central services fail.
- Prevent offline operation from silently extending expired authority.

ARCHITECTURE AND SYSTEM IMPLICATIONS

- Separate local state, central state, cache, replica, evidence, backup, and synchronized record.
- Define freshness, ordering, conflict, replay, idempotency, clock, identity, and authority-expiry semantics.
- Secure boot or equivalent, update, attestation, physical access, local encryption, quarantine, and recovery require explicit ownership.
- Account for bandwidth, power, storage, thermal, and physical failure constraints.

OWNER COCKPIT AND USER-CONTROL REQUIREMENTS

- Show site, node, workload, connectivity, policy freshness, health, authority, data, evidence, and lifecycle.
- Expose local and central actions, physical consequence, approval, actor, and recovery path.
- Permit authorized stop, isolate, revoke, resynchronize, restore, update, rollback, replace, and retire actions.
- Export node histories, synchronization records, decisions, evidence, dissent, and continuity state.

REQUIRED OUTPUTS AND ARTIFACTS

- Edge topology, site and node catalog, local-central authority model, data and synchronization contracts, and deployment model.
- Offline, update, physical security, privacy, safety, observability, capacity, recovery, migration, and retirement plans.
- Verification, incident, evidence, risk, decision, and handoff package.

EVIDENCE REQUIREMENTS

- Claims identify site, node, workload, connectivity, policy age, version, configuration, method, time, actor, and limitation.
- Lab, simulation, telemetry, deployment, synchronization, or drill evidence proves only its declared sites and conditions.
- Offline, safety, security, freshness, recovery, and physical evidence identifies coverage and validity.
- Missing, stale, or contradictory node evidence constrains status.

VERIFICATION AND ACCEPTANCE TESTS

- Trace requirements through node allocation, data flow, policy, runtime, evidence, recovery, and retirement.
- Exercise disconnect, stale authority, duplicate or conflicting data, clock drift, node loss, update rollback, and physical access failure.
- Verify local actions cannot exceed authority, bypass safety or privacy controls, or erase audit history.
- Verify restore, resynchronization, quarantine, and safe return to service.
- Reproduce representative edge claims from the exact node and site baseline.

FAILURE STATES AND ANTI-PATTERNS

- Treating central availability as edge availability or offline continuation as authorized autonomy.
- Ignoring site conditions, physical access, stale policy, clock, power, storage, bandwidth, and update dependencies.
- Allowing local caches, models, or operators to bypass authority, privacy, safety, or evidence controls.

RECOVERY AND RECONSIDERATION

- On node compromise, stale policy, unsafe condition, synchronization conflict, or evidence loss, isolate the node and preserve records.
- Identify safe local operation, authority, site owner, affected users, evidence, and review condition.
- Use manual control, reduced mode, quarantine, restore, resynchronization, rollback, replacement, or retirement.
- Reopen on site, device, workload, policy, connectivity, model, or lifecycle change.

COMPLETION CRITERIA

- Nodes, sites, authority, connectivity, data, synchronization, safety, security, evidence, recovery, and lifecycle are explicit.
- Offline, stale, disconnected, physical, migration, and retirement behavior is addressed or justified.
- Exact validity and unresolved gaps are recorded; CONTENT-COMPLETE does not imply safe field operation.

HANDOFF AND CONTINUITY REQUIREMENTS

- Preserve site and node identities, versions, policy, synchronization, incidents, decisions, evidence, dissent, and gaps.
- Hand off to Cloud, Infrastructure, Network, Communications, Security, Data, Robotics, Embedded, Operations, Testing, and Retirement.
- Record changes requiring reopening Application, Platform, Safety, Reliability, Resilience, or Governance architecture.

RELATED CODEX SECTIONS

- 019-036 Requirements through Cloud Architecture; 038-044 Distributed through Runtime Architecture.
- 045-060 Security through Hardware Architecture; 068-074 embedded and robotics engineering; 301-406 operations and lifecycle.

SOURCE BASIS AND UNRESOLVED SOURCE GAPS

- Based on the Canonical Section Contract, shared governance files, Sections 019-036, Owner Cockpit Requirements, Status and Evidence Lexicon, and local architecture baseline.
- No universal edge, site, device, connectivity, safety, or offline standard is supplied. Actual sites, node inventory, physical authority, recovery targets, and acceptance evidence remain unresolved.

038

SECTION 038 - DISTRIBUTED SYSTEMS ARCHITECTURE

CODEX/03_SECTIONS/038_DISTRIBUTED_SYSTEMS_ARCHITECTURE.txt

SHA-256 c15b920e118741f8e6e552995fd981911570de23acb6abc009c46c57c0947ea8

8,243 BYTES / PRESERVED SOURCE IDENTITY

SECTION 038 — Distributed Systems Architecture

STATUS: CONTENT-COMPLETE

PURPOSE

Define the behavior and governance of cooperating components that communicate across time, space, failure, trust, and ownership boundaries.

REQUIRED OUTCOME

- Explicit node, service, data, message, time, consistency, failure, authority, and ownership models.
- Contracts for ordering, delivery, duplication, partition, retry, idempotency, quorum, recovery, and evidence.
- Traceability from requirements through distributed behavior, implementation, tests, operations, and retirement.

WHY THIS MATTERS

Distributed components cannot assume shared time, memory, availability, or truth. Hidden partitions, duplicate messages, stale reads, partial commits, and correlated failures create behavior that happy-path tests do not reveal.

APPLICABILITY

- Components communicate over process, machine, region, organizational, or provider boundaries.
- Consistency, ordering, latency, partition, concurrency, or partial failure affects material outcomes.

NON-APPLICABILITY CONDITIONS

- No distributed boundary or material timing, consistency, failure, data, or authority effect changes.
- An approved baseline covers exact topology, workload, and change with no material impact.
- The artifact is exploratory and clearly not a correctness or resilience claim.

Record baseline, scope, owner, evidence date, assumptions, and reopening trigger.

PREREQUISITES AND DEPENDENCIES

- Sections 019-037 provide requirements, process, function, application, software, network, cloud, data, and operational context.
- Time, identity, message, storage, failure, observability, and recovery authorities are known or marked open.

REQUIRED INPUTS

- Nodes, services, actors, data, messages, events, clocks, partitions, workloads, consistency needs, failure domains, and recovery objectives.
- Contracts, retries, limits, security, privacy, observability, deployment, migration, testing, and retirement requirements.

HUMAN RESPONSIBILITIES AND AUTHORITY

Owners approve consistency, availability, quorum, data conflict, external effect, risk, and recovery tradeoffs. Humans approve autonomous behavior, change, incident response, and compensation and retain stop, isolate, and restore authority.

AI RESPONSIBILITIES AND BOUNDARIES

AI may inspect traces and designs, identify race or failure scenarios, draft contracts and tests, and analyze bounded telemetry. It must preserve timing and topology context and cannot declare distributed correctness, resolve data conflict, or change live coordination without authority.

APPLICABLE EXISTING AGENT ROLES

- Architect, Analyst, Researcher, Observer, Planner, Implementer, Tester, Verifier, Security Auditor, Governance Reviewer, Critic, Documentation Writer, Memory Curator, Optimizer, and Red Team.

REQUIRED ACTIVITIES

1. Define nodes, services, boundaries, actors, ownership, data, messages, clocks, and failure domains.
2. State consistency, availability, ordering, delivery, retry, quorum, and authority assumptions.
3. Model normal, concurrent, partitioned, delayed, duplicated, reordered, degraded, recovery, and retirement behavior.
4. Define idempotency, deduplication, compensation, transaction, timeout, cancellation, and evidence semantics.
5. Validate with domain, data, security, operations, reliability, and user authorities.
6. Test partitions, clock drift, duplicate and lost messages, split brain, partial commit, overload, failover, and restore.
7. Baseline and hand off exact models, evidence, decisions, dissent, and gaps.

HUMAN-AI INTERACTION REQUIREMENTS

- Show component, message, time, data, authority, actor, and external-effect context for consequential actions.
- Require review for conflict resolution, quorum changes, autonomous retries, cross-boundary writes, and irreversible effects.
- Preserve stop, evidence, diagnosis, manual recovery, and compensation during partial failure.

ARCHITECTURE AND SYSTEM IMPLICATIONS

- Define explicit message identity, causality, ordering, delivery, replay, expiry, and idempotency.
- Separate authoritative state, replica, cache, event, projection, log, and evidence.
- Model partitions, correlated failure, split brain, backpressure, overload, and clock uncertainty.
- Make compensation and audit append-only; do not treat destructive reversal as rollback.

OWNER COCKPIT AND USER-CONTROL REQUIREMENTS

- Show topology, state, partition, lag, message health, authority, evidence freshness, and recovery mode.
- Expose actions, blast radius, actor, approvals, conflict decisions, and compensation path.
- Permit authorized pause, isolate, drain, fail over, reconcile, restore, compensate, and retire actions.
- Export traces, messages, decisions, evidence, dissent, and continuity record.

REQUIRED OUTPUTS AND ARTIFACTS

- Distributed context and topology, consistency and failure model, message and state contracts, timing assumptions, and recovery design.
- Traceability, test and drill basis, observability model, decision, risk, evidence, and handoff registers.

EVIDENCE REQUIREMENTS

- Claims identify topology, workload, data, timing, configuration, failure injection, method, result, and limits.
- A local test, simulation, trace, or successful transaction proves only its bounded schedule and conditions.
- Evidence must distinguish no failure observed from failure impossible and nominal from partitioned behavior.
- Missing clocks, traces, message history, or conflict evidence constrains status.

VERIFICATION AND ACCEPTANCE TESTS

- Trace representative outcomes across nodes, messages, state, evidence, recovery, and retirement.
- Exercise delay, loss, duplication, reorder, partition, clock drift, stale read, split brain, overload, and partial commit.
- Verify idempotency, deduplication, authority expiry, compensation, audit, and user-visible status.
- Disable nodes, stores, networks, models, providers, and observability and verify safe behavior.
- Reproduce claims from exact topology and trace package.

FAILURE STATES AND ANTI-PATTERNS

- Assuming shared time, memory, availability, or exactly-once behavior without evidence.
- Treating retry as recovery, replication as consistency, or eventual convergence as universal correctness.
- Hiding partial commits, stale state, split brain, duplicate effects, or unbounded queues.
- Allowing AI to resolve conflicts or issue repeated external effects without authority and idempotency.

RECOVERY AND RECONSIDERATION

- On partition, divergence, duplicate effect, lost evidence, or authority conflict, halt affected writes where safe and preserve history.
- Identify safe reads and operations, owner, authority, affected actors, evidence, and review condition.
- Use approved fencing, drain, reconciliation, compensation, restore, failover, or reduced mode.
- Reopen when topology, timing, consistency, provider, data, or lifecycle assumptions change.

COMPLETION CRITERIA

- Topology, ownership, timing, contracts, consistency, failure, compensation, evidence, recovery, and lifecycle are explicit.
- Partition, concurrency, overload, migration, and retirement behavior is addressed or justified.
- Exact evidence limits and unresolved gaps are recorded.

HANDOFF AND CONTINUITY REQUIREMENTS

- Preserve node, message, state, trace, decision, compensation, evidence, dissent, and open-gap identities.
- Hand off to Network, Communications, Integration, API, Event, Runtime, Data, Reliability, Resilience, Testing, and Operations.
- Record changes requiring reopening Software, Platform, Cloud, Security, or Governance architecture.

RELATED CODEX SECTIONS

- 019-037 Requirements through Edge Architecture; 039-044 Network through Runtime Architecture.
- 045-060 Security through Hardware Architecture; 130-145 distributed and real-time engineering; 301-406 operations and lifecycle.

SOURCE BASIS AND UNRESOLVED SOURCE GAPS

- Based on the Canonical Section Contract, shared governance files, Sections 019-037, Owner Cockpit Requirements, Status and Evidence Lexicon, and local architecture baseline.
- No universal consistency, clock, transaction, delivery, or failure model is supplied. Actual workloads, schedules, contracts, recovery objectives, and evidence remain unresolved.

039

SECTION 039 - NETWORK ARCHITECTURE

CODEX/03_SECTIONS/039_NETWORK_ARCHITECTURE.txt

SHA-256 a21f8308b255a4f860416c271744fb00b0780dd2d48a4e8dcfe38e5a57b0de16

8,550 BYTES / PRESERVED SOURCE IDENTITY

SECTION 039 — Network Architecture

STATUS: CONTENT-COMPLETE

PURPOSE

Define network zones, paths, identities, trust boundaries, addressing, routing, transport, performance, availability, monitoring, and lifecycle for authorized communication.

REQUIRED OUTCOME

- A network context and topology with zones, endpoints, flows, owners, dependencies, controls, failure domains, and lifecycle.
- Explicit connectivity, segmentation, identity, authorization, encryption, naming, routing, capacity, observability, and recovery contracts.
- Traceability from application, platform, data, security, operations, tests, evidence, and retirement.

WHY THIS MATTERS

Network paths are control boundaries as well as transport. Overbroad reachability, ambiguous trust, unmanaged DNS, hidden egress, or untested failover can turn a local defect into broad compromise or outage.

APPLICABILITY

- Systems communicate across hosts, zones, sites, clouds, devices, suppliers, or public networks.
- Reachability, latency, segmentation, routing, naming, availability, privacy, or interception has material consequence.

NON-APPLICABILITY CONDITIONS

- No material path, endpoint, zone, identity, routing, exposure, or lifecycle effect changes.
- An approved baseline covers exact topology and change with no material impact.
- The artifact is exploratory and not a connectivity, security, or availability claim.

Record baseline, scope, owner, evidence date, assumptions, and reopening trigger.

PREREQUISITES AND DEPENDENCIES

- Sections 019-038 provide requirements, applications, data, distributed behavior, cloud, infrastructure, security, and operations context.
- Addressing, identity, DNS, certificates, providers, physical links, monitoring, incident, and recovery authorities are known or open.

REQUIRED INPUTS

- Endpoints, zones, users, services, data classes, trust, flows, protocols, ports, latency, capacity, availability, jurisdictions, and lifecycle.
- Firewall, routing, DNS, certificate, identity, segmentation, logging, detection, backup, failover, maintenance, and retirement obligations.

HUMAN RESPONSIBILITIES AND AUTHORITY

Network owners approve zones, reachability, exposure, routing, identity, certificates, providers, maintenance, exceptions, risk, and retirement. Workload and security owners approve required flows and residual risk. Humans retain isolate, revoke, reroute, restore, and retire authority.

AI RESPONSIBILITIES AND BOUNDARIES

AI may analyze authorized topology, flows, logs, capacity, and policy drift, and draft bounded rule or test proposals. It must preserve endpoint and time context and must not alter routes, firewalls, DNS, certificates, or exposure or declare security without authority.

APPLICABLE EXISTING AGENT ROLES

- Architect, Analyst, Researcher, Observer, Planner, Implementer, Tester, Verifier, Security Auditor, Governance Reviewer, Critic, Documentation Writer, Memory Curator, Optimizer, and Red Team.

REQUIRED ACTIVITIES

1. Inventory endpoints, zones, links, providers, owners, flows, trust boundaries, and failure domains.
2. Define addressing, naming, routing, segmentation, identity, authorization, encryption, and exposure rules.
3. Model normal, congested, partitioned, failover, maintenance, emergency, migration, and retirement modes.
4. Define flow contracts, observability, filtering, alerting, capacity, and recovery.
5. Validate with application, platform, cloud, security, privacy, operations, and site owners.
6. Test unauthorized reachability, egress, spoofing, DNS failure, certificate expiry, congestion, partition, and failover.
7. Baseline topology, evidence, decisions, open gaps, and handoff.

HUMAN-AI INTERACTION REQUIREMENTS

- Show endpoint, zone, identity, data, path, authority, actor, exposure, and consequence for network changes.
- Require human review for public exposure, segmentation, privileged routes, certificate, DNS, and emergency changes.
- Preserve out-of-band status, evidence, isolate, console, and recovery paths when network automation fails.
- Prevent chat or break-glass shortcuts from bypassing audit and policy.

ARCHITECTURE AND SYSTEM IMPLICATIONS

- Separate intended, declared, provisioned, observed, and historical topology.
- Treat DNS, certificates, routing, firewall, identity, egress, proxies, and telemetry as governed dependencies.
- Enforce least reachability, segmentation, authentication, authorization, encryption, logging, rate limits, and safe failover.
- Define addressing, naming, compatibility, migration, rollback, and retirement semantics.

OWNER COCKPIT AND USER-CONTROL REQUIREMENTS

- Show topology, zones, paths, exposure, health, traffic, policy, certificates, capacity, owner, authority, and evidence freshness.
- Expose shared-impact and public-impact actions with approvals, actor, scope, and recovery.
- Permit authorized block, isolate, reroute, rotate, fail over, restore, rollback, and retire actions.
- Export topology, flow records, decisions, evidence, dissent, and continuity state.

REQUIRED OUTPUTS AND ARTIFACTS

- Network context, topology, zone and trust model, flow catalog, addressing and naming plan, identity and exposure model.
- Routing, segmentation, certificate, DNS, capacity, observability, incident, failover, migration, and retirement designs.
- Verification, evidence, risk, decision, and handoff package.

EVIDENCE REQUIREMENTS

- Claims identify path, endpoint, zone, protocol, configuration, time, workload, method, actor, result, and limitation.
- Reachability tests, scans, telemetry, diagrams, provider status, or failover drills prove only their scope and time.
- Security, performance, availability, and recovery evidence states coverage, traffic, conditions, and validity.
- Missing flow inventory or stale topology constrains status.

VERIFICATION AND ACCEPTANCE TESTS

- Trace application and data flows through zones, controls, paths, telemetry, operation, recovery, and retirement.
- Verify least reachability, segmentation, identity, certificate, DNS, egress, logging, and failover.
- Exercise congestion, route loss, DNS failure, certificate expiry, provider loss, spoofing, unauthorized path, and partition.
- Confirm emergency access is bounded, attributable, time-limited, and audited.
- Reproduce representative claims from exact topology and evidence.

FAILURE STATES AND ANTI-PATTERNS

- Treating a diagram, ping, scan, or provider status as proof of secure or reliable connectivity.
- Allowing any-to-any access, unmanaged egress, shared credentials, stale DNS, or unowned certificates.
- Hiding proxy, supplier, management, monitoring, or out-of-band dependencies.

RECOVERY AND RECONSIDERATION

- On exposure, partition, congestion, certificate or DNS failure, compromise, or evidence loss, restrict affected paths and preserve records.
- Identify safe connectivity, owner, authority, communications, evidence, and review condition.
- Use approved block, isolate, alternate path, credential or certificate rotation, failover, restore, or retirement.
- Reopen on endpoint, zone, provider, flow, identity, or lifecycle change.

COMPLETION CRITERIA

- Endpoints, zones, paths, identities, policies, capacity, evidence, recovery, and lifecycle are explicit.
- Exposure, partition, congestion, maintenance, migration, and retirement behavior is addressed or justified.
- Exact validity and unresolved gaps are recorded; CONTENT-COMPLETE does not imply secure or available networking.

HANDOFF AND CONTINUITY REQUIREMENTS

- Preserve topology, flows, endpoints, certificates, DNS, decisions, incidents, evidence, dissent, and gaps.
- Hand off to Cloud, Infrastructure, Communications, Integration, API, Security, Identity, Operations, Testing, and Retirement.
- Record changes requiring reopening Distributed, Platform, Application, Data, Reliability, or Governance architecture.

RELATED CODEX SECTIONS

- 019-038 Requirements through Distributed Systems Architecture; 040-044 Communications through Runtime Architecture.
- 045-060 Security through Hardware Architecture; 195 Network Security; 284 Network Engineering; 301-406 operations and lifecycle.

SOURCE BASIS AND UNRESOLVED SOURCE GAPS

- Based on the Canonical Section Contract, shared governance files, Sections 019-038, Owner Cockpit Requirements, Status and Evidence Lexicon, and local architecture baseline.
- No universal network, carrier, protocol, segmentation, certificate, or jurisdiction standard is supplied. Actual topology, flows, owners, exposure, capacity, and acceptance evidence remain unresolved.

040

SECTION 040 - COMMUNICATIONS ARCHITECTURE

CODEX/03_SECTIONS/040_COMMUNICATIONS_ARCHITECTURE.txt

SHA-256 b3bcff90ff48df2de0bc687c9a6381a964083a03c2280305799a24121699538e

8,964 BYTES / PRESERVED SOURCE IDENTITY

SECTION 040 — Communications Architecture

STATUS: CONTENT-COMPLETE

PURPOSE

Define human, machine, service, event, voice, text, visual, and emergency communication channels, contracts, meaning, identity, delivery, accessibility, privacy, and continuity.

REQUIRED OUTCOME

- A communications landscape with audiences, senders, receivers, channels, content, timing, identity, authority, delivery, evidence, and lifecycle.
- Explicit normal, delayed, failed, emergency, multilingual, accessible, offline, and recovery behavior.
- Traceability from information, processes, applications, networks, events, operations, governance, tests, and retirement.

WHY THIS MATTERS

Communication can create commitments, warnings, records, consent, instructions, or harm. A delivered message is not necessarily understood, authorized, accessible, timely, or complete.

APPLICABILITY

- Systems or organizations exchange consequential human or machine communication.
- Delivery, interpretation, identity, accessibility, privacy, timing, records, or channel failure affects outcomes.

NON-APPLICABILITY CONDITIONS

- No material channel, audience, message meaning, identity, timing, accessibility, privacy, or lifecycle effect changes.
- An approved baseline covers exact channel and change with no material impact.
- The artifact is exploratory and not a delivery, comprehension, or notice claim.

Record baseline, scope, owner, evidence date, assumptions, and reopening trigger.

PREREQUISITES AND DEPENDENCIES

- Sections 019-039 provide requirements, information, processes, applications, network, distributed, security, and operations context.
- Content, identity, accessibility, privacy, records, carrier, localization, and emergency authorities are known or open.

REQUIRED INPUTS

- Audiences, affected parties, purposes, channels, messages, languages, timing, urgency, consent, jurisdiction, and lifecycle.
- Delivery, identity, authorization, templates, event, notification, logging, records, accessibility, failure, escalation, and recovery requirements.

HUMAN RESPONSIBILITIES AND AUTHORITY

Communication owners approve purpose, audience, wording, urgency, channel, consent, commitments, notices, escalation, emergency use, and records. Humans resolve ambiguity and retain authority to pause, correct, retract, escalate, and communicate during failure.

AI RESPONSIBILITIES AND BOUNDARIES

AI may classify authorized messages, draft content, translate, summarize, route, detect delivery anomalies, and propose templates. It must preserve source, audience, channel, authority, and uncertainty. It must not invent notices, send consequential messages, impersonate, suppress dissent, or declare comprehension without approval and evidence.

APPLICABLE EXISTING AGENT ROLES

- Architect, Researcher, Analyst, Observer, Planner, Coordinator, Documentation Writer, User Experience Reviewer, Governance Reviewer, Security Auditor, Tester, Verifier, Critic, Memory Curator, and Red Team.

REQUIRED ACTIVITIES

1. Define audiences, purpose, message types, channels, identities, urgency, languages, and owners.
2. Model message creation, review, approval, delivery, receipt, comprehension, correction, escalation, and retention.
3. Define contracts for content, identity, authorization, timing, ordering, duplication, delivery, evidence, and opt-out.
4. Address accessibility, localization, privacy, security, records, emergency, offline, and degraded channels.
5. Validate with users, operators, content, domain, accessibility, privacy, security, legal, and support authorities.
6. Test delivery, delay, duplication, wrong audience, translation, inaccessible channel, provider loss, correction, and recovery.
7. Baseline and hand off content, channel, evidence, decisions, and gaps.

HUMAN-AI INTERACTION REQUIREMENTS

- Identify sender, reviewer, AI or automation, recipient, channel, authority, data, and external consequence.
- Require human review for legal, safety, emergency, rights-affecting, privacy, security, or externally binding messages.
- Provide accessible status, alternative channel, correction, appeal, and human contact paths.
- Never let fluent generation, delivery receipt, open rate, or model confidence substitute for truth or comprehension.

ARCHITECTURE AND SYSTEM IMPLICATIONS

- Separate source message, template, generated draft, approved content, transmitted message, receipt, response, record, and evidence.
- Enforce identity, consent, authorization, rate, audience, privacy, accessibility, and retention across all channels.
- Define ordering, deduplication, expiry, retry, fallback, translation, and emergency override semantics.
- Preserve audit and correction when providers, networks, models, or channels fail.

OWNER COCKPIT AND USER-CONTROL REQUIREMENTS

- Show message, audience, sender, channel, approval, status, urgency, language, accessibility, authority, and evidence freshness.
- Expose send, hold, correct, retract, escalate, alternate-channel, and recovery actions with actor and impact.
- Permit authorized pause, revoke, quarantine, resend, and retire actions.
- Export message history, content versions, delivery evidence, decisions, dissent, and continuity state.

REQUIRED OUTPUTS AND ARTIFACTS

- Communications context, audience and channel catalog, message and content contracts, identity and approval model.
- Accessibility, localization, privacy, security, delivery, records, emergency, observability, incident, and recovery designs.
- Verification, evidence, decision, risk, and handoff package.

EVIDENCE REQUIREMENTS

- Claims identify message, version, audience, channel, sender, approval, time, method, result, and limitations.

- Delivery, receipt, open, response, survey, or simulation evidence proves only its bounded claim and not comprehension or truth.
- Accessibility and localization evidence identifies population, method, coverage, and limitations.
- Failed, delayed, wrong-audience, inaccessible, or missing evidence remains visible and constrains status.

VERIFICATION AND ACCEPTANCE TESTS

- Trace a consequential notice from source through authoring, approval, delivery, receipt, response, record, and retention.
- Exercise wrong recipient, duplicate, delayed, out-of-order, translated, inaccessible, spoofed, retracted, and provider-loss paths.
- Verify identity, consent, authorization, audit, opt-out, emergency, accessibility, and alternate-channel behavior.
- Disable channel, model, network, carrier, identity, and evidence services and verify safe fallback.
- Reproduce representative claims from the exact content and delivery package.

FAILURE STATES AND ANTI-PATTERNS

- Treating delivery, open rate, or polished wording as comprehension or truth.
- Sending without identity, approval, audience validation, accessibility, privacy, or correction.
- Using one channel or language for materially different populations without explicit justification.
- Allowing AI to impersonate, commit, threaten, or suppress without authority.

RECOVERY AND RECONSIDERATION

- On wrong audience, false content, lost delivery, channel compromise, or evidence failure, pause affected communication and preserve versions.
- Identify affected parties, owner, authority, correction, notification, evidence, and review condition.
- Use approved retraction, correction, alternate channel, manual contact, quarantine, or incident response.
- Reopen on audience, channel, message purpose, language, law, provider, or lifecycle change.

COMPLETION CRITERIA

- Audiences, channels, content, identity, authority, accessibility, delivery, evidence, correction, recovery, and lifecycle are explicit.
- Failure, emergency, localization, offline, and retirement paths are addressed or justified.
- Exact evidence limits and unresolved gaps are recorded; CONTENT-COMPLETE does not imply understood or accepted communication.

HANDOFF AND CONTINUITY REQUIREMENTS

- Preserve content versions, audiences, channels, approvals, delivery records, incidents, evidence, dissent, and gaps.
- Hand off to Information, Network, Integration, API, Event, Security, Privacy, Accessibility, Content, Operations, Testing, and Records.
- Record changes requiring reopening Application, Data, Governance, Safety, or Compliance architecture.

RELATED CODEX SECTIONS

- 019-039 Requirements through Network Architecture; 041-044 Integration through Runtime Architecture.
- 045-060 Security through Hardware Architecture; 088-103 voice, conversational, accessibility, and localization design; 313-318 observability.

SOURCE BASIS AND UNRESOLVED SOURCE GAPS

- Based on the Canonical Section Contract, shared governance files, Sections 019-039, Owner Cockpit Requirements, Status and Evidence Lexicon, and local architecture baseline.
- No universal channel, notice, accessibility, localization, carrier, records, or emergency standard is supplied. Actual audiences, consent, authority, content, and comprehension evidence remain unresolved.

041

SECTION 041 - INTEGRATION ARCHITECTURE

CODEX/03_SECTIONS/041_INTEGRATION_ARCHITECTURE.txt

SHA-256 9b7aca4d7be6b673a4b837bf7edffb76694d0f8e18b4646bb409f263c5705db3

8,582 BYTES / PRESERVED SOURCE IDENTITY

SECTION 041 — Integration Architecture

STATUS: CONTENT-COMPLETE

PURPOSE

Define how applications, services, data, devices, organizations, and providers cooperate through governed interfaces, mappings, orchestration, synchronization, and failure handling.

REQUIRED OUTCOME

- An integration landscape with participants, ownership, trust, contracts, data, protocols, transformations, timing, dependencies, and lifecycle.
- Explicit delivery, error, retry, compensation, reconciliation, observability, security, privacy, and recovery behavior.
- Traceability from requirements, processes, functions, applications, data, network, APIs, events, tests, operations, and retirement.

WHY THIS MATTERS

Integration turns local behavior into cross-boundary consequence. A successful connection can still corrupt meaning, duplicate effects, leak data, or leave an unreconciled partial result.

APPLICABILITY

- Two or more bounded systems, services, domains, organizations, devices, or providers exchange data, commands, events, or decisions.
- Mapping, orchestration, synchronization, identity, timing, or failure affects material outcomes.

NON-APPLICABILITY CONDITIONS

- No material cross-boundary exchange, contract, transformation, authority, or lifecycle effect changes.
- An approved integration baseline covers exact participants and change with no material impact.
- The artifact is exploratory and not an interoperability, delivery, or data-quality claim.

Record baseline, scope, owners, evidence date, assumptions, and reopening trigger.

PREREQUISITES AND DEPENDENCIES

- Sections 019-040 provide requirements, applications, data, networks, communication, distributed behavior, security, and operations context.
- Participant, data, identity, contract, provider, records, privacy, and recovery authorities are known or open.

REQUIRED INPUTS

- Participants, data, commands, events, schemas, mappings, protocols, identity, authority, timing, delivery, error, volume, and lifecycle needs.
- Contracts, access, transformation, reconciliation, observability, migration, testing, support, and retirement obligations.

HUMAN RESPONSIBILITIES AND AUTHORITY

Integration owners approve participants, purpose, data sharing, mappings, external effects, service commitments, exceptions, and change. Domain and data owners retain authority over meaning and use. Humans retain stop, quarantine, reconcile, compensate, and retire authority.

AI RESPONSIBILITIES AND BOUNDARIES

AI may compare schemas, draft mappings, identify incompatibility, analyze traces, and propose tests. It must preserve source and target meaning, versions, permissions, and uncertainty. It must not approve mappings, transfer restricted data, issue external commands, or declare reconciliation without authority and evidence.

APPLICABLE EXISTING AGENT ROLES

- Architect, Analyst, Researcher, Implementer, Planner, Coordinator, Tester, Verifier, Security Auditor, Governance Reviewer, Critic, Documentation Writer, Memory Curator, Optimizer, and Red Team.

REQUIRED ACTIVITIES

1. Inventory participants, owners, flows, contracts, data, identities, providers, and failure domains.
2. Define purpose, authority, schema, mappings, transformations, timing, delivery, idempotency, and compatibility.
3. Model normal, delayed, duplicate, partial, incompatible, unauthorized, degraded, migration, and retirement paths.
4. Define reconciliation, compensation, evidence, audit, privacy, security, and operational controls.
5. Validate with domain, data, application, security, privacy, operations, and supplier owners.
6. Test failures, retries, replay, corruption, drift, provider loss, and recovery.
7. Baseline the integration and hand off open decisions and evidence limits.

HUMAN-AI INTERACTION REQUIREMENTS

- Show source, target, actor, data, command, model, provider, authority, and external effect for each consequential exchange.
- Require human review for new data sharing, mappings, transformations, commands, or irreversible effects.
- Preserve manual status, evidence, quarantine, correction, and recovery paths.
- Do not let AI-generated mapping or orchestration become binding without approval.

ARCHITECTURE AND SYSTEM IMPLICATIONS

- Separate source, transformed, mapped, delivered, acknowledged, reconciled, compensated, and evidence states.
- Define identity, schema, version, ordering, delivery, retry, deduplication, timeout, and compensation contracts.
- Enforce privacy, authorization, minimization, audit, validation, and rate controls at integration boundaries.
- Preserve append-only evidence and durable recovery independent of the failed participant where feasible.

OWNER COCKPIT AND USER-CONTROL REQUIREMENTS

- Show participants, flows, versions, status, lag, errors, data scope, authority, evidence, and lifecycle.
- Expose send, pause, retry, quarantine, reconcile, compensate, restore, and retire actions with actor and impact.
- Permit authorized stop, revoke, narrow, and recovery actions.
- Export contracts, mappings, traces, decisions, evidence, dissent, and continuity state.

REQUIRED OUTPUTS AND ARTIFACTS

- Integration context, participant and flow catalog, contract and mapping records, identity and trust model.
- Error, retry, reconciliation, compensation, observability, security, privacy, migration, support, and retirement designs.
- Test, evidence, decision, risk, acceptance, and handoff package.

EVIDENCE REQUIREMENTS

- Claims identify source, target, contract version, data, environment, timing, method, actor, result, and limitation.
- Connection, sample transfer, schema match, acknowledgement, or dashboard proves only its bounded claim.
- Reconciliation evidence identifies population, rules, exceptions, duplicates, missing records, and validity.
- Failed, partial, stale, contradictory, or missing evidence constrains integration status.

VERIFICATION AND ACCEPTANCE TESTS

- Trace a representative cross-boundary outcome through mapping, transfer, processing, acknowledgement, reconciliation, and record.
- Exercise duplicate, lost, delayed, reordered, malformed, unauthorized, stale, partial, incompatible, and replayed exchanges.
- Verify access, minimization, audit, idempotency, compensation, quarantine, and recovery.
- Disable each participant, provider, network, schema registry, credential, and evidence path and verify safe behavior.
- Reproduce claims from exact contracts and evidence.

FAILURE STATES AND ANTI-PATTERNS

- Treating connectivity or acknowledgement as successful integration or correct outcome.
- Hiding transformations, shared credentials, manual reconciliation, provider dependencies, or partial effects.
- Allowing retries to duplicate external consequences or AI to infer mapping authority.

RECOVERY AND RECONSIDERATION

- On mismatch, leakage, corruption, duplicate effect, participant loss, or evidence failure, pause affected exchange and preserve records.
- Identify safe continuation, owners, authority, affected data, evidence, communication, and review condition.
- Use approved quarantine, replay, reconciliation, compensation, restore, fallback, migration, or retirement.
- Reopen on participant, contract, data, policy, provider, or lifecycle change.

COMPLETION CRITERIA

- Participants, purpose, contracts, mappings, authority, controls, evidence, failure, recovery, and lifecycle are explicit.
- Partial, incompatible, delayed, duplicate, migration, and retirement behavior is addressed or justified.
- Exact validity and unresolved gaps are recorded.

HANDOFF AND CONTINUITY REQUIREMENTS

- Preserve participants, contracts, mappings, versions, traces, decisions, evidence, dissent, incidents, and gaps.
- Hand off to API, Event, Data, Network, Security, Privacy, Operations, Testing, Migration, and Retirement.
- Record changes requiring reopening Application, Distributed, Information, or Governance architecture.

RELATED CODEX SECTIONS

- 019-040 Requirements through Communications Architecture; 042-044 API through Runtime Architecture.
- 045-060 Security through Hardware Architecture; 128-131 API, integration, and distributed engineering; 301-406 operations and lifecycle.

SOURCE BASIS AND UNRESOLVED SOURCE GAPS

- Based on the Canonical Section Contract, shared governance files, Sections 019-040, Owner Cockpit Requirements, Status and Evidence Lexicon, and local architecture baseline.
- No universal integration, interchange, data contract, supplier, or reconciliation standard is supplied. Actual participants, mappings, authority, data agreements, and acceptance evidence remain unresolved.

042

SECTION 042 - API ARCHITECTURE

CODEX/03_SECTIONS/042_API_ARCHITECTURE.txt

SHA-256 05a31d1a385ddb753f733a488a762203c3d12c80578e20ff12b3c8ed394ce573

8,376 BYTES / PRESERVED SOURCE IDENTITY

SECTION 042 — API Architecture

STATUS: CONTENT-COMPLETE

PURPOSE

Define externally consumable application interfaces, resources, operations, schemas, identity, authorization, versioning, limits, errors, observability, and lifecycle.

REQUIRED OUTCOME

- An API inventory and contract model with purpose, consumers, ownership, resources, operations, data, authority, versions, compatibility, and lifecycle.
- Explicit validation, error, idempotency, pagination, rate, audit, privacy, security, accessibility, and recovery behavior.
- Traceability from requirements, functions, applications, data, integrations, tests, operations, and retirement.

WHY THIS MATTERS

An API is a public promise and a control boundary. Ambiguous schemas, weak authorization, unsafe retries, undocumented deprecation, or misleading errors can cause durable external effects.

APPLICABILITY

- A system exposes or consumes programmatic operations, data, events, tools, or administrative interfaces.
- API contracts affect identity, rights, data, compatibility, availability, cost, or external state.

NON-APPLICABILITY CONDITIONS

- No material interface, operation, schema, authority, consumer, or lifecycle effect changes.
- An approved API baseline covers exact contract and change with no material impact.
- The artifact is exploratory and not an availability, compatibility, or security claim.

Record baseline, scope, owner, evidence date, assumptions, and reopening trigger.

PREREQUISITES AND DEPENDENCIES

- Sections 019-041 provide requirements, application, data, integration, network, security, privacy, and operations context.
- Consumer, identity, schema, version, rate, records, accessibility, and support authorities are known or open.

REQUIRED INPUTS

- Consumers, operations, resources, schemas, identity, authorization, data classes, timing, volume, errors, versions, and lifecycle.
- Validation, audit, rate, idempotency, pagination, observability, migration, deprecation, support, and recovery requirements.

HUMAN RESPONSIBILITIES AND AUTHORITY

API owners approve purpose, consumers, operations, data exposure, authorization, versioning, quotas, commitments, exceptions, and retirement. Data and domain owners approve meaning and sharing. Humans retain revoke, throttle, pause, correct, and retire authority.

AI RESPONSIBILITIES AND BOUNDARIES

AI may inspect contracts, propose schemas, identify breaking changes, draft tests, and analyze bounded traffic. It must preserve version, consumer, authority, and data context. It must not expose data, change a contract, grant access, execute privileged operations, or declare compatibility without authority.

APPLICABLE EXISTING AGENT ROLES

- Architect, Analyst, Researcher, Implementer, Tester, Verifier, Security Auditor, Governance Reviewer, Critic, Documentation Writer, Memory Curator, Planner, Optimizer, and Red Team.

REQUIRED ACTIVITIES

1. Inventory APIs, consumers, resources, operations, schemas, identities, dependencies, owners, and lifecycle.
2. Define semantics, validation, authorization, data minimization, limits, errors, idempotency, and audit.
3. Model synchronous, asynchronous, long-running, paged, streamed, batch, admin, and recovery operations.
4. Define versioning, compatibility, deprecation, migration, notification, support, and retirement.
5. Validate with consumers, data, security, privacy, accessibility, operations, support, and domain owners.
6. Test malformed, unauthorized, replayed, concurrent, throttled, stale, partial, and dependency-loss requests.
7. Baseline and hand off contract, evidence, decisions, and open gaps.

HUMAN-AI INTERACTION REQUIREMENTS

- Show caller, operation, data, tool, model, provider, authority, consent, and external effect.
- Require review for privileged, rights-affecting, externally binding, data-sharing, or destructive operations.
- Preserve human status, error explanation, appeal, manual, evidence, and recovery paths.
- AI-generated API calls or schemas remain proposals until authorized.

ARCHITECTURE AND SYSTEM IMPLICATIONS

- Separate contract, implementation, gateway, authorization, request, response, side effect, audit, and evidence.
- Define schema, version, identity, authorization, validation, idempotency, rate, timeout, retry, error, and compatibility semantics.
- Enforce controls consistently across UI, API, batch, event, administrative, and emergency paths.
- Preserve append-only audit and compensation for external effects.

OWNER COCKPIT AND USER-CONTROL REQUIREMENTS

- Show API version, consumer, operation, authority, data scope, rate, status, errors, evidence, and lifecycle.
- Expose approve, deny, revoke, throttle, pause, retry, compensate, deprecate, and retire actions.
- Show actor, request, impact, approval, and recovery path for consequential operations.
- Export contracts, traffic evidence, decisions, dissent, incidents, and continuity state.

REQUIRED OUTPUTS AND ARTIFACTS

- API catalog, context, resource and operation model, schemas, security and data contract, version and lifecycle plan.
- Error, rate, observability, testing, migration, deprecation, support, compensation, and recovery design.
- Evidence, decision, risk, acceptance, and handoff package.

EVIDENCE REQUIREMENTS

- Claims identify API, operation, version, caller, environment, data, method, date, result, and limitation.
- Specification, request success, schema validation, load test, or gateway metric proves only its declared scope.
- Compatibility and security evidence states consumer, version, threat, coverage, and validity.
- Missing, stale, failed, or contradictory evidence constrains contract status.

VERIFICATION AND ACCEPTANCE TESTS

- Trace requirements through API operation, authorization, implementation, consumer, evidence, and retirement.

- Exercise malformed, unauthorized, replayed, duplicate, concurrent, oversized, paged, streamed, throttled, stale, and partial requests.
- Verify data minimization, audit, idempotency, error truthfulness, accessibility, compatibility, and compensation.
- Disable identity, gateway, backend, network, model, provider, and evidence paths and verify safe behavior.
- Reproduce representative claims from exact contract and test package.

FAILURE STATES AND ANTI-PATTERNS

- Treating an API specification or successful request as proof of secure, compatible, or correct behavior.
- Over-broad data exposure, implicit authorization, unsafe retries, unbounded pagination, or undocumented breaking change.
- Allowing AI, admin, or internal callers to bypass public controls.

RECOVERY AND RECONSIDERATION

- On contract break, data leak, abuse, overload, dependency loss, or evidence failure, restrict the operation and preserve records.
- Identify affected consumers, owner, authority, communication, evidence, and safe alternative.
- Use throttle, revoke, fallback, compensation, version rollback, migration, or retirement.
- Reopen on consumer, schema, data, identity, policy, provider, or lifecycle change.

COMPLETION CRITERIA

- Operations, consumers, schemas, authority, data, compatibility, errors, limits, evidence, recovery, and lifecycle are explicit.
- Invalid, abusive, partial, migration, and retirement behavior is addressed or justified.
- Exact validity and unresolved gaps are recorded.

HANDOFF AND CONTINUITY REQUIREMENTS

- Preserve API identities, versions, schemas, consumers, decisions, traffic evidence, incidents, dissent, and gaps.
- Hand off to Integration, Event, Network, Security, Identity, Data, Testing, Operations, and Retirement.
- Record changes requiring reopening Application, Software, Distributed, Privacy, or Governance architecture.

RELATED CODEX SECTIONS

- 019-041 Requirements through Integration Architecture; 043-044 Event-Driven and Runtime Architecture.
- 045-060 Security through Hardware Architecture; 128-129 API and integration engineering; 255-300 delivery; 301-406 operations and lifecycle.

SOURCE BASIS AND UNRESOLVED SOURCE GAPS

- Based on the Canonical Section Contract, shared governance files, Sections 019-041, Owner Cockpit Requirements, Status and Evidence Lexicon, and local architecture baseline.
- No universal API style, protocol, identity, rate, accessibility, or compatibility standard is supplied. Actual consumers, data agreements, threat model, service objectives, and acceptance evidence remain unresolved.

043

SECTION 043 - EVENT DRIVEN ARCHITECTURE

CODEX/03_SECTIONS/043_EVENT_DRIVEN_ARCHITECTURE.txt

SHA-256 8a2487e84fd00f658a3ad37983a29295d8f7b0b709eb36cf5b3ef6ffc6b8c005

8,389 BYTES / PRESERVED SOURCE IDENTITY

SECTION 043 — Event-Driven Architecture

STATUS: CONTENT-COMPLETE

PURPOSE

Define events, producers, consumers, brokers, schemas, ordering, delivery, retention, replay, projection, authorization, and lifecycle for event-driven behavior.

REQUIRED OUTCOME

- An event catalog with meaning, producer, consumer, identity, schema, causality, timing, retention, authority, evidence, and owner.
- Explicit delivery, duplication, ordering, replay, poison, backpressure, privacy, security, reconciliation, and recovery behavior.
- Traceability from requirements, processes, applications, data, APIs, distributed systems, operations, tests, and retirement.

WHY THIS MATTERS

Events can trigger work long after their source action and can be replayed, duplicated, delayed, or consumed by unknown parties. Without explicit semantics, a harmless retry can become a repeated external effect or an irreversible disclosure.

APPLICABILITY

- Systems publish or consume events, notifications, streams, commands, projections, or change records.
- Event meaning, timing, replay, retention, consumer behavior, or delivery affects material outcomes.

NON-APPLICABILITY CONDITIONS

- No event boundary, consumer, schema, replay, retention, authority, or lifecycle effect changes.
- An approved event baseline covers exact topics and change with no material impact.
- The artifact is exploratory and not a delivery, ordering, or outcome claim.

Record baseline, scope, owner, evidence date, assumptions, and reopening trigger.

PREREQUISITES AND DEPENDENCIES

- Sections 019-042 provide requirements, data, APIs, integrations, distributed behavior, security, and operations context.
- Event, schema, broker, retention, privacy, records, identity, and consumer owners are known or open.

REQUIRED INPUTS

- Event purpose, schema, producer, consumer, identity, causality, ordering, timing, volume, retention, sensitivity, and lifecycle.
- Delivery, replay, deduplication, poison handling, backpressure, audit, access, migration, testing, and retirement obligations.

HUMAN RESPONSIBILITIES AND AUTHORITY

Event owners approve event meaning, producers, consumers, data sharing, retention, replay, external effects, and retirement. Domain and data owners approve semantics and use. Humans retain pause, revoke, quarantine, replay, compensate, and retire authority.

AI RESPONSIBILITIES AND BOUNDARIES

AI may inspect event catalogs and traces, detect schema drift, draft consumers and tests, and analyze bounded streams. It must preserve event identity, causality, and data permission. It must not publish, replay, infer meaning, or trigger external effects without authority.

APPLICABLE EXISTING AGENT ROLES

- Architect, Analyst, Researcher, Implementer, Tester, Verifier, Security Auditor, Governance Reviewer, Critic, Documentation Writer, Memory Curator, Planner, Coordinator, Optimizer, and Red Team.

REQUIRED ACTIVITIES

1. Inventory events, producers, consumers, brokers, schemas, topics, identities, owners, and lifecycle.
2. Define meaning, causality, ordering, delivery, retention, replay, access, and compatibility.
3. Model normal, duplicate, delayed, lost, poison, out-of-order, partitioned, replayed, and retired behavior.
4. Define consumer isolation, idempotency, backpressure, dead-letter, reconciliation, audit, and compensation.
5. Validate with domain, data, application, integration, security, privacy, operations, and consumer owners.
6. Test schema evolution, replay, loss, duplication, poison, lag, overload, provider loss, and recovery.
7. Baseline and hand off catalog, evidence, decisions, and gaps.

HUMAN-AI INTERACTION REQUIREMENTS

- Show event, producer, consumer, data, model, tool, authority, replay state, and external effect.
- Require human review for event meaning, new consumers, sensitive data, replay, and external actions.
- Preserve status, evidence, quarantine, manual handling, stop, and recovery paths.
- AI-generated consumers remain proposed until authorized and tested.

ARCHITECTURE AND SYSTEM IMPLICATIONS

- Separate source event, broker record, consumer offset, projection, side effect, replay, dead letter, and evidence.
- Define event identity, causality, schema, ordering, retention, replay, deduplication, expiry, and access semantics.
- Enforce privacy, authorization, minimization, audit, and deletion across topics, logs, replicas, caches, and replays.
- Use append-only history and compensation for external effects.

OWNER COCKPIT AND USER-CONTROL REQUIREMENTS

- Show event, topic, producer, consumers, schema, lag, retention, sensitivity, authority, evidence, and lifecycle.
- Expose publish, pause, drain, quarantine, replay, reset, reconcile, compensate, and retire actions.
- Require approval and show actor, population, external effect, and recovery for consequential actions.
- Export event history, offsets, traces, decisions, evidence, dissent, and continuity state.

REQUIRED OUTPUTS AND ARTIFACTS

- Event catalog, context, schema and ownership records, producer-consumer map, broker and retention model.
- Delivery, replay, dead-letter, privacy, security, observability, migration, reconciliation, compensation, and retirement designs.
- Test, evidence, decision, risk, acceptance, and handoff package.

EVIDENCE REQUIREMENTS

- Claims identify event, schema, producer, consumer, topic, time, configuration, method, result, and limitation.
- Publish, consume, lag, replay, or broker evidence proves only its bounded scope and schedule.
- Ordering, delivery, duplication, retention, privacy, and recovery evidence identifies workload and coverage.
- Missing offsets, schema lineage, or consumer evidence constrains status.

VERIFICATION AND ACCEPTANCE TESTS

- Trace an event from source through broker, consumer, projection, side effect, evidence, retention, and retirement.

- Exercise loss, duplicate, reorder, delay, poison, replay, schema incompatibility, lag, overload, and unauthorized consumption.
- Verify idempotency, access, retention, deletion, dead-letter, compensation, and audit.
- Disable broker, producer, consumer, schema, identity, model, and evidence services and verify safe behavior.
- Reproduce claims from exact event package.

FAILURE STATES AND ANTI-PATTERNS

- Treating publish or broker acknowledgement as successful outcome.
- Assuming exactly-once, ordered, complete, or private delivery without bounded evidence.
- Retaining sensitive events indefinitely or allowing replay to repeat external effects.
- Hiding unknown consumers, poison queues, lag, dead letters, or schema drift.

RECOVERY AND RECONSIDERATION

- On poison, leakage, schema break, replay error, lag, or evidence failure, pause affected topics or consumers and preserve history.
- Identify safe consumers, owner, authority, affected records, evidence, and review condition.
- Use quarantine, drain, replay with fencing, reconciliation, compensation, restore, migration, or retirement.
- Reopen on event meaning, schema, consumer, retention, policy, provider, or lifecycle change.

COMPLETION CRITERIA

- Meaning, producers, consumers, schemas, delivery, replay, retention, authority, evidence, recovery, and lifecycle are explicit.
- Duplicate, lost, delayed, poison, migration, and retirement paths are addressed or justified.
- Exact validity and unresolved gaps are recorded.

HANDOFF AND CONTINUITY REQUIREMENTS

- Preserve event identities, schemas, offsets, consumers, replays, decisions, evidence, incidents, dissent, and gaps.
- Hand off to Integration, API, Data, Distributed, Network, Security, Privacy, Operations, Testing, and Records.
- Record changes requiring reopening Application, Information, or Governance architecture.

RELATED CODEX SECTIONS

- 019-042 Requirements through API Architecture; 044 Runtime Architecture; 045-060 Security through Hardware Architecture.
- 131-145 distributed, real-time, media, and streaming engineering; 153-161 data engineering; 301-406 operations and lifecycle.

SOURCE BASIS AND UNRESOLVED SOURCE GAPS

- Based on the Canonical Section Contract, shared governance files, Sections 019-042, Owner Cockpit Requirements, Status and Evidence Lexicon, and local architecture baseline.
- No universal event, broker, ordering, retention, replay, or privacy standard is supplied. Actual event catalog, consumers, data agreements, recovery objectives, and acceptance evidence remain unresolved.

044

SECTION 044 - RUNTIME ARCHITECTURE

CODEX/03_SECTIONS/044_RUNTIME_ARCHITECTURE.txt

SHA-256 c4857143447ce5d5e5c615d13b0d635c2ec092b15c3aed9a392e4599ef007289

8,938 BYTES / PRESERVED SOURCE IDENTITY

SECTION 044 — Runtime Architecture

STATUS: CONTENT-COMPLETE

PURPOSE

Define the execution environment and behavior of software, services, jobs, agents, models, schedulers, resources, state, observability, control, and lifecycle after deployment.

REQUIRED OUTCOME

- A runtime model with processes, threads or tasks, services, resources, scheduling, state, dependencies, identities, environments, and owners.
- Explicit startup, shutdown, scaling, concurrency, resource limits, upgrade, failure, recovery, and evidence behavior.
- Traceability from application and software design through platform, infrastructure, operations, testing, incident, and retirement.

WHY THIS MATTERS

Runtime behavior depends on timing, scheduling, resource pressure, configuration, provider behavior, and interactions absent from static design. A deployed artifact can be correct in isolation and unsafe under load, restart, drift, or partial failure.

APPLICABILITY

- Software executes over time in managed, hosted, embedded, desktop, mobile, edge, or distributed environments.
- Scheduling, state, resources, concurrency, runtime configuration, or failure affects material outcomes.

NON-APPLICABILITY CONDITIONS

- No material execution, state, resource, schedule, dependency, or lifecycle effect changes.
- An approved runtime baseline covers exact artifact, environment, workload, and change with no material impact.
- The artifact is exploratory and not a runtime readiness or performance claim.

Record baseline, scope, owner, evidence date, assumptions, and reopening trigger.

PREREQUISITES AND DEPENDENCIES

- Sections 019-043 provide requirements, applications, software, platform, infrastructure, distributed, network, API, event, security, and operations context.
- Runtime, resource, scheduler, configuration, identity, model, provider, observability, and recovery authorities are known or open.

REQUIRED INPUTS

- Artifacts, processes, tasks, state, resources, schedules, workload, concurrency, limits, dependencies, environments, and lifecycle.
- Startup, shutdown, health, scaling, upgrade, logging, tracing, metrics, access, backup, restore, failover, and retirement requirements.

HUMAN RESPONSIBILITIES AND AUTHORITY

Runtime owners approve resource limits, scheduling, autonomy, configuration, upgrades, scaling, risk, and recovery. Application and platform owners retain responsibility for behavior and data. Humans retain pause, drain, revoke, restart, restore, and retire authority.

AI RESPONSIBILITIES AND BOUNDARIES

AI may inspect runtime telemetry, identify resource or scheduling anomalies, draft configurations and tests, and recommend bounded actions. It must preserve workload and environment context. It must not change runtime state, kill work, grant authority, or declare reliability without delegation and evidence.

APPLICABLE EXISTING AGENT ROLES

- Architect, Analyst, Researcher, Observer, Planner, Implementer, Tester, Verifier, Security Auditor, Governance Reviewer, Critic, Documentation Writer, Memory Curator, Optimizer, and Red Team.

REQUIRED ACTIVITIES

1. Inventory runtime artifacts, processes, tasks, state, resources, schedules, identities, dependencies, and owners.
2. Define startup, readiness, liveness, shutdown, cancellation, timeout, concurrency, limits, and ownership.
3. Model normal, overloaded, restarted, upgraded, degraded, partitioned, offline, recovery, and retirement behavior.
4. Define configuration, secrets, observability, audit, data, security, privacy, accessibility, and support controls.
5. Validate with application, platform, infrastructure, operations, security, data, and user authorities.
6. Test load, restart, kill, leak, timeout, race, stale configuration, dependency loss, restore, and upgrade rollback.
7. Baseline and hand off runtime evidence, decisions, gaps, and continuity state.

HUMAN-AI INTERACTION REQUIREMENTS

- Show workload, runtime, resource, identity, authority, model, provider, action, and consequence.
- Require human review for killing or migrating work, changing limits, granting runtime access, or changing autonomous behavior.
- Preserve status, logs, evidence, manual control, stop, and recovery paths when runtime automation fails.
- Prevent chat or administrative shortcuts from bypassing runtime controls.

ARCHITECTURE AND SYSTEM IMPLICATIONS

- Separate desired configuration, deployed artifact, runtime instance, process state, telemetry, log, trace, metric, and evidence.
- Define scheduling, resource isolation, health, timeout, cancellation, retry, idempotency, restart, and drain semantics.
- Preserve identity, authorization, secrets, audit, observability, graceful degradation, and safe shutdown across runtime paths.
- Invalidate evidence when artifacts, configuration, workload, provider, or environment changes.

OWNER COCKPIT AND USER-CONTROL REQUIREMENTS

- Show artifact, runtime, environment, workload, resources, state, health, limits, dependencies, owner, authority, and evidence freshness.
- Expose start, stop, drain, pause, restart, scale, revoke, rollback, restore, and retire actions with impact.
- Permit authorized manual intervention and explain inferred versus observed health.
- Export runtime state, telemetry, traces, decisions, evidence, dissent, incidents, and continuation state.

REQUIRED OUTPUTS AND ARTIFACTS

- Runtime context, execution and resource model, lifecycle and scheduling contract, configuration and observability model.
- Health, scaling, security, privacy, resilience, recovery, upgrade, rollback, support, and retirement design.
- Test, evidence, decision, readiness, incident, and handoff package.

EVIDENCE REQUIREMENTS

- Claims identify artifact, runtime, environment, workload, configuration, resource, time, method, actor, result, and limitation.
- Process exit, health signal, benchmark, log, trace, or dashboard proves only its bounded observation.

- Load, concurrency, recovery, and availability evidence identifies workload, coverage, failure injection, and validity.
- Missing or stale telemetry constrains runtime status.

VERIFICATION AND ACCEPTANCE TESTS

- Trace requirements and application behavior through runtime configuration, execution, telemetry, evidence, recovery, and retirement.
- Exercise overload, restart, kill, timeout, race, resource exhaustion, stale config, dependency loss, upgrade, rollback, and restore.
- Verify identity, authorization, secrets, audit, health semantics, graceful shutdown, cancellation, and safe fallback.
- Disable scheduler, runtime provider, model, network, data, and evidence services and verify safe behavior.
- Reproduce representative claims from exact runtime package.

FAILURE STATES AND ANTI-PATTERNS

- Treating process existence, liveness, uptime, or log volume as correctness or reliability proof.
- Ignoring scheduling, resource exhaustion, memory or state leaks, stale configuration, restart, and upgrade behavior.
- Allowing automated remediation to kill, migrate, or alter work without authority, evidence, and compensation.

RECOVERY AND RECONSIDERATION

- On crash, overload, unsafe state, configuration drift, dependency loss, or evidence failure, stop or drain affected work and preserve state.
- Identify safe operation, owner, authority, affected users, evidence, communication, and review condition.
- Use approved restart, reduced mode, drain, rollback, restore, failover, migration, or retirement.
- Reopen on artifact, runtime, scheduler, provider, workload, configuration, or lifecycle change.

COMPLETION CRITERIA

- Execution, state, scheduling, resources, identity, configuration, evidence, recovery, and lifecycle are explicit.
- Load, restart, degradation, upgrade, migration, and retirement are addressed or justified.
- Exact validity and unresolved gaps are recorded; CONTENT-COMPLETE does not imply runtime readiness.

HANDOFF AND CONTINUITY REQUIREMENTS

- Preserve artifact and runtime identities, configurations, telemetry, traces, incidents, decisions, evidence, dissent, and gaps.
- Hand off to Platform, Infrastructure, Cloud, Distributed, Network, Security, Operations, Testing, Reliability, and Retirement.
- Record changes requiring reopening Software, Application, Data, or Governance architecture.

RELATED CODEX SECTIONS

- 019-043 Requirements through Event-Driven Architecture; 045-060 Security through Hardware Architecture.
- 112 Runtime Engineering; 130-145 distributed, real-time, performance, and streaming engineering; 301-406 operations and lifecycle.

SOURCE BASIS AND UNRESOLVED SOURCE GAPS

- Based on the Canonical Section Contract, shared governance files, Sections 019-043, Owner Cockpit Requirements, Status and Evidence Lexicon, and local architecture baseline.
- No universal runtime, scheduler, resource, workload, or service-level standard is supplied. Actual runtime targets, configurations, limits, recovery objectives, and acceptance evidence remain unresolved.

045

SECTION 045 - SECURITY ARCHITECTURE

CODEX/03_SECTIONS/045_SECURITY_ARCHITECTURE.txt

SHA-256 a48a3a3fb715a50168d32bc7d0ec80f8e3768900b19f4edf29ca88d11e024a4f

8,441 BYTES / PRESERVED SOURCE IDENTITY

SECTION 045 — Security Architecture

STATUS: CONTENT-COMPLETE

PURPOSE

Define the security objectives, trust boundaries, assets, threats, controls, identities, detection, response, recovery, and lifecycle protections for the system of interest.

REQUIRED OUTCOME

- A security context and threat model covering assets, actors, trust, attack paths, controls, residual risk, evidence, and owners.
- Enforceable confidentiality, integrity, availability, accountability, privacy, safety, and recovery controls across all material paths.
- Traceability from requirements through design, implementation, operation, incidents, verification, and retirement.

WHY THIS MATTERS

Security is a property of the whole socio-technical system, not a checklist or perimeter. Unmodeled identities, suppliers, logs, recovery tools, AI prompts, or physical paths can defeat a strong local control.

APPLICABILITY

- Any system handles assets, identities, data, capabilities, external interfaces, physical effects, or consequential decisions.
- Security compromise or control failure could cause material harm, loss, rights impact, or governance failure.

NON-APPLICABILITY CONDITIONS

- No asset, trust boundary, identity, data, external effect, control, evidence, or lifecycle security condition changes.
- An approved security baseline covers exact scope and change with no material impact.
- The artifact is exploratory and not a security, compliance, or absence-of-vulnerability claim.

Record baseline, scope, owner, evidence date, assumptions, and reopening trigger.

PREREQUISITES AND DEPENDENCIES

- Sections 019-044 provide requirements, architecture, data, interfaces, runtime, operations, and recovery context.
- Security, identity, privacy, safety, legal, incident, supplier, and acceptance authorities are attributable or marked open.

REQUIRED INPUTS

- Assets, actors, data, identities, trust boundaries, threats, abuse cases, controls, dependencies, environments, and criticality.
- Vulnerability, detection, response, recovery, evidence, testing, supplier, model, tool, and retirement requirements.

HUMAN RESPONSIBILITIES AND AUTHORITY

Security owners approve protection objectives, residual risk, exceptions, access, incident severity, disclosure, recovery, and retirement. Asset and domain owners approve business impact. Humans retain authority to stop, isolate, revoke, contain, restore, and accept risk.

AI RESPONSIBILITIES AND BOUNDARIES

AI may inventory authorized assets, model threats, analyze evidence, draft controls and tests, and identify anomalies. It must preserve scope, evidence, and uncertainty. It must not declare secure, suppress findings, accept risk, expose secrets, or execute containment beyond delegation.

APPLICABLE EXISTING AGENT ROLES

- Security Auditor, Red Team, Architect, Analyst, Researcher, Observer, Tester, Verifier, Governance Reviewer, Critic, Planner, Implementer, Documentation Writer, Memory Curator, and Coordinator.

REQUIRED ACTIVITIES

1. Define assets, actors, trust boundaries, impact, objectives, and ownership.
2. Identify threats, abuse cases, attack paths, assumptions, dependencies, and control gaps.
3. Design preventive, detective, corrective, compensating, and recovery controls.
4. Allocate identity, authorization, cryptography, secrets, isolation, logging, alerting, response, and evidence controls.
5. Validate with asset owners, users, operators, privacy, safety, legal, suppliers, and affected parties.
6. Test misuse, bypass, privilege, injection, exfiltration, tampering, denial, recovery, and retirement paths.
7. Baseline risk, decisions, evidence, exceptions, dissent, and handoff.

HUMAN-AI INTERACTION REQUIREMENTS

- Show actor, asset, authority, tool, data, model, provider, threat, control, and consequence.
- Require human review for risk acceptance, exceptions, disclosure, containment, privilege, and irreversible security actions.
- Preserve independent stop, evidence, incident, appeal, and recovery paths when AI fails.
- Never treat model confidence or a clean scan as proof of security.

ARCHITECTURE AND SYSTEM IMPLICATIONS

- Enforce security at identity, data, interface, runtime, network, physical, supplier, management, and recovery boundaries.
- Separate policy, decision, enforcement, telemetry, evidence, incident, and compensation records.
- Bind controls to assets, actors, scope, conditions, expiry, evidence, and revocation.
- Preserve append-only audit and non-destructive compensation for recovery.

OWNER COCKPIT AND USER-CONTROL REQUIREMENTS

- Show assets, threats, controls, findings, exceptions, risk, actor, authority, evidence freshness, and lifecycle.
- Expose approve, deny, revoke, isolate, contain, investigate, restore, compensate, and retire actions.
- Explain control state, uncertainty, affected users, and evidence that would change the decision.
- Export threat model, decisions, findings, evidence, dissent, incidents, and continuity state.

REQUIRED OUTPUTS AND ARTIFACTS

- Security context, asset and trust catalog, threat model, control architecture, risk register, and exception records.
- Identity, access, data, secrets, cryptographic, monitoring, incident, recovery, supplier, AI, and retirement designs.
- Verification, evidence, acceptance, and handoff package.

EVIDENCE REQUIREMENTS

- Claims identify asset, scope, threat, control, configuration, environment, method, date, result, and limitation.
- Scan, review, test, attestation, incident, or provider claim proves only its coverage and time.
- Control evidence distinguishes existence, operation, and effectiveness; unknown remains unknown.
- Failed, stale, suppressed, contradictory, or missing findings constrain security status.

VERIFICATION AND ACCEPTANCE TESTS

- Trace assets and threats through controls, enforcement, evidence, incident, recovery, and retirement.
- Exercise unauthorized access, privilege escalation, injection, tampering, exfiltration, replay, denial, insider, supplier, and recovery paths.
- Verify controls cannot be bypassed through UI, API, bulk, admin, AI, offline, or emergency paths.
- Disable detection, identity, key, network, provider, model, and evidence services and verify safe containment.
- Reproduce representative security claims from exact baseline and evidence.

FAILURE STATES AND ANTI-PATTERNS

- Treating compliance paperwork, scan results, encryption, or perimeter controls as complete security.
- Hiding assets, threats, residual risk, exceptions, false positives, supply-chain, recovery, or human factors.
- Allowing convenience, AI, emergency access, or owner pressure to bypass controls or audit.

RECOVERY AND RECONSIDERATION

- On compromise, control failure, risk change, evidence loss, or authority conflict, contain affected scope and preserve immutable records.
- Identify safe work, affected parties, owner, authority, disclosure, evidence, and review condition.
- Use approved isolation, credential rotation, patch, restore, compensation, disclosure, or retirement paths.
- Reopen on asset, threat, control, provider, data, model, population, or lifecycle change.

COMPLETION CRITERIA

- Assets, threats, controls, owners, residual risk, evidence, response, recovery, and lifecycle are explicit.
- Boundary, abuse, supplier, AI, incident, and retirement behavior is addressed or justified.
- Exact validity, open findings, and acceptance authority are recorded.

HANDOFF AND CONTINUITY REQUIREMENTS

- Preserve threat, asset, control, finding, exception, incident, decision, evidence, dissent, and gap identities.
- Hand off to Identity, Privacy, Governance, Compliance, Safety, Reliability, Operations, Testing, and Retirement.
- Record changes requiring reopening all affected architecture and risk sections.

RELATED CODEX SECTIONS

- 019-044 Requirements through Runtime Architecture; 046-060 Identity through Hardware Architecture.
- 189-217 security, threat, incident, fraud, and trust engineering; 246-252 security, compliance, and safety testing; 301-406 operations and lifecycle.

SOURCE BASIS AND UNRESOLVED SOURCE GAPS

- Based on the Canonical Section Contract, shared governance files, Sections 019-044, Owner Cockpit Requirements, Status and Evidence Lexicon, and local architecture baseline.
- No universal threat taxonomy, control catalog, risk appetite, jurisdiction, or assurance regime is supplied. Actual assets, threats, owners, evidence, and acceptance remain unresolved.

046

SECTION 046 - IDENTITY ARCHITECTURE

CODEX/03_SECTIONS/046_IDENTITY_ARCHITECTURE.txt

SHA-256 cbc1983355403d6151a36d0c4778ef8b76020351870cee4ab4493284c1b74574

8,331 BYTES / PRESERVED SOURCE IDENTITY

SECTION 046 — Identity Architecture

STATUS: CONTENT-COMPLETE

PURPOSE

Define how people, organizations, services, devices, agents, roles, credentials, and delegated authorities are identified, linked, verified, represented, changed, and retired.

REQUIRED OUTCOME

- An identity landscape with subjects, identifiers, attributes, authorities, trust, proofing, federation, lifecycle, and owners.
- Explicit authentication, delegation, impersonation, recovery, revocation, privacy, audit, and evidence behavior.
- Traceability from requirements through authorization, applications, data, operations, incidents, tests, and retirement.

WHY THIS MATTERS

Identity is the basis for authority and accountability. Ambiguous subjects, shared credentials, stale roles, weak recovery, or untracked delegation can make every downstream control unreliable.

APPLICABILITY

- People or non-human actors access resources, make decisions, invoke tools, or create records.
- Identity, federation, attributes, roles, credentials, proofing, or delegation affects material rights or risk.

NON-APPLICABILITY CONDITIONS

- No identity, subject, credential, role, delegation, access, audit, or lifecycle effect changes.
- An approved identity baseline covers exact scope and change with no material impact.
- The artifact is exploratory and not an authentication or attribution claim.

Record baseline, scope, owner, evidence date, assumptions, and reopening trigger.

PREREQUISITES AND DEPENDENCIES

- Sections 019-045 provide requirements, security, applications, data, interfaces, operations, and governance context.
- Subject, directory, credential, federation, privacy, legal, support, and revocation authorities are known or open.

REQUIRED INPUTS

- Subjects, identifiers, attributes, roles, resources, proofing, authentication, federation, delegation, recovery, jurisdictions, and lifecycle.
- Credential, secret, session, audit, privacy, accessibility, break-glass, support, and retirement requirements.

HUMAN RESPONSIBILITIES AND AUTHORITY

Identity owners approve subject classes, proofing, role meaning, delegation, federation, recovery, exceptions, and retirement. Resource owners approve access. Humans retain authority to revoke, suspend, correct, investigate, and restore identity.

AI RESPONSIBILITIES AND BOUNDARIES

AI may inventory identity flows, detect anomalies, compare role definitions, draft tests, and recommend review. It must preserve subject and evidence context. It must not infer identity, approve access, impersonate, reset credentials, or declare proofing sufficient without authority.

APPLICABLE EXISTING AGENT ROLES

- Architect, Analyst, Researcher, Security Auditor, Governance Reviewer, Tester, Verifier, Critic, Observer, Implementer, Documentation Writer, Memory Curator, Planner, and Red Team.

REQUIRED ACTIVITIES

1. Inventory subjects, identifiers, attributes, roles, credentials, directories, providers, resources, and owners.
2. Define proofing, authentication, federation, delegation, session, recovery, revocation, and audit semantics.
3. Model joiner, mover, leaver, service, device, agent, emergency, compromised, and retired states.
4. Minimize identity data and allocate privacy, security, accessibility, support, and continuity controls.
5. Validate with users, resource owners, security, privacy, legal, HR or equivalent, support, and domain authorities.
6. Test spoofing, replay, privilege, stale role, federation loss, recovery abuse, revocation, and break-glass.
7. Baseline and hand off identity records, evidence, decisions, dissent, and gaps.

HUMAN-AI INTERACTION REQUIREMENTS

- Show subject, proof, credential, role, authority, delegation, resource, actor, and consequence.
- Require human approval for privileged access, delegation, impersonation, recovery, exception, and irreversible identity changes.
- Preserve non-AI recovery, status, appeal, revocation, and evidence paths.
- Never let behavioral similarity, model confidence, or convenience substitute for identity proof.

ARCHITECTURE AND SYSTEM IMPLICATIONS

- Separate subject identity, identifier, attributes, credential, session, role, authorization decision, audit, and evidence.
- Bind credentials and delegated capability to scope, purpose, expiry, audience, and revocation.
- Enforce identity consistently across UI, API, batch, events, admin, AI, offline, and emergency paths.
- Preserve historical attribution without retaining unnecessary sensitive identity data.

OWNER COCKPIT AND USER-CONTROL REQUIREMENTS

- Show subject, role, sessions, credentials, delegated authority, resources, provider, status, evidence, and lifecycle.
- Expose approve, deny, revoke, suspend, recover, correct, investigate, and retire actions with actor and scope.
- Explain access and denial without disclosing protected information.
- Export identity history, decisions, evidence, dissent, incidents, and continuity state.

REQUIRED OUTPUTS AND ARTIFACTS

- Identity context, subject and identifier catalog, proofing and federation model, credential and role lifecycle.
- Delegation, session, recovery, revocation, privacy, security, accessibility, audit, support, and retirement designs.
- Verification, evidence, decision, risk, acceptance, and handoff package.

EVIDENCE REQUIREMENTS

- Claims identify subject class, provider, proofing, credential, role, scope, time, method, actor, result, and limitation.
- Login, token, directory, attestation, or anomaly evidence proves only its bounded claim.
- Access evidence distinguishes authentication from authorization and attribution.
- Stale, failed, inaccessible, or missing identity evidence constrains status.

VERIFICATION AND ACCEPTANCE TESTS

- Trace identity from proofing through session, authorization, action, audit, revocation, recovery, and retirement.

- Exercise spoofing, replay, credential theft, stale role, federation loss, delegated expiry, reset abuse, and break-glass.
- Verify least privilege, separation, attribution, privacy, accessibility, audit, and revocation across all paths.
- Disable directory, identity provider, key, network, model, and evidence services and verify safe behavior.
- Reproduce representative identity claims from exact records.

FAILURE STATES AND ANTI-PATTERNS

- Treating a valid credential as valid authority or a login as proof of safe attribution.
- Shared accounts, perpetual roles, unbounded delegation, weak recovery, hidden impersonation, or stale directories.
- Allowing AI or admin convenience to bypass proofing, consent, authorization, or audit.

RECOVERY AND RECONSIDERATION

- On compromise, proofing dispute, provider loss, stale role, or evidence failure, suspend affected authority and preserve records.
- Identify safe access, owner, subject impact, authority, evidence, communication, and review condition.
- Use approved credential rotation, reproofing, manual recovery, restricted mode, revocation, or retirement.
- Reopen on subject, role, provider, policy, population, or lifecycle change.

COMPLETION CRITERIA

- Subjects, identifiers, proofing, credentials, roles, authority, delegation, recovery, evidence, and lifecycle are explicit.
- Compromise, provider loss, recovery, break-glass, migration, and retirement are addressed or justified.
- Exact validity and unresolved gaps are recorded.

HANDOFF AND CONTINUITY REQUIREMENTS

- Preserve identity, role, credential, delegation, session, decision, evidence, incident, dissent, and gap identities.
- Hand off to Security, Privacy, Governance, Compliance, Application, API, Operations, Testing, and Records.
- Record changes requiring reopening all affected access and architecture sections.

RELATED CODEX SECTIONS

- 019-045 Requirements through Security Architecture; 047-060 Privacy through Hardware Architecture.
- 200-205 identity, authentication, authorization, cryptography, keys, and secrets management; 246-251 testing and compliance.

SOURCE BASIS AND UNRESOLVED SOURCE GAPS

- Based on the Canonical Section Contract, shared governance files, Sections 019-045, Owner Cockpit Requirements, Status and Evidence Lexicon, and local architecture baseline.
- No universal proofing, federation, role, privacy, accessibility, or identity assurance standard is supplied. Actual subjects, providers, authorities, recovery, and evidence remain unresolved.

047

SECTION 047 - PRIVACY ARCHITECTURE

CODEX/03_SECTIONS/047_PRIVACY_ARCHITECTURE.txt

SHA-256 1d6a9c876c97794b58f896b371076eb74050d5daa9d73852b093e1935c0cc360

8,888 BYTES / PRESERVED SOURCE IDENTITY

SECTION 047 — Privacy Architecture

STATUS: CONTENT-COMPLETE

PURPOSE

Define how personal, sensitive, inferred, linked, generated, and contextual information is collected, used, shared, protected, explained, corrected, retained, and deleted within authorized purposes.

REQUIRED OUTCOME

- A data-subject, purpose, processing, sharing, rights, risk, control, and lifecycle model.
- Enforceable minimization, purpose limitation, transparency, access, correction, objection, portability, retention, deletion, and breach-response behavior as applicable.
- Traceability from requirements through data, identity, applications, security, operations, evidence, and retirement.

WHY THIS MATTERS

Privacy harm can arise from inference, linkage, context, retention, or access even when a system has no obvious breach. Privacy Architecture makes purpose and human impact visible instead of treating consent text or masking as complete protection.

APPLICABILITY

- Data relates to identifiable people, households, devices, behavior, location, biometrics, sensitive attributes, or inferred interests.
- Collection, use, generation, sharing, retention, or deletion affects rights, dignity, autonomy, safety, or trust.

NON-APPLICABILITY CONDITIONS

- No personal, sensitive, linked, inferred, or contextual data or privacy obligation is affected.
- An approved baseline covers exact data, purpose, population, jurisdiction, and change with no material impact.
- The artifact is synthetic or exploratory and makes no claim about lawful or privacy-safe processing.

Record baseline, scope, owner, evidence date, assumptions, jurisdiction, and reopening trigger.

PREREQUISITES AND DEPENDENCIES

- Sections 019-046 provide requirements, data, identity, security, information, application, and operational context.
- Legal, privacy, data, security, records, user, supplier, and rights authorities are known or marked open.

REQUIRED INPUTS

- Subjects, data, purposes, lawful basis or authority, populations, jurisdictions, recipients, processing, inference, models, retention, and rights.
- Notices, consent, access, correction, objection, portability, deletion, security, breach, records, and evidence requirements.

HUMAN RESPONSIBILITIES AND AUTHORITY

Privacy and accountable owners approve purposes, collection, sharing, sensitive processing, retention, rights handling, exceptions, transfers, risk, and acceptance. Humans resolve conflicts and retain authority to restrict, correct, withdraw, delete, notify, and stop processing.

AI RESPONSIBILITIES AND BOUNDARIES

AI may inventory data uses, map flows, identify minimization or retention gaps, and draft privacy tests. It must preserve subject, purpose, jurisdiction, provenance, and uncertainty. It must not infer consent, approve new use, identify a person, deny rights, or expose personal data without authority.

APPLICABLE EXISTING AGENT ROLES

- Privacy and Governance Reviewer, Security Auditor, Architect, Researcher, Analyst, Observer, Tester, Verifier, Critic, Documentation Writer, Memory Curator, Planner, and Red Team.

REQUIRED ACTIVITIES

1. Inventory subjects, data, purposes, processing, recipients, jurisdictions, risks, owners, and lifecycle.
2. Define minimization, purpose limitation, transparency, consent or authority, access, correction, objection, portability, retention, and deletion.
3. Map source, transformation, inference, model, prompt, tool, sharing, storage, logs, backups, and evidence copies.
4. Assess re-identification, linkage, discrimination, surveillance, disclosure, inference, and secondary-use risk.
5. Validate with affected people, domain, privacy, legal, security, accessibility, records, and operations authorities.
6. Test rights, access, correction, deletion, restricted processing, breach, model, cache, backup, and recovery paths.
7. Baseline decisions, evidence, dissent, open gaps, and handoff.

HUMAN-AI INTERACTION REQUIREMENTS

- Show subject, purpose, data, recipient, model, provider, authority, decision, and consequence.
- Require human review for sensitive, rights-affecting, inferred, cross-border, or irreversible processing.
- Let people obtain understandable notice, access, correction, objection, export, deletion, and human appeal as applicable.
- Never let fluency, anonymization claims, or model confidence replace privacy analysis.

ARCHITECTURE AND SYSTEM IMPLICATIONS

- Bind data access to purpose, subject, authority, minimization, retention, and deletion.
- Separate source, derived, inferred, generated, cached, logged, backed-up, archived, and evidence copies.
- Propagate restrictions and deletion through indexes, replicas, logs, models, prompts, exports, vendors, and recovery media.
- Preserve privacy-safe status and manual rights paths during provider or model failure.

OWNER COCKPIT AND USER-CONTROL REQUIREMENTS

- Show data, subject, purpose, recipient, jurisdiction, authority, retention, access, risk, and evidence freshness.
- Expose inspect, correct, restrict, object, export, delete, withdraw, notify, and appeal actions within authority.
- Explain collection, use, inference, sharing, denial, retention, and deletion state in accessible language.
- Export processing records, decisions, evidence, dissent, rights requests, and continuity state.

REQUIRED OUTPUTS AND ARTIFACTS

- Privacy context, data and purpose inventory, subject and rights model, processing and sharing map, risk assessment.
- Minimization, notice, access, correction, retention, deletion, model, supplier, breach, and evidence designs.
- Verification, decision, acceptance, and handoff package.

EVIDENCE REQUIREMENTS

- Claims identify subject, data, purpose, jurisdiction, processing, authority, recipient, method, date, and limitation.
- Notice, consent, masking, access log, deletion job, assessment, or vendor claim proves only its declared scope.
- Rights, minimization, inference, retention, and deletion evidence identifies population, copies, exceptions, and validity.
- Missing, stale, contradictory, or inaccessible evidence constrains privacy status.

VERIFICATION AND ACCEPTANCE TESTS

- Trace a subject's data from collection through use, sharing, inference, retention, access, correction, and deletion.
- Exercise unauthorized use, purpose change, linkage, re-identification, sensitive inference, cross-border transfer, cache, backup, and model retention.
- Verify rights, minimization, transparency, access, correction, deletion, audit, and appeal across all entry paths.
- Disable privacy service, identity, model, provider, data catalog, and evidence store and verify safe restriction.
- Reproduce representative privacy claims from exact records.

FAILURE STATES AND ANTI-PATTERNS

- Treating consent text, masking, pseudonyms, or a deletion button as complete privacy protection.
- Collecting first and finding purpose later; hiding inference, linkage, retention, or vendor copies.
- Allowing AI to infer permission, deny rights, or make sensitive decisions without authority and review.

RECOVERY AND RECONSIDERATION

- On unauthorized use, rights failure, inference harm, retention uncertainty, or evidence loss, restrict processing and preserve records.
- Identify subjects, owner, authority, notification, evidence, remedy, and review condition.
- Use quarantine, access restriction, correction, withdrawal, deletion, notification, provider remediation, or retirement.
- Reopen on purpose, population, law, jurisdiction, model, provider, or lifecycle change.

COMPLETION CRITERIA

- Subjects, purposes, data, authority, rights, risks, controls, evidence, retention, deletion, and lifecycle are explicit.
- Inference, linkage, supplier, breach, rights, model, and recovery behavior is addressed or justified.
- Exact validity, unresolved legal questions, and acceptance authority are recorded.

HANDOFF AND CONTINUITY REQUIREMENTS

- Preserve processing, subject, purpose, rights, decision, evidence, request, incident, dissent, and gap identities.
- Hand off to Data, Information, Identity, Security, Governance, Compliance, Records, AI, Testing, and Operations.
- Record changes requiring reopening all affected data, identity, security, and governance sections.

RELATED CODEX SECTIONS

- 019-046 Requirements through Identity Architecture; 048-060 Governance through Hardware Architecture.
- 155-158 data governance, quality, integration, and migration; 218 Privacy Engineering; 391-395 retention and deletion.

SOURCE BASIS AND UNRESOLVED SOURCE GAPS

- Based on the Canonical Section Contract, shared governance files, Sections 019-046, Owner Cockpit Requirements, Status and Evidence Lexicon, and local architecture baseline.
- No universal privacy law, lawful basis, rights regime, anonymization standard, or cross-border rule is supplied. Actual populations, purposes, jurisdictions, data inventory, and legal determinations remain unresolved.

048

SECTION 048 - GOVERNANCE ARCHITECTURE

CODEX/03_SECTIONS/048_GOVERNANCE_ARCHITECTURE.txt

SHA-256 c36a9a77e989948b6f201bf8c5f894adccdd5d1dbbded37a1a92edffb5f806e7

9,343 BYTES / PRESERVED SOURCE IDENTITY

SECTION 048 — Governance Architecture

STATUS: CONTENT-COMPLETE

PURPOSE

Define how authority, policy, delegation, decision rights, evidence, oversight, exceptions, accountability, change, and lifecycle are structured and enforced across the system.

REQUIRED OUTCOME

- A governance model with authorities, policies, decision records, evaluation, approvals, delegations, controls, evidence, appeals, and lifecycle.
- Default-deny resolution for unresolved authority or policy conflict, with durable audit and replayable decision history.
- Traceability from mission and requirements through architecture, implementation, operation, incident, compensation, and retirement.

WHY THIS MATTERS

Governance is the boundary between capability and permission. Without explicit authority and immutable decisions, policy becomes informal, exceptions become invisible, and an AI or operator can turn a recommendation into an unreviewed commitment.

APPLICABILITY

- A system makes decisions, grants authority, changes policy, invokes tools, manages records, affects rights, or creates external consequence.
- Multiple owners, policies, jurisdictions, suppliers, agents, or lifecycle stages require coordination and precedence.

NON-APPLICABILITY CONDITIONS

- No decision right, policy, authority, delegation, exception, evidence, or lifecycle governance effect changes.
- An approved governance baseline covers exact scope and change with no material impact.
- The artifact is exploratory and not an authority, compliance, or approval claim.

Record baseline, scope, authority, evidence date, assumptions, and reopening trigger.

PREREQUISITES AND DEPENDENCIES

- Sections 001-047 provide objectives, authority, requirements, architecture, security, identity, privacy, and operational context.
- Policy sources, decision owners, capability tokens, evaluation, approval, audit, record, and appeal authorities are known or open.

REQUIRED INPUTS

- Objectives, invariants, policies, roles, authorities, resources, actions, mutation classes, risk, approval thresholds, evidence, and lifecycle.
- Decision, evaluation, delegation, quorum, audit, trace, artifact, override, compensation, and retirement requirements.

HUMAN RESPONSIBILITIES AND AUTHORITY

Authorized humans define policy, precedence, values, decision rights, risk appetite, quorum, exceptions, overrides, acceptance, and compensation. They approve, deny, revoke, investigate, appeal, supersede, and retire governance decisions. No AI recommendation creates authority.

AI RESPONSIBILITIES AND BOUNDARIES

AI may classify requests, retrieve policy, identify conflicts, draft evaluations, summarize evidence, and recommend decisions. It must bind claims to policy and evaluation evidence, preserve uncertainty, and default to escalation or denial. It must not approve mutations, invent policy, bypass quorum, or replace immutable decisions.

APPLICABLE EXISTING AGENT ROLES

- Governance Reviewer, Decision Arbiter Advisory, Architect, Analyst, Researcher, Critic, Verifier, Security Auditor, Planner, Coordinator, Documentation Writer, Memory Curator, and Red Team.

REQUIRED ACTIVITIES

1. Identify authorities, policies, resources, actions, mutation classes, risks, and decision owners.
2. Define precedence, evaluation, delegation, capability scope, approval, quorum, override, expiry, revocation, and denial.
3. Model decision, policy, evidence, audit, trace, artifact, session, approval, incident, and compensation records.
4. Bind mutation-capable actions to policy evaluation, capability, trace, decision, and audit records.
5. Define conflict, appeal, reconsideration, supersession, emergency, migration, and retirement behavior.
6. Validate with owners, affected parties, security, privacy, legal, safety, accessibility, and operations authorities.
7. Test expired or revoked authority, policy mismatch, missing evidence, quorum failure, replay, and destructive rollback attempts.
8. Baseline and hand off the governance model and unresolved gaps.

HUMAN-AI INTERACTION REQUIREMENTS

- Show intent, action, resource, policy, evaluation, capability scope, authority, decision, approval, trace, and consequence.
- Require complete governance binding for every mutation-capable action and quorum proof where applicable.
- Keep decisions immutable, audit append-only, resolution conservative, and rollback compensation-based.
- Let humans inspect evidence, dissent, conflict, denial reason, appeal, and reconsideration path.

ARCHITECTURE AND SYSTEM IMPLICATIONS

- Represent policies, evaluations, decisions, approvals, capabilities, bindings, audits, traces, artifacts, and compensations as attributable records.
- Enforce governance at execution boundaries, not only in prompts, UI, documentation, or local checks.
- Preserve immutable history, precedence, lifecycle state, expiry, revocation, replay, and append-only audit.
- Ensure every terminal operation emits durable success, denial, failure, cancellation, or recovery evidence.

OWNER COCKPIT AND USER-CONTROL REQUIREMENTS

- Show intent, action, resource, mutation class, policy, evaluation, capability, authority, decision, quorum, trace, and status.
- Expose approve, deny, revoke, pause, investigate, appeal, reconsider, compensate, supersede, and retire actions.
- Explain missing binding, policy conflict, expired capability, denied resolution, and evidence gap.
- Export the complete decision and audit chain without hidden conversation state.

REQUIRED OUTPUTS AND ARTIFACTS

- Governance context, authority and policy catalog, precedence model, evaluation and decision schemas, delegation and capability model.
- Approval, quorum, audit, trace, artifact, override, appeal, incident, compensation, and lifecycle records.
- Verification, compliance, evidence, acceptance, and continuity handoff package.

EVIDENCE REQUIREMENTS

- Each decision identifies intent, subject, action, resource, policy, evaluation, capability, authority, trace, evidence, outcome, and expiry.
- A prompt, local check, approval message, or successful mutation without binding is not governance evidence.
- Audit is append-only and replayable; missing, contradictory, stale, or unverifiable records constrain status.
- Overrides and compensations link to the prior immutable decision and preserve reason and authority.

VERIFICATION AND ACCEPTANCE TESTS

- Attempt mutation with missing, mismatched, expired, revoked, over-broad, or policy-inconsistent binding and verify denial.
- Verify quorum, precedence, immutable decision, append-only audit, trace, terminal event, artifact linkage, and compensation.
- Exercise conflicting policies, operator override, emergency path, replay, session expiry, tool failure, and partial mutation.
- Confirm read-only actions do not receive unnecessary authority while mutation actions cannot bypass governance.
- Reproduce the decision and audit chain independently from exported records.

FAILURE STATES AND ANTI-PATTERNS

- Local-only policy checks, unsigned or mutable decisions, missing trace, incomplete binding, or silent overrides.
- Treating approval, model confidence, capability possession, or successful execution as sufficient governance.
- Destructive rollback, hidden audit mutation, quorum bypass, authority inheritance, or default-allow conflict resolution.

RECOVERY AND RECONSIDERATION

- On binding failure, policy conflict, revoked capability, audit loss, or partial mutation, deny or halt and preserve records.
- Identify safe read-only work, decision owner, evidence, affected resources, and appeal or reconsideration path.
- Recover only through a new authorized decision and compensation linked to the prior immutable decision.
- Reopen governance when policy, authority, capability, resource, mutation class, or lifecycle changes.

COMPLETION CRITERIA

- Authority, policy, evaluation, binding, decision, quorum, audit, trace, artifact, conflict, appeal, compensation, and lifecycle are explicit.
- Default-deny, immutable decision, append-only audit, replay, and non-destructive recovery are verified.
- Exact unresolved policy and authority gaps are recorded; CONTENT-COMPLETE does not imply approval.

HANDOFF AND CONTINUITY REQUIREMENTS

- Preserve policy, evaluation, capability, binding, decision, approval, quorum, audit, trace, artifact, incident, compensation, dissent, and gap identities.
- Hand off to Compliance, Security, Identity, Privacy, Safety, Operations, Testing, AI, and Retirement sections.
- Record changes requiring reopening every affected policy, authority, architecture, and lifecycle section.

RELATED CODEX SECTIONS

- 001-020 shared governance, roles, evidence, continuity, requirements, and systems engineering; 021-047 architecture.
- 049-060 Compliance through Hardware Architecture; 183 AI Governance; 200-208 identity and threat; 225-227 records, audit, and controls; 405-406 lifecycle and retirement.

SOURCE BASIS AND UNRESOLVED SOURCE GAPS

- Based on the Canonical Section Contract, shared governance files, uploaded governing requirements, Sections 001-047, Owner Cockpit Requirements, Status and Evidence Lexicon, and local architecture baseline.

- Specific policy catalog, signature mechanism, quorum thresholds, capability-token format, jurisdiction, retention, and external authority mappings are not supplied and remain unresolved.

049

SECTION 049 - COMPLIANCE ARCHITECTURE

CODEX/03_SECTIONS/049_COMPLIANCE_ARCHITECTURE.txt

SHA-256 9cd229d080c553356b5cece978530cdb3f5a679ff2196f50d71c35639d6317bf

8,078 BYTES / PRESERVED SOURCE IDENTITY

SECTION 049 — Compliance Architecture

STATUS: CONTENT-COMPLETE

PURPOSE

Define how applicable laws, regulations, contracts, standards, policies, controls, evidence, assessments, findings, exceptions, and reporting obligations are identified and maintained across the lifecycle.

REQUIRED OUTCOME

- A jurisdictional obligation register mapped to scope, controls, owners, evidence, validity, assessment, and remediation.
- Clear distinction between designed, implemented, operating, effective, assessed, certified, and accepted states.
- Traceability from obligations through governance, security, privacy, safety, implementation, operation, audit, and retirement.

WHY THIS MATTERS

Compliance is not a decorative label or a single audit event. An obligation can apply only to a population, data class, location, version, or process, and evidence can expire or prove less than the claim made from it.

APPLICABILITY

- Law, regulation, contract, certification, policy, or standard affects the system, organization, data, users, suppliers, or lifecycle.
- Non-compliance could create legal, financial, safety, rights, trust, or operational consequence.

NON-APPLICABILITY CONDITIONS

- No applicable obligation, jurisdiction, contract, control, evidence, or lifecycle condition changes.
- An approved register covers exact scope, jurisdiction, version, and change with no material impact.
- The artifact is exploratory and not a compliance, certification, or legal conclusion.

Record source, effective date, scope, authority, evidence date, assumptions, and reopening trigger.

PREREQUISITES AND DEPENDENCIES

- Sections 001-048 provide authority, requirements, architecture, security, identity, privacy, and governance context.
- Qualified legal, compliance, records, privacy, security, safety, and domain authorities are identified or gaps are recorded.

REQUIRED INPUTS

- Jurisdictions, populations, products, data, activities, suppliers, contracts, standards, policies, controls, assessments, evidence, and expiry.
- Exceptions, remediation, reporting, notification, retention, audit, and retirement obligations.

HUMAN RESPONSIBILITIES AND AUTHORITY

Qualified human authorities determine applicability, interpretation, evidence sufficiency, exceptions, legal risk, certification, disclosure, and acceptance. AI and operators cannot make legal determinations or certify compliance. Humans can restrict, remediate, disclose, appeal, and retire.

AI RESPONSIBILITIES AND BOUNDARIES

AI may inventory obligations, map controls, compare evidence, identify gaps, and draft assessment material. It must preserve source, jurisdiction, version, and uncertainty. It must not invent law, claim certification, suppress findings, or accept exceptions without authority.

APPLICABLE EXISTING AGENT ROLES

- Governance Reviewer, Researcher, Analyst, Security Auditor, Critic, Verifier, Documentation Writer, Memory Curator, Planner, Architect, and Red Team.

REQUIRED ACTIVITIES

1. Identify authoritative obligations, scope, jurisdiction, effective dates, and owners.
2. Map obligations to policies, controls, processes, data, systems, roles, tests, and evidence.
3. Define assessment, sampling, validity, exception, remediation, notification, and reporting rules.
4. Track design, implementation, operation, effectiveness, assessment, certification, and acceptance separately.
5. Validate with qualified legal, compliance, privacy, security, safety, records, and domain authorities.
6. Test evidence integrity, control operation, expiry, scope, exception, and reporting paths.
7. Baseline the register and hand off unresolved determinations.

HUMAN-AI INTERACTION REQUIREMENTS

- Show obligation source, scope, jurisdiction, control, evidence, owner, authority, and claim level.
- Require human review for interpretation, legal conclusion, exception, certification, notification, and disclosure.
- Preserve findings, dissent, uncertainty, appeal, and evidence history.
- Never let an automated score or generated report become a compliance decision without authority.

ARCHITECTURE AND SYSTEM IMPLICATIONS

- Represent obligations, controls, evidence, assessments, findings, exceptions, decisions, and validity as versioned records.
- Bind compliance status to evidence and scope; enforce controls at runtime where applicable.
- Preserve immutable decisions, append-only audit, retention, access, and replayable assessment history.

OWNER COCKPIT AND USER-CONTROL REQUIREMENTS

- Show obligation, jurisdiction, scope, owner, control, evidence age, finding, exception, deadline, and status.
- Expose investigate, remediate, restrict, notify, appeal, accept, supersede, and retire actions with authority.
- Explain why a claim is unknown, conditional, failed, expired, or unverified.
- Export register, mappings, evidence, decisions, dissent, findings, and continuity state.

REQUIRED OUTPUTS AND ARTIFACTS

- Obligation register, applicability decisions, control map, evidence index, assessment plan, exception and remediation records.
- Reporting, notification, audit, retention, supplier, certification, and retirement handoff package.

EVIDENCE REQUIREMENTS

- Each claim identifies obligation, source, jurisdiction, scope, version, control, evidence, method, date, owner, and limitation.
- A policy, scan, attestation, test, certificate, or audit report proves only its stated scope and validity.
- Expired, missing, contradictory, or disputed evidence constrains compliance status.

VERIFICATION AND ACCEPTANCE TESTS

- Trace obligations through controls, implementation, operation, evidence, assessment, finding, remediation, and retirement.

- Introduce wrong jurisdiction, expired evidence, out-of-scope sample, false attestation, missing control, and exception conflict.
- Verify immutable decisions, audit, evidence provenance, reporting, notification, and appeal.
- Independently reproduce representative claims from the register and evidence package.

FAILURE STATES AND ANTI-PATTERNS

- Treating certification, checklist completion, policy presence, or a green score as universal compliance.
- Applying one jurisdiction or standard to all populations without applicability analysis.
- Hiding exceptions, expired evidence, scope limits, legal uncertainty, or dissent.

RECOVERY AND RECONSIDERATION

- On obligation conflict, evidence failure, control failure, or changed law, restrict claims and preserve records.
- Identify qualified authority, affected scope, safe work, evidence, notification, and review date.
- Use remediation, compensating control, exception, disclosure, suspension, or retirement only through authorized decisions.
- Reopen on law, contract, population, data, control, provider, or lifecycle change.

COMPLETION CRITERIA

- Obligations, applicability, controls, owners, evidence, validity, findings, exceptions, remediation, and lifecycle are explicit.
- Scope, jurisdiction, expiry, assessment, reporting, and retirement are addressed or justified.
- No legal or certification claim exceeds supplied authority and evidence.

HANDOFF AND CONTINUITY REQUIREMENTS

- Preserve sources, versions, applicability, controls, evidence, findings, exceptions, decisions, dissent, and gaps.
- Hand off to Governance, Security, Privacy, Safety, Records, Audit, Testing, Operations, and Retirement.
- Record changes requiring reopening affected architecture and obligation mappings.

RELATED CODEX SECTIONS

- 001-048 governance, authority, requirements, architecture, security, privacy, and governance; 050-060 safety through hardware.
- 220-227 compliance, policy, records, audit, and controls; 251 compliance testing; 405-406 lifecycle governance and retirement.

SOURCE BASIS AND UNRESOLVED SOURCE GAPS

- Based on the Canonical Section Contract, shared governance files, Sections 001-048, Owner Cockpit Requirements, Status and Evidence Lexicon, and local architecture baseline.
- No jurisdiction, legal advice, contract set, certification scope, control catalog, or assessment authority is supplied. Actual applicability and evidence remain unresolved.

050

SECTION 050 - SAFETY ARCHITECTURE

CODEX/03_SECTIONS/050_SAFETY_ARCHITECTURE.txt

SHA-256 f80c5d7f6bacf067645da58bc347e3a818a32774ee7c46176505ad180a9d88d4

8,521 BYTES / PRESERVED SOURCE IDENTITY

SECTION 050 — Safety Architecture

STATUS: CONTENT-COMPLETE

PURPOSE

Define hazards, harms, safety constraints, protective functions, human oversight, operating limits, fail-safe behavior, evidence, and lifecycle controls for systems that can affect people, property, environment, or public safety.

REQUIRED OUTCOME

- A hazard and harm model with affected parties, operating conditions, controls, safe states, evidence, owners, and residual risk.
- Explicit prevention, detection, mitigation, stop, emergency, recovery, and change-assurance behavior.
- Traceability from mission and requirements through design, operation, incidents, testing, and retirement.

WHY THIS MATTERS

Safety failure can be physical, social, environmental, or cumulative. A passing nominal test does not establish safe behavior under uncertainty, misuse, degraded sensing, human fatigue, model drift, or recovery.

APPLICABILITY

- A system can affect health, bodily safety, property, environment, public services, rights, or high-consequence decisions.
- Failure, misuse, delay, automation, or human interaction could create material harm.

NON-APPLICABILITY CONDITIONS

- No physical, human, environmental, rights, or high-consequence effect changes.
- An approved safety baseline covers exact scope, population, environment, and change with no material impact.
- The artifact is exploratory and not a safety-case or safe-operation claim.

Record hazard scope, owner, evidence date, assumptions, and reopening trigger.

PREREQUISITES AND DEPENDENCIES

- Sections 019-049 provide requirements, architecture, operations, security, privacy, compliance, and governance context.
- Qualified safety, domain, user, accessibility, legal, security, operational, and emergency authorities are identified or gaps recorded.

REQUIRED INPUTS

- Hazards, harms, affected parties, use and misuse cases, operating modes, environment, human factors, controls, limits, and lifecycle.
- Safety requirements, assurance levels, fail-safe behavior, alarms, stop authority, incident, recovery, evidence, and retirement obligations.

HUMAN RESPONSIBILITIES AND AUTHORITY

Safety authorities approve hazard analysis, risk, operating limits, safeguards, human oversight, emergency action, residual risk, and acceptance. Humans retain stop, override, evacuation, quarantine, investigation, and retirement authority. AI cannot accept safety risk.

AI RESPONSIBILITIES AND BOUNDARIES

AI may identify hazards, analyze scenarios, draft safety cases and tests, and summarize evidence. It must preserve assumptions, uncertainty, population, and operating context. It must not declare safe, suppress hazards, issue physical actions, or replace qualified judgment.

APPLICABLE EXISTING AGENT ROLES

- Safety Auditor, Red Team, Architect, Researcher, Analyst, Observer, Tester, Verifier, Governance Reviewer, Critic, Planner, Documentation Writer, Memory Curator, and User Experience Reviewer.

REQUIRED ACTIVITIES

1. Identify hazards, harms, affected parties, use, misuse, environments, and lifecycle.
2. Analyze initiating conditions, severity, likelihood, uncertainty, common cause, human factors, and exposure.
3. Define prevention, detection, mitigation, stop, alarm, interlock, manual fallback, emergency, and recovery controls.
4. Allocate controls and evidence to functions, systems, people, AI, devices, operations, and suppliers.
5. Validate with qualified specialists, users, operators, maintainers, emergency, accessibility, legal, and affected parties.
6. Test nominal, boundary, misuse, degraded, sensor-loss, model-loss, human-error, emergency, and recovery scenarios.
7. Baseline safety case, evidence, decisions, dissent, residual risk, and handoff.

HUMAN-AI INTERACTION REQUIREMENTS

- Show hazard, actor, model, data, authority, confidence limit, control, operating mode, and consequence.
- Require human approval for safety decisions, autonomy, overrides, exceptions, emergency actions, and residual risk.
- Preserve independent stop, alarm, manual, status, evidence, appeal, and recovery paths.
- Treat uncertainty or model confidence as a reason for bounded action, not permission.

ARCHITECTURE AND SYSTEM IMPLICATIONS

- Separate hazard, risk, control, safe state, evidence, incident, and decision records.
- Make safety constraints enforceable at the action boundary and independent of a nonessential model or provider.
- Design fail-safe, graceful degradation, interlock, redundancy, isolation, and recovery semantics.
- Preserve immutable decisions, append-only incident evidence, and compensation-only reversal.

OWNER COCKPIT AND USER-CONTROL REQUIREMENTS

- Show hazards, operating mode, safeguards, alerts, authority, affected parties, evidence, residual risk, and lifecycle.
- Expose stop, isolate, alarm, override, evacuate, investigate, recover, and retire actions with approval and impact.
- Explain uncertainty, unsafe condition, denial, and safe recovery in accessible terms.
- Export safety case, decisions, evidence, incidents, dissent, and continuity state.

REQUIRED OUTPUTS AND ARTIFACTS

- Hazard analysis, safety case, operating limits, control and safe-state model, human oversight, emergency, and recovery plans.
- Assurance, test, evidence, incident, risk, decision, acceptance, and retirement package.

EVIDENCE REQUIREMENTS

- Claims identify hazard, control, population, environment, mode, configuration, method, date, result, and limitation.
- Simulation, inspection, demonstration, test, incident, or certification evidence proves only its bounded conditions.
- Safety evidence distinguishes hazard identified, control designed, control implemented, and risk accepted.
- Missing, inconclusive, stale, or contradictory evidence constrains safety status.

VERIFICATION AND ACCEPTANCE TESTS

- Trace hazards through requirements, controls, implementation, operation, evidence, incident, recovery, and retirement.
- Exercise misuse, sensor loss, stale data, model failure, human error, communication loss, overload, emergency, and recovery.
- Verify stop, interlock, safe state, manual control, alarm, audit, accessibility, and authority revocation.
- Disable model, provider, network, sensor, operator, power, and evidence services and verify safe response.
- Reproduce safety claims from exact safety case and evidence.

FAILURE STATES AND ANTI-PATTERNS

- Treating a risk score, certification, nominal demo, or model confidence as proof of safety.
- Hiding uncertainty, near misses, human factors, unsafe overrides, inaccessible controls, or recovery dependency.
- Allowing optimization or automation to weaken stop authority or safeguards.

RECOVERY AND RECONSIDERATION

- On hazard, near miss, control failure, uncertainty, or evidence loss, enter the approved safe state and preserve records.
- Identify affected people, authority, communication, evidence, investigation, and review condition.
- Use manual control, isolation, reduced mode, repair, restore, compensation, or retirement through authorized decisions.
- Reopen on hazard, environment, population, model, control, policy, or lifecycle change.

COMPLETION CRITERIA

- Hazards, harms, controls, limits, authorities, evidence, safe states, recovery, and lifecycle are explicit.
- Misuse, degraded, emergency, human, AI, supplier, and retirement behavior is addressed or justified.
- Exact residual risk and unresolved evidence are recorded; CONTENT-COMPLETE does not imply safe operation.

HANDOFF AND CONTINUITY REQUIREMENTS

- Preserve hazards, controls, safety case, incidents, decisions, evidence, dissent, residual risk, and gaps.
- Hand off to Reliability, Resilience, Security, Privacy, Compliance, Operations, Testing, Robotics, Embedded, and Retirement.
- Record changes requiring reopening all affected safety and architecture sections.

RELATED CODEX SECTIONS

- 019-049 Requirements through Compliance Architecture; 051-060 Reliability through Hardware.
- 182 AI Safety; 208 Threat Modeling; 252 Safety Testing; 243-245 reliability, resilience, and chaos testing; 326-330 continuity and recovery.

SOURCE BASIS AND UNRESOLVED SOURCE GAPS

- Based on the Canonical Section Contract, shared governance files, Sections 019-049, Owner Cockpit Requirements, Status and Evidence Lexicon, and local architecture baseline.
- No universal hazard taxonomy, safety standard, assurance level, risk threshold, emergency regime, or qualified authority is supplied. Actual hazards, populations, operating conditions, and acceptance evidence remain unresolved.

051

SECTION 051 - RELIABILITY ARCHITECTURE

CODEX/03_SECTIONS/051_RELIABILITY_ARCHITECTURE.txt

SHA-256 02e975dcaf4f43cff43f7d53a5c02ec5cf7f690b661c3dd3337d78d945c0a9d1

7,593 BYTES / PRESERVED SOURCE IDENTITY

SECTION 051 — Reliability Architecture

STATUS: CONTENT-COMPLETE

PURPOSE

Define the probability, predictability, durability, availability, failure-rate, maintainability, and evidence structures needed for a system to perform required functions over declared conditions and time.

REQUIRED OUTCOME

- Reliability objectives, failure models, dependencies, margins, measures, controls, tests, maintenance, and lifecycle are explicit.
- Claims distinguish design intent, prediction, demonstration, measurement, operational history, and accepted residual risk.

WHY THIS MATTERS

Reliability is contextual and time-dependent. A system that works once or has high uptime can still fail too often, recover too slowly, or degrade users through hidden maintenance and common-cause dependencies.

APPLICABILITY

- Failure frequency, durability, availability, service continuity, maintainability, or evidence validity materially affects outcome.
- Components operate over time, load, environmental stress, change, or maintenance cycles.

NON-APPLICABILITY CONDITIONS

- No reliability, durability, availability, maintenance, dependency, or lifecycle claim changes.
- An approved baseline covers exact workload, environment, and change with no material impact.
- The artifact is exploratory and not a reliability prediction or guarantee.

Record baseline, scope, owner, conditions, evidence date, assumptions, and reopening trigger.

PREREQUISITES AND DEPENDENCIES

- Sections 019-050 provide requirements, safety, security, operations, architecture, and recovery context.
- Failure data, workload, environment, maintenance, supplier, test, and acceptance authorities are known or open.

REQUIRED INPUTS

- Reliability objectives, workload, environment, failure modes, dependencies, repair, maintenance, capacity, margins, and lifecycle.
- Telemetry, incident, test, supplier, backup, restore, failover, and evidence requirements.

HUMAN RESPONSIBILITIES AND AUTHORITY

Owners approve objectives, assumptions, margins, maintenance burden, residual risk, service commitments, exceptions, and acceptance. Humans decide tradeoffs and retain stop, degrade, repair, failover, restore, and retire authority.

AI RESPONSIBILITIES AND BOUNDARIES

AI may analyze failure and incident data, identify trends, model scenarios, and draft reliability tests. It must preserve workload and uncertainty. It must not forecast certainty, suppress failures, change service, or accept reliability risk without authority.

APPLICABLE EXISTING AGENT ROLES

- Architect, Analyst, Researcher, Observer, Tester, Verifier, Critic, Planner, Implementer, Optimizer, Security Auditor, Governance Reviewer, Documentation Writer, Memory Curator, and Red Team.

REQUIRED ACTIVITIES

1. Define outcomes, conditions, workload, time horizon, reliability objectives, and owners.
2. Identify failure modes, common causes, dependencies, margins, maintenance, repair, and evidence.
3. Design redundancy, monitoring, graceful degradation, repair, replacement, and recovery controls.
4. Define measures, sampling, confidence, thresholds, invalidation, and reporting.
5. Validate with users, operators, maintainers, safety, security, suppliers, and domain authorities.
6. Test stress, aging, failure, restart, dependency loss, maintenance, failover, restore, and change.
7. Baseline evidence, decisions, residual risk, dissent, and gaps.

HUMAN-AI INTERACTION REQUIREMENTS

- Show reliability claim, conditions, workload, data, model, recommendation, uncertainty, and consequence.
- Require review for maintenance, degradation, service commitments, risk acceptance, and automated remediation.
- Preserve status, evidence, manual control, safe degradation, and recovery paths.

ARCHITECTURE AND SYSTEM IMPLICATIONS

- Separate target, predicted, tested, measured, operational, and accepted reliability states.
- Model common-cause failure, dependency correlation, maintenance, repair, spares, and evidence freshness.
- Instrument failure and outcome, not only request success or uptime.
- Link change, configuration, incident, and maintenance to validity of reliability claims.

OWNER COCKPIT AND USER-CONTROL REQUIREMENTS

- Show objectives, conditions, failure history, margins, dependencies, maintenance, evidence, residual risk, and lifecycle.
- Expose degrade, repair, failover, restore, investigate, accept, and retire actions with authority.
- Explain observed versus forecast reliability and evidence limits.
- Export measures, incidents, tests, decisions, evidence, dissent, and continuity state.

REQUIRED OUTPUTS AND ARTIFACTS

- Reliability context, objectives, failure model, dependency and maintenance plan, measures, evidence, and test basis.
- Degradation, failover, repair, restore, change, supplier, acceptance, and retirement package.

EVIDENCE REQUIREMENTS

- Claims identify workload, environment, configuration, period, failure definition, method, sample, result, confidence, and limitation.
- Prediction, simulation, test, drill, and operational history remain distinct.
- Missing, censored, stale, biased, or contradictory data constrains reliability claims.

VERIFICATION AND ACCEPTANCE TESTS

- Trace objectives through failure modes, controls, tests, operation, incidents, recovery, and retirement.
- Exercise load, aging, dependency loss, correlated failure, maintenance, failover, restore, and change.
- Verify measures capture failed outcomes, degraded service, recovery, and user impact.
- Reproduce representative reliability claims from exact evidence.

FAILURE STATES AND ANTI-PATTERNS

- Treating uptime, a benchmark, redundancy, or a prediction as general reliability proof.
- Ignoring common-cause failure, maintenance, repair time, hidden dependencies, or user harm during degradation.
- Optimizing a metric while silently changing the failure definition.

RECOVERY AND RECONSIDERATION

- On reliability breach, unknown condition, evidence invalidation, or common-cause failure, narrow service and preserve history.
- Identify owner, authority, affected users, safe mode, evidence, communications, and review condition.
- Use repair, degradation, failover, restore, compensation, or retirement.
- Reopen on workload, environment, configuration, dependency, objective, or lifecycle change.

COMPLETION CRITERIA

- Objectives, conditions, failure, dependencies, measures, evidence, maintenance, recovery, and lifecycle are explicit.
- Degraded, correlated, repaired, migrated, and retired behavior is addressed or justified.
- Exact confidence, validity, residual risk, and gaps are recorded.

HANDOFF AND CONTINUITY REQUIREMENTS

- Preserve objectives, failure data, measures, incidents, decisions, evidence, dissent, residual risk, and gaps.
- Hand off to Safety, Resilience, Runtime, Operations, Testing, Capacity, Availability, Reliability Management, and Retirement.
- Record changes requiring reopening affected architecture and evidence baselines.

RELATED CODEX SECTIONS

- 019-050 Requirements through Safety Architecture; 052-060 Resilience through Hardware.
- 243-245 reliability, resilience, and chaos testing; 301, 321-330 reliability and continuity operations; 324 Reliability Management.

SOURCE BASIS AND UNRESOLVED SOURCE GAPS

- Based on the Canonical Section Contract, shared governance files, Sections 019-050, Owner Cockpit Requirements, Status and Evidence Lexicon, and local architecture baseline.
- No universal reliability target, failure distribution, workload, environmental profile, or statistical confidence standard is supplied. Actual objectives and evidence remain unresolved.

052

SECTION 052 - RESILIENCE ARCHITECTURE

CODEX/03_SECTIONS/052_RESILIENCE_ARCHITECTURE.txt

SHA-256 ebd0638bab0a33a13a6183fd9d60352978b747a4d3d0448c6ff1038dbaab9917

8,084 BYTES / PRESERVED SOURCE IDENTITY

SECTION 052 — Resilience Architecture

STATUS: CONTENT-COMPLETE

PURPOSE

Define how a system anticipates, absorbs, adapts to, recovers from, and learns from disruption while preserving protected outcomes, authority, evidence, and human control.

REQUIRED OUTCOME

- A disruption and dependency model with thresholds, modes, controls, decision rights, recovery, adaptation, and learning.
- Verified continuity, degradation, failover, restoration, communication, and compensation paths.

WHY THIS MATTERS

Resilience is not merely redundancy. It includes people, suppliers, data, decisions, communications, recovery, and the ability to operate safely when assumptions fail.

APPLICABILITY

- Disruption, degradation, dependency loss, crisis, attack, change, or uncertainty could affect material outcomes.
- The system must continue, recover, adapt, or retire while preserving people, rights, safety, data, or accountability.

NON-APPLICABILITY CONDITIONS

- No protected outcome, dependency, disruption, recovery, adaptation, or lifecycle obligation changes.
- An approved resilience baseline covers exact scope and change with no material impact.
- The artifact is exploratory and not a continuity or recovery claim.

Record baseline, scope, owner, evidence date, assumptions, and reopening trigger.

PREREQUISITES AND DEPENDENCIES

- Sections 019-051 provide requirements, operations, safety, security, reliability, continuity, and recovery context.
- Business continuity, incident, supplier, data, communications, safety, and owner authorities are known or open.

REQUIRED INPUTS

- Protected outcomes, dependencies, disruption scenarios, thresholds, modes, people, resources, data, recovery objectives, and lifecycle.
- Failover, restore, communication, manual operation, compensation, exercise, learning, and evidence requirements.

HUMAN RESPONSIBILITIES AND AUTHORITY

Owners set protected outcomes, priorities, thresholds, recovery tradeoffs, crisis authority, communications, exceptions, and acceptance. Humans decide when to degrade, fail over, restore, compensate, learn, or retire and retain stop authority.

AI RESPONSIBILITIES AND BOUNDARIES

AI may identify scenarios, correlate signals, simulate disruption, draft runbooks, and recommend bounded responses. It must preserve evidence and uncertainty. It must not declare resilience, choose crisis priorities, suppress warnings, or execute consequential recovery without authority.

APPLICABLE EXISTING AGENT ROLES

- Architect, Analyst, Researcher, Observer, Planner, Coordinator, Tester, Verifier, Security Auditor, Safety Auditor, Governance Reviewer, Critic, Documentation Writer, Memory Curator, Optimizer, and Red Team.

REQUIRED ACTIVITIES

1. Identify protected outcomes, people, dependencies, disruption scenarios, and owners.
2. Define thresholds, modes, safe degradation, failover, restore, communication, and recovery authority.
3. Model technical, human, supplier, data, financial, environmental, and governance disruption.
4. Design redundancy, diversity, isolation, manual fallback, evidence, and learning loops.
5. Validate with owners, users, operators, safety, security, privacy, suppliers, and continuity specialists.
6. Exercise disruption, adaptation, recovery, communication, authority loss, and return to service.
7. Baseline lessons, decisions, evidence, dissent, residual risk, and gaps.

HUMAN-AI INTERACTION REQUIREMENTS

- Show scenario, protected outcome, actor, model, authority, mode, evidence, and consequence.
- Require human review for crisis priorities, service reduction, failover, compensation, notification, and retirement.
- Preserve independent status, stop, communication, evidence, and manual recovery paths.

ARCHITECTURE AND SYSTEM IMPLICATIONS

- Separate planned, active, degraded, recovering, restored, learned, superseded, and retired states.
- Model common-cause, cascading, compound, slow-onset, and adversarial disruption.
- Keep recovery evidence and decisions append-only and link compensation to prior decisions.
- Ensure alternate people, data, communications, identity, tools, suppliers, and facilities are usable, not merely listed.

OWNER COCKPIT AND USER-CONTROL REQUIREMENTS

- Show disruption, protected outcomes, mode, dependencies, thresholds, owner, authority, evidence, and residual risk.
- Expose degrade, isolate, fail over, restore, communicate, compensate, investigate, and retire actions.
- Explain tradeoffs, uncertainty, affected parties, and return-to-service conditions.
- Export exercise, incident, decision, evidence, dissent, lesson, and continuity state.

REQUIRED OUTPUTS AND ARTIFACTS

- Resilience context, dependency and disruption model, mode and threshold design, continuity and recovery plan.
- Communication, manual operation, supplier, data, exercise, learning, compensation, evidence, and retirement package.

EVIDENCE REQUIREMENTS

- Claims identify scenario, scope, workload, mode, dependency, configuration, exercise or incident, method, date, result, and limitation.
- Plan, simulation, tabletop, drill, incident, or recovery evidence proves only its exercised conditions.
- Evidence must show protected outcome, user impact, decision, recovery, and residual risk, not just technical return.
- Missing, partial, stale, or failed exercise evidence constrains resilience status.

VERIFICATION AND ACCEPTANCE TESTS

- Trace protected outcomes through dependencies, controls, modes, recovery, communication, evidence, and retirement.
- Exercise provider, region, identity, data, network, staffing, facility, model, communication, and authority loss.
- Verify safe degradation, manual fallback, failover, restore, compensation, audit, accessibility, and return-to-service.
- Confirm lessons produce governed changes and immutable decisions.
- Reproduce representative resilience claims from exact exercise package.

FAILURE STATES AND ANTI-PATTERNS

- Treating redundancy, a plan, tabletop, or restored process as proof of resilient outcome.
- Ignoring staff, supplier, communications, data, identity, physical, financial, or governance dependencies.
- Returning to service without evidence, owner decision, user communication, or residual-risk review.

RECOVERY AND RECONSIDERATION

- On disruption or evidence failure, enter the approved mode, preserve records, and protect people and critical outcomes.
- Identify authority, affected parties, safe work, evidence, communications, recovery owner, and review condition.
- Use failover, reduced service, manual operation, restore, substitution, compensation, or retirement.
- Reopen on disruption, dependency, population, objective, authority, or lifecycle change.

COMPLETION CRITERIA

- Outcomes, scenarios, dependencies, modes, thresholds, authority, evidence, recovery, learning, and lifecycle are explicit.
- Compound disruption, human, supplier, communication, data, and retirement paths are addressed or justified.
- Exact exercise limits, residual risk, and unresolved gaps are recorded.

HANDOFF AND CONTINUITY REQUIREMENTS

- Preserve scenarios, dependencies, exercises, incidents, decisions, evidence, lessons, dissent, residual risk, and gaps.
- Hand off to Reliability, Safety, Security, Operations, Continuity, Disaster Recovery, Backup, Restore, Failover, Testing, and Retirement.
- Record changes requiring reopening affected architecture and recovery baselines.

RELATED CODEX SECTIONS

- 019-051 Requirements through Reliability Architecture; 053-060 Accessibility through Hardware.
- 243-245 reliability, resilience, and chaos testing; 326-331 continuity, disaster recovery, backup, restore, failover, and readiness; 324 Reliability Management.

SOURCE BASIS AND UNRESOLVED SOURCE GAPS

- Based on the Canonical Section Contract, shared governance files, Sections 019-051, Owner Cockpit Requirements, Status and Evidence Lexicon, and local architecture baseline.
- No universal resilience objective, disruption catalogue, recovery authority, continuity standard, or exercise evidence is supplied. Actual protected outcomes and recovery targets remain unresolved.

053

SECTION 053 - ACCESSIBILITY ARCHITECTURE

CODEX/03_SECTIONS/053_ACCESSIBILITY_ARCHITECTURE.txt

SHA-256 862a52c79871c9a4854d4c298b7df2d9d62a1ea72d2d2422f4fdea96d4ef5b87

8,203 BYTES / PRESERVED SOURCE IDENTITY

SECTION 053 — Accessibility Architecture

STATUS: CONTENT-COMPLETE

PURPOSE

Define how people with different abilities, devices, languages, contexts, and support needs can perceive, understand, navigate, operate, contribute to, correct, and recover from the system.

REQUIRED OUTCOME

- An accessibility model covering users, tasks, modalities, assistive technology, content, interaction, feedback, errors, alternatives, and evidence.
- Accessibility controls are architectural properties across interfaces, data, communication, operations, support, and recovery, not a late visual check.

WHY THIS MATTERS

Inaccessible design can deny essential service, hide errors, remove agency, or make recovery impossible. Automated conformance signals do not capture real task completion, cognitive load, or support needs.

APPLICABILITY

- People interact with, are affected by, or must recover from a system, service, content, device, or process.
- Accessibility, accommodation, language, device, environmental, or assistive-technology differences affect outcome.

NON-APPLICABILITY CONDITIONS

- No human, affected-party, interface, content, communication, support, or recovery interaction changes.
- An approved baseline covers exact population, modalities, environment, and change with no material impact.
- The artifact is exploratory and not a conformance or usability claim.

Record population, scope, owner, evidence date, assumptions, and reopening trigger.

PREREQUISITES AND DEPENDENCIES

- Sections 001-052 provide user, information, application, communication, safety, security, privacy, and operations context.
- Affected users, accessibility specialists, legal, content, support, and product authorities are identified or gaps recorded.

REQUIRED INPUTS

- User populations, tasks, modalities, assistive technologies, content, controls, errors, alternatives, languages, environments, and lifecycle.
- Accessibility, accommodation, testing, support, privacy, security, safety, procurement, and evidence requirements.

HUMAN RESPONSIBILITIES AND AUTHORITY

Owners approve accessibility outcomes, supported populations, accommodations, exceptions, residual burden, and acceptance. Affected users and qualified specialists must influence material decisions. Humans retain authority to block release, require remediation, and provide support or alternative access.

AI RESPONSIBILITIES AND BOUNDARIES

AI may inspect content and interaction artifacts, identify likely barriers, draft alternatives and tests, and summarize user evidence. It must preserve population and context. It must not infer disability, declare access, speak for users, or remove human accommodation without authority.

APPLICABLE EXISTING AGENT ROLES

- User Experience Reviewer, Researcher, Analyst, Observer, Architect, Tester, Verifier, Critic, Governance Reviewer, Documentation Writer, Memory Curator, Planner, and Red Team.

REQUIRED ACTIVITIES

1. Identify populations, tasks, modalities, assistive technologies, context, barriers, and owners.
2. Define perceivable, operable, understandable, robust, alternative, error, status, and recovery behavior.
3. Model keyboard, screen reader, voice, touch, visual, cognitive, motor, sensory, offline, and support paths as applicable.
4. Allocate accessibility to content, APIs, data, runtime, communications, operations, and procurement.
5. Validate with affected users and qualified accessibility, legal, design, content, security, and support authorities.
6. Test real tasks, errors, dynamic updates, authentication, generated content, localization, degraded mode, and recovery.
7. Baseline evidence, exceptions, decisions, dissent, and gaps.

HUMAN-AI INTERACTION REQUIREMENTS

- Provide accessible status, focus, error, explanation, correction, stop, appeal, and recovery paths.
- Identify generated content, confidence, source, modality, and human contact when material.
- Require human review for accessibility exceptions, accommodation changes, and high-impact decisions.
- Never treat automated scanning or model fluency as proof of access.

ARCHITECTURE AND SYSTEM IMPLICATIONS

- Preserve semantic structure, alternative representations, input flexibility, status, timing, focus, localization, and assistive-technology contracts.
- Ensure accessibility survives authentication, API, batch, event, admin, AI, offline, and recovery paths.
- Design accessible evidence, support, export, audit, and owner controls.

OWNER COCKPIT AND USER-CONTROL REQUIREMENTS

- Show supported populations, modalities, known barriers, evidence, exceptions, owner, and lifecycle.
- Expose remediation, accommodation, alternate path, support, stop, correction, and appeal actions.
- Explain unavailable or degraded accessible paths and preserve alternatives.
- Export findings, user evidence, decisions, dissent, accommodations, and continuity state.

REQUIRED OUTPUTS AND ARTIFACTS

- Accessibility context, user and task model, modality and assistive-technology map, interaction and content requirements.
- Accommodation, alternative, support, testing, evidence, exception, remediation, and lifecycle package.

EVIDENCE REQUIREMENTS

- Claims identify population, task, modality, device, assistive technology, environment, method, date, result, and limitation.
- Automated scan, checklist, demo, or conformance statement proves only its bounded coverage.
- User research and task evidence states sample, support, barriers, and unresolved interpretation.
- Missing, stale, or contradictory accessibility evidence constrains status.

VERIFICATION AND ACCEPTANCE TESTS

- Complete representative tasks with keyboard, screen reader, voice, touch, zoom, reduced motion, alternative input, and support paths as applicable.
- Exercise errors, timeouts, dynamic updates, authentication, generated content, localization, offline, denial, and recovery.
- Verify status, focus, contrast, semantics, alternatives, captions, transcripts, plain language, and export behavior as relevant.
- Reproduce claims from exact population and evidence package.

FAILURE STATES AND ANTI-PATTERNS

- Treating automated conformance, visual polish, or a single device as accessibility proof.
- Hiding inaccessible errors, time limits, authentication, generated content, support, or recovery.
- Treating accommodations as user burden or requiring disclosure beyond necessity.

RECOVERY AND RECONSIDERATION

- On inaccessible path, assistive-technology failure, or evidence gap, provide an equivalent supported path and preserve the issue record.
- Identify affected users, owner, accommodation, evidence, support, and review condition.
- Use manual support, alternate channel, correction, pause, remediation, or retirement.
- Reopen on population, modality, content, device, provider, policy, or lifecycle change.

COMPLETION CRITERIA

- Populations, tasks, modalities, barriers, alternatives, evidence, accommodations, support, recovery, and lifecycle are explicit.
- Dynamic, generated, denied, degraded, localization, and retirement paths are addressed or justified.
- Exact evidence limits and unresolved barriers are recorded.

HANDOFF AND CONTINUITY REQUIREMENTS

- Preserve populations, tasks, findings, accommodations, evidence, decisions, dissent, incidents, and gaps.
- Hand off to Content, Information, Application, API, Communication, Privacy, Security, Testing, Operations, and Compliance.
- Record changes requiring reopening affected architecture and user evidence.

RELATED CODEX SECTIONS

- 001-052 governance, user, information, application, security, privacy, safety, and resilience architecture; 054-060 Content through Hardware.
- 075-106 human-computer interaction, UX, accessibility, content, prototyping, and validation; 247 Accessibility Testing; 250 Localization Testing.

SOURCE BASIS AND UNRESOLVED SOURCE GAPS

- Based on the Canonical Section Contract, shared governance files, Sections 001-052, Owner Cockpit Requirements, Status and Evidence Lexicon, and local architecture baseline.
- No universal accessibility standard, population, legal scope, assistive-technology matrix, accommodation authority, or task evidence is supplied. Actual barriers and acceptance remain unresolved.

054

SECTION 054 - CONTENT ARCHITECTURE

CODEX/03_SECTIONS/054_CONTENT_ARCHITECTURE.txt

SHA-256 df7611d0817b84023ef56bc0829a06235edf8d96e9cc464257adf8bb3015e0ad

7,921 BYTES / PRESERVED SOURCE IDENTITY

SECTION 054 — Content Architecture

STATUS: CONTENT-COMPLETE

PURPOSE

Define the lifecycle, structure, ownership, voice, format, metadata, rights, accessibility, localization, publication, correction, retention, and retirement of content.

REQUIRED OUTCOME

- A content model with types, audiences, purposes, owners, sources, workflows, versions, rights, channels, evidence, and lifecycle.
- Controlled authoring, review, publication, correction, withdrawal, localization, accessibility, and archival behavior.

WHY THIS MATTERS

Content can instruct, persuade, record, disclose, or create obligation. Unowned content becomes stale, inaccessible, legally risky, or impossible to correct, especially when generated at scale.

APPLICABILITY

- Content is created, edited, generated, translated, published, rendered, retained, or retired for users, operators, systems, or records.
- Meaning, rights, accessibility, truth, timing, or publication affects material outcomes.

NON-APPLICABILITY CONDITIONS

- No material content, audience, meaning, rights, accessibility, channel, evidence, or lifecycle effect changes.
- An approved content baseline covers exact type, audience, channel, and change with no material impact.
- The artifact is exploratory and not an approved publication or rights claim.

Record baseline, scope, owner, evidence date, assumptions, and reopening trigger.

PREREQUISITES AND DEPENDENCIES

- Sections 016, 019, 030, 040, and 053 provide domain, information, requirements, communications, and accessibility context.
- Content, legal, rights, privacy, accessibility, localization, records, and publication authorities are known or open.

REQUIRED INPUTS

- Audiences, purpose, content types, source, rights, voice, language, channel, accessibility, review, evidence, retention, and lifecycle.
- Editorial workflow, generated content, citation, correction, withdrawal, localization, support, and governance requirements.

HUMAN RESPONSIBILITIES AND AUTHORITY

Content owners approve purpose, meaning, voice, rights, audience, publication, corrections, sensitive claims, exceptions, and retirement. Humans retain final editorial and legal authority and can pause, retract, correct, or archive.

AI RESPONSIBILITIES AND BOUNDARIES

AI may draft, classify, summarize, translate, check structure, and identify stale or duplicate content. It must preserve sources, rights, audience, version, and uncertainty. It must not publish, invent citations, copy restricted material, or make consequential claims without review.

APPLICABLE EXISTING AGENT ROLES

- Documentation Writer, Researcher, Analyst, User Experience Reviewer, Governance Reviewer, Verifier, Critic, Memory Curator, Tester, and Red Team.

REQUIRED ACTIVITIES

1. Inventory content types, sources, owners, audiences, rights, channels, versions, and lifecycle.
2. Define structure, metadata, voice, evidence, citations, review, approval, accessibility, and localization.
3. Model draft, review, approved, published, corrected, withdrawn, archived, superseded, and retired states.
4. Validate content with users, domain, legal, rights, privacy, accessibility, localization, and support authorities.
5. Test findability, comprehension, truth, citation, accessibility, translation, correction, withdrawal, and recovery.
6. Baseline content model, decisions, evidence, dissent, and gaps.

HUMAN-AI INTERACTION REQUIREMENTS

- Label source, draft, transformation, translation, inference, generation, review, and publication state.
- Show provenance, author, reviewer, rights, audience, date, and correction path.
- Require human review for legal, safety, rights, privacy, public, or externally binding content.
- Preserve non-AI authoring, correction, status, and withdrawal paths.

ARCHITECTURE AND SYSTEM IMPLICATIONS

- Separate source, draft, approved version, rendered copy, translation, cache, publication, record, and evidence.
- Enforce rights, access, provenance, accessibility, localization, retention, correction, and withdrawal across channels.
- Preserve versions and links when content is corrected, retracted, superseded, or deleted.

OWNER COCKPIT AND USER-CONTROL REQUIREMENTS

- Show content type, source, owner, audience, state, rights, review, accessibility, language, freshness, and lifecycle.
- Expose draft, review, approve, publish, correct, retract, archive, and retire actions with authority.
- Explain generated or blocked content and preserve an accessible alternative.
- Export content versions, provenance, decisions, evidence, dissent, and continuity state.

REQUIRED OUTPUTS AND ARTIFACTS

- Content model, type and metadata catalog, editorial workflow, rights and source register, channel and localization plan.
- Accessibility, review, generation, citation, correction, withdrawal, records, evidence, and retirement package.

EVIDENCE REQUIREMENTS

- Claims identify content, source, version, author, reviewer, audience, channel, date, method, rights, and limitation.
- Publication, approval, citation, accessibility, or engagement evidence proves only its bounded claim.
- Generated and translated content records source, tool, configuration, review, and limitations.
- Missing, stale, disputed, or inaccessible evidence constrains publication status.

VERIFICATION AND ACCEPTANCE TESTS

- Trace content from source through authoring, review, publication, user action, correction, retention, and retirement.

- Exercise wrong version, broken citation, rights restriction, inaccessible rendering, translation error, stale content, generation failure, and retraction.
- Verify approval, provenance, accessibility, localization, records, correction, withdrawal, and audit.
- Reproduce representative claims from exact content package.

FAILURE STATES AND ANTI-PATTERNS

- Treating publication, engagement, polish, or generated fluency as truth or comprehension.
- Unowned stale content, missing sources, inaccessible formats, rights leakage, hidden edits, or irreversible publication.
- Allowing AI to invent sources, imitate authority, or publish without review.

RECOVERY AND RECONSIDERATION

- On false, harmful, stale, inaccessible, or unauthorized content, pause distribution and preserve prior versions.
- Identify affected audiences, owner, correction, notification, evidence, and review condition.
- Use correction, retraction, alternate format, access restriction, archive, or retirement.
- Reopen on source, audience, law, language, channel, model, or lifecycle change.

COMPLETION CRITERIA

- Types, audiences, sources, rights, review, accessibility, localization, publication, evidence, correction, and lifecycle are explicit.
- Generated, translated, disputed, stale, withdrawn, and retired content behavior is addressed or justified.
- Exact limitations and unresolved source gaps are recorded.

HANDOFF AND CONTINUITY REQUIREMENTS

- Preserve content, versions, provenance, rights, approvals, corrections, decisions, evidence, dissent, and gaps.
- Hand off to Information, Knowledge, AI, Accessibility, Privacy, Security, Communications, Records, Testing, and Operations.
- Record changes requiring reopening affected information and governance sections.

RELATED CODEX SECTIONS

- 016, 019, 030, 040, and 053 Domain, Requirements, Information, Communications, and Accessibility Architecture; 055-060 Knowledge through Hardware.
- 097-103 content, strategy, writing, localization, and accessibility design; 341-342 documentation and knowledge management; 391-395 retention and deletion.

SOURCE BASIS AND UNRESOLVED SOURCE GAPS

- Based on the Canonical Section Contract, shared governance files, Sections 001-053, Owner Cockpit Requirements, Status and Evidence Lexicon, and local architecture baseline.
- No universal editorial, rights, accessibility, localization, records, or publication standard is supplied. Actual content owners, sources, audiences, and acceptance evidence remain unresolved.

055

SECTION 055 - KNOWLEDGE ARCHITECTURE

CODEX/03_SECTIONS/055_KNOWLEDGE_ARCHITECTURE.txt

SHA-256 3e99a9aa8813bc90f192eba82c9823900fc56450a205a88d7fef1c98816b9524

7,879 BYTES / PRESERVED SOURCE IDENTITY

SECTION 055 — Knowledge Architecture

STATUS: CONTENT-COMPLETE

PURPOSE

Define how knowledge is represented, sourced, curated, related, retrieved, taught, cited, corrected, governed, and retired for human and machine use.

REQUIRED OUTCOME

- A knowledge landscape with concepts, sources, provenance, authority, versions, relationships, retrieval contexts, users, and lifecycle.
- Explicit distinction between source fact, interpretation, inference, generated answer, uncertainty, and accepted knowledge.

WHY THIS MATTERS

Knowledge systems can create false authority by blending sources, inference, memory, and generation. Retrieval quality, citations, and fluency do not by themselves establish truth, currency, or suitability for a decision.

APPLICABILITY

- A system stores, retrieves, teaches, summarizes, links, recommends, or generates knowledge for people or machines.
- Knowledge quality, provenance, access, currency, interpretation, or correction affects outcomes.

NON-APPLICABILITY CONDITIONS

- No material knowledge source, meaning, retrieval, provenance, audience, access, or lifecycle effect changes.
- An approved baseline covers exact domain, sources, audience, and change with no material impact.
- The artifact is exploratory and not a truth, expertise, or answer-quality claim.

Record baseline, scope, owner, evidence date, assumptions, and reopening trigger.

PREREQUISITES AND DEPENDENCIES

- Sections 016, 030, 047, and 054 provide domain, information, privacy, content, and accessibility context.
- Domain experts, source authorities, rights, records, privacy, security, accessibility, and user authorities are known or open.

REQUIRED INPUTS

- Concepts, sources, evidence, authorities, versions, relationships, audiences, questions, retrieval, citations, access, correction, and lifecycle.
- Curation, review, training, generation, retention, withdrawal, and uncertainty requirements.

HUMAN RESPONSIBILITIES AND AUTHORITY

Knowledge owners and domain experts approve meaning, source authority, curation, conflict resolution, publication, and retirement. Humans decide what is accepted knowledge and retain correction, withdrawal, and appeal authority.

AI RESPONSIBILITIES AND BOUNDARIES

AI may classify sources, propose relations, retrieve context, summarize, identify conflicts, and draft knowledge records. It must preserve provenance and uncertainty. It must not fabricate sources, resolve expert conflict as fact, disclose restricted knowledge, or claim expertise without authority.

APPLICABLE EXISTING AGENT ROLES

- Researcher, Analyst, Documentation Writer, Architect, User Experience Reviewer, Governance Reviewer, Verifier, Critic, Memory Curator, Tester, Planner, and Red Team.

REQUIRED ACTIVITIES

1. Inventory knowledge domains, sources, concepts, authorities, audiences, and lifecycle.
2. Define ontology or vocabulary, provenance, confidence, version, conflict, citation, access, and review.
3. Model source, derived, inferred, generated, superseded, disputed, and retired knowledge states.
4. Design retrieval, ranking, context, explanation, correction, withdrawal, and fallback paths.
5. Validate with domain experts, users, privacy, security, accessibility, legal, records, and support.
6. Test ambiguity, stale source, conflicting authority, missing citation, restricted source, retrieval failure, and correction.
7. Baseline and hand off evidence, decisions, dissent, and gaps.

HUMAN-AI INTERACTION REQUIREMENTS

- Show source, authority, date, context, confidence limit, transformation, model, and generated status.
- Require human review for canonical knowledge, sensitive disclosure, high-impact advice, and conflict resolution.
- Provide citation, correction, appeal, non-AI, and recovery paths.
- Never treat a fluent answer, rank, or citation count as truth.

ARCHITECTURE AND SYSTEM IMPLICATIONS

- Separate source repository, knowledge record, index, embedding, retrieval result, generated response, citation, and evidence.
- Preserve lineage, source access, version, expiry, withdrawal, and deletion across indexes, caches, models, and prompts.
- Bind retrieval and generation to purpose, audience, permission, provenance, and freshness.

OWNER COCKPIT AND USER-CONTROL REQUIREMENTS

- Show knowledge item, source, authority, version, date, access, confidence, conflicts, model, and lifecycle.
- Expose review, correct, dispute, restrict, withdraw, supersede, archive, and retire actions.
- Explain why a source or answer was included, omitted, ranked, or blocked.
- Export knowledge graph or catalog, lineage, decisions, evidence, dissent, and continuity state.

REQUIRED OUTPUTS AND ARTIFACTS

- Knowledge context, concept and source catalog, ontology or vocabulary, provenance and authority model.
- Retrieval, ranking, citation, generation, correction, conflict, access, privacy, accessibility, and lifecycle design.
- Evidence, decision, verification, acceptance, and handoff package.

EVIDENCE REQUIREMENTS

- Claims identify source, authority, concept, version, date, context, retrieval, transformation, model, method, and limitation.
- A citation, retrieval result, expert statement, benchmark, or answer proves only its bounded claim.
- Conflicting, stale, inaccessible, withdrawn, or missing sources constrain knowledge status.

VERIFICATION AND ACCEPTANCE TESTS

- Trace representative answers or decisions to source, authority, retrieval, transformation, citation, correction, and retirement.
- Exercise ambiguity, conflict, stale source, restricted source, prompt injection, missing citation, retrieval loss, and withdrawal.

- Verify permission, provenance, explanation, correction, accessibility, deletion, and fallback.
- Reproduce representative claims from exact knowledge and evidence package.

FAILURE STATES AND ANTI-PATTERNS

- Treating retrieval, citation, expertise label, or fluent generation as truth.
- Blending fact, inference, opinion, and generated content without labels or provenance.
- Hiding source conflict, stale knowledge, rights, access, or correction paths.

RECOVERY AND RECONSIDERATION

- On false answer, source conflict, injection, provenance loss, restricted disclosure, or stale knowledge, restrict use and preserve history.
- Identify affected users, domain owner, evidence, correction, communication, and review condition.
- Use source withdrawal, manual review, restricted mode, correction, reindexing, rollback, or retirement.
- Reopen on source, domain, audience, model, policy, or lifecycle change.

COMPLETION CRITERIA

- Concepts, sources, authority, provenance, retrieval, generation, uncertainty, access, correction, evidence, and lifecycle are explicit.
- Conflict, stale, restricted, generated, inaccessible, and retired knowledge behavior is addressed or justified.
- Exact limitations and unresolved source gaps are recorded.

HANDOFF AND CONTINUITY REQUIREMENTS

- Preserve source, concept, relation, version, retrieval, citation, decision, evidence, dissent, and gap identities.
- Hand off to Information, Content, AI, Machine Learning, Privacy, Security, Accessibility, Testing, Operations, and Records.
- Record changes requiring reopening Domain, Data, Governance, or AI architecture.

RELATED CODEX SECTIONS

- 016, 030, 047, and 054 Domain, Information, Privacy, and Content Architecture; 056-060 AI through Hardware.
- 171-177 NLP, knowledge graph, ontology, and semantic engineering; 342 Knowledge Management; 180 Model Evaluation.

SOURCE BASIS AND UNRESOLVED SOURCE GAPS

- Based on the Canonical Section Contract, shared governance files, Sections 001-054, Owner Cockpit Requirements, Status and Evidence Lexicon, and local architecture baseline.
- No universal ontology, source authority, evidence threshold, retrieval standard, or domain corpus is supplied. Actual knowledge owners, sources, permissions, and acceptance remain unresolved.

056

SECTION 056 - AI ARCHITECTURE

CODEX/03_SECTIONS/056_AI_ARCHITECTURE.txt

SHA-256 4a6fdf0287b97acb60898df0c08393584a518941d8004acadeb43ef629bcf5fe

9,547 BYTES / PRESERVED SOURCE IDENTITY

SECTION 056 — AI Architecture

STATUS: CONTENT-COMPLETE

PURPOSE

Define the governed structure of AI capabilities, models, agents, prompts, tools, context, data, evaluation, human oversight, deployment, operation, and retirement.

REQUIRED OUTCOME

- An AI system context with purpose, capability, model, provider, data, prompt, tool, authority, user, risk, evaluation, monitoring, fallback, and lifecycle.
- Explicit distinction between assistance, recommendation, decision, action, and external consequence.
- Traceability from requirements and governance through implementation, runtime, evidence, incidents, compensation, and retirement.

WHY THIS MATTERS

AI can generate plausible but unsupported content, act through tools, amplify data bias, drift after change, and obscure responsibility behind confidence or automation. Architecture must make capability and authority separate and inspectable.

APPLICABILITY

- A model, agent, classifier, generator, recommender, planner, tool-calling system, or AI provider affects people, data, decisions, workflows, or external state.
- AI behavior has material safety, privacy, security, legal, financial, accessibility, reliability, or governance consequence.

NON-APPLICABILITY CONDITIONS

- No model, agent, provider, prompt, AI-derived output, tool, data, authority, or lifecycle effect changes.
- An approved AI baseline covers exact use, model, data, population, and change with no material impact.
- The artifact is exploratory and not an accuracy, safety, alignment, or autonomy claim.

Record baseline, scope, owner, evidence date, assumptions, and reopening trigger.

PREREQUISITES AND DEPENDENCIES

- Sections 019-055 provide requirements, architecture, data, privacy, security, safety, knowledge, and governance context.
- Model, provider, data, policy, capability, evaluation, red-team, human-review, and incident authorities are known or open.

REQUIRED INPUTS

- Purpose, users, affected parties, model, provider, version, prompt, context, data, tools, permissions, autonomy, environment, and lifecycle.
- Quality, safety, security, privacy, fairness, accessibility, cost, latency, monitoring, fallback, appeal, evaluation, and retirement requirements.

HUMAN RESPONSIBILITIES AND AUTHORITY

AI owners and affected-domain authorities approve use case, capability, model, data, autonomy, tools, human review, risk, release, incident, and retirement. Humans retain final authority over decisions, external effects, rights, safety, privacy, and compensation.

AI RESPONSIBILITIES AND BOUNDARIES

AI may analyze authorized inputs, generate bounded proposals, retrieve context, invoke explicitly permitted tools, and report uncertainty. It must identify source and confidence limits, preserve trace and evidence, and stop on missing authority or policy. It must not self-authorize, conceal uncertainty, bypass evaluation, or treat output as fact or permission.

APPLICABLE EXISTING AGENT ROLES

- Architect, Researcher, Analyst, Governance Reviewer, Security Auditor, Red Team, Tester, Verifier, Critic, Planner, Implementer, Documentation Writer, Memory Curator, and User Experience Reviewer.

REQUIRED ACTIVITIES

1. Define purpose, capability, users, affected parties, authority, autonomy, and prohibited use.
2. Inventory model, provider, prompt, context, data, tools, permissions, dependencies, and lifecycle.
3. Model input, retrieval, generation, reasoning, tool call, decision, action, human review, and external effect.
4. Define evaluation, monitoring, drift, abuse, privacy, security, safety, accessibility, cost, fallback, and incident controls.
5. Bind every mutation-capable tool action to policy evaluation, capability, trace, immutable decision, and audit.
6. Validate with domain, users, safety, security, privacy, legal, accessibility, operations, and governance authorities.
7. Test normal, adversarial, ambiguous, prompt-injected, unavailable, stale, biased, degraded, recovery, and retirement paths.
8. Baseline decisions, evidence, dissent, limitations, and handoff.

HUMAN-AI INTERACTION REQUIREMENTS

- Show intent, model, provider, prompt, context, source, tool, capability, authority, evaluation, decision, trace, and consequence.
- Require human approval for rights, safety, privacy, security, policy, external commitments, and irreversible mutations.
- Preserve status, explanation, correction, appeal, non-AI, stop, evidence, and recovery paths.
- Default-deny on uncertainty, missing binding, policy conflict, expired capability, or failed evaluation.

ARCHITECTURE AND SYSTEM IMPLICATIONS

- Separate model, provider, prompt, context, retrieval, output, tool invocation, decision, action, audit, and evidence.
- Treat model and provider change as architecture change that can invalidate evaluation and controls.
- Enforce capability scope, policy evaluation, trace, immutable decision, quorum where applicable, and append-only audit at execution.
- Preserve compensation-only rollback and terminal success, denial, failure, cancellation, and recovery events.

OWNER COCKPIT AND USER-CONTROL REQUIREMENTS

- Show use case, model, provider, version, data, prompt, tools, authority, evaluation, uncertainty, incidents, and lifecycle.
- Expose approve, deny, constrain, pause, revoke, inspect, appeal, compensate, supersede, and retire actions.
- Explain source, uncertainty, policy, denial, tool effect, and human decision point.
- Export prompts, context records, traces, decisions, evaluations, audits, evidence, dissent, and continuity state.

REQUIRED OUTPUTS AND ARTIFACTS

- AI context, use-case and capability catalog, model and provider record, data and prompt contract, tool and authority model.
- Evaluation, safety, security, privacy, fairness, accessibility, monitoring, incident, fallback, compensation, and retirement design.
- Decision, governance binding, audit, verification, evidence, acceptance, and handoff package.

EVIDENCE REQUIREMENTS

- Claims identify use case, model, provider, version, prompt, context, data, tools, permissions, evaluation, environment, method, result, and limitation.
- Benchmark, sample output, user rating, tool success, or provider statement proves only its declared scope.
- Safety, fairness, privacy, security, reliability, and autonomy evidence states population, task, threat, coverage, and validity.
- Missing, stale, adversarial, contradictory, or unrepresentative evidence constrains AI status.

VERIFICATION AND ACCEPTANCE TESTS

- Trace use case through model, data, prompt, context, evaluation, output, human review, tool, decision, audit, and retirement.
- Exercise hallucination, ambiguity, prompt injection, data leakage, bias, jailbreak, tool misuse, replay, expired capability, provider loss, and model drift.
- Verify default-deny, mutation binding, policy evaluation, immutable decision, quorum, audit, trace, compensation, and terminal stream semantics.
- Disable model, provider, retrieval, tool, identity, policy, network, and evidence services and verify safe fallback.
- Reproduce representative claims from exact run, evaluation, decision, and evidence records.

FAILURE STATES AND ANTI-PATTERNS

- Treating fluency, benchmark, confidence, retrieval, or automation as truth, authority, safety, or acceptance.
- Hidden prompts, tools, context, provider changes, data leakage, stale evaluations, or unowned human review.
- Letting an agent mutate external state without complete governance binding and immutable decision.

RECOVERY AND RECONSIDERATION

- On unsafe output, policy conflict, failed evaluation, tool misuse, leakage, drift, provider loss, or audit gap, stop affected execution and preserve traces.
- Identify safe read-only work, affected parties, owner, authority, evidence, communication, and review condition.
- Use constraint, revoke, fallback, human review, compensation, restore, replacement, or retirement through new authorized decisions.
- Reopen on model, provider, prompt, data, tool, policy, population, or lifecycle change.

COMPLETION CRITERIA

- Purpose, capability, model, data, prompt, tool, authority, evaluation, human review, evidence, failure, recovery, and lifecycle are explicit.
- Adversarial, denied, degraded, mutation, compensation, and retirement behavior is addressed or justified.
- Exact limits and unresolved evidence are recorded; CONTENT-COMPLETE does not imply safe or approved AI.

HANDOFF AND CONTINUITY REQUIREMENTS

- Preserve use case, model, provider, prompt, context, tool, trace, evaluation, decision, audit, incident, evidence, dissent, and gaps.
- Hand off to Machine Learning, AI Safety, AI Governance, Security, Privacy, Knowledge, Testing, Operations, and Retirement.
- Record changes requiring reopening every affected architecture and evaluation baseline.

RELATED CODEX SECTIONS

- 019-055 Requirements through Knowledge Architecture; 057-060 Machine Learning through Hardware Architecture.
- 167-188 machine learning, AI engineering, model evaluation, alignment, safety, governance, MLOps, prompt, HITL; 180-186 evaluation and operations.

SOURCE BASIS AND UNRESOLVED SOURCE GAPS

- Based on the Canonical Section Contract, uploaded governing requirements, shared governance files, Sections 001-055, Owner Cockpit Requirements, Status and Evidence Lexicon, and local architecture baseline.
- No universal model, provider, evaluation, safety, fairness, privacy, autonomy, or sector rule is supplied. Actual use cases, data, capability tokens, policy bindings, human authorities, and acceptance evidence remain unresolved.

057

SECTION 057 - MACHINE LEARNING ARCHITECTURE

CODEX/03_SECTIONS/057_MACHINE_LEARNING_ARCHITECTURE.txt

SHA-256 aa8c6f027f28c1fe6bc61c19ed840fcce111e7b401f3b45deb1f05a359fc38e5

8,430 BYTES / PRESERVED SOURCE IDENTITY

SECTION 057 — Machine Learning Architecture

STATUS: CONTENT-COMPLETE

PURPOSE

Define the governed lifecycle and structure of datasets, features, training, evaluation, models, serving, monitoring, human review, and retirement for machine learning systems.

REQUIRED OUTCOME

- A machine learning lifecycle model with purpose, data, labels, features, model, pipeline, evaluation, deployment, drift, owners, risks, evidence, and lifecycle.
- Reproducible lineage and bounded claims for performance, fairness, robustness, safety, privacy, and operational behavior.

WHY THIS MATTERS

Machine learning behavior depends on data, labels, population, objective, threshold, environment, and feedback. A high aggregate score can conceal subgroup harm, leakage, drift, or an invalid deployment context.

APPLICABILITY

- A learned model influences classification, prediction, ranking, generation, recommendation, control, resource allocation, or decisions.
- Training, data, evaluation, drift, or model change affects material outcomes.

NON-APPLICABILITY CONDITIONS

- No learned artifact, training data, feature, evaluation, model output, deployment, or lifecycle effect changes.
- An approved baseline covers exact task, population, model, data, and change with no material impact.
- The artifact is research or synthetic and not a production, fairness, safety, or accuracy claim.

Record baseline, scope, owner, evidence date, assumptions, population, and reopening trigger.

PREREQUISITES AND DEPENDENCIES

- Sections 019-056 provide requirements, data, privacy, security, safety, AI, knowledge, operations, and governance context.
- Data, domain, evaluation, safety, privacy, security, model, deployment, and human-review authorities are known or open.

REQUIRED INPUTS

- Task, objective, population, labels, datasets, features, model, training, evaluation, thresholds, deployment, feedback, and lifecycle.
- Quality, fairness, privacy, security, robustness, interpretability, monitoring, drift, rollback, retraining, and retirement requirements.

HUMAN RESPONSIBILITIES AND AUTHORITY

Model and domain owners approve task, data purpose, labels, metrics, thresholds, evaluation, deployment, monitoring, retraining, risk, and retirement. Humans decide whether evidence supports use and retain authority over consequential outcomes.

AI RESPONSIBILITIES AND BOUNDARIES

AI may analyze authorized datasets, propose features and tests, summarize evaluations, and identify drift. It must preserve lineage, population, split, metric, and uncertainty. It must not fabricate labels, hide subgroup results, approve deployment, or infer safety from aggregate performance.

APPLICABLE EXISTING AGENT ROLES

- Architect, Researcher, Analyst, Implementer, Tester, Verifier, Security Auditor, Governance Reviewer, Critic, Documentation Writer, Memory Curator, and Red Team.

REQUIRED ACTIVITIES

1. Define task, outcome, population, prohibited use, owners, and lifecycle.
2. Inventory data, labels, features, models, code, configurations, dependencies, and provenance.
3. Define training, validation, test, deployment, threshold, calibration, and monitoring contracts.
4. Assess leakage, bias, drift, robustness, privacy, security, explainability, accessibility, and feedback loops.
5. Validate with domain, data, users, privacy, security, safety, accessibility, operations, and governance authorities.
6. Test distribution shift, adversarial input, missing data, subgroup performance, retraining, rollback, and recovery.
7. Baseline lineage, evaluation, decisions, evidence, dissent, limitations, and gaps.

HUMAN-AI INTERACTION REQUIREMENTS

- Show task, population, data, model, metric, threshold, confidence, human review, authority, and consequence.
- Require human approval for training data purpose, deployment, threshold, retraining, and high-impact use.
- Preserve manual review, explanation, appeal, stop, status, evidence, and recovery paths.
- Never treat a score, benchmark, or model card as universal validity.

ARCHITECTURE AND SYSTEM IMPLICATIONS

- Preserve lineage from source data and label through feature, training run, model artifact, deployment, output, feedback, and retirement.
- Separate training, validation, test, shadow, canary, production, rollback, and archived model states.
- Bind model change to evaluation invalidation, approval, monitoring, and rollback or compensation semantics.
- Protect datasets, features, weights, prompts, logs, and outputs under privacy, security, retention, and deletion controls.

OWNER COCKPIT AND USER-CONTROL REQUIREMENTS

- Show task, population, dataset, model, version, metric, subgroup results, drift, owner, authority, evidence, and lifecycle.
- Expose review, constrain, pause, rollback, retrain, revoke, investigate, appeal, and retire actions.
- Explain evidence limits, uncertainty, threshold, data change, and human decision point.
- Export lineage, runs, evaluations, decisions, audits, evidence, dissent, and continuity state.

REQUIRED OUTPUTS AND ARTIFACTS

- ML context, data and label catalog, feature and model lineage, experiment and evaluation records, deployment and monitoring plan.
- Fairness, privacy, security, robustness, safety, human review, drift, retraining, rollback, and retirement package.

EVIDENCE REQUIREMENTS

- Claims identify task, population, data, split, feature, model, version, metric, threshold, environment, method, result, and limitation.
- Benchmark, validation, model card, user rating, or provider claim proves only its declared population and condition.
- Subgroup, drift, leakage, robustness, privacy, and deployment evidence remains visible and bounded.
- Missing, stale, biased, leaked, or unrepresentative evidence constrains status.

VERIFICATION AND ACCEPTANCE TESTS

- Trace a claim from requirement through data, label, feature, run, model, evaluation, deployment, output, monitoring, and retirement.
- Exercise leakage, shift, missingness, adversarial input, subgroup harm, threshold change, retraining, model swap, and provider loss.
- Verify lineage, access, privacy, fairness, robustness, human review, rollback, audit, and appeal.
- Reproduce representative evaluations from exact code, data, configuration, and environment.

FAILURE STATES AND ANTI-PATTERNS

- Treating aggregate accuracy, benchmark, model card, or training completion as production validity.
- Hiding data leakage, subgroup failure, drift, feedback loops, label uncertainty, or threshold effects.
- Retraining or replacing a model without re-evaluation, approval, lineage, and rollback.

RECOVERY AND RECONSIDERATION

- On drift, bias, leakage, unsafe output, data issue, evaluation failure, or evidence loss, restrict model use and preserve runs.
- Identify affected population, owner, authority, evidence, communication, remedy, and review condition.
- Use human review, threshold restriction, rollback, model substitution, retraining, compensation, or retirement.
- Reopen on task, population, data, feature, model, provider, policy, or lifecycle change.

COMPLETION CRITERIA

- Task, population, data, lineage, model, evaluation, thresholds, human review, evidence, monitoring, recovery, and lifecycle are explicit.
- Drift, adversarial, subgroup, retraining, rollback, and retirement behavior is addressed or justified.
- Exact validity and unresolved evidence are recorded; CONTENT-COMPLETE does not imply model fitness.

HANDOFF AND CONTINUITY REQUIREMENTS

- Preserve datasets, labels, features, runs, models, evaluations, decisions, evidence, incidents, dissent, and gaps.
- Hand off to Data, AI, Security, Privacy, Safety, MLOps, Testing, Operations, Governance, and Retirement.
- Record changes requiring reopening AI, Data, Privacy, Safety, or Governance architecture.

RELATED CODEX SECTIONS

- 019-056 Requirements through AI Architecture; 058-060 Robotics through Hardware Architecture.
- 164-186 data science, ML, deep learning, AI engineering, evaluation, alignment, safety, governance, MLOps, and model operations.

SOURCE BASIS AND UNRESOLVED SOURCE GAPS

- Based on the Canonical Section Contract, shared governance files, Sections 019-056, Owner Cockpit Requirements, Status and Evidence Lexicon, and local architecture baseline.
- No universal task, population, metric, fairness, robustness, privacy, or deployment standard is supplied. Actual datasets, labels, owners, evaluations, and acceptance evidence remain unresolved.

058

SECTION 058 - ROBOTICS ARCHITECTURE

CODEX/03_SECTIONS/058_ROBOTICS_ARCHITECTURE.txt

SHA-256 9db76c1ee7aec285c188a5e6d5bf99c8fbc9bdf5497eba3fdacc2a02ffd46d24

8,171 BYTES / PRESERVED SOURCE IDENTITY

SECTION 058 — Robotics Architecture

STATUS: CONTENT-COMPLETE

PURPOSE

Define the governed structure of robots, actuators, sensors, controllers, autonomy, human interaction, environments, safety, maintenance, and lifecycle.

REQUIRED OUTCOME

- A robotics context with physical world, tasks, bodies, sensors, actuators, control, autonomy, human authority, hazards, and recovery.
- Explicit safe states, envelopes, communications, evidence, manual control, update, service, and retirement behavior.

WHY THIS MATTERS

Robotic systems can create immediate physical consequence under uncertainty, latency, sensor failure, environmental change, and human proximity. Simulation or nominal motion does not establish safe real-world operation.

APPLICABILITY

- Software or AI senses, plans, controls, or affects physical motion, force, environment, or equipment.
- Physical action, human proximity, autonomy, or failure could create material harm.

NON-APPLICABILITY CONDITIONS

- No physical sensing, actuation, autonomy, environment, human interaction, or lifecycle effect changes.
- An approved baseline covers exact robot, task, environment, and change with no material impact.
- The artifact is simulation or research and explicitly not a field-safety claim.

Record robot, task, environment, owner, evidence date, assumptions, and reopening trigger.

PREREQUISITES AND DEPENDENCIES

- Sections 019-057 provide requirements, safety, AI, ML, edge, embedded, data, security, and operations context.
- Qualified robotics, safety, human factors, domain, maintenance, accessibility, security, and legal authorities are identified or gaps recorded.

REQUIRED INPUTS

- Robot, task, environment, sensors, actuators, controller, autonomy, human interaction, hazards, limits, communications, and lifecycle.
- Stop, interlock, manual control, update, maintenance, inspection, evidence, recovery, and retirement requirements.

HUMAN RESPONSIBILITIES AND AUTHORITY

Safety and domain owners approve tasks, operating envelope, autonomy, controls, training, maintenance, exceptions, and acceptance. Humans retain immediate stop, manual control, isolation, recovery, and retirement authority.

AI RESPONSIBILITIES AND BOUNDARIES

AI may propose paths, analyze sensor or telemetry data, simulate scenarios, and identify hazards. It must preserve environment and uncertainty. It must not exceed task or capability bounds, suppress alarms, issue physical action without authority, or declare safe motion.

APPLICABLE EXISTING AGENT ROLES

- Architect, Security Auditor, Researcher, Analyst, Observer, Tester, Verifier, Governance Reviewer, Critic, Planner, Documentation Writer, Memory Curator, and Red Team.

REQUIRED ACTIVITIES

1. Define task, environment, people, robot, physical effect, owner, autonomy, and lifecycle.
2. Model sensors, actuators, controller, energy, communications, control loops, workspace, and hazards.
3. Define operating envelope, safe state, stop, interlock, manual control, degraded, recovery, and maintenance.
4. Assess AI and ML data, model, update, security, privacy, accessibility, and human factors.
5. Validate with users, operators, safety, domain, maintenance, security, privacy, accessibility, and emergency authorities.
6. Test physical, simulated, degraded, sensor-loss, communication-loss, adversarial, human, and recovery conditions.
7. Baseline safety case, evidence, decisions, dissent, residual risk, and gaps.

HUMAN-AI INTERACTION REQUIREMENTS

- Show robot, task, environment, sensor, model, authority, trajectory, force, human proximity, and consequence.
- Require human approval for autonomy, task change, workspace, overrides, updates, and physical action.
- Preserve independent stop, manual, alarm, status, evidence, and recovery controls.
- Default-deny when localization, perception, authority, or safety evidence is missing.

ARCHITECTURE AND SYSTEM IMPLICATIONS

- Separate mission, plan, controller, sensor input, actuator command, physical state, evidence, and decision.
- Enforce speed, force, workspace, identity, authority, interlock, and safe-stop constraints at control boundary.
- Account for latency, calibration, drift, power, thermal, mechanical, communication, and human factors.
- Preserve immutable decisions and append-only safety and incident evidence.

OWNER COCKPIT AND USER-CONTROL REQUIREMENTS

- Show robot, task, mode, location, sensors, actuators, autonomy, envelope, operator, alerts, evidence, and lifecycle.
- Expose stop, manual, isolate, reset, inspect, recover, update, rollback, and retire actions with authority.
- Explain uncertainty, blocked action, safe state, and recovery conditions accessibly.
- Export telemetry, commands, decisions, evidence, incidents, dissent, and continuity state.

REQUIRED OUTPUTS AND ARTIFACTS

- Robotics context, task and environment model, control and safety architecture, autonomy and human-control model.
- Sensor, actuator, update, maintenance, security, privacy, test, evidence, recovery, acceptance, and retirement package.

EVIDENCE REQUIREMENTS

- Claims identify robot, task, environment, configuration, payload, model, sensor, test method, date, result, and limitation.
- Simulation, bench, demonstration, drill, or field evidence proves only its declared conditions.
- Safety evidence identifies envelope, hazards, controls, human role, failures, and residual risk.
- Missing calibration, field, human, or failure evidence constrains status.

VERIFICATION AND ACCEPTANCE TESTS

- Trace task through plan, control, physical action, safety control, evidence, recovery, and retirement.
- Exercise obstacle, sensor loss, localization failure, stale model, communication loss, power loss, human intrusion, and update failure.
- Verify independent stop, interlock, manual control, authority, audit, privacy, and recovery.

- Disable model, provider, network, sensor, actuator, operator, and evidence path and verify safe behavior.
- Reproduce representative safety and performance claims from exact test package.

FAILURE STATES AND ANTI-PATTERNS

- Treating simulation, path success, sensor confidence, or nominal motion as safe operation proof.
- Hiding workspace, human proximity, calibration, latency, power, maintenance, or manual-control assumptions.
- Allowing AI or remote operator convenience to bypass safety envelope or stop authority.

RECOVERY AND RECONSIDERATION

- On unsafe motion, sensor fault, collision risk, authority loss, or evidence failure, stop or enter safe state and preserve records.
- Identify people, operator, authority, safe recovery, evidence, communication, and review condition.
- Use manual recovery, isolation, repair, rollback, reduced task, compensation, or retirement.
- Reopen on robot, task, environment, model, controller, sensor, policy, or lifecycle change.

COMPLETION CRITERIA

- Robot, task, environment, control, autonomy, hazards, human authority, evidence, recovery, and lifecycle are explicit.
- Real-world, degraded, emergency, maintenance, update, and retirement paths are addressed or justified.
- Exact limits and unresolved safety evidence are recorded; CONTENT-COMPLETE does not imply safe deployment.

HANDOFF AND CONTINUITY REQUIREMENTS

- Preserve robot, mission, model, controller, sensor, command, incident, decision, evidence, dissent, and gaps.
- Hand off to Embedded, Hardware, Edge, AI, Safety, Security, Reliability, Testing, Operations, and Retirement.
- Record changes requiring reopening all affected architecture and safety baselines.

RELATED CODEX SECTIONS

- 019-057 Requirements through Machine Learning Architecture; 059-060 Embedded Systems and Hardware.
- 073-074 Control Systems and Robotics Engineering; 182 AI Safety; 198-199 embedded and hardware security; 252 Safety Testing.

SOURCE BASIS AND UNRESOLVED SOURCE GAPS

- Based on the Canonical Section Contract, shared governance files, Sections 019-057, Owner Cockpit Requirements, Status and Evidence Lexicon, and local architecture baseline.
- No universal robot, task, safety envelope, human factors, certification, or field evidence is supplied. Actual hardware, environments, authorities, and acceptance remain unresolved.

059

SECTION 059 - EMBEDDED SYSTEMS ARCHITECTURE

CODEX/03_SECTIONS/059_EMBEDDED_SYSTEMS_ARCHITECTURE.txt

SHA-256 b751b88d51ae1543ae9e9ee2d03aaf66a5c71e1c14792da6ba92055b64ebe313

8,515 BYTES / PRESERVED SOURCE IDENTITY

SECTION 059 — Embedded Systems Architecture

STATUS: CONTENT-COMPLETE

PURPOSE

Define the structure and lifecycle of processors, firmware, real-time behavior, sensors, actuators, buses, power, memory, updates, diagnostics, safety, and physical integration in embedded systems.

REQUIRED OUTCOME

- A hardware-software context with timing, resources, interfaces, state, control, identity, security, safety, maintenance, and lifecycle.
- Explicit boot, runtime, fault, update, recovery, calibration, manufacturing, service, and disposal behavior.

WHY THIS MATTERS

Embedded systems combine code with physical constraints and often persist in the field for years. A software fix can change timing, energy, safety, or secure-boot assumptions, while a hardware fault can corrupt software evidence and control.

APPLICABILITY

- Software is integrated with dedicated processors, firmware, sensors, actuators, buses, power, or physical equipment.
- Timing, resource, update, physical, safety, security, or service constraints have material consequence.

NON-APPLICABILITY CONDITIONS

- No firmware, hardware interface, timing, resource, power, physical, update, or lifecycle effect changes.
- An approved baseline covers exact device, firmware, environment, and change with no material impact.
- The artifact is simulation or prototype and explicitly not field-readiness evidence.

Record device, version, scope, owner, evidence date, assumptions, and reopening trigger.

PREREQUISITES AND DEPENDENCIES

- Sections 019-058 provide requirements, software, hardware, robotics, safety, security, edge, data, and operations context.
- Hardware, firmware, electrical, safety, manufacturing, service, security, and field authorities are known or gaps recorded.

REQUIRED INPUTS

- Device, processor, firmware, interfaces, timing, memory, power, thermal, sensors, actuators, environment, users, and lifecycle.
- Boot, update, calibration, diagnostic, safety, security, privacy, manufacturing, maintenance, recovery, and retirement requirements.

HUMAN RESPONSIBILITIES AND AUTHORITY

Embedded and product owners approve timing, safety, secure update, field service, calibration, exceptions, and acceptance. Humans retain authority to stop, isolate, update, rollback, replace, recall, and retire devices.

AI RESPONSIBILITIES AND BOUNDARIES

AI may inspect authorized designs, analyze telemetry, draft firmware tests, identify resource or timing risks, and propose diagnostics. It must preserve device and configuration context. It must not flash devices, alter safety code, bypass secure boot, or declare field safety without authority.

APPLICABLE EXISTING AGENT ROLES

- Architect, Implementer, Researcher, Analyst, Security Auditor, Tester, Verifier, Governance Reviewer, Critic, Planner, Documentation Writer, Memory Curator, and Red Team.

REQUIRED ACTIVITIES

1. Define device, environment, purpose, users, physical effect, timing, resources, owners, and lifecycle.
2. Model processor, memory, buses, firmware, boot, sensors, actuators, power, thermal, and diagnostics.
3. Define real-time, degraded, fail-safe, update, calibration, recovery, manufacturing, service, and disposal states.
4. Allocate safety, security, privacy, accessibility, observability, evidence, and field-support controls.
5. Validate with hardware, firmware, electrical, safety, security, manufacturing, service, and affected-user authorities.
6. Test timing, resource exhaustion, power loss, corruption, bus fault, update failure, recovery, and environmental stress.
7. Baseline configuration, evidence, decisions, dissent, and gaps.

HUMAN-AI INTERACTION REQUIREMENTS

- Show device, firmware, hardware, sensor, authority, update, data, physical effect, and safety state.
- Require human approval for field updates, calibration, secure-boot changes, safety behavior, and irreversible actions.
- Preserve physical stop, diagnostic, manual, status, evidence, and recovery controls.
- Default-deny on unknown device identity, firmware provenance, or safety condition.

ARCHITECTURE AND SYSTEM IMPLICATIONS

- Separate source, build, signed image, installed firmware, boot state, runtime state, telemetry, calibration, and evidence.
- Define timing, interrupt, watchdog, memory, power, thermal, bus, update, rollback, and fault containment semantics.
- Enforce secure identity, signed updates, least privilege, diagnostics, logging, and privacy across field paths.
- Preserve service history and non-destructive compensation or replacement records.

OWNER COCKPIT AND USER-CONTROL REQUIREMENTS

- Show device identity, hardware, firmware, boot, calibration, mode, health, authority, update, evidence, and lifecycle.
- Expose stop, isolate, diagnose, update, rollback, replace, recall, restore, and retire actions with authorization.
- Explain unsafe, degraded, unknown, and service-required states accessibly.
- Export configurations, commands, telemetry, decisions, evidence, incidents, dissent, and continuity state.

REQUIRED OUTPUTS AND ARTIFACTS

- Embedded context, hardware-software partition, timing and resource model, interface and boot contract, update and diagnostics plan.
- Safety, security, calibration, manufacturing, service, field, evidence, recovery, acceptance, and retirement package.

EVIDENCE REQUIREMENTS

- Claims identify device, hardware revision, firmware image, configuration, environment, workload, method, date, result, and limitation.
- Build, bench, simulation, certification, field telemetry, or update evidence proves only its declared scope.

- Timing, power, thermal, safety, security, update, and recovery evidence identifies conditions and coverage.
- Missing provenance, calibration, field, or failure evidence constrains status.

VERIFICATION AND ACCEPTANCE TESTS

- Trace requirements through hardware-software allocation, image, device state, physical behavior, evidence, recovery, and retirement.
- Exercise boot failure, image tamper, power loss, memory exhaustion, timing overrun, bus fault, sensor loss, update interruption, and rollback.
- Verify device identity, secure update, watchdog, safe state, diagnostics, audit, privacy, and field recovery.
- Reproduce representative claims from exact hardware, firmware, configuration, and test package.

FAILURE STATES AND ANTI-PATTERNS

- Treating build success, bench test, signed image, or nominal telemetry as field safety or reliability proof.
- Ignoring timing, calibration, power, thermal, bus, physical access, service, and update dependencies.
- Allowing AI or remote administration to bypass secure boot, safety, provenance, or audit.

RECOVERY AND RECONSIDERATION

- On unsafe state, image failure, hardware fault, timing fault, unauthorized update, or evidence loss, enter safe mode and preserve records.
- Identify device owner, operator, authority, affected users, evidence, service, communication, and review condition.
- Use diagnostic, rollback, manual service, replacement, recall, restore, compensation, or retirement.
- Reopen on hardware, firmware, environment, model, update, supplier, policy, or lifecycle change.

COMPLETION CRITERIA

- Device, hardware, firmware, timing, resources, interfaces, controls, evidence, service, recovery, and lifecycle are explicit.
- Boot, fault, update, field, service, degraded, and retirement behavior is addressed or justified.
- Exact limits and unresolved evidence are recorded; CONTENT-COMPLETE does not imply field readiness.

HANDOFF AND CONTINUITY REQUIREMENTS

- Preserve device, hardware, firmware, image, calibration, command, telemetry, incident, decision, evidence, dissent, and gaps.
- Hand off to Hardware, Software, Robotics, Security, Safety, Testing, Manufacturing, Operations, and Retirement.
- Record changes requiring reopening affected architecture and certification baselines.

RELATED CODEX SECTIONS

- 019-058 Requirements through Robotics Architecture; 060 Hardware Architecture.
- 068-074 firmware, embedded, IoT, telecommunications, control, and robotics engineering; 198-199 embedded and hardware security; 232, 252 hardware and safety testing.

SOURCE BASIS AND UNRESOLVED SOURCE GAPS

- Based on the Canonical Section Contract, shared governance files, Sections 019-058, Owner Cockpit Requirements, Status and Evidence Lexicon, and local architecture baseline.
- No universal processor, real-time, safety, secure-boot, update, certification, environmental, or field-service standard is supplied. Actual devices, suppliers, authorities, and evidence remain unresolved.

060

SECTION 060 - HARDWARE ARCHITECTURE

CODEX/03_SECTIONS/060_HARDWARE_ARCHITECTURE.txt

SHA-256 85a2164aad47f3446470dfee97f17b00afd616c18b03ed5472076158c2689fc8

8,989 BYTES / PRESERVED SOURCE IDENTITY

SECTION 060 — Hardware Architecture

STATUS: CONTENT-COMPLETE

PURPOSE

Define the physical components, interfaces, materials, energy, thermal, mechanical, electrical, manufacturing, service, safety, security, and lifecycle structure of hardware that realizes approved outcomes.

REQUIRED OUTCOME

- A hardware context with components, interfaces, tolerances, environments, suppliers, manufacturing, service, failure, evidence, owners, and lifecycle.
- Traceability from requirements through electrical, mechanical, embedded, software, testing, operation, maintenance, disposal, and retirement.

WHY THIS MATTERS

Physical design fixes constraints that software cannot wish away. Tolerance, heat, energy, supply, materials, manufacturing variation, serviceability, and disposal determine whether a design is safe, reliable, maintainable, and honest about its limits.

APPLICABILITY

- Physical components, equipment, devices, materials, energy, packaging, or facilities realize a material function or interface.
- Hardware failure, variation, manufacture, service, supply, environment, or disposal affects consequence.

NON-APPLICABILITY CONDITIONS

- No physical component, material, interface, energy, environment, supplier, manufacturing, service, or lifecycle effect changes.
- An approved baseline covers exact hardware, environment, and change with no material impact.
- The artifact is conceptual or simulated and explicitly not a certification or field-readiness claim.

Record hardware, scope, owner, evidence date, assumptions, environment, and reopening trigger.

PREREQUISITES AND DEPENDENCIES

- Sections 019-059 provide requirements, software, embedded, robotics, safety, security, resilience, and operations context.
- Hardware, electrical, mechanical, manufacturing, supplier, environmental, safety, service, and disposal authorities are known or open.

REQUIRED INPUTS

- Functions, loads, materials, dimensions, tolerances, power, thermal, environmental, interfaces, users, manufacturing, maintenance, and lifecycle.
- Safety, security, reliability, accessibility, testing, certification, supply, repair, recycling, disposal, and evidence requirements.

HUMAN RESPONSIBILITIES AND AUTHORITY

Hardware and product owners approve design, materials, tolerances, suppliers, manufacturing, safety, service, exceptions, certification, and retirement. Humans retain authority to halt manufacture, quarantine, recall, repair, replace, dispose, and retire.

AI RESPONSIBILITIES AND BOUNDARIES

AI may analyze authorized designs, tolerances, bills of material, test data, and supplier evidence, and draft alternatives or tests. It must preserve revision and physical context. It must not approve a design, substitute material, change manufacturing, certify safety, or direct physical action without authority.

APPLICABLE EXISTING AGENT ROLES

- Architect, Researcher, Analyst, Security Auditor, Tester, Verifier, Governance Reviewer, Critic, Planner, Implementer, Documentation Writer, Memory Curator, and Red Team.

REQUIRED ACTIVITIES

1. Define product, environment, loads, users, physical effects, owners, suppliers, and lifecycle.
2. Decompose components, materials, interfaces, energy, thermal, mechanical, electrical, and environmental constraints.
3. Analyze tolerance, variation, failure, hazards, service, manufacturing, supply, security, and disposal.
4. Allocate controls for safety, reliability, accessibility, embedded integration, testing, evidence, and recovery.
5. Validate with qualified hardware, electrical, mechanical, manufacturing, safety, security, service, and affected-user authorities.
6. Test prototype, production, environmental, stress, fault, aging, service, recovery, and disposal conditions.
7. Baseline revision, evidence, decisions, dissent, residual risk, and gaps.

HUMAN-AI INTERACTION REQUIREMENTS

- Show component, revision, material, supplier, load, environment, authority, evidence, and physical consequence.
- Require human approval for material, supplier, tolerance, safety, manufacturing, recall, and irreversible changes.
- Preserve manual inspection, stop, quarantine, traceability, evidence, repair, and recovery paths.
- Do not treat optimization, simulation, or generated design as certification.

ARCHITECTURE AND SYSTEM IMPLICATIONS

- Separate requirement, design, drawing, revision, prototype, production unit, field configuration, test, and evidence.
- Define interface, tolerance, energy, thermal, material, manufacturing, calibration, service, and disposal semantics.
- Preserve configuration and serial traceability through manufacturing, field service, recall, replacement, and retirement.
- Protect hardware and design records, supplier data, diagnostics, and evidence under access and retention controls.

OWNER COCKPIT AND USER-CONTROL REQUIREMENTS

- Show hardware revision, serial or batch, components, materials, supplier, environment, health, authority, evidence, and lifecycle.
- Expose inspect, quarantine, repair, replace, recall, update, restore, dispose, and retire actions with approval.
- Explain unsafe, out-of-tolerance, unknown, service, and recovery conditions.
- Export drawings, revisions, test results, decisions, evidence, incidents, dissent, and continuity state.

REQUIRED OUTPUTS AND ARTIFACTS

- Hardware context, component and interface model, materials and tolerance record, environmental and failure analysis, supplier and manufacturing plan.
- Safety, security, embedded integration, testing, service, repair, recall, disposal, evidence, acceptance, and retirement package.

EVIDENCE REQUIREMENTS

- Claims identify hardware revision, unit or batch, material, configuration, environment, load, method, date, result, and limitation.

- Simulation, drawing, prototype, supplier statement, certification, inspection, or field test proves only its declared scope.
- Production, environmental, safety, reliability, service, and disposal evidence identifies sample and coverage.
- Missing revision, provenance, manufacturing, or field evidence constrains status.

VERIFICATION AND ACCEPTANCE TESTS

- Trace requirements through components, interfaces, manufacturing, embedded behavior, physical outcome, evidence, service, and retirement.
- Exercise tolerance, thermal, power, vibration, corrosion, aging, misuse, component loss, supplier change, counterfeit, repair, and disposal.
- Verify revision control, material identity, safety, accessibility, security, serviceability, recall, and evidence integrity.
- Reproduce representative claims from exact drawings, unit or batch, configuration, and test package.

FAILURE STATES AND ANTI-PATTERNS

- Treating a drawing, prototype, simulation, supplier claim, or certification as universal field proof.
- Ignoring manufacturing variation, tolerance stack, thermal, supply, service, counterfeit, disposal, or physical access.
- Allowing AI or schedule pressure to substitute materials, skip tests, or conceal out-of-tolerance units.

RECOVERY AND RECONSIDERATION

- On defect, hazard, counterfeit, supplier change, evidence loss, or out-of-tolerance condition, quarantine affected units and preserve traceability.
- Identify owner, authority, affected users, communication, evidence, service, and review condition.
- Use inspection, repair, replacement, recall, redesign, compensation, secure disposal, or retirement.
- Reopen on component, material, supplier, revision, environment, firmware, policy, or lifecycle change.

COMPLETION CRITERIA

- Components, interfaces, materials, tolerances, suppliers, manufacturing, environment, evidence, service, recovery, and lifecycle are explicit.
- Fault, variation, recall, repair, disposal, embedded, and retirement paths are addressed or justified.
- Exact limits and unresolved evidence are recorded; CONTENT-COMPLETE does not imply certified or field-ready hardware.

HANDOFF AND CONTINUITY REQUIREMENTS

- Preserve revisions, units, batches, materials, suppliers, drawings, tests, incidents, decisions, evidence, dissent, and gaps.
- Hand off to Embedded, Software, Robotics, Electrical, Mechanical, Manufacturing, Security, Safety, Testing, Operations, and Retirement.
- Record changes requiring reopening affected architecture, certification, and lifecycle baselines.

RELATED CODEX SECTIONS

- 019-059 Requirements through Embedded Systems Architecture; 061-074 engineering and control sections.
- 198-199 embedded and hardware security; 232 Hardware Testing; 252 Safety Testing; 351-355 manufacturing and device provisioning; 399-401 hardware disposal.

SOURCE BASIS AND UNRESOLVED SOURCE GAPS

- Based on the Canonical Section Contract, shared governance files, Sections 019-059, Owner Cockpit Requirements, Status and Evidence Lexicon, and local architecture baseline.
- No universal hardware, material, environmental, manufacturing, certification, service, recycling, or disposal standard is supplied. Actual designs, suppliers, units, authorities, and acceptance evidence remain unresolved.